

Optimizing Classical Compiler Passes

Table of Contents

Bril Phase Selection and Ordering	2
MLGO: a Machine Learning Guided Compiler Optimizations Framework	8
Optimizing LLVM Pass Sequences with Shackleton: A Linear Genetic Programming Framework	19
Learning Compiler Pass Orders using Coreset and Normalized Value Prediction	29
Efficient compiler optimization by modeling passes dependence	42
Automatic Tuning of Compilers Using Machine Learning	60
END	164

CS 6120 Schedule Syllabus Slack GitHub Blog Projects

The CS 6120 Course Blog

Bril Phase Selection and Ordering

by Horace He & Qian Huang November 19, 2019

Background

Modern compilers are often organized in terms of independent "passes". That is, a compiler will consist of tens or even hundreds of optimization passes that each do a clearly defined task. Along with these many passes, however, comes the problem of ordering them. For example, it isn't obvious whether one should run constant folding or copy propagation first. Although some of these may have well defined answers, it's likely that one can find an example in which the ordering performs poorly.

We set out to try and build a ML-based phase selection/ordering system. Due to some amount of problems/discoveries, we ended up only analyzing the effects of different orders and generating an optimal sequence of optimizations.

Design Overview

Optimizations

To set up the phase ordering task, we selected and implemented several optimization passes:

- For Dead Code Elimination (DCE), we used the given implementation directly.
- For data flow analysis based optimization passes, we implemented Copy Propagation and Constant Folding, as defined in the lecture. Each pass will gather data flow analysis information first then modify the instructions in place.
- For control flow graph optimizations, we implemented Branch Removal, Unreachable Code Elimination, CFG cleaning and Tail Merging. We refer to this [slide](#) for more details.
 - Branch Removal

If we can decide the guard value to be true or false, then we can eliminate one of the two branches accordingly.
 - Unreachable Code Elimination(UCE)

The basic blocks unreachable from the entry block will be removed.

- CFG cleaning

We can simplify CFG by several transformations that eliminate useless edges and combine some basic blocks. Specifically, we can 1) replace a branch instruction with two identical destinations to a jump instruction; 2) merging empty blocks; 3) merging two blocks with only one edge in between under some cases.

- Tail Merging

If the predecessor or successors share a same section of code in the end or beginning correspondingly, we can change the jump to reuse the same segment of code.

In general, the optimizations we selected are tailored for the simple structure of Bril, which has a lot of constant calculation and branches that do not allow falling through. Passes like DCE and UCE will likely to remove more codes, but other passes might or might not generate new opportunities for DCE and UCE. Thus we need to order passes properly to obtain best performance, especially under the case where we can only apply a limited number of passes.

Random Program Generator

In order to analyze programs in Bril, one obstacle was that we didn't have a large enough corpus of code to analyze. To do statistical analysis or any kind of optimization techniques, we need a substantial training set to evaluate our ordering methods.

To create this training set, we created a random Bril program generator. Our program generator evaluates each decision independently, and doesn't take any care to preserve program wide structure. We follow this flow chart at each step.

1. First, we randomly chose whether we want to insert a binary operation, a constant, or an id.
2. Next, given the instruction type, we decide whether it should be a boolean or an integer.
3. Finally, we choose whether this instruction should write to a new identifier or an already existing identifier. If it's an already existing identifier we uniformly choose one from all the identifiers of the correct type.

One thing that we've ignored so far is control flow. Considering that several of our optimizations are control flow optimizations, we clearly need to handle them somehow. However, introducing arbitrary control flow is fairly tricky. For example, we

may construct non-terminating programs, which make evaluating them/checking our optimizations for bugs tricky.

Thus, we settled on constructing a Directed Acyclic Graph. Every fixed interval, we insert a label. In addition to that label, we have a chance of adding a jump/branch to a succeeding label.

We note that there are a lot of hand set probabilities in this generation process that have the ability to heavily affect the types of programs generated. For example, introducing a lot of control flow makes it more difficult for other optimizations to be applied. Thus, we select a mix of random constants.

Phase Ordering Analysis

So, first of all, how much does the phase ordering even matter? To evaluate the effectiveness of our optimizations, we will use # of instructions as a proxy.

We will use three different test sets for this. One where we have a lot of control flow, one with no control flow, and one with intermediate control flow. These will be called CFLow, CFMid, and CFHigh.

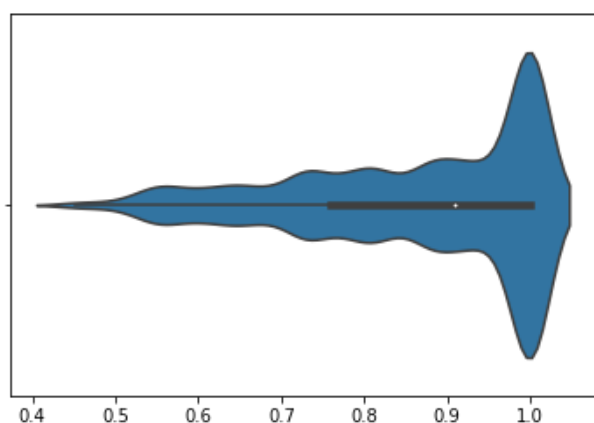
Then, we apply 200 random permutations of the 7 optimizations mentioned above. We record the worst and the best performing permutation, and report that here.

Dataset	Avg Low	Avg High	Avg Ratio
CFLow	18.7	26.3	.72
CFMid	34.0	39.0	.87
CFHigh	52.7	54.7	.96

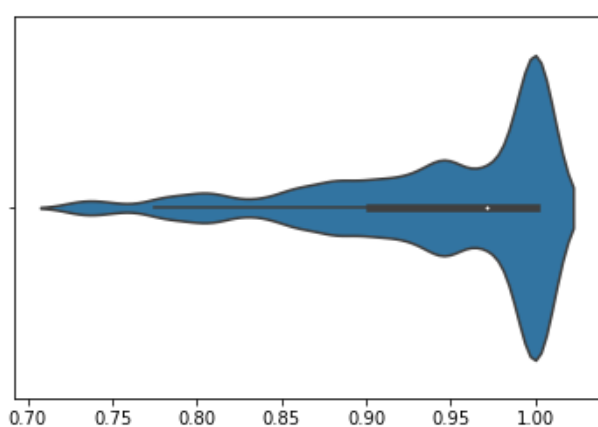
As we can see, especially on the programs with low amounts of control flow, there can be a huge difference between the best and worst permutation.

However, merely the min and max does not provide much information about how the distribution of possible permutations performs. To show that, we will normalize all permutation performances by the worst performing permutations and plot the distribution as a violin plot. We see that there is a tail of permutations that performs substantially better than the majority of permutations, many of which perform as bad as the worst permutation.

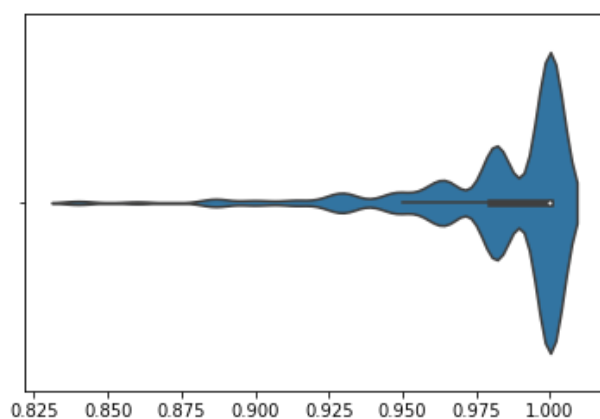
CFLow:



CFMid:



CFHigh:



Phase Ordering Heuristics

However, questions remain about the nature of the good performing permutations. Is there a single true permutation? Or does the ideal permutation vary among programs? To answer that, we generated random permutations and evaluated them across all 3 datasets. We report the percentage (# instrs of optimized program/#

instrs of unoptimized program) of the best single permutation across a given dataset, the average percentage for optimizing the permutation per program, and the percentage for the best overall permutation across all three datasets.

Dataset	Best Permutatation	Best Individual Permutation	Overall Permutation
CFLow	35.81	35.32	36.15
CFMid	59.28	59.24	59.28
CFHigh	77.60	77.54	77.61

Somewhat disappointingly, we find that there are permutations that always perform very close to the optimal permutation for any program. On average, the optimal permutations reduced the code length to 57.68% of the original code length.

Doing so, we also find that there are many permutations with the best performance as the best permutation (selected by best average performance across all 3 datasets). We were curious if there was essentially some subset of the optimizations that mattered. We analyze all of the "optimal" permutations for common subsequences, and find that all of the optimal permutations had the following order of permutations:

1. Copy Propagation
2. Constant Folding
3. Branch Removal
4. Dead Code Elimination

We find that for our datasets, these are the only optimizations that matter. Removing the other 3 optimizations still provides the same performance across our dataset.

However, note that per program, there is still occasionally performance improvements to be found. Of our 150 random programs, there are 10 programs in which we can find a better order than the fixed dataset wide one. We're unable to isolate any particular pattern these programs/permutations have.

Future Work

Originally we were interested in using ML to choose optimization orders. However, because of how effective a fixed optimization order is, we decided that we were unlikely to get substantive improvements from a ML based ordering system. We suspect that this is due to the low amount of optimizations we have, as well as the lack of more "ambiguous" optimizations. Optimizations like constant folding or dead code elimination always improves performance of code (we think...). Thus, it's

possible that there is a fixed optimization order that achieves almost optimal performance. However, the implementation cost of adding more optimization passes (including verifying their correctness) limited us to only 7 optimizations.

Future work would include more optimization passes. If adding more optimization passes gave more ambiguity in the optimization ordering, we could then explore program specific orderings.

Another aspect of optimization passes that we completely ignored is the fact that we can run each optimization pass more than once. For example, dead code elimination is often run many times due to its computational cheapness and the fact that dead code is often generated by other optimization passes. It could be worth exploring that as well.

Horace He is a senior undergraduate studying CS & Math. He's interested in the intersection of machine learning and compilers. This past summer, he worked on PyTorch at Facebook, as well as automatically scheduling Halide programs.

Dian Huang is a junior undergraduate studying CS & Math. She's mostly interested in machine learning and algorithms. She's curious about the compiler optimizations people perform in practice.

This is the course blog for [CS 6120](#), the PhD-level compilers course in the computer science department at Cornell. You can subscribe to [posts on the blog](#) with [RSS](#).

Cornell University — Department of Computer Science

MLGO: a Machine Learning Guided Compiler Optimizations Framework

Mircea Trofin*
Google, Inc.
mtrofin@google.com

Yundi Qian*
Google, Inc.
yundi@google.com

Eugene Brevdo
Google, Inc.
ebrevdo@google.com

Zinan Lin
Carnegie Mellon University
zinanl@andrew.cmu.edu

Krzysztof Choromanski
Google, Inc.
kchoro@google.com

David Li
Google, Inc.
davidxl@google.com

Abstract

Leveraging machine-learning (ML) techniques for compiler optimizations has been widely studied and explored in academia. However, the adoption of ML in general-purpose, industry strength compilers has yet to happen.

We propose MLGO¹, a framework for integrating ML techniques systematically in an industrial compiler – LLVM. As a case study, we present the details and results of replacing the heuristics-based inlining-for-size optimization in LLVM with machine learned models. To the best of our knowledge, this work is the first full integration of ML in a complex compiler pass in a real-world setting. It is available in the main LLVM repository.

We use two different ML algorithms: Policy Gradient and Evolution Strategies, to train the inlining-for-size model, and achieve up to 7% size reduction, when compared to state of the art LLVM -Oz. The same model, trained on one corpus, generalizes well to a diversity of real-world targets, as well as to the same set of targets after months of active development. This property of the trained models is beneficial to deploy ML techniques in real-world settings.

1 Introduction

Previous work [13, 25] has shown promise in replacing compiler optimization heuristics with machine-learned policies. Heuristics are algorithms that, empirically, produce reasonably optimal results for hard problems, within pragmatic constraints (e.g. "reasonably fast"). In the compiler case, heuristics are widely used in optimization passes, even those leveraging profile feedback, such as inlining and register allocation. Such passes have significant impact on the performance of a broad variety of programs. These problems are often NP-hard and searching for optimal solutions may require exponential time or memory. Reinforcement Learning (RL) is a family of machine learning techniques that may be applied to find increasingly optimal solutions through an automated iterative exploration and training process.

Our focus is ahead-of-time (AOT) compilers, specifically, C/C++. In a real-world setting, we expect two main benefits from machine learning techniques: first, heuristics are human-trained based on a human-manageable set of benchmarks and regression cases. Machine learning easily scales to large corpora of training examples - which we expect to increase the likelihood of obtaining policies that generalize well. This is important because, as we will explore in detail, we do not want to retrain policies too frequently (it is an adoption blocker), nor do we want to train 'online', while the compiler is running in production (it would affect determinism). Second, heuristics are human-written code that needs to be maintained. This places a downward pressure on the number of program properties ("features") and the combinations between them that can be practically leveraged. We believe using more features and feature combinations would result in better optimization decisions. ML scales well with the addition of features, and can discover profitable feature combinations. While ML techniques may be able to address these two points, a trade-off is that maintaining and evolving them requires practices and approaches different from those used for heuristics.

As pointed out, applying ML to compiler optimizations has been explored by academia, but it has not been adopted in production environments. To explore why, we chose a pilot optimization problem and approached it with the intention to deploy in production. The goal of the pilot is to inform problem framing and design choices. Other than performing better than the tip-of-tree production compiler, we did not aim to advance the state of the art for the pilot problem.

The chosen problem is inlining-for-size in LLVM, and in particular, the inlining decision heuristics. The expectation was that this would offer representative challenges - size optimization is important for real-world scenarios, such as mobile software, and inlining is a particularly challenging optimization (see Section 2.2).

We chose size rather than speed for the pilot because size is relatively easy to measure and non-noisy, which we expected to aid in rapid prototyping by removing one source of potential problems (noisy rewards). We acknowledge that translating our experience to performance problems may

*These authors contributed equally.

¹We welcome your feedback! Please open an issue at <https://github.com/google/ml-compiler-opt> with the label *paper*.

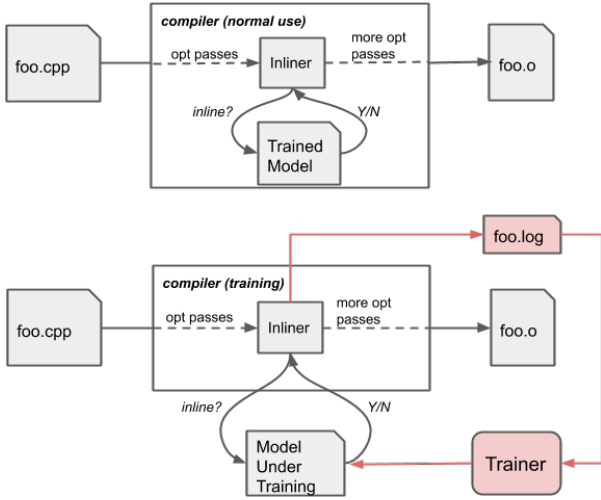


Figure 1. MLGO Overview

seem non-immediate at a first glance, but we believe that not to be the case: for example, inlining for speed can be understood partially as a "for-size" problem, with respect to the instruction working set (more in Section 8).

From a very high level, our MLGO framework separates the *use* of the compiler from the *training* of policies as shown in Figure 1. Day-to-day production use is unchanged; as an implementation detail, a trained model embedded in the compiler is used to make decisions (in this case, inlining) that were previously handled by a manual heuristic. Training happens separately using a large, representative corpus of intermediate representation (IR) modules. The training process is iterative, each step using an updated policy, so the policy is not embedded in the compiler. During training, the inliner produces a log that records the inlining process (features, decisions, etc). The logs are collected and fed to the training algorithm to produce a new model.

The paper is organized as follows: Section 2 provides an overview of the relevant ML techniques and the LLVM inliner. Section 3 gives an overview of MLGO: our framing of the problem of applying ML to compiler optimizations, and our methodology. Section 4 describes the policy training in MLGO for the inlining problem. Section 5 details the implementation, in LLVM, of our pilot project. The results are presented in section 6 and the related work is described in Section 7. Finally in Section 8, we discuss our plans for applying the lessons learned so far to speed problems, as well as next steps in ML techniques that we are considering. Section 9 concludes the paper.

A note: throughout this paper, we use interchangeably the terms **policy** and **model**. A compiler optimization policy is a decision rule that takes actions inside the optimization pipeline (e.g., "should we inline this call graph edge or not?").

"Model" refers to a neural network implementing an optimization policy. Also, we use the term **heuristic** to refer to manually-crafted decision rules.

2 Background

2.1 Machine Learning Techniques for Replacing Compiler Optimization Heuristics

There are two characteristics that make reinforcement learning (RL) a suitable tool for replacing compiler optimization heuristics: 1) there are no examples showing optimal strategies for these heuristics — in the inlining problem, we don't know whether inlining or not for a certain call site is the optimal choice; 2) we can efficiently explore different strategies, and improve strategies from those experiences. The absence of examples ("labels") means we cannot use supervised learning. In contrast, RL is an area of machine learning that learns from trial and error instead of given labels. It has proven its success in robotics, playing Atari games, playing the game of Go, etc [16, 18, 24]. In RL, an agent (i.e., the compiler) learns by repeatedly interacting with the environment (i.e., compiling) and gradually improves its policy (i.e., decision rules). More specifically, by compiling software again and again with different strategies, the compiler will come up with better and better policy on its own with RL algorithms.

Previous work has shown Evolution Strategies (ES) to be a competitive alternative to RL algorithms in MuJoCo and Atari tasks [8, 22]. Motivated by this, we also tried this method in compiler optimization problems. ES are a class of black box optimization techniques. Like RL, ES training is also able to gradually improve the strategy with trial and error, and thus is also a suitable tool for compiler optimization problems.

2.2 High-level Overview of the Current LLVM Inlining Pass

Today's LLVM inliner is a pass operating on a strongly-connected component (SCC) of the static call graph in a module², at a time in bottom up order. The inlined callee's call sites are added to a work list and iteratively considered for inlining in a top down fashion. A pipeline of optimizations (Inst combine, scalar replacement of aggregates (SROA), loop optimizations, etc) is then applied on each function in the SCC³, after the SCC was processed. The effects of these optimizations impact inlining decisions of call sites in the SCC calling into the last one.

LLVM inlining consists of many heuristics: the choice of call site traversal, the set of "cleanup" function passes run on functions after they are modified because call sites were

²in LTO or ThinLTO mode, a module can consist of IR from multiple source files

³The DAG walk is repeated up to a set of times if de-virtualization happens in cleanups

inlined, the timing of these cleanups, and finally, the decision to inline or not a specific call site.

The decision to inline or not a call site is itself built on top of a rich set of heuristics. The compiler first computes the static "cost" of the callee post inlining by traversing the callee body simulating post-inline cleanup passes. If some call site arguments are known to be constant at compile time, that information is used to evaluate what instructions / basic blocks would be simplified, should the inlining be carried out. The computed cost is then compared with a threshold. The threshold value is based on things like call site hotness, inline keyword, etc. Bonuses are also given to callees with a single basic block or high percentage of SIMD instructions. In certain cases, the compiler may also choose to defer inlining if inlining the caller itself to its own callers first may result in better savings - it may be better to make a local non-optimal decision that, later, would open the opportunity for better optimizations due to more context being available, such as call parameters propagating from callers further up in the call graph.

These sets of heuristics have been tuned for years and our pilot project replaces the manual decision process described above with ML models.

3 MLGO

MLGO is a set of guidelines and requirements derived from our understanding of the problem of leveraging ML techniques for replacing manual optimization heuristics. We start with our understanding of the participating personas and their scenarios, which then motivates MLGO guidelines and design decisions.

3.1 Persona: Compiler User

This user wants to benefit from improved compiler optimizations. They care about: correctness and performance of the generated code; compilation determinism (i.e. identical output for identical input) - to leverage incremental builds; avoiding the added cost and complexity of new infrastructure requirements on build and release pipelines, such as new compiler run-time dependencies or new steps (like training); and timeliness of the build, as it impacts hardware resource planning and developer productivity⁴. **Our goal is to introduce no changes to this user.**

To achieve this, the MLGO guidelines are:

1. To maintain correctness guarantees, we replace heuristics, not semantics-preserving code. For example, we change the decision making process for carrying out function inlining, not how the inlining action is implemented. This is along the insight of separation of correctness vs policy observed earlier in [27]

⁴There are non-ML driven optimization alternatives that trade off significantly increasing compilation time for improved optimizations. An ML alternative needs to be competitive to justify its other trade-offs

2. 'Online' training - meaning, training while the compiler is executing in production - is an anti-goal for us: it would hurt determinism and compilation performance. Instead, policy training happens offline. Trained policies are embedded in the compiler as statically linked native code, and the resulting compiler is subjected to the same release process it currently is. Build and release infrastructures and pipelines of targets using the compiler do not need to be changed. Build determinism using the ML-enabled compiler is ensured because the policies are fixed - no training happens when the compiler runs, only inference. While native compilation doesn't guarantee timeliness, it eliminates one source of concern. Due engineering diligence still needs to be applied to ensure timely feature extraction, for instance.

3. We require ML techniques that yield policies that generalize well over different code bases and code changes, and do not need frequent retraining. The compiler user doesn't *have* to worry about policy training (although they are free to do so and potentially get better results). This is akin to how, in the context of manual heuristics, a compiler user doesn't have to fine-tune passes (or author code in them) to get reasonable results. In particular, we do not see automated tuning of existing heuristic parameters as a viable solution. Tuning parameters have been available in compilers for a long time, and the experience has been that a set of values does not translate well from target to target. The policy, while adjustable, is still dominated by combinations/evaluations identified manually. In addition, requiring re-tuning would complicate product build and release pipelines, which we want to avoid on behalf of our user.

We refer to this use of policies as **release mode** (since it is encountered by users of a released/shipped compiler).

3.2 Persona: Compiler Engineer

This user wants to drive better optimizations in the compiler, diagnose regressions, and incorporate findings:

1. **Policy Creation.** The engineer wants to incorporate ML techniques in a compiler optimization pass.
2. **Policy Improvement.** Here, they investigate a specific regression encountered in production, or want to improve a ML-enabled pass.
3. **"The Ship Blocker".** The engineer must quickly resolve a ship-blocking regression introduced by a hot patch, and caused by a misbehaving RL-enabled policy.

In all of these cases, the *compiler engineer* improves a policy through repeated exploration and training (see Section 4.4). They want flexibility in replacing the model under training, and have less concern with timeliness and determinism, especially since models under training may use small random

perturbations, to facilitate exploration. We refer to this use of policies as **development mode**. Here, models are loaded via a command line option, the compiler may have extra runtime dependencies, and model evaluation may involve changes to the runtime behavior of the compiler — because, for example, the model evaluators may be multi-threaded and/or JIT-ing.

Because of the tension between heuristic code complexity and hypothesized ability to improve the heuristic by incorporating more features (as discussed in the introduction), MLGO forgoes goals of human comprehensibility of the resulting policy (in contrast to [25]). Instead, we focus on developing and evaluating alternative methodologies to address the above scenarios. We discuss our current understanding of the trade-offs, and expect that more clarity will arise as we apply the approach through the lifetime of a number of diverse projects. Section 4 will detail our experience with developing *Policy Creation*. We have less experience, at this point, with *Policy Improvement* and *"The Ship Blocker"*, and derive our direction from experiences in other domains.

Policy Improvement is currently (i.e. for manual heuristics) an iterative engineering process. The trigger is typically regressions identified in the field. The compiler engineer diagnoses the problem, hypothesizes a solution, then ensures that the solution does not introduce regressions in some corpus of benchmarks; if regressions happen, the process is repeated. In the MLGO methodology, we envision a gradual process. We do not believe it presents significant negative trade-offs compared to the state of the art:

1. Start by incorporating regression use-case(s) into training corpus and retrain the policy.
2. If that fails, hypothesize missing features. This requires some manual diagnosing of the current policy behavior. While we treat the policy as a black box, we do observe its effects, and can formulate hypotheses as to what information may be missing - since the information we provide (features) is also observable. The needed skill set is close to what compiler engineers currently employ for manual heuristic development, and, just like for manual heuristics, evolving the feature set is likely an iterative process. Typically, adding features and retraining shouldn't result in regressions for the previous training corpus, which is a benefit of our approach over the manual heuristic case. We should note that, if applying feature auto-extraction [10, 13] proves feasible in production, this step collapses to the previous step.
3. If the above also fails, involve an ML expert to investigate alternative training algorithms. This is akin to today's (rare) full pass rewrites (for example: new register allocation pass). The difference is the need for cross-disciplinary interaction. Our hope is that, with time and experience, MLGO offer a reusable library of

best practices and training solutions available "off the shelf" to compiler engineers.

Ship blockers are those cases where the compiler engineer doesn't have the luxury to do deep investigations into compiler behavior, since they are on a tight time budget. Assuming the pathological case is identified (i.e. which compilation unit causes the compiler to misbehave), in the case of manual heuristics, the levers of control are: trial-and-error with different compilation flag values (change policy thresholds, for instance); modify user code (use inlining directives, for instance); or disable the specific optimization for a specific module.

In MLGO, the picture is similar. Other than policy thresholds, the control levers available to the engineer are the same. In addition, the engineer may choose to revert to an earlier version of the policy, or manual heuristics for the problematic module, and, if needed, experiment with threshold flags. Specific to ML-based policies, we are exploring with local training and overfitting: as we will detail, our experience so far shows that it is possible to train a policy on a single modern, multi-core workstation, and obtain a reasonably good result within a day. An engineer could attempt to specialize a policy to overfit for the pathological case, and compile that case with the specialized policy (while compiling the rest of the project with the non-overfitted policy). This is similar to "experimenting with flags", with the exception that the exploration is directed by a training algorithm and more likely to quickly converge to a solution. The trade-off is that changing heuristic flag values does not require a training infrastructure — even if that infrastructure could be run locally.

4 MLGO Policy Training

In this section, we show how we use reinforcement learning (RL) and evolution strategies (ES) to train inlining policies in the MLGO framework. Sections 4.1 and 4.2 present how we train the inlining policy for the inlining-for-size problem with RL and ES algorithms, and Section 4.3 compares the pros and cons of the two algorithms. Section 4.4 concludes this section by giving an overview of our policy training infrastructure⁵.

4.1 Reinforcement Learning

4.1.1 RL Problem Formulation

RL aims to find an optimal policy for a Markov Decision Process (MDP). MDP is a mathematical framework that models sequential decision making — in inlining-for-size problem, we make sequential decisions whether to inline or not. An MDP can be represented by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$ with state space $\mathcal{S}$, action space $\mathcal{A}$, state transition distribution $\mathcal{P}(s'|s, a)$, and reward function $\mathcal{R}(s, a)$. In the MDP formalism, at time t , the agent observes the state $s_t \in \mathcal{S}$ of the

⁵<https://github.com/google/ml-compiler-opt>

environment, then decides to take an action $a_t \in \mathcal{A}$. It also receives the reward $r_t = \mathcal{R}(s_t, a_t)$. The environment state then transitions to $s_{t+1} \in \mathcal{S}$ by sampling from the probability distribution $\mathcal{P}(s_{t+1}|s_t, a_t)$. This process repeats until the agent reaches a termination state at time T . The agent's decisions are a function (we call it policy) $\pi = Pr(a|s)$ that maps observed state s to a distribution over actions. In our case, π is a neural network and we call it policy network. RL algorithms aim to find the optimal policy π^* to maximize the total reward⁶ $R = \sum_{t=0}^T r_t$.

We first formulate the inlining-for-size problem as an MDP. The inlining pass traverses over the call sites in the call graph in a deterministic order and decides at each call site whether to inline or not. Every inlining operation changes the call graph. We treat this as a sequential decision process, and we formulate it into an MDP as:

state $\mathcal{S}$: we define the current call graph and the call site being visited to be the state.

action $\mathcal{A}$: $\mathcal{A} = \{0, 1\}$, where 1 means inline and 0 means do not inline.

state transition probability $\mathcal{P}$: unlike usual MDPs, the state transition is deterministic (no randomness) in the inlining problem. After an action is taken (inline or not inline), the compiler determines what the next state is (updates the call graph and decides the next call site to visit).

reward $\mathcal{R}$: reward is defined to be the native size reduction after the action is taken. If $a = 0$ (do not inline), the reward is 0 since nothing changes; if $a = 1$ (do inline), the reward is defined as:

$$S(Caller_{before}) - S(Caller_{after}) + \begin{cases} S(Callee), & \text{callee deleted} \\ 0, & \text{callee remains} \end{cases} \quad (1)$$

where $S(f)$ is the native size of function f . Note that we do not actually know what the native size would be for a certain function while performing inlining, since inlining operates at the IR level. The definition for reward here is not practical for training. We will discuss how we tackle this challenge next.

4.1.2 Policy Gradient Algorithm

Policy Gradient (PG) [28] is a family of RL algorithms derived from REINFORCE [34]. Though we use Proximal Policy Optimization (PPO) [23], we first briefly introduce REINFORCE — as PPO is an enhancement to REINFORCE and they work in very similar ways.

On a high level, all PG algorithms gradually improve the policy π_θ by computing the gradients of the parameters θ in the policy network w.r.t. the total reward R , and then update θ with the gradient to improve the policy. With $J(\theta)$ denoting the expected reward under policy π_θ , the gradient

$\nabla_\theta J(\theta)$ in REINFORCE is computed as:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^T R \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \quad (2)$$

Here $\mathbb{E}$ is an expectation over the policy π_θ being applied to an inlining pass. In practice, this expectation is approximated with Monte Carlo methods — with n trajectories⁷ collected from compiling with policy π_θ , the parameter θ is updated with:

$$\theta \leftarrow \theta + \alpha \frac{1}{n} \sum_{i=1}^n \left\{ \sum_{t=0}^T R_i \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \right\} \quad (3)$$

where α is the learning rate. As θ is updated, the policy $\pi(\theta)$ tends to evolve in the direction that increases the total reward. Algorithm 1 describes the process — as training progresses, the policy gradually improves on its own by iterating between two stages: 1) compile with a new policy and collect fresh trajectories; 2) update policy network parameters θ .

Algorithm 1 MLGO PG Training Algorithm

- 1: Initialize θ
 - 2: **for** iteration = 1, 2, ... **do**
 - 3: Compile with policy π_θ to collect n trajectories
 - 4: Update θ using Equation 3
 - 5: **end for**
-

The details of training with PPO, which has several additional terms in the loss function, are available in [23]. One core improvement of PPO is to subtract a baseline B from the reward to reduce the variance. Equation 2 is modified as:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^T (R - B) \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \quad (4)$$

Here the baseline B describes what the R is supposed to be (irrelevant to policy). By subtracting it, $R - B$ provides better information about the effectiveness of the policy π_θ .

The total reward R in these equations can be replaced with the returns following action a_t : $\sum_{t'=t}^T r_{t'}$. In this case, the baseline B is a value network $V(s_t)$ predicting the future returns $\sum_{t'=t}^T r_{t'}$ from the state s_t . We choose to use the total reward R since: 1) it is directly available in the inlining-for-size problem, while partial returns would have to be approximated; 2) it is difficult to build the value network $V(s_t)$ with the reduced state. We will discuss the details in the next section.

⁶In general, the total discounted reward is $R = \sum_{t=0}^T \gamma^t r_t$, where γ is the discounting factor; but we take $\gamma = 1$ so we ignore it here.

⁷A trajectory is defined as $(s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$, and total reward $R = \sum_{t=0}^T r_t$

4.1.3 Challenges and Implementation Details

We run into two challenges when applying PPO to the inlining-for-size problem: 1) complex state space; 2) impractical reward definition.

Complex state space: Our MDP formulation defines the state as the current call graph and the call site being visited. Unfortunately, encoding and processing a call graph at each decision point may not be computationally practical for a general-purpose compiler to afford.

Impractical reward definition: It is difficult to know a function’s native size $S(f)$ during the inlining pass because native code lowering happens in a later pass, and because its structure may change due to more of its call sites being inlined.

To tackle the first challenge, we approximate the true state by distilling the state space to 11 numerical features as listed in Table 1. These features describe the local call site and provide some global information about the call graph. Section 5.1 details the features we use. We considered, but rejected for now, the use of (IR) code embedding techniques[3]; this allows us to minimize additional computational/memory costs. We plan to consider such techniques in the future.

Type	Feature
caller feature	caller_basic_block_count
	caller_conditionally_executed_blocks
	caller_users
callee feature	callee_basic_block_count
	callee_conditionally_executed_blocks
	callee_users
call site feature	callsite_height
	cost_estimate
	number_constant_params
call graph feature	edge_count
	node_count

Table 1. Features for Inlining for Size

One drawback of the simplified state is that it greatly reduces information available to the policy — it only contains a part of the local call site information, and limited global call graph information. We do not expect this to hurt the policy network because it is roughly the same information available to the current inlining heuristic. However, this reduced state vector does not allow us to build the value network $V(s_t)$ baseline — at a certain time t , the simplified state s_t is not informative enough to predict the future return $\sum_{t'=t}^T r_{t'}$.

A simple approach to side-stepping the lack of partial reward information and the side effect of reduced state representation is to use the total reward R instead of the partial return as shown in Equation 2. While the per-step reward is difficult to get, the total reward is the sum of the individual

(unknown) rewards — it is relatively easy to evaluate: evaluate native size with / without inlining, and subtract. In the total reward setup, the baseline B is defined as the estimated native size reduction of the module after the inlining pass. We can use the native size reduction under the heuristic policy as the baseline B .

Using the total reward instead of partial rewards has its drawbacks: 1) more data needs to be collected to achieve the same performance; 2) the final model quality may be worse.

4.1.4 Warmstart with Behavioral Cloning Policy

Instead of having the RL algorithm learn from the scratch (initialize θ randomly), we facilitate training by initializing θ from some "warmstart" policy. To facilitate the RL training, we need to have a "warmstart" policy that already performs reasonably well. An intuitive idea is the heuristic inlining decisions in LLVM. Therefore, we train the warmstart policy to imitate the heuristic inlining decisions in LLVM using behavioral cloning algorithm[4]. The behavioral cloning algorithm essentially views the problem as a supervised learning problem where the features are the same as the RL training and the label is the heuristic inlining decision — it trains a neural network that makes inlining decisions as close as the heuristic inliner does. In this way, we get a policy that makes decisions similar to LLVM’s current inlining heuristics and thus can serve as the warmstart policy to make our RL training much faster.

4.2 Evolution Strategies

Previous work has shown that ES, as a gradient-free black box optimization technique, is a competitive alternative to RL algorithms on MuJoCo and Atari tasks [22].

ES focuses on black-box optimization problems of the form $\max_{\theta} F(\theta)$, where F can be any *black-box function* that can be evaluated. Given θ , we essentially assume we have an oracle that calculates $F(\theta)$. In our specific case, θ are the parameters of the policy network π_{θ} and $F(\theta)$ is the total reward R — the native size reduction after inlining under policy π_{θ} for a certain module.

Instead of directly optimizing $F(\theta)$, ES focuses on optimizing $J(\theta)$, which is a smoothed version of $F(\theta)$:

$$\max_{\theta} J(\theta) = \max_{\theta} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} F(\theta + \sigma \epsilon). \quad (5)$$

Here $\mathcal{N}(0, I)$ denotes the multivariate normal distribution with zero mean and identity covariance matrix. Similar to PG, ES also takes the gradient of the parameter θ w.r.t. $J(\theta)$:

$$\nabla_{\theta} J(\theta) = \frac{1}{\sigma} \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} \{F(\theta + \sigma \epsilon) \epsilon\}, \quad (6)$$

and uses Monte Carlo approximation of the gradient to update θ to improve the policy:

$$\theta \leftarrow \theta + \alpha \frac{1}{n\sigma} \sum_{i=1}^n \{F(\theta + \sigma \epsilon_i) \epsilon_i\} \quad (7)$$

where α is the learning rate, and ε_i are vectors sampled from $\mathcal{N}(0, I)$.

Algorithm 2 describes the ES algorithm. Similar to PG, ES also iterates between data collection and policy update to gradually improve the policy.

Algorithm 2 MLGO ES Training Algorithm

```

1: initialize  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Sample  $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_n \sim \mathcal{N}(0, I)$ 
4:   Compile with policy  $\pi_{\theta+\sigma\varepsilon_i}$  to get  $F(\theta + \sigma\varepsilon_i)$ 
5:   Update  $\theta$  based on Equation 7
6: end for
  
```

4.3 PG v.s. ES

While the policy gradient algorithm and the evolution strategies algorithm are similar on a high level, they are different in many ways and have their pros/cons.

Complexity: The key advantage of ES is that it is conceptually simpler: 1) it requires less engineering complexity — unlike the PG algorithm where we need the logged trajectory during inlining ($s_1, a_1, s_2, a_2, \dots, s_T, a_T$) and the total reward R for training, the ES algorithm only needs the total reward. Thus we do not need to log the trajectory while doing compilation; this reduces both engineering complexity and storage/network requirements; 2) it has less requirements on the problem structure as long as there is an oracle telling the total reward $F(\theta)$ under the policy parameters θ . As a result, it is easier to apply ES to other compiler optimization problems as PG requires formulating the optimization problem as an MDP.

Sample Efficiency: Sample efficiency quantifies the amount of data required for training. The key advantage of PG is that it has much higher sample efficiency than ES. In the inlining-for-size problem, we observed that even though PG is trained using total reward, over 20X computational resources are needed to train an ES policy of similar quality. In problems where partial reward information is available after every decision point, we expect the sample efficiency gap to be even larger.

4.4 Training Infrastructure

PG and ES algorithms are very similar in policy training on a high level — both of them improve the policy π_θ by iterating between compiling with policy π_θ to collect data and update parameters θ . Figure 2 demonstrates their training workflow. Before the training, we prepare an IR corpus consisting of pre-inlining IR files extracted from some software. At each iteration, the trainer sends the policy π_θ to data collector, the data collector samples several IR files from the IR corpus, does compilation to collect training data, and sends the training data back to the trainer for training. The training

is done after several iterations, and the trainer exports the trained policy. We use TF-Agents[29] — an RL library in TensorFlow[1] for training and the policy is in the format of TensorFlow SavedModel.

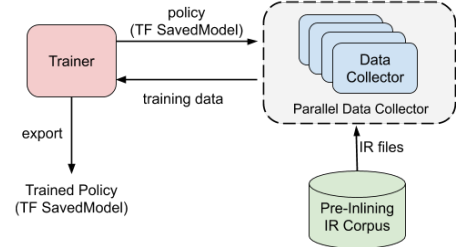


Figure 2. System Overview: Policy Training

The bottleneck of training for the inlining problem is data collection. Therefore, the data collection is carried out in a parallel way to improve the overall training efficiency.

Figure 3 details how the data collector module works. It is supported by the development mode in MLGO framework. It takes a pre-inlining IR file and a policy (optional) as inputs, conducts inlining on the IR file based on the policy, has the post-inlining IR file optimized by other opt passes after inlining, converts the optimized IR file into native code, and gets the native size of this module. The native size, together with the log file generated during inlining by MLGO that contains the trajectory ($s_1, a_1, s_2, a_2, \dots, s_T, a_T$), composes the output of the data collector module — training data. If the policy is not given, the inliner will conduct the current heuristic inlining and log the trace. It has two use-cases as discussed in Section 4.1: 1) collect data to train the warmstart policy with behavioral cloning algorithm; 2) use the heuristic inlining as the baseline. The ES algorithm only needs the reward (native size) for training so the log file is not needed.

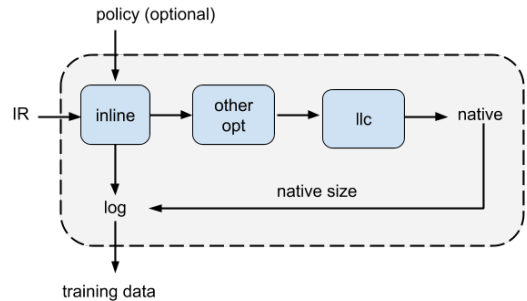


Figure 3. Data Collection for Inlining-for-Size

5 LLVM Implementation

We implemented the pilot project in LLVM⁸, together with reusable support for release and development modes, as well as continuous integration build bots.

5.1 RL Driven Inlining (for Size) in LLVM

We introduced an abstraction for the inline decision-making policy, the *InlineAdvisor*, and a module analysis, *InlineAdvisorAnalysis*, that may be used to retrieve the *InlineAdvisor*. The analysis can not be accidentally invalidated by other passes. This is necessary, since the inliner pass is interleaved with the execution of function passes, as previously discussed, and we want to track module-wide features throughout the performance of inlining and related passes over a module. Instead, the analysis is managed explicitly - see *ModuleInlinerWrapperPass* (`llvm/Transforms/IPO/Inliner.h`). The specific implementation of the advisor is chosen through a LLVM flag (`-enable-ml-inliner`). By default, the implementation is the manual heuristic. Passing `'release'` or `'development'` to the flag selects the respective mode, if the compiler was built with support for that mode.

Feature extraction is modeled as a separate analysis, *FunctionPropertiesAnalysis*, and reused by the release and development implementations. The full feature set is captured in `llvm/Analysis/InlineModelFeatureMaps.h`. We capture some call site-local information, as well as global information, such as module-wide number of functions and static calls, caller/-callee user counts; position in the original call graph, as the distance of the call site to the farthest SCC; and an estimate of removed instructions given the call site context.

We use TensorFlow [1] as the model training and inference framework. In both modes, the interface between LLVM and the model is defined solely in terms of input and output tensor specifications (tensor name, type, and shape). The internal structure of the model is an implementation detail. This means that during training, the compiler engineer is free to explore hyper-parameters or add/modify hidden layers. Also, ingesting a new model with a different internal structure, in release mode, is just a matter of recompiling LLVM.

Refer to `lib/Analysis/{MLInlineAdvisor | ReleaseModeModelRunner | DevelopmentModeInlineAdvisor}.cpp` for more implementation details. At a high level, both release and development modes:

- Handle user inlining directives and correctness aspects (these are done without model evaluation)
- Extract the features *associated* with a call site and form fixed-sized tensors (primitive data type vectors), and efficiently maintain the module-wide features
- Pass the tensors to the model evaluator, and request it perform an evaluation

- Take the result of the evaluation as advice (i.e. inline/don't) and make that available to the inliner pass.
- It is possible that a policy misbehaves in unforeseen circumstances (which, as a note, should then be incorporated into the training loop). The resulting IR, while correct, could become increasingly expensive to process by subsequent passes. To avoid this, we set a hard threshold to the amount by which the number of instructions may grow in a compilation unit.

5.2 Release Mode Implementation in LLVM

The model is encoded in the TensorFlow serialization format, *SavedModel* [31], which is compiled into native code by the `saved_model_cli` tool [32]. To use this tool, we added a build rule (see `llvm/cmake/modules/TensorFlowCompile.cmake`) to the LLVM build system. Applying the rule to a model generates a header file and an object file. From here, the model may be consumed as a C function; for simplicity, the *SavedModel* compiler provides a thin C++ wrapper, exposing plain C/C++ APIs (primitive types), which is compiled as part of the LLVM build process. The *SavedModel* is checked in as source⁹.

To build with support for the release mode, the *SavedModel* compiler must be available during the build time of LLVM. The compiler may be installed through a python pip package¹⁰. Once installed, its location is provided to the LLVM build via the `TENSORFLOW_AOT_PATH` cmake flag. Specifying that flag also defines a conditional compilation flag, `HAVE_TF_AOT`, which enables the compilation as part of the Analysis component of the support for release mode.

Note that this mechanism would build the release mode implementations of *all* optimization passes that have RL-driven policies, meaning that implementers just need to reuse the same mechanisms - conditional compilation flag, build rule, etc - to plug in a ML-based policy replacement.

5.3 Development Mode Implementation in LLVM

As discussed, in development mode, we want to support loading models from the command line. For the development mode, the build time dependency is to the TensorFlow C API library, instead of the TensorFlow pip package. Model loading, initialization, and evaluation is performed via a reusable C++ API wrapper (see `lib/Analysis/TFUtils.cpp`) that simplifies the programming model of this aspect of development mode implementations.

⁹The *SavedModel* separates the evaluation graph structure from the value of the trained weights used for evaluation. The graph is stored as text. The weights/serialized float arrays are stored as a binary blob. Their evolution (due to training) does not diff well, so the compactness of a binary format is more economical for the project repository.

¹⁰See the buildbot setup script available at https://github.com/google/ml-compiler-opt/blob/58bf347286c21519b3cc418f659c485cbb7ad82f/buildbot/buildbot_init.sh

⁸Code references are made in the context of commit 71059257bd4.

In addition to facilitating a different model ingestion mechanism, the development mode is responsible for producing traces necessary for training ("training logs")¹¹. These logs capture the succession of feature values observed when the policy is asked to make a decision, and the decision made afterwards ("trajectories"). Training logs may be produced for both the heuristic policy (for bootstrapping training - "warmstart") as well as for the ML policy currently under training. Exploration - i.e. deviating from policy, with the purpose of finding new learning opportunities - is delegated to a TensorFlow mechanism that introduces some randomness in decisions. This mechanism is an implementation detail of the model as produced by the training algorithm, and is outside the control of the compiler. Care must be taken to remove such randomness before shipping a model, and re-validate its effectiveness. We encode the training logs as textual SequenceExamples[30] proto-buffers, the typical abstraction Tensorflow training algorithms would expect. We produce a textual output to avoid an additional dependency to LLVM, and to simplify diagnostics and testing of the feature.

Analogous to the release mode, enabling development mode in LLVM requires the dependency be made available to the build system. In this case we use the TENSORFLOW_C_API flag, which in turn defines the HAVE_TF_API conditional compilation flag. More details may be obtained from the previously noted build bot scripts. Also similar to the release mode, this mechanism enables all cases that have the TensorFlow C API library dependency. Unlike release mode, the development mode's use of the TensorFlow C library is a run-time dependency, and needs to be on the loader path.

6 Evaluation

6.1 Compilation Overhead

Model evaluation in release mode has **fixed cost**, both in terms of compiler run-time memory utilization, as well as CPU utilization. This is because models are fixed size graphs connecting functional operators, taking fixed sized inputs, using constant weights, and producing fixed sized outputs. For the current model, we observed 0.65% increase in memory utilization at run-time. When inlining a large IR module (33MB), we measured a 10% increase in **inlining** time, mostly attributable to feature extraction; since inlining tends to represent 10-15% of **total** compile time, the net contribution of the release mode is only 1%. Finally, clang binary size increase due to the inclusion of the compiled model was 115KB, representing 0.08% size increase.

We did not formally measure the overhead of the development mode, mainly because timeliness is less of a concern here, and also because model evaluation may happen through a variety of means, including JIT-ing, which makes measurements more unstable. We did want to validate that

¹¹RL algorithms require these. ES algorithms do not

	PG	ES	ES (L)
Size Reduction	4.95%	3.74%	5.94%
Parallelism in Data Collection	100	488	488
Training Time	~12h	~60h	~150h

Table 2. Policy Gradient v.s. Evolution Strategies

the solution is practically usable in training loops, and observed 26K IR modules being inlined in parallel on a 72 thread machine, 192GB RAM, in around 10 minutes, and without going past half of the available RAM¹².

6.2 Inlining for Size Results

We trained the inlining for size policy on an internal search application containing over 28000 IR modules with a variety of different code patterns¹³. The rich set of patterns will improve generalizability, across both time and software domain, of the trained policy. As mentioned, this is important for real-world deployment.

We trained the policy using both PG and ES on the internal search software. Table 2 compares their effectiveness in terms of reduction of the .text section compared with heuristic-driven -Oz. We trained 3 policies: PG and ES with a 2 hidden layer (40, 20) neural network, and ES(L) with a deeper 4 layer (20, 20, 20, 20) neural network¹⁴. We can see that: 1) PG has better sample efficiency than ES — it consumes ~5% training resources of ES (100 * 12 v.s. 488 * 60); 2) better policies may be achieved with a larger neural network at the cost of more training resources; 3) we can train a reasonable PG policy on a single multi-core (e.g. 72 threads) high-performance machine well within a day.

6.2.1 Generalizability across Software

We deploy the trained PG and ES policies to a wide range of software to evaluate their generalizability. Figure 4 shows how the 3 models we trained on the search application perform on 3 different internal applications and on Clang¹⁵. Figure 5 shows their effectiveness on SPEC 2006. We can see that all the 3 policies show good generalizability — they are able to reduce the native size to some extent. Policy effectiveness is ES(L)>PG>ES for most software, which is the same as what we see on the search application. It also suggests good generalizability as a policy performs better on a certain software is likely to also perform better on other software.

¹²Anecdotally, we were able to build Fuchsia using a development mode clang, and timeliness was not a noticeable issue.

¹³We also have an end-to-end demo at <https://github.com/google/ml-compiler-opt/blob/main/docs/demo/demo.md> that trains on publicly available code and achieves similar performance.

¹⁴Detailed hyper-parameters at https://github.com/google/ml-compiler-opt/tree/main/compiler_opt/rl/inlining/gin_configs

¹⁵Specifically, clang @4ca60915bcc (2020/8/28) building clang @d469133f95b (2020/4/25).

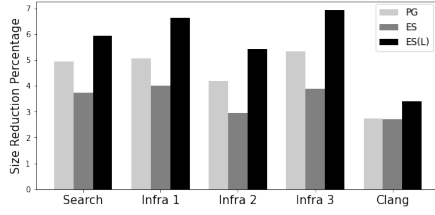


Figure 4. Generalizability across Software

6.2.2 Generalizability across Time

To evaluate the trained policies’ generalizability across time, we deploy the 3 trained policies on the same software as in Figure 4 4 months later. We also use the LLVM 4 months later¹⁶. Both the software and the compiler have been under active development in that period. Figure 6 demonstrates the results. We can see that their effectiveness may degrade somewhat (compared with Figure 4), but they still have decent wins compared with the current -Oz.

7 Related Work

There have been many academic efforts in using machine learning and related techniques to replace hand-crafted heuristics in compilers. Our contribution is identifying the problem framing and design constraints that enable applying these techniques to production.

Wang and O’Boyle [33] present an extensive survey of the use of machine learning in compiler optimizations. Most, however, employ supervised learning techniques, which, as explained, are not in our scope. The closest, Cavazos et al - [6] used unsupervised learning to automatically tune the inlining parameters (thresholds) of a research Java Virtual Machine (JVM), which features a very simple manually written heuristic. In subsequent work, Simon et al. [25] construct a heuristic as a decision tree, to address maintainability and evolvability. While the ML techniques are similar to what we are using in MLGO, both parameter tuning and direct policy comprehensibility are counter to our goals, as described in section 3.

Adams et al. [2] employed ML to train a cost model to automatically schedule Halide programs for image processing. With runtime sampling, the cost model is used to find the optimal schedule parameters using beam search. Similarly, Chen et al. [7] used deep learning to train a statistic model for TensorFlow programs.

Inlining-specific, Dean et al. [11] build a database of observed decisions and their effects, and consult it in subsequent compiler runs - while this is not learning, it is a precursor of efforts in this area. In [9], Cooper et al. presented a scheme to parameterize the inline heuristics (decision tree) and the hill-climbing parameter space search.

¹⁶Clang selfhost @4ca60915bcc (2020/8/28).

Haj-Ali et al. [13] use reinforcement learning to instrument source code with pragma directives to drive the vectorization pass. The policy does not replace a compiler heuristic, rather it informs one, by augmenting source code as a pre-build step. This does not make the technique transparently deployable for compiler users. That being said, we currently see no fundamental reason their solution cannot be adapted to MLGO. The main practical issue we see is understanding trade-offs of automated feature extraction, which we intend to explore as a next step as well.

Supervised learning is used by Stephenson and Amarasinghe [26] to predict loop unrolling factors and by Eliot et al. [19] to train a local (single basic block) instruction scheduler. It uses a machine model to predict the so called preference relationship given a partial schedule and two candidate/ready instructions. Cummins et al. [10] automatically extract features from source code, and use supervised learning to learn heuristics for predicting optimal mapping for heterogeneous parallelism and GPU thread coarsening factors.

Instruction scheduling is a hard problem in the compiler that extensively uses heuristics. The application of learning to instruction scheduling within straight line code has been explored by Moss et al. [20] and McGovern et al. [17].

Data prefetching plays a similar role in bringing *data* into the processor without stalls. In [15], Hashemi et al. treated the memory prefetching strategies as an n-gram classification problem in natural language processing, and used LSTM based Recurrent Neural Network (RNN) to do the prediction. Peled et al. [21] define the notion of *semantic locality* and use reinforcement learning techniques to build a context-based memory prefetcher that approximates semantic locality.

Another approach to optimize programs without dealing with specific optimizations is *super-optimization*. This refers to the process of finding a better version of a given program that is semantically equivalent. Early efforts in super-optimization relied on brute force search. Recent efforts have focused on using stochastic search to improve the efficiency. Bunel et al. [5] have used reinforcement learning to optimize stochastic search based super-optimization techniques.

Milepost GCC [12] is a self-tuning GCC-based compiler, where program features are used to predict compiler flags beneficial to some goal (such as performance or size). It does not use ML-trained policies as part of its implementation.

8 Future Directions

8.1 Applying MLGO to Speed Optimizations

The immediately-observable difference between our pilot project and speed problems is that the reward is measured differently: speed is measured through benchmark runs, which are more time consuming and more noisy than size measurements. Using benchmark runs results as reward for speed optimization will have difficulties scaling, so our current

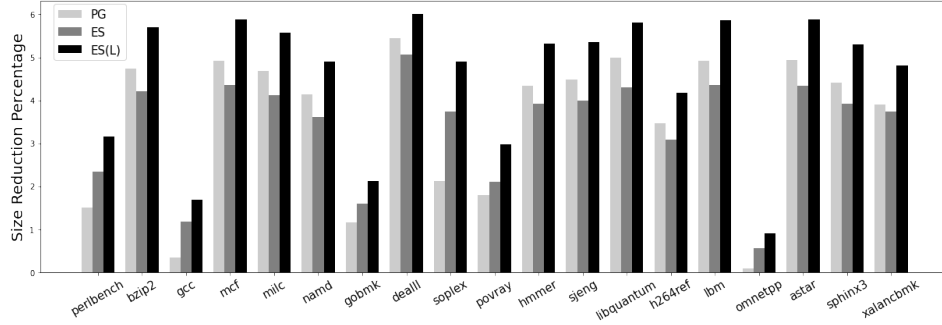


Figure 5. SPEC 2006 Size Reduction

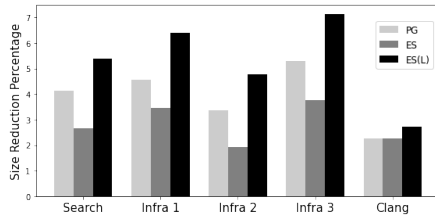


Figure 6. Generalizability across Time

preference is to avoid benchmark runs altogether, and focus instead on using problem-specific reward approximations.

For register allocation, for example, a natural reward is calculating, per function, the block frequency-weighted sum of introduced moves. For inlining for speed, we plan to use a linear combination of a per-critical call graph estimate of working set (i.e. cache lines needed for execution) and dynamic instruction count. Both approaches require profiling information for carrying out the analysis, which we assume as a pre-requisite for workloads that are concerned with speed.

8.2 ML Techniques

There are multiple directions to pursue in terms of the ML techniques:

Richer State Representations: instead of using the 11 numerical features to represent the state, we can have richer state representations. For example, we can use code embedding techniques [3] to embed the caller/callee to get more detailed information about the call site; or we can use graph neural network techniques [14] on the neighboring sub-graph of the call site to get more information about the call graph.

PG with Partial Reward: PG with partial reward would greatly improve the sample efficiency and trainability. However, there are two challenges to tackle: 1) find an efficient way to encode the global call graph information into state; 2) train a supervised model to predict a function’s native size from its IR.

9 Conclusion

We investigated the problem of leveraging ML techniques for compiler optimization in a real-world setting. We proposed a particular understanding of the problem space, and derived the MLGO framework. We applied it to inlining-for-size and described the resulting implementation, available in LLVM as a build-time opt-in, as well as the training methodology, two training algorithms and their trade-offs, and results. We are currently applying the same principles to addressing inlining for speed and register allocation policies, and hope that, through our experience, as well as that of the community, we can further refine MLGO and eventually mature it to a solution that compiler engineers can broadly apply and leverage machine learning for compiler optimizations in real-world settings.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*. 265–283.
- [2] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-mao Li, Michael Charbi, Benoit Steiner, Steven Johnson, Kayvon Fatahian, Fredo Durand, and Jonathan Regan-kelley. 2019. Learning to optimize Halide with tree search and random programs (*ACM Transactions on Graphics*). 2–12.
- [3] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29.
- [4] Michael Bain and Claude Sammut. 1995. A Framework for Behavioural Cloning.. In *Machine Intelligence 15*. 103–129.
- [5] Rudy Bunel, Alban Desmaison, M Pawan Kumar, Philip HS Torr, and Pushmeet Kohli. 2016. Learning to superoptimize programs. *arXiv preprint arXiv:1611.01787* (2016).
- [6] John Cavazos and Michael FP O’Boyle. 2005. Automatic tuning of inlining heuristics. In *SC’05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*. IEEE, 14–14.
- [7] Tianqi Chen, Lianmin Zheng, Eddie Q. Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. *CoRR* abs/1805.08166 (2018). arXiv:1805.08166 <http://arxiv.org/abs/1805.08166>

Optimizing LLVM Pass Sequences with Shackleton: A Linear Genetic Programming Framework

Hannah Peeler
hannah.peeler@arm.com
Arm Ltd., Cambridge, UK
USA

Shuyue Stella Li
sli136@jhu.edu
Department of Computer Science,
Johns Hopkins University
Baltimore, MD, USA

Andrew N. Sloss
andrew.sloss@arm.com
Arm Ltd., Cambridge, UK
USA

Kenneth N. Reid
ken@kenreid.co.uk
Department of Animal Science,
Michigan State University
East Lansing, MI, USA

Yuan Yuan*
yyuan@msu.edu
Department of Computer Science and
Engineering, Michigan State
University
East Lansing, MI, USA

Wolfgang Banzhaf
banzhafw@msu.edu
Department of Computer Science and
Engineering, Michigan State
University
East Lansing, MI, USA

Abstract

In this paper we introduce Shackleton as a generalized framework enabling the application of linear genetic programming to a variety of use cases. We also explore here a novel application for this class of methods: optimizing sequences of LLVM optimization passes. The algorithm underpinning Shackleton is discussed, with an emphasis on the effects of different features unique to the framework when applied to LLVM pass sequences. Combined with analysis of different hyperparameter settings, we report the results on automatically optimizing pass sequences using Shackleton for two software applications at differing complexity levels. Finally, we reflect on the advantages and limitations of our current implementation and lay out a path for further improvements. These improvements aim to surpass hand-crafted solutions with an automatic discovery method for an optimal pass sequence.

CCS Concepts: • **Computing methodologies** → **Genetic programming**; • **Software and its engineering** → *Compilers*.

Keywords: Evolutionary Algorithms, Genetic Programming, Compiler Optimization, Parameter Tuning, Metaheuristics

1 INTRODUCTION

Linear Genetic Programming (LGP) [5] is categorized within the super-set of Genetic Programming (GP) methodologies. These methodologies were established in the early 1990s as a means to generate algorithms and programs automatically, steered by an evolutionary process [4, 11]. Historically, the proliferation of LGP (and the wider umbrella of Evolutionary Algorithms (EAs) it sits under) has been hindered by the need to tailor each implementation to its target application and the limited selection of potential applications [16]. As

a result, there has been and continues to be an interest in frameworks designed to make EAs and LGP methods more accessible to a wider audience. With such frameworks, experimentation demonstrates how these methods fare in new problem spaces. However, the finer details of their implementation - their coding language, provided examples, overall structure - bring trade-offs that tend toward complexity and difficulty in comprehension [8]. In this paper, we introduce the Shackleton Framework, a generic LGP framework that intends to tackle some of these shortcomings.

Shackleton is a generalized framework that allows for the exploration of applying LGP to novel use cases with minimal background knowledge. The aim of the initial creation of Shackleton and this paper is two-fold: (i) to explore the capabilities and usability of a generic LGP framework written entirely in the C language, and (ii) to assess the performance of such a framework on the optimization of a practical and complex use-case: the optimization of LLVM compiler optimization pass sequences. The choice of C as the programming language for this project is motivated by its efficiency and its rarity as a language for EA methods (explored more in Section 2). Compiler optimization passes are also a rare target for EAs despite the creation of effective pass sequences being a problem-space exploration problem. Through creation of the framework and hyperparameter experimentation, we explore the implications of these choices and provide an analysis of the optimization sequences that Shackleton’s evolutionary scheme generates.

This paper describes the framework in detail, with an emphasis on what distinguishes it from other LGP methods. Section 2 discusses related work and important background topics to understand. Section 3 introduces Shackleton in full, outlines the problem space to be explored, and the hyperparameter tuning methodology used. Section 4 shares the setup and methodology, results, and discussion on four

*Now at Beihang University, Beijing, China

distinct experiments. Finally, section 5 concludes and summarizes this paper, and outlines potential future applications.

2 Background and Related Work

The work presented in this paper is a combination of method, implementation, and target application, but is built upon structures created by other researchers and software developers.

Due to the often problem-specific nature of EAs, a common task explored is the creation of EA frameworks that decrease barriers to experimentation. Some example frameworks are generalized, covering many methods. No matter the method, these frameworks offer advantages over writing entire solutions from scratch. One such framework is the Distributed EAs in Python (DEAP) package [8], which utilizes the increasingly popular Python language with a wide range of features. There are other frameworks that focus specifically on GP but with more novel language choices, such as PushGP [21] and its derivatives: PyshGP [17] written in Python, and Clojush [20] written in Clojure. ‘Push’ itself is a language written specifically to support GP experimentation and operators. Shackleton shares this attribute with Push, but is written entirely in C to leverage its level of memory control and runtime efficiency. Paired with Shackleton’s customizability through the integrated editor tool, our framework is unique amongst GP frameworks as well as being specifically a *linear* GP framework.

While Shackleton is by design a general framework that can be used with multiple target applications, it was designed with a target of compiler optimization sequences in mind. A compiler completes the sole task of translating computer code written in one source language to a target language in an executable form. In contrast, an *optimizing* compiler tries to also minimize or maximize attributes of the outputted computer program in the target language to achieve some benefit at execution time [1]. For much of the history of optimized compilation, decisions regarding optimization were made based on a priori static information. Recent research aims to improve this by targeting machine learning based compilation, leveraging the exploration capabilities and increase in popularity of machine learning techniques at the compilation stage [25]. GP and GAs as methods are no exception to this. A genetic algorithm approach has been applied to GNU Compiler Collection (GCC), a compiler similar to LLVM but differentiated by its fixed number of supported languages and reuse constraints [3, 10]. Further, there is precedent for applying GP specifically to a compiler context to leverage GP’s optimization potential for well-known compiler heuristics [22]. Shackleton builds on this motivation by adding another layer of specificity: optimizing a particular stage of the LLVM Compiler Project’s core framework.

2.1 Linear Genetic Programming

Shackleton as a system relies on a method called LGP which descends from a long line of computational concepts and techniques. To understand LGP, it is important to understand the evolution of methods that resulted in its definition. LGP shares a common ancestor with methods like *cellular automata* and *neural computation*, that are *nature-inspired* or *bio-inspired computation*. This term encompasses a wide variety of methods ranging from those that simulate biological natural phenomena in digital forms to methods that actively employ natural materials in their execution. Within the class of methods that takes inspiration from biology and aims to simulate some biological phenomena, we find a subset of methods known as Evolutionary Computation (EC) or EAs [19].

GP is an “EC technique that automatically solves problems without requiring the user to know or specify the form or structure of the solution in advance” [18]. The Field Guide to GP offers that “at the most abstract level GP is a *systematic, domain-independent* method for getting computers to solve problems *automatically* starting from a *high-level statement* of what needs to be done”. This compelling idea spawns a myriad of different sub-methods and projects. LGP is one such method where programs are linear sequences of instructions, leveraging the fact that computers often represent and run programs in a linear fashion. This same concept is used in Shackleton.

2.2 The LLVM Project

The LLVM Project is a collection of modular and reusable compiler and tool-chain technologies. LLVM utilizes its own intermediate representation (LLVM IR) in its core libraries and processes, allowing its compilation process to be largely source- and target-independent [14]. During the compilation process, the LLVM IR is optimized using sequences of LLVM passes, optimizations that traverse some portion of the program to either collect information (analysis passes) or transform the program (transform passes). This process and specifically the included passes and their order in the final pass sequence is central to the work discussed in this paper. Creating effective pass sequences typically requires prior knowledge of LLVM. Without this knowledge, the next alternative is to use often sub-optimal default pass sequences. Shackleton can create custom pass sequences for a specific application automatically, without the need for domain-specific or LLVM-specific knowledge.

For the purposes of this work, Shackleton will not explore the entirety of the LLVM optimization pass search space. Analysis passes that are targeted for debugging or visualization purposes are omitted by design. This omission is to avoid conflicts with automation of the framework. All utility passes are also omitted, due to their small number, their

considerable focus on visualization, and odd function compared to analysis and transform passes. Additional passes are omitted on a case-by-case basis depending on observed behavior in experimentation. Custom sequences created by the framework are compared against all the default sequences provided in a base LLVM installation.

3 Methods

3.1 Problem Space

Shackleton is designed to be applied to various use cases, and here we focus on the LLVM use case as described above. The framework takes in the source code of a target program, and outputs the near-optimal optimization pass sequence that the LLVM compiler can use to optimize this program. In our experiments, Ant Colony Optimization for the Traveling Salesman Problem (ACOTSP) [23] and the Backtrack Algorithm for the Subset Sum Problem (SSP) [15] are used as the target programs.

The Ant Colony Optimization (ACO) algorithm is inspired by the foraging behavior of some ant species [7], in which the path with more visits would be reinforced after each iteration. In the traveling salesman problem, a graph of cities and the distance between each pair of cities are given, and the goal is to find a path in the topology that minimizes total distance travelled.

In the SSP, we are given a set of N numbers from 1 to 1,000,000, and we are asked to find a subset that sums up to a random number X (where $1,000,000 < X < N \times 1,000,000$) [15][6][2][12]. Although it might run into halting states, the backtracking algorithm approximates the solution in a reasonable time. Both ACOTSP and SSP are NP-complete problems, as they do not have polynomial-time solutions. Therefore, optimizing the algorithms from the compiler level is of great importance.

3.2 The Shackleton Framework

Shackleton is a generic GP framework that aims to make GP easier for a myriad of uses. Currently, the main target of Shackleton is to use the framework for optimization of LLVM pass sequences that ultimately optimize executable code. The source code for Shackleton is publicly available on [Arm's GitHub page](#).

Shackleton is underpinned by a generic data structure that represents what will be evolved in Shackleton. Any object type can reside in this structure, an attractive attribute given how often problem-specific implementations are needed to achieve favorable results [13]. A unique offering of Shackleton is the editor tool that enables new use cases to be formulated for the tool with ease: this may be explored in more detail in supplemental materials.

3.2.1 Genetic Programming Design in Shackleton. We now describe the implementation of LGP in the Shackleton Framework in detail, which includes elitism, new individuals,

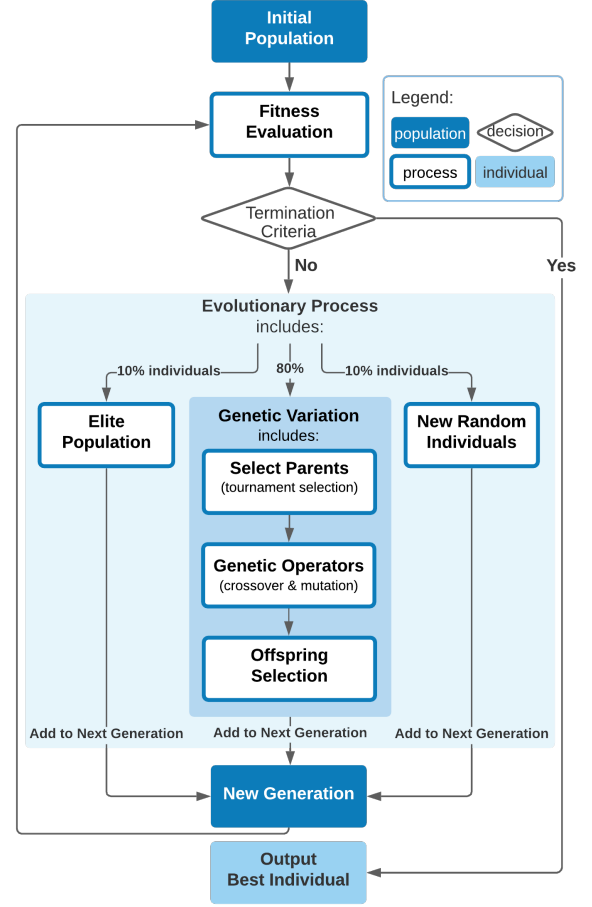


Figure 1. Novel Genetic Programming

and a special offspring selection mechanism as shown in FIG 1. In the LLVM flag use case of Shackleton, each chromosome consists of a sequence of optimization flags. First, a population of random individuals is generated. For each sequence, its fitness is calculated by compiling the program with that sequence of passes and averaging the program runtime over 40 runs. In each generation, a portion of the individuals with the best fitness scores are chosen as the elite group and copied over to the next generation unchanged. Then, in the special offspring selection process, a tournament-style selection approach is used to select parent individuals. Genetic operators including one-point crossover and one-point substitution mutation are applied to their genomes to produce a ‘nest’ of 4 offspring individuals; in each nest, the two best performing individuals out of the four offspring and the two parents are selected to join the next generation. This special offspring selection mechanism is inspired by Tacket’s brood selection [24]. Finally, some new individuals are randomly generated to be included in the next generation in order to preserve

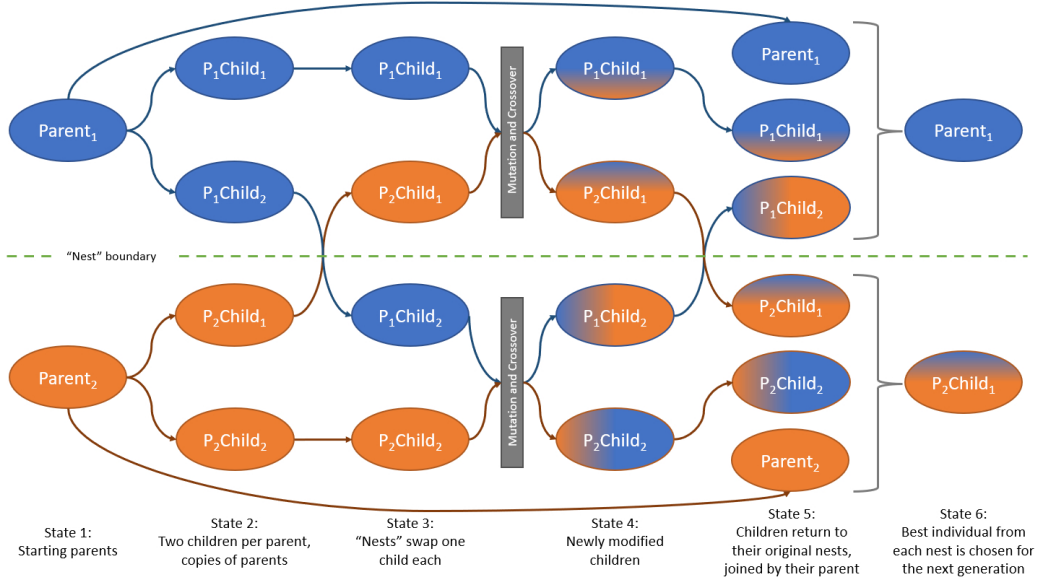


Figure 2. Offspring Selection Process. This does not show a deterministic outcome that will always occur, but rather a possible outcome from a single iteration of the stochastic process. The coloration represents the genetic identity of an individual: the same color means identical genetic materials, and gradients represent a mixing of material from multiple individuals.

diversity in the population. The fitness of the new generation is then evaluated and evolution continues. Throughout this process, the genes that would not contribute to faster programs are gradually eliminated from the population, as they are more likely to possess genomes with worse fitness values. The program stops when the termination criteria are met and it outputs individuals with the best fitness scores as the final solution. For the experiments that are run in this paper, the termination criterion is when the number of generations reaches the set limit, but other implementations (e.g., converging fitness values) are also available and can be further explored.

3.3 Hyperparameter Tuning

Shackleton’s algorithm requires that a selection of hyperparameters be set ahead of runtime. These hyperparameters affect the structure of the evolutionary process (i.e., number of generations, population size, and tournament size) and the balance of the application of evolutionary operators (i.e., mutation, and crossover). There are additional hyperparameters such as the percentage of elite individuals carried to the next generation, nest size in the special offspring selection mechanism, length of each individual, as well as different fitness metrics. It is often unclear what the optimal combination of hyperparameters is for a given application or implementation.

In the experimental section of this paper, we first test out extreme combinations of the number of generations and

number of individuals to determine the effect of these parameters. Then, all hyperparameter values are fixed at a moderate level to explore three algorithmic designs that consist of different ways to calculate the fitness value of each individual. Lastly, the effects of crossover rate, mutation rate, and tournament size are explored in the hyperparameter tuning process using the Taguchi Method [9]. With the Taguchi Design, as few as 16 sets of parameter combinations are needed to explore 4 levels of the three parameters, which would require 64 experiments in a regular factorial design. We chose these specific hyperparameters because they were found, through preliminary testing, to influence the optimization significantly.

The experiments were conducted on HPCC nodes running CentOS Linux version 7 and Clang version 8.0.0. We utilized the Backtrack Algorithm (from SSP), which contains recursive loops that are potential sites of optimization. Preliminary testing found this problem to be of adequate complexity for our experimentation purposes, considering both the runtime and human-verifiable testability.

4 Experiments

4.1 Number of Generations vs. Population Size

4.1.1 Experiment Setup. The number of generations and population size directly determine the number of runtime evaluations. These two hyperparameters are roughly proportional to how many individuals will appear throughout the evolutionary process. Therefore, in order to reduce the overhead runtime of Shackleton, it is essential to determine

Table 1. Number of Generations vs. Population Size
Experiment Parameters

Experiment Nr.	Number of Generations	Population Size
0	50	40
1	250	8
2	200	10
3	10	200
4	8	250
5	4	500

how much these two factors influence the search process of the optimal solution. In our first experiment, we tracked the fitness of the best individual in each generation with extreme settings of number of generations and population size. This is to see whether more computing power and runtime should be allocated to a large number of generations or a larger population size, when their product is fixed. The values are set such that each parameter setting has a constant product, so the same number of total individuals are represented in the entire run of Shackleton.

Table 1 shows the combinations of parameters used for this experiment. Six trials were run for each setting and other hyperparameters are kept constant.

4.1.2 Results. FIG 3 plots the control set up with 50 generations and a population size of 40; FIG 4 and FIG 5 plot two representative cases of a setting with a large *num_generation* value and a setting with a large *population_size* value. The *y*-axis is the runtime of the sample use case - the SSP with the Backtrack Algorithm - in seconds, which we use as the fitness of the individuals, and the *x*-axis is the generation number. The runtime of the program with no optimization and with default LLVM optimization flags (first 8 data points of each trend line) is plotted with the runtime of the individuals across generations. FIG 6 shows the percentage improvement of the parameter settings compared to each of the baseline optimization levels.

4.1.3 Discussion. By exploring the extreme ends of the search space, this experiment shows that more computing power and runtime should be allocated to the population size rather than the number of generations. In the hyperparameter setting plotted in FIG 4, a small population size of 10 is allowed to undergo 200 generations in search for the optimal solution, but the fitness of the best individual is noisy and does not tend to converge, with a final percent improvement of 4.08%. On the other hand, a large population size of 200 shows converging fitness within just 10 generations and a final percent improvement of 5.25%. This result gives support to the claim that the greater diversity in a large population is crucial in the search for an optimal solution.

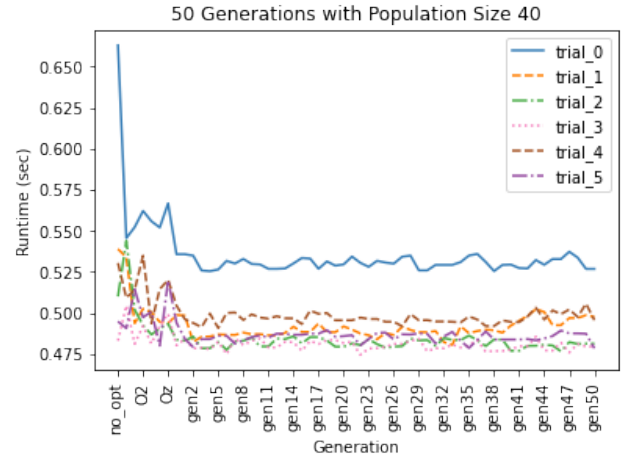


Figure 3. gen=50, pop=40

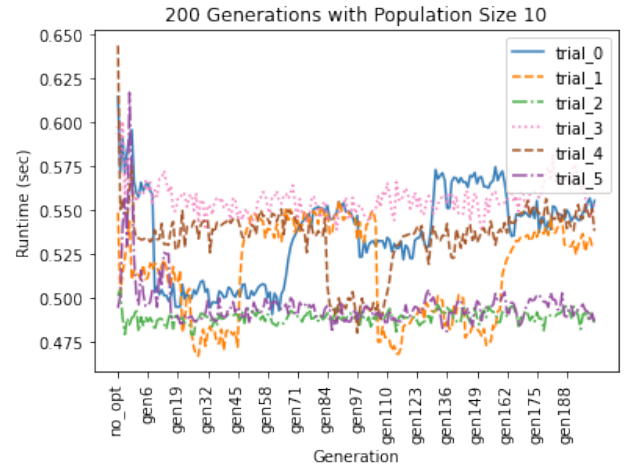


Figure 4. gen=250, pop=8

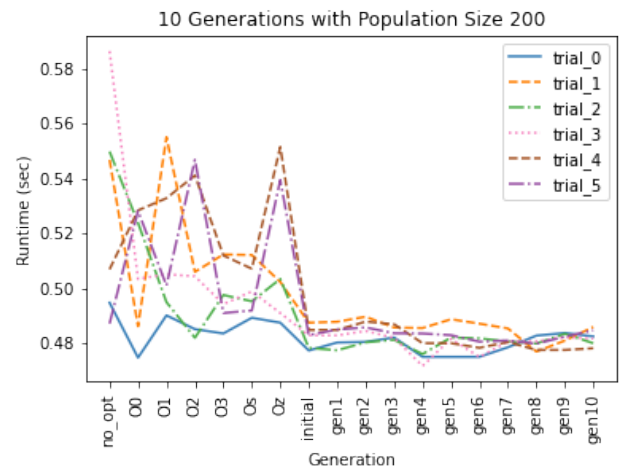


Figure 5. gen=10, pop=200

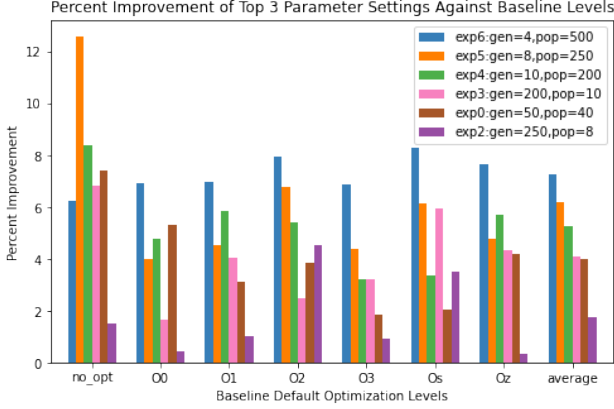


Figure 6. Improvement over Baseline

Table 2. Fitness Calculation Experiment Setup

Experiment Nr.	Fitness	Hashing	% Improvement
6	runtime	Yes	11.29
7	time+var	Yes	1.27
8	runtime	No	3.09

4.2 Fitness Calculation

4.2.1 Experiment Setup. The framework is implemented with three algorithmic designs that feature different ways of calculating the fitness scores, as well as re-evaluating repeated individuals. In calculating the runtime of each individual, the optimization sequence is applied to the source code and the runtime of the code is taken as the average across 40 runs. Observations from preliminary testing indicate that runtime across different runs can vary considerably. Therefore, we first hypothesize that using the sum of the average runtime and the variance as the fitness might prefer individuals with more consistent runtime and lead to better optimization. We also hypothesize that for the same optimization sequence, storing the runtime into a hash table instead of re-doing the 40 runs each time an individual appears in the population could decrease overall runtime. This section explores the impacts of these two potential improvements in terms of algorithmic design.

For each fitness calculation, all hyperparameters are kept at a moderate level as shown in Table 3. Table 2 highlights the different designs.

4.2.2 Results. The fitness across generations and the overhead runtime of Shackleton are recorded as the key metrics of this experiment. FIG 7 shows the percent improvement of fitness compared to the default LLVM optimization levels. Fitness evaluation using only the runtime shows a significant advantage compared to using both the runtime and the variance across runs. FIG 8 shows the overhead runtime of each design taken as the average across six trials.

Table 3. Default Hyperparameter Settings

Hyperparameter	Value
number of generations	50
population size	40
crossover probability	50
mutation probability	50
tournament size	4
elite percentage	10
nest size	6

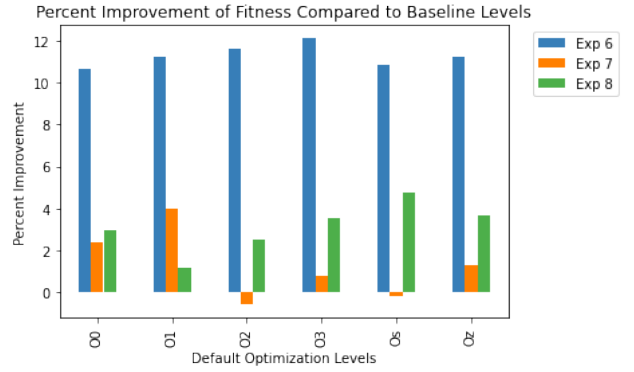


Figure 7. Percent Improvement of Different Fitness Calculations

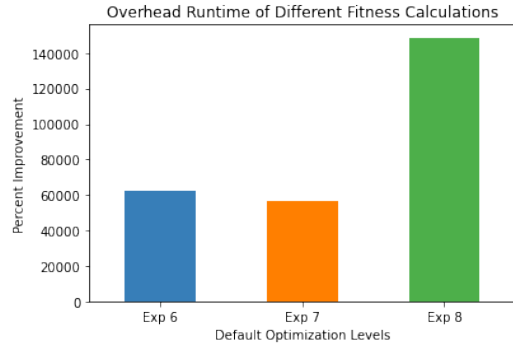


Figure 8. Overhead Runtime of Different Fitness Calculations

4.2.3 Discussion. We compared experiment #6 and experiment #7, which differ only in that the fitness includes the variance of the runs in experiment #7. Experiment #6 is able to achieve a larger percentage improvement. This implies that the variance amongst runs of a program is not relevant to the optimization sequence applied to it, but rather due to the inherent fluctuations of clock speed caused by sharing of resources on the same computing cluster. Therefore, including the variance into the fitness function would not help

Table 4. Parameter Tuning Experiment Setup

Exp. Nr.	Crossover	Mutation	Tournament Size	Improvement
09	20	20	2	5.44%
10	20	40	4	0.50%
11	20	60	6	7.04%
12	20	80	8	1.91%
13	40	20	4	4.88%
14	40	40	2	3.49%
15	40	60	8	5.76%
16	40	80	6	4.90%
17	60	20	6	8.31%
18	60	40	8	4.18%
19	60	60	2	3.43%
20	60	80	4	10.88%
21	80	20	8	5.62%
22	80	40	6	5.53%
23	80	60	4	7.21%
24	80	80	2	6.83%

reduce the noise in the data and could distract the population from moving toward a sequence with shorter runtime.

For experiment #8, where the fitness is re-evaluated and averaged every time an individual appears in the population, the more comprehensive fitness calculation greatly increases Shackleton’s overhead runtime. However, as shown by the percent improvement after the genetic algorithm, this did not lead to a more effective evolution process. Therefore, the improvement of looking up the fitness value from a hash table helps considerably with the overall runtime and does not sacrifice performance. In future experiments, the fitness will be calculated the same way as it is in experiment #6.

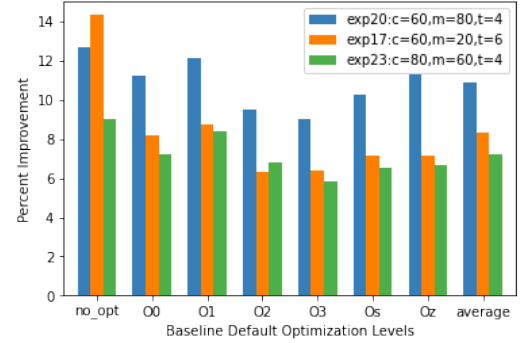
4.3 Hyperparameter Tuning

4.3.1 Experiment Setup. In this experiment, the optimal hyperparameter combination for percent crossover, percent mutation, and tournament size is explored using a Taguchi design [9]. Other hyperparameters are kept constant: 50 generations, population size = 40, elite percentage = 15, nest size = 6, and individual size = 0 (When individual size is set to 0, individuals will be randomly generated with length between 10 and 90. Single-point mutation keeps the length constant, and crossover changes the length of the offspring). The three independent hyperparameters are tested across 4 equidistant levels; the Taguchi Method effectively reduces the number of combinations needed to 16 compared to $4^3 = 64$ in a regular factorial grid search design.

4.3.2 Results. Table 5 shows the three hyperparameter combinations that yield the highest percent improvement in terms of runtime when compared against each default LLVM optimization level. From Table 4, it can be concluded that

Table 5. Top Hyperparameter Combinations

Baseline	Top 1	Top 2	Top 3
No opt	(60,80,4)	(60,20,6)	(80,60,4)
Improvement	19.12%	18.20%	10.43%
O0	(60,80,4)	(40,20,4)	(60,20,6)
Improvement	17.17%	11.13%	9.17%
O1	(60,80,4)	(20,60,6)	(80,60,4)
Improvement	18.31%	10.83%	10.46%
O2	(60,80,4)	(80,60,4)	(80,80,2)
Improvement	15.27%	8.31%	7.88%
O3	(60,80,4)	(20,60,6)	(60,20,6)
Improvement	14.53%	8.24%	7.18%
Os	(60,80,4)	(80,80,2)	(60,20,6)
Improvement	16.01%	8.22%	8.14%
Oz	(60,80,4)	(60,20,6)	(80,60,4)
Improvement	18.83%	8.19%	8.00%
Average	(60,80,4)	(60,20,6)	(80,60,4)
Improvement	17.04%	9.69%	8.57%

Figure 9. Top 3 Hyperparameter Combinations

the combination (60,80,4) from experiment 20 - crossover probability 60%, mutation probability 80%, and tournament size of 4 - is the optimal hyperparameter combination as it is consistently the top-performing combination compared against any default optimization level. Examining the top 3 hyperparameter combinations, we can see that (60,20,6) and (80,60,4) also give consistently good optimization. FIG 9 and 10 shows the percent improvement in terms of runtime compared with the baseline optimizations levels using the above three best hyperparameter combinations. The overhead runtime averaged over six trials for each parameter setting is plotted in FIG 11, with the top three performing hyperparameter combinations in blue and others in orange. Lastly, a Taguchi Analysis [9] is performed to analyze the importance factor of the three hyperparameters, and the results are shown in Table 6.

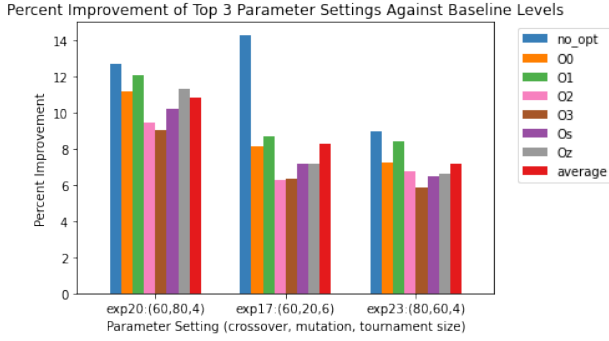


Figure 10. Top 3 Hyperparameter Combinations

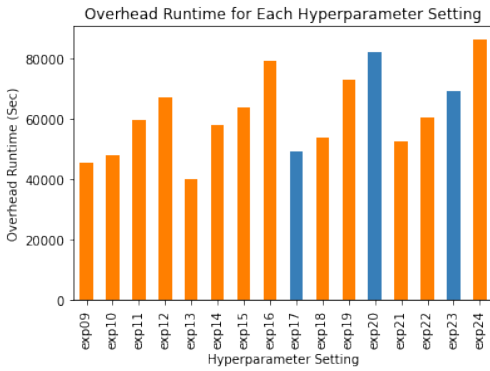


Figure 11. Overhead Runtime of Shackleton Framework

Table 6. Taguchi Analysis - Improvement Signal-to-Noise Ratio

	Crossover	Mutation	Tournament Size
L1	-7.92	2.14	7.09
L2	3.26	-4.94	-13.00
L3	5.01	7.44	4.08
L4	3.87	-0.41	6.06
Δ	12.93	12.38	20.10

Table 7. Taguchi Analysis - Overhead Runtime Signal-to-Noise Ratio

	Crossover	Mutation	Tournament Size
L1	48.84	49.71	44.25
L2	46.63	47.53	48.60
L3	43.39	43.78	42.20
L4	43.88	41.71	47.68
Δ	5.45	8.00	6.40

4.3.3 Discussion. From Table 5, it can be observed that the top hyperparameter configuration when compared to any baseline optimization level is when crossover probability equals 60, mutation probability equals 80, and tournament size equals 4. Most hyperparameter combinations that made it to the top 3 when compared to any baseline level have a high rate of crossover, high rate of mutation, and a moderate tournament size. Parents from the previous generation are selected in a tournament style, where n individuals will be randomly chosen from the entire population, and the individual with the best fitness is taken as the origin of a genetic operator. A higher tournament size will result in the parent being selected to have a higher fitness, but setting it too high will reduce population diversity as it results in the same individuals being selected every time. Crossover and mutation are genetic operators that make changes from the original parents. High crossover and mutation probabilities contribute to the diversity of the offspring population. Therefore, a moderate tournament size ensures that ‘good enough’ individuals are selected as parents. High crossover and mutation probabilities maintain diversity in the population so that the search can keep moving towards the optimal solution.

From FIG 11, the overhead runtime roughly correlates to the level of the mutation probability. However, the Taguchi Analysis [9] in Table 6 shows that tournament size has the largest effect on the optimization process. A larger range means that a change of this hyperparameter can cause a larger effect on the measurement output. Table 7 shows the same analysis done with respect to the overhead runtime of the Shackleton Framework. The range of the signal-to-noise ratio for each hyperparameter is fairly similar, implying that each hyperparameter has a similar level of impact on the overhead runtime. The above information should also be taken into consideration when choosing the best hyperparameter combination.

Looking at both the performance and runtime metrics and the results from the Taguchi Analysis, experiment #17 (crossover probability = 60, mutation probability = 20, and tournament size = 6) is among the top 3 hyperparameter combinations with highest percent improvement and has a reasonable runtime. The high crossover probability maintains the diversity in the population; low mutation probability contributes to the fast overhead runtime; and a large tournament size selects efficient parents without reducing diversity.

4.4 Robustness Analysis

4.4.1 Experiment Setup. Even though the hyperparameter settings are problem-specific and can be set by the user, it is important to make sure that most (if not all) hyperparameter combinations will speed up the execution of the target source code. In this experiment, each of the 16 hyperparameter combinations from Table 4 is run 30 times, and

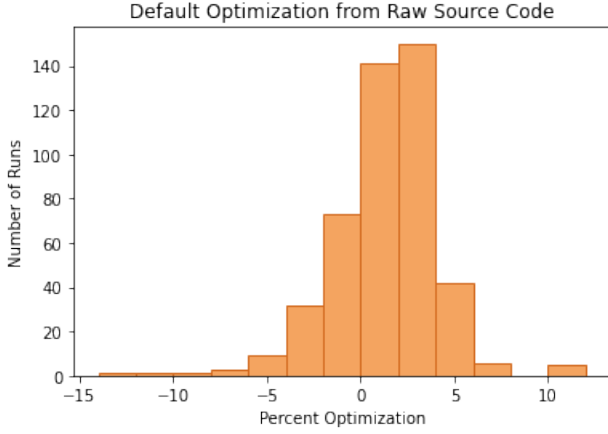


Figure 12. Percent Optimization of All Hyperparameter Settings (compared to raw source code)

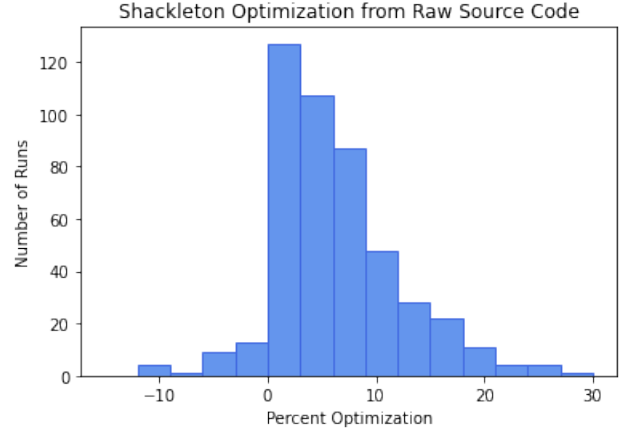


Figure 13. Percent Optimization of All Hyperparameter Settings (compared to raw source code)

Table 8. Percent Improvement

Comparison	Average % Improvement	Additional %
O2-raw	1.40	74.09
Shackleton-raw	6.20	93.88
Shackleton-O2	4.77	90.17

the data from each of these $16 \times 30 = 480$ runs are collected to examine the robustness of Shackleton.

4.4.2 Results. Two metrics are of importance in this evaluation - percent optimization from the raw source code and the percent improvement of the automatically-generated sequence compared to the default optimization. We first want to ensure that Shackleton will produce optimization sequences that speed up the execution of the target source code. Then, we want the automatically-generated sequences to perform better than the default LLVM optimization.

In FIG 12, the percent optimization produced by the default LLVM -O2 sequence is plotted; 74% of these runs produce faster code compared to the unoptimized source code. The histogram in FIG 13 shows the percent optimization produced by the Shackleton-generated sequence; 94% of these runs produce faster code compared to the unoptimized source code. Lastly, the histogram in FIG 14 shows the percent improvement compared to the default -O2 LLVM compiler flag; 90% of the Shackleton-generated optimization runs are faster than the default optimization runs. Outlier values (more than 3 standard deviations from the mean) are removed from the analysis because these data points are not taken as an average among trials, making it more prone to system fluctuations. The experimental results are summarized in Table 8.

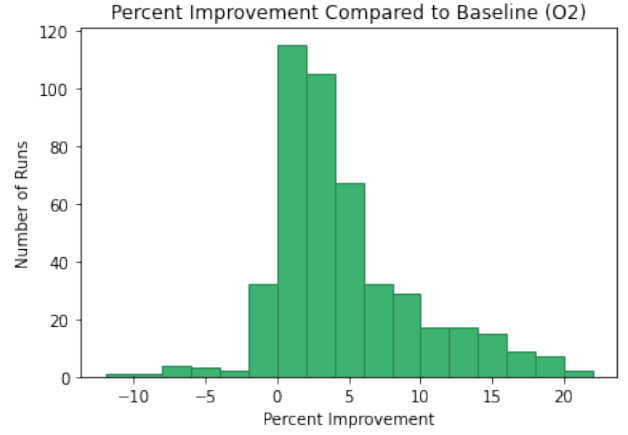


Figure 14. Percent Improvement of All Hyperparameter Settings (compared to O2)

4.4.3 Discussion. Shackleton’s use of stochastic elements in its evolutionary scheme mean that speed improvements over the raw source code after optimization are not always guaranteed. However, the experiments shown here indicate that Shackleton consistently produces optimized code that is faster and offers a larger average improvement than that produced by the default LLVM optimization sequences. These experiments are not exhaustive and notably omit some hyperparameters. More comprehensive analysis of these omitted hyperparameters and additional scaling of the parameters shown here could offer additional insights into benefits and limiting factors. Based on our experiments, with fine-tuned hyperparameters the optimization pass sequences automatically generated by Shackleton can be expected to achieve even better performance.

5 CONCLUSION AND FUTURE WORK

The existing predefined LLVM optimization levels (-Ox) are used for general purpose program optimization and are not problem specific. Targeted runtime optimization of a program by a hand-crafted LLVM pass or a pass sequence would require expert knowledge in both LLVM and the program to be optimized. In this paper, we presented the Shackleton Framework, which is able to automatically generate near-optimal LLVM optimization flag sequences. The Shackleton Framework, without any prior knowledge of the compiler, the optimization passes, nor the program, consistently produces optimization pass sequences that achieve significant runtime improvement over default optimization options. We first proposed an elitist approach and a special offspring selection mechanism for a more greedy genetic algorithm, while a new random offspring mechanism ensures diversity in the population. We also performed hyperparameter tuning and explored the effect of each hyperparameter on the optimization results and overhead runtime.

There are numerous opportunities to further improve and analyze the Shackleton Framework. Shackleton currently only supports a relatively small subset of LLVM optimization passes and only compares against the default optimization passes for analysis. Widening the scope of passes included and allowing for comparison against bespoke sequences fit for a specific use case could offer benefits beyond what is shown here. LLVM also allows programmers to create their own passes to leverage attributes of their specific applications more effectively than the base set of passes. This benefit of the open-source framework could be integrated into Shackleton to generate better sequences faster. In addition to benefits for the LLVM use case, the generic Shackleton Framework could be improved by allowing for hyperparameter tuning beyond what is currently supported. Notably, the brood/nest size and selection percentage for the special offspring selection mechanism are currently fixed. Allowing these parameters to vary or be configured by hand would offer better control over the evolutionary process and may provide more refined results. As an open source framework, Shackleton is designed to be experimented with and expanded upon. With further improvements, the pass sequences automatically generated by the Shackleton Framework could potentially surpass hand-crafted pass sequences while simultaneously freeing developers' time to pursue other advancements for compilers.

Acknowledgments

This work was generously funded by the EnSURE (Engineering Summer Undergraduate Research Experience) program at Michigan State University (MSU) as well as the John R. Koza Endowment. We also gratefully acknowledge The Institute of Cyber-Enabled Research (ICER) at MSU for providing

the hardware infrastructure that made the computation required to complete this work possible.

References

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, 75 Arlington Street Suite 300 Boston MA 02116 USA.
- [2] Michael Alekhnovich, Allan Borodin, Joshua Buresh-Oppenheimer, Russell Impagliazzo, Avner Magen, and Toniann Pitassi. 2011. Toward a model for backtracking and dynamic programming. *Computational Complexity* 20, 4 (2011), 679–740.
- [3] Prathibha A. Ballal, H. Sarojadevi, and Harsha P S. 2015. Article: Compiler Optimization: A Genetic Algorithm Approach. *International Journal of Computer Applications* 112, 10 (February 2015), 9–13.
- [4] Wolfgang Banzhaf, Peter Nordin, Robert E. Keller, and Frank Francone. 1998. *Genetic Programming – An Introduction*. Morgan Kaufmann, Morgan Kaufmann Publishers 340 Pine Street, 6th Floor San Francisco, CA 94104 USA.
- [5] Markus F. Brameier and Wolfgang Banzhaf. 2007. *Linear Genetic Programming*. Springer, Salmon Tower Building, New York City, New York, USA.
- [6] Pinar Civicioglu. 2013. Backtracking search optimization algorithm for numerical optimization problems. *Appl. Math. Comput.* 219, 15 (2013), 8121–8144.
- [7] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. 2006. Ant colony optimization. *IEEE Computational Intelligence magazine* 1, 4 (2006), 28–39.
- [8] Félix-Antoine Fortin, François-Michel De Rainville, Marc-André Gardner, Marc Parizeau, and Christian Gagné. 2012. DEAP: Evolutionary Algorithms Made Easy. *Journal of Machine Learning Research* 13 (jul 2012), 2171–2175.
- [9] Alessandro Freddi and Mario Salmon. 2019. *Introduction to the Taguchi Method: From Conceptualization to First Prototyping with Examples and Case Studies*. Springer, Salmon Tower Building New York City, New York, USA, 159–180. https://doi.org/10.1007/978-3-319-95342-7_7
- [10] William von Hagen. 2006. *The Definitive Guide to GCC, Second Edition (Definitive Guide)*. Apress, USA.
- [11] John R. Koza. 1992. *Genetic Programming*. MIT Press, 12th floor of One Broadway, in Cambridge, MA 02142.
- [12] Jeffrey C Lagarias and Andrew M Odlyzko. 1985. Solving low-density subset sum problems. *Journal of the ACM (JACM)* 32, 1 (1985), 229–246.
- [13] Alexander Leonhardi, Wolfgang Reissenberger, Tim Schmelmer, Karsten Weicker, and Nicole Weicker. 1998. Development of problem-specific evolutionary algorithms. In *Parallel Problem Solving from Nature – PPSN V*, Agoston E. Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 388–397.
- [14] llvm-admin team. 2021. The LLVM Compiler Infrastructure. <https://llvm.org>.
- [15] Parth Shirish Nandedkar. 2019. SubsetSum-BacktrackAlgorithm. <https://github.com/parthnan/SubsetSum-BacktrackAlgorithm>.
- [16] Mihai Oltean and Crina Grosan. 2003. A Comparison of Several Linear GP Techniques. *Complex Systems* 14 (01 2003), 282–313.
- [17] Eddie Pantridge. 2020. pyshgp. <https://github.com/erp12/pyshgp>.
- [18] Riccardo Poli, William Langdon, and Nicholas Mcphee. 2008. *A Field Guide to Genetic Programming*. Lulu Enterprises, UK Ltd, 73a High Street, Egham, United Kingdom, TW20 9HE.
- [19] Grzegorz Rozenberg, Thomas Bäck, and Joost N. Kok (Eds.). 2012. *Handbook of Natural Computing*. Springer-Verlag Berlin Heidelberg, Berlin, Heidelberg.
- [20] Lee Spector. 2015. Clojush. <https://github.com/lspector/Clojush>.
- [21] Lee Spector. Last accessed 2022. Evolutionary Computing with Push. <http://faculty.hampshire.edu/lspector/push.html>

Learning Compiler Pass Orders using Coreset and Normalized Value Prediction

Youwei Liang^{*1} Kevin Stone^{*2} Ali Shameli² Chris Cummins² Mostafa Elhoushi² Jiadong Guo²
Benoit Steiner² Xiaomeng Yang² Pengtao Xie¹ Hugh Leather² Yuandong Tian²

Abstract

Finding the optimal pass sequence of compilation can lead to a significant reduction in program size and/or improvement in program efficiency. Prior works on compilation pass ordering have two major drawbacks. They either require an excessive budget (in terms of compilation steps) at compile time or fail to generalize to unseen programs. In this paper, for code-size reduction tasks, we propose a novel pipeline to find program-dependent pass sequences within 45 compilation calls. It first identifies a *coreset* of 50 pass sequences via greedy optimization of a submodular function, and then learns a policy with Graph Neural Network (GNN) to pick the optimal sequence by predicting the normalized values of the pass sequences in the coreset. Despite its simplicity, our pipeline outperforms the default `-Oz` flag by an average of 4.7% over a large collection (4683) of unseen code repositories from diverse domains across 14 datasets. In comparison, previous approaches like reinforcement learning on the raw pass sequence space may take days to train due to sparse reward, and may not generalize well in held-out ones from different domains. Our results demonstrate that existing human-designed compiler flags can be improved with a simple yet effective technique that transforms the raw action space into a small one with denser rewards.

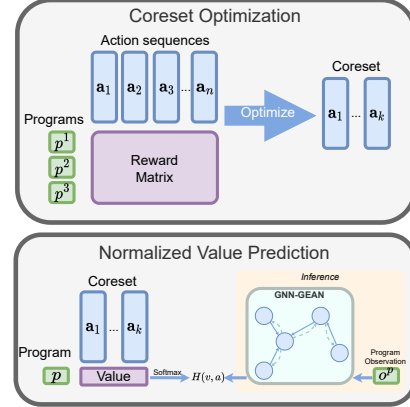


Figure 1. A depiction of our main contributions. (Top) **Coreset Optimization**: A process for discovering a small set of pass sequences (*coreset*) that generalizes. (Bottom) **Normalized Value Prediction** A process where our model learns to predict the normalized value of pass sequences from the coreset. (Bottom inset) Our model, Graph Edge Attention Network (**GEAN**), for encoding program observations.

size of the search space. For example, the LLVM compiler has 124 different compilation flags. If the pass sequences have a length of 45, then the possible number of sequences ($124^{45} \sim 10^{94}$) is already more than the atoms in the universe ($\sim 10^{80}$ (Planck Collaboration et al., 2016)).

In recent years, machine learning (ML)-guided pass ordering has emerged as an interesting field to replace this laborious process (Wang & O’Boyle, 2018). Along this line, many works show promising results using various optimization and/or ML techniques (e.g., reinforcement learning (Haj-Ali et al., 2020a), language modelling (Cummins et al., 2017), evolutionary algorithms (Kulkarni & Cavazos, 2012), etc).

However, there are several limitations. Some previous works (e.g., MLGO (Trofin et al., 2021), MLGoPerf (Ashouri et al., 2022)) run adaptive search algorithms to optimize a set of programs for many hours. While this achieves strong performance gain, the procedure can be slow and does not distill knowledge from past experience and requires searching from scratch for unseen programs. Recent works like Autophase (Haj-Ali et al., 2020b) learn a *pass policy* via re-

1. Introduction

For more efficient execution with fewer resources (e.g., memory, CPU, and storage), applying the right ordering for compiler optimization passes to a given program, i.e., *pass ordering*, is an important yet challenging problem. Manual efforts require expert knowledge and are time-consuming, error-prone, and often yield sub-par results, due to the huge

^{*}Equal second author contribution, listed alphabetically.

¹University of California, San Diego ²Meta AI. Correspondence to: Kevin Stone <kevinleestone@meta.com>.

inforcement learning, and applies it to unseen programs without further search procedure, and GO (Zhou et al., 2020) fine-tunes the models for unseen programs. These approaches work for unseen programs from the same/similar domain, but can still be quite slow in the training stage, and do not show good generalization to programs from very different domains.

In this work, we propose a novel pass ordering optimization pipeline to reduce the *code size* of a program. As a first contribution, we formulate the search for a *universal* core set of pass sequences (termed **coreset**) as an optimization problem of a submodular reward function and use a greedy algorithm to approximately solve it. The resulting coreset consists of 50 pass sequences with a total number of passes of 625. Importantly, it leads to very strong performance across programs from diverse domains, ranging from the Linux Kernel to BLAS. Specifically, for one unseen program in the evaluation set, there exists one of the 50 pass sequences that leads to an average code size reduction of 5.8% compared to the default `-Oz` setting, across 10 diverse codebases (e.g. Cbench (Fursin, 2014), MiBench (Guthaus et al., 2001), NPB (Bailey et al., 1995), CHStone (Hara et al., 2008), and Anghabench (Da Silva et al., 2021)) of over one million programs in total. Considering the huge search space of compiler flags, this is a very surprising finding.

While it is time-consuming to find an optimal pass sequence with an exhaustive enumeration of the core subset of pass sequences, as a second contribution, we find that the (near) optimal pass sequence can be directly predicted with high accuracy via our *Graph Edge Attention Network* (GEAN), a graph neural network (GNN) architecture adapted to encode the augmented ProGraML (Cummins et al., 2021) graphs of programs. Therefore, we can run a few pass sequences selected by the model on an unseen program to obtain a good code size reduction. This enables us to find a good flag configuration that leads to 4.7% improvement on average, with just 45 compilation passes, a reasonable trade-off between the cost spent on trying compilation passes and the resulting performance gain.

We compare our approach with extensive baselines, including reinforcement learning (RL) -based methods such as PPO, Q-learning, and behavior cloning. We find that RL-based approaches operating on the original compiler pass space often suffer from unstable training (due to inaccurate value estimation) and sparse reward space and fail to generalize to unseen programs at inference. In comparison, our approach transforms the vast action space into a smaller one with much more densely distributed rewards. In this transformed space, approaches as simple as behavior cloning can be effective and generalizable to unseen programs.

2. Related Work

Graph structured data are present in numerous applications and it has been shown that taking advantage of this data can help us train very effective machine learning models. (Brauckmann et al., 2020) use abstract syntax trees and control flow graphs for learning compiler optimization goals. They show that using such graphs allows them to outperform state-of-the-art in the task of heterogeneous OpenCL mapping. (Guo et al., 2020) uses a transformer based model with a graph guided masked attention that incorporates the data flow graph into the training. They achieve state of the art performance in four tasks including code search, clone detection, code translation, and code refinement.

As a contender to graph neural networks, (Mialon et al., 2021) uses transformers to process graphs. They show that if we effectively encode positional and local sub-structures of graphs and feed them to the transformer, then the transformer can outperform the classical GNN models. They test their model on classification and regression tasks and achieve state of the art performance. In (Srinivas et al., 2020), they used an unsupervised model to learn embeddings of high dimensional pixel data using contrastive learning. They then use this embedding for downstream reinforcement learning tasks.

3. Methodology

3.1. Action space

The CompilerGym framework (Cummins et al., 2022) provides a convenient interface for the compiler pass ordering problem. The default environment allows choosing one of 124 discrete actions at each step corresponding to running a specific compiler pass. In this work we will use the term pass interchangeably with action. We fix episode lengths to 45 steps to match the setup in (Haj-Ali et al., 2020b). Given that our trajectories have a length of 45 steps, this means we have $124^{45} \sim 1.6 \times 10^{94}$ possible pass sequences to explore. To find an optimal pass sequence for a program, we can apply some existing reinforcement learning methods including Q learning like DQN (Mnih et al., 2015) and policy gradient like PPO (Schulman et al., 2017).

Pass Sequences However for this problem it turns out that certain pass sequences are good at optimizing many different programs (where “good” is defined as better than the compiler default `-Oz`). We found that constraining the action space to a learned set of pass sequences enables state of the art performance and also significantly reduces the challenge of exploration. This allows us to cast the problem as one of supervised learning over this set of pass sequences. We use the following algorithm to find a good set of pass sequences.

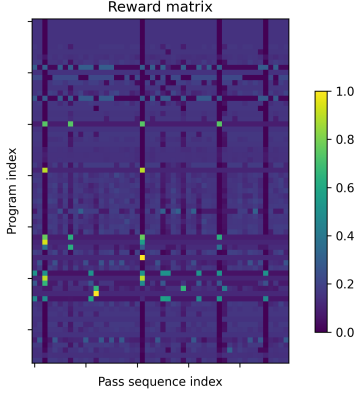


Figure 2. An exemplar reward matrix for 67 programs and 50 pass sequences. Most of the pass sequences do not lead to strong rewards, except for a few. On the other hand, certain pass sequences (i.e., columns) can lead to high rewards for multiple programs simultaneously and thus are good candidates for the coreset.

Suppose we have N programs and M promising pass sequences. Let $R = [r_{ij}] \in \mathbb{R}^{N \times M}$ be the reward matrix, in which $r_{ij} > 0$ is the *ratio* of the codesize of i -th program if applied with j -th pass sequence, compared to $-\infty$. $r_{ij} > 1$ means that the j -th pass sequence does *better* than $-\infty$ in codesize reduction for i -th program, and $r_{ij} < 1$ means it performs worse. The reward matrix is normalized per row, by the maximum reward for each program, so that the optimal pass sequence has reward of 1 for each program.

Then we aim to pick a subset S of K pass sequences, called the *coreset*, from all M pass sequences, so that the overall saving $J(S)$ is maximized:

$$\max_{|S| \leq K} J(S) = \sum_{i=1}^N \max_{j \in S} r_{ij} \quad (1)$$

Finding M candidate pass sequences. Note that there can be exponential number of pass sequences, and we cannot construct the entire reward matrix, instead we seed a list of candidate action trajectories. For this, we run a random policy on a subset of M (17500) selected training programs. For each selected program, we run E (200) episodes and pick the best pass sequence as one of the M candidates. If part of the best pass sequence leads to the same state, they are truncated so that the sequence becomes shorter. If multiple pass sequences yield the same reward we only retain the first after ordering them length-lexicographically. On average these last two steps reduce the length of the candidate pass sequences by 80%.

We then construct the reward r_{ij} by applying the j -th best pass sequence to program i , and comparing it with $-\infty$.

Finding the best coreset S with greedy algorithm. As

a function defined in subsets, $J(S)$ can be proven to be a nonnegative and monotone submodular function (See Appendix). While maximizing a submodular function is NP-hard (Ward & Živný, 2016), the following *greedy algorithm* is proven to be an efficient approximate algorithm that leads to fairly good solutions (Nemhauser et al., 1978). Starting from $S_0 = \emptyset$, at each iteration t , it picks a new pass sequence j_t as follows:

$$j_t := \arg \max_{j \notin S_{t-1}} J(S_{t-1} \cup \{j\}) \quad (2)$$

And $S_t \leftarrow S_{t-1} \cup \{j_t\}$ until we pick K pass sequences. We set $K = 50$ in this paper.

Given the discovered coreset S , we define a **generalized action** as a pass sequence in S . Applying a generalized action to a program means that we roll out the corresponding pass sequence on the program and return the best program state (i.e., having the highest cumulative reward) (it is feasible because we can cache the program state for each step).

3.2. Normalized Value Prediction

After discovering the “good” pass sequences (i.e., the coreset), we can turn the problem of the sequential decision-making on compiler passes into a problem of supervised learning. The target is to train a model to predict the best pass sequence conditioned on the program, where the label of the program is the index of the pass sequence that results in the largest code size reduction. However, one important observation we have is that there are typically multiple pass sequences in the coreset that all result in the largest code size reduction (see Figure 3 for the examples). Therefore, instead of using the single-class classification method with a cross entropy loss, we leverage the fact we have access to the values for all pass sequences. We predict the softmax normalized value of each pass sequence with a cross entropy loss detailed below. This approach is similar to behavior cloning (Pomerleau, 1988) but with soft targets over the coreset.

For a program p , we roll out all pass sequences in the coreset on it, obtaining a reward r_j^p for the j -th sequence (i.e., the highest cumulative reward observed during the rollout of the pass sequence), which forms a value vector $\mathbf{r}^p = [r_1^p, \dots, r_K^p]$. Then, the normalized values of the pass sequences are defined by

$$\mathbf{v}^p = \text{Softmax}(\mathbf{r}^p / T) \quad (3)$$

where T is a temperature parameter.

For an initial observation o^p of a program, our model outputs a probability distribution, $\mathbf{a} = f(o^p)$, over the pass sequences. The target of the training is to make $\mathbf{a}$ close to the normalized values of the pass sequences. To this end, we use the Kullback–Leibler (KL) divergence to supervise the

model, which can be reduced to the following cross entropy loss up to a constant term.

$$\mathcal{L}(\mathbf{a}^p, \mathbf{v}^p) = - \sum_{j=1}^K a_j^p \log v_j^p \quad (4)$$

3.3. Program Representations

Since we use the CompilerGym (Cummins et al., 2022) environments for program optimization, we exploit the program representations from CompilerGym, where program source code is converted to LLVM Intermediate Representation (IR) (Lattner & Adve, 2004) and several representations are constructed from the IR, including the ProGraML graph (Cummins et al., 2021), the Autophase feature (Haj-Ali et al., 2020b), and the Inst2vec feature (Ben-Nun et al., 2018). We choose to use the LLVM environment from CompilerGym because the LLVM ecosystem is a popular compiler infrastructure that powers Swift, Rust, Clang, and more.

Autophase We use the Autophase features Haj-Ali et al. (2020b) to build some baseline models for comparison, which will be detailed in Section 4.2. The Autophase feature is a 56-dimension integer feature vector summarizing the LLVM IR representation, and it contains integer counts of various program properties such as the number of instructions and maximum loop depth. we use an MLP to encode it and output a program representation.

ProGraML As a part of our main method, we leverage ProGraML (Cummins et al., 2021) graphs for training GNN models. ProGraML is a graph-based representation that encodes semantic information of the program which includes control flow, data flow, and function call flow. This representation has the advantage that it is not a fixed size - it does oversimplify large programs - and yet it is still a more compact format than the original IR format. We list three bullet points of the ProGraML graph below.

- **Node features** Each node in a ProGraML graph have 4 features described in Table 1. The “text” feature is a textual representation and the main feature that captures the semantics of a node. For example, it tells us what an “instruction” node does (e.g., it can be `alloca`, `store`, `add`, etc).
- **Edges features** Each edge in a ProGraML graph have 2 features described in Table 1.
- **Type graph** There is an issue with the ProGraML graph. Specifically, a node of type variable/constant node can end up with a long textual representation (for the “text” feature) if it is a composite data structure. For example, a struct (as in C/C++) containing dozens of data members needs to include all the

members in its “text” feature. In other words, the current ProGraML representation does not automatically break down composite data types into their basic components. Since there is an unbounded number of possible structs, this prevents 100% vocabulary coverage on any IR with structs (or other composite types). To address this issue, we propose to expand the node representing a composite data type into a type graph. Specifically, a pointer node is expanded into this type graph: `[variable] <- [pointer] <- [pointed-type]`, where [...] denotes a node and <- denotes an edge connection. A struct node is expanded into a type graph where all its members are represented by individual nodes (which may be further expanded into their components) and connected to a struct node. An array is expanded into this type graph: `[variable] <- [array] <- [element-type]`. The newly added nodes are categorized as type nodes and the edges connecting the type nodes are type edges. The type nodes and type edges constitute the type sub-graphs in the ProGraML graphs. In this manner, we break down the composite data structures into the type graphs that consist of only primitive data types such as `float` and `i32`.

3.4. Network Architecture

Since the Autophase feature can be encoded by a simple MLP, we discuss only the network architectures for encoding the ProGraML graphs in this section.

We use a graph neural network (GNN) as the backbone to encode the ProGraML graphs and output a graph-level representation. The GNN encodes the graph via multiple layers of message passing and outputs a graph-level representation by a global average pooling over the node features. The goal of graph encoding is to use the structure and relational dependencies of the graph to learn an embedding that allows us to learn a better policy. To this end, we experimented with several different GNN architectures such as Graph Convolutional Network (GCN) (Kipf & Welling, 2017), Gated Graph Convolutions Network (GGC) (Li et al., 2015), Graph Attention Network (GAT) (Brody et al., 2022), Graph Isomorphism Network (GIN) (Xu et al., 2019). To better capture the rich semantics of node/edge features in the ProGraML graphs, we propose Graph Edge Attention Network (GEAN), a variant of the graph attention network (Veličković et al., 2017). These GNNs leverage both the node and edge features, so we start by presenting how to embed the node and edge features.

Node embedding For the “text” features of the nodes, we build a vocabulary that maps from text to integer. The vocabulary covers all the text fields of the nodes in the graphs in the training set. The final vocabulary consists of

	Feature	Description
Node	type	One of {instruction, variable, constant, type }
	text	Semantics of the node
	function	Integer position in function
	block	Integer position in IR basic block
Edge	flow	Edge type. One of {call, control, data, type }
	position	Integer edge position in flow branching

Table 1. Features in the ProGraML graph representation which we augment with type information (changes highlighted). We ablate the augmentations in Section 4.5.

117 unique textual representations, and we add an additional item “unknown” to the vocabulary which denotes any text features that may be encountered at inference time and we have not seen before. The i -th textual representation is embedded using a learnable vector $v_i \in \mathbb{R}^d$, where d is the embedding dimension. The “type” feature is not used because it can be inferred from the “text” feature.

Edge embedding The edge embedding is the sum of three types of embedding as the following.

- **Type embedding** We have 4 types of edge flows, so we use 4 learnable vectors to represent them.
- **Position embedding** The “position” feature of an edge is a non-negative integer which does not have an upper bound. We truncate any edge positions larger than 32 to 32 and use a set of 32 learnable vectors to represent the edge positions.
- **Block embedding** We use the block indices of the two nodes connected by the edge to construct a new edge feature. The motivation is that whether the edge goes beyond an IR basic block can influence program optimization. Suppose the block indices of the source node and the target node of an edge are respectively b_i and b_j . We get the relative position of the two nodes with respect to IR basic blocks in the following way: $p_{block} = \text{sign}(b_i - b_j)$. If the edge connects two nodes in the same IR basic block, then p_{block} is 0. And $p_{block} = \pm 1$ indicates the edge goes from a block to the next/previous block. There are 3 possible values for p_{block} , so it is embedded using 3 learnable vectors.

The final embedding of an edge is the sum of its type, position, and block embedding vectors.

Graph mixup We note that the ProGraML graphs of two programs can be composed into a single graph without affecting the semantics of the two programs. And their value vectors can be added up to correctly represent the value vector of the composite graph. In this manner, we can enrich the input space to the GNNs and mitigate model overfitting for the normalized value prediction method.

Graph Edge Attention Network We introduce the GEAN in this paragraph and defer its mathematical details to the Appendix. There are two main differences between the GAT and GEAN. 1) GEAN adopts a dynamic edge representation. Specifically, GAT uses the node-edge-node feature to calculate the attention for neighborhood aggregation, while GEAN uses the node-edge-node feature to calculate not only the attention but also a new edge representation. Then, the updated edge representation is sent to the next layer for computation. Note that GAT uses the same edge embedding in each layer. We conduct an ablation study showing that the edge representation in GEAN improves the generalization of the model. 2) GAT treats the graph as an undirected graph while GEAN encodes the node-edge-node feature to output an updated node-edge-node feature, where the two updated node features represent the feature to be aggregated in the source node and the target node, respectively. This ensures that the directional information is preserved in the neighborhood aggregation.

3.5. Dataset Preparation

Overfitting issues could happen if training is performed on a small subset of programs, or the set of programs is not diverse enough. To mitigate this we find it helpful to create an aggregate dataset that uses many different public datasets as curated by CompilerGym. CompilerGym gives us access to 14 different datasets constructed using two different methods.

- **Curated** These are small collections of hand-picked programs. They are curated to be distinct from one another without overlap and are not useful for training. Typically programs are larger as they may comprise multiple source files combined into a single program. These are commonly used for evaluating compiler optimization improvements.
- **Uncurated** These are comprised of individual compiler IRs from building open source repositories such as Linux and Tensorflow. We also include synthetically generated programs, targeted for compiler testing (not optimization).

For our aggregate dataset we decided to holdout the entirety of the four curated datasets for use as an out-of-domain test set. This is important because they represent the types of programs we expect to see in the wild. We also split the uncurated datasets into train, validation, and test programs.

3.6. Evaluation

For all our metrics and rewards we leverage the IR instruction count as value we are trying to minimize. We also report metrics on each CompilerGym dataset as well as

Type	Dataset	Train	Val	Test
Uncurated	anghabench-v1	70,7000	1,000	2,000
	blas-v0	133	28	29
	github-v0	7,000	1,000	1,000
	linux-v0	4,906	1,000	1,000
	opencv-v0	149	32	32
	poj104-v1	7,000	1,000	1,000
	tensorflow-v0	415	89	90
	clgen-v0	697	149	150
	csmith-v0	222	48	48
	llvm-stress-v0	697	149	150
Curated	cbench-v1	0	0	11
	chstone-v0	0	0	12
	mibench-v1	0	0	40
	npb-v0	0	0	121
Total	-	728,219	4,495	4,683

Table 2. CompilerGym dataset types and training splits. The hand curated datasets are used solely to evaluate generalization to real world program domains at test time.

the mean over datasets to get a single number to compare overall results.

- The mean percent improved over $-Oz$ (**MeanOverOz**) defined as following:

$$\bar{I}^{Oz} = \text{MeanOverOz} := \frac{1}{|\mathcal{P}|} \sum_p \frac{I_p^{Oz} - I_p^{\pi_\theta}}{I_p^{Oz}}, \quad (5)$$

where p is a specific program from the set of programs $\mathcal{P}$ in the dataset. I_p^{Oz} is the number of IR instructions in the program after running the default compiler pass $-Oz$. $I_p^{\pi_\theta}$ is the number of IR instructions in the program after applying the policy under consideration. We can think of this as a simple average of the percent improvement over $-Oz$.

- We also look compare the geometric mean (**GMeanOverOz**) of final sizes across all programs relative to $-Oz$ to give a weighted comparison that is insensitive to outliers.

$$\bar{I}_G^{Oz} = \text{GMeanOverOz} := \left(\prod_p \frac{I_p^{Oz}}{I_p^{\pi_\theta}} \right)^{\frac{1}{|\mathcal{P}|}} \quad (6)$$

4. Experiments

4.1. Experimental Setup

For the methods in Table 3, we search over a set of hyperparameters, including the temperature T in Eq. 3, number of layers, the embedding dimension of the node/edge features, and the output dimension in the hidden layers in the MLPs. We select the best model of each method based on

the validation metric (validation MeanOverOz in 45 steps) and report the best model’s metrics on the test set.

4.2. Baseline Methods

Oracle We consider a brute-force search over the coreset in order to find the best pass sequence for a given program. This gives us an upper-bound of the downstream policy network. In our case the coreset has a total of 50 sequences (625 total passes).

Top-45 We also consider how well we would do if the oracle is only allowed to use the most popular pass sequences but limited to 45 passes. We use 45 passes because this is maximum allowed for our all other baselines and our proposed method.

RL-PPO We reproduce the Autophase (Haj-Ali et al., 2020b) pipeline by using the state-of-the-art RL algorithm PPO (Schulman et al., 2017) to learn a policy model. We have two program representations for training the RL models, including the Autophase feature and the ProGraML graphs (note that Haj-Ali et al. (2020b) only used the Autophase feature). The Autophase/ProGraML feature is sent to a GNN/MLP for feature encoding, which outputs a program-level embedding. Following Haj-Ali et al. (2020b), we add an additional action history feature to the RL pipeline, which is a histogram of previously applied passes. The vector of the histogram of action history is divided by 45 (i.e., the number of the total passes in our budget) for normalization. A 2-layer MLP is used to encode the action history to obtain a feature vector, which is concatenated with the program embedding extracted from the ProGraML graph or the Autophase feature. The concatenated feature is sent to a 2-layer MLP to output the action probability for the policy network. The value network (i.e., the critic) in our PPO pipeline mimics the policy network (i.e., the actor) in feature encoding and outputs a scalar to estimate the state values. The state values are the expectation of the discounted cumulative rewards where the reward in each step is the improvement over $-Oz$: $(I_p^{(t)} - I_p^{\pi_\theta})/I_p^{(t)}$, where $I_p^{(t)}$ denotes the current IR instruction count of the program p at time step t . This reward is reasonable since it makes the value approximation Markovian. At inference, an action is sampled from the output of the learned policy network at each time step until the total number of steps reaches 45.

Q-value-rank We consider each pass sequence in the coreset as a *generalized action* and train a Q network to predict the value of each generalized action. Recall that the value vector r^p is the highest cumulative reward observed during the rollout of each pass sequence in the coreset on program p . The Q-value-rank model is trained to approximate the value vector using a mean squared loss.

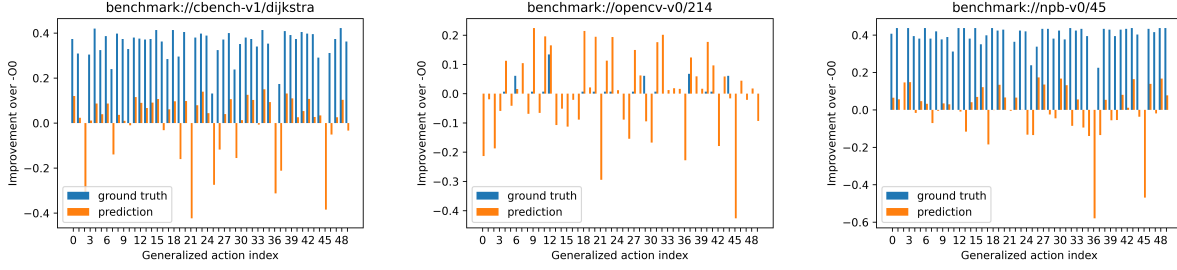


Figure 3. GEAN-Q-value-rank: ground truth of rewards and model predictions over the 50 generalized actions for three benchmarks.

BC We consider learning a standard behavior cloning model to predict the best pass sequences from the coreset, where the best pass sequence is defined as the following. As in the previous bullet point, the value vector is denoted by r^p . If there is only one i such that $r_i^p = \max_{j \in n} r_j^p$, then the classification label is i . If there are multiple such i 's (multiple pass sequences) that achieve the largest reward $\max_{j \in n} r_j^p$, then we order the corresponding pass sequences by length-lexicographic ordering. The classification label is selected to be the first one after the ordering. This ensures that our definition for the best pass sequence (among the coreset) for a program is unique and consistent. We use a standard cross entropy loss to train the single-class classification model.

NVP This is the normalized value prediction method described in Section 3.2.

The last three methods (i.e., Q-value-rank, BC, and NVP) share the same **inference protocol**. Note that they all output a vector α of length 50 whose entries correspond to the pass sequences in the coreset. At inference, we roll out the pass sequences with the highest values in α one by one until our budget of 45 passes is reached. Since the pass sequence has an average length of 12.5, typically 3 or 4 pass sequences are applied (anything beyond 45 passes will be truncated). For BC and NVP, we also tried sampling pass sequences using the model output α , but that resulted in worse performance. Therefore, we stick to selection by maximum values.

4.3. Main Results

In Table 3 we present the main results of our experiments comparing our proposed method -NVP to various baselines. The test programs were completely held-out during both data-driven learning phases (pass sequence search and model training).

The results show that our model achieves strong performance over the prior method (Autophase-RL-PPO) proposed in (Haj-Ali et al., 2020b). Additionally, we can see that both the GEAN model and the normalized value prediction over the discovered coreset are needed to achieve the best performance within 45 passes. See Figure 6 in the

Method	#passes	$\bar{I}^{Oz}$	$\bar{I}_G^{Oz}$
Compiler (-Oz)	-	0%	1.000
Autophase-RL-PPO	45	-16.3%	0.960
GCN-RL-PPO	45	-12.2%	0.987
GGC-RL-PPO	45	-8.5%	1.000
GIN-RL-PPO	45	-11.3%	0.991
GAT-RL-PPO	45	-10.3%	0.999
GEAN-RL-PPO	45	-10.0%	0.997
GEAN-Q-value-rank	45	-0.3%	1.043
Autophase-BC	45	2.6%	1.043
GEAN-BC	45	2.1%	1.038
Autophase-NVP (Ours)	45	3.8%	1.054
GCN-NVP (Ours)	45	4.4%	1.058
GGC-NVP (Ours)	45	4.1%	1.056
GIN-NVP (Ours)	45	4.2%	1.056
GAT-NVP (Ours)	45	4.1%	1.055
GEAN-NVP (Ours)	45	4.7%	1.062
Top-45	45	-7.5%	0.992
Oracle	625	5.8%	1.075

Table 3. Evaluation results on **held-out test set** averaged over all datasets. $\bar{I}^{Oz}$ denotes per-program **MeanOverOz**, and $\bar{I}_G^{Oz}$ denotes **GMeanOverOz** over all programs. All methods except Compiler and Oracle baselines use 45 compiler optimization passes.

Appendix for a visualization of the improvement in program size over the 45 passes on programs from the holdout set.

The Oracle shows strong performance but requires a large number of interactions with the compiler. But, this shows that the pass sequence search generalizes to new unseen programs. This is somewhat unsurprising given that the compiler's built-in hand-tuned pass list (-Oz) works reasonably well for most programs.

The performance of Top-45 by itself is weak showing that in order to achieve good results in a reasonable number of passes (45) we need to leverage a general policy and

search to select the most likely candidate pass sequences to evaluate.

4.4. Why Did the RL-PPO Baseline Fail?

We provide an empirical analysis of why the RL-PPO approaches obtain much lower performance compared to our NVP approaches. We *hypothesize* two possible reasons for the failures of RL-PPO. **1) Inaccurate state-value estimation results in a high variance in training.** In the PPO algorithm, we have a policy network (the actor) to output a probability distribution over the actions. And we have a value network (the critic) for estimating the state values, where the approximation is based on regressing the cumulative reward of trajectories sampled from the current policy (Schulman et al., 2017). The update of the policy network is based on the state value outputted by the value network. Inaccurate state value estimation results in a high variance in training the policy network. Due to the stochastic nature of the value estimation that stems from the Monte Carlo sampling in cumulative reward regression, it is difficult to analyze how accurately the value network approximates the ground truth state values (which are unknown even for the programs in the training set). We alleviate this issue by analyzing the Q-value-rank approach (as introduced in Section 4.2), which can be seen as a simplified version of the value approximation in PPO. The Q-value-rank approach is simpler because the values to estimate are deterministic (i.e., the value vector r^p is fixed for a program p). Moreover, since we consider the 50 pass sequences in our coreset as 50 generalized actions, the Q-value-rank approach can be seen as the value approximation in a PPO pipeline where a trajectory consists of only a single step over the 50 generalized actions. In this sense, the Q-value-rank approach is a simplified version of the regular value estimation in PPO. Figure 3 shows that the value estimation is inaccurate for programs in the held-out test set even for Q-value-rank approach. Therefore, it is even more challenging to estimate the state values in PPO. The inaccuracy leads to a high variance in training the policy network. **2) The reward is very sparse.** As shown in Figure 2, the rewards are very sparse. Therefore, the good states (i.e., the program states with a higher chance to be optimized in code size reduction) are rarely seen by the value/policy network during rollouts. Then, the value network does not have a good value estimation for those good states, and the policy network does not converge to output a good policy for them. We conjecture these two issues are the main reason for why the RL-PPO methods obtain the worst performance as shown in Table 3.

4.5. Ablation Studies

Ablation for GEAN-NVP We perform 3 ablation experiments for GEAN-NVP, where we remove graph mixup,

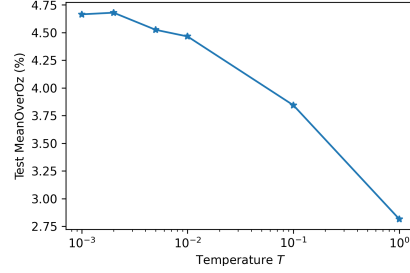


Figure 4. The effect of temperature on GEAN-NVP.

Method	Test MeanOverOz
GEAN-NVP	4.7% (0.0%)
- graph mixup	4.4% (-0.3%)
- edge embedding	4.4% (-0.3%)
- type graph	-5.3% (-10.0 %)

Table 4. Ablation on GEAN-NVP components.

mask the edge embedding, and remove the type graph, respectively. The results in Table 4 show that the test MeanOverOz metric drops after removing any of the three components. Specifically, the performance drops significantly after removing the type graph, which validates its importance.

The effect of the temperature The temperature parameter T in Eq. 3 controls how sharp the target distribution is. The distribution tends to be sharper as the temperature decreases. To analyze the influence of the temperature on the generalization of the model, We vary the temperature T in training the GEAN-NVP model and report the results in Figure 4.

5. Conclusions

In this paper, we develop a pipeline for program size reduction under limited compilation passes. We find that it is a great challenge to approximate the state values (i.e., the maximum code size reduction) for a diverse set of programs, so existing state-of-the-art methods such as proximal policy optimization (PPO) fail to obtain good performances. To tackle this problem, we propose a search algorithm that discovers a good set of pass sequences (i.e., the coreset), which generalizes well to unseen programs. Moreover, we propose to train a GNN to approximate the normalized state values of programs over the coreset, for which we propose a variant of the graph attention network, termed GEAN. Our pipeline of coreset discovery and normalized value prediction via GEAN perform significantly better than the PPO baselines.

A. Proofs

Lemma A.1. *The objective $J(S)$ defined in Eqn. 1*

$$\max_{|S| \leq K} J(S) = \sum_{i=1}^N \max_{j \in S} r_{ij} \quad (7)$$

(with the additional definition $J(\emptyset) = 0$), is a nonnegative and monotone submodular function.

Proof. Since $r_{ij} > 0$, it is clear that $J(S) \geq 0$ is nonnegative.

To incorporate the special case $J(\emptyset) = 0$, note that $J(S)$ can be written as

$$\max_{|S| \leq K} J(S) = \sum_{i=1}^N \max \left(\max_{j \in S} r_{ij}, 0 \right). \quad (8)$$

Let $\hat{r}_{ij} = r_{ij}$ and $\hat{r}_{i,0} = 0$, then in order to prove $J(S)$ is monotone and submodular, by additivity, we only need to prove $J_i(S) := \max_{j \in S \cup \{0\}} \hat{r}_{ij}$ is monotone and submodular.

For any $A \subseteq B$, it is clear that

$$J_i(A) = \max_{j \in A \cup \{0\}} \hat{r}_{ij} \leq \max_{j \in B \cup \{0\}} \hat{r}_{ij} = J_i(B) \quad (9)$$

So $J_i(S)$ is monotone.

To prove submodularity, for any $A \subseteq B$, we compare the quantity of $J_i(A \cup \{j\}) - J_i(A)$ and $J_i(B \cup \{j\}) - J_i(B)$ for $j \notin B$.

Case 1: $\hat{r}_{ij}$ is a maximum over the subset B . In this case, then $\hat{r}_{ij}$ is also a maximum over the subset A . Then $J_i(A \cup \{j\}) = J_i(B \cup \{j\}) = \hat{r}_{ij}$, since $J_i(A) \leq J_i(B)$, we have:

$$J_i(A \cup \{j\}) - J_i(A) \geq J_i(B \cup \{j\}) - J_i(B) \quad (10)$$

Case 2: $\hat{r}_{ij}$ is a maximum over A but not in B . Then $J_i(A \cup \{j\}) - J_i(A) \geq 0$, but $J_i(B \cup \{j\}) - J_i(B) = 0$. So Eqn. 10 still holds.

Case 3: $\hat{r}_{ij}$ is neither a maximum in A or in B . Then both $J_i(A \cup \{j\}) - J_i(A) = 0$ and $J_i(B \cup \{j\}) - J_i(B) = 0$. So Eqn. 10 still holds.

By definition of submodularity (Eqn. 10), we know $J_i(S)$ is submodular and so does $J(S)$. $\square$

B. GEAN Encoding

Our Graph Edge Attention Network (GEAN) has the following key features.

Attention with edge features We modify the attention mechanism in GAT to output an edge embedding and two node features for neighborhood aggregation. For clarity, we show a table containing the notations used in the GNN in Table 5. The feature update process can be mathematically defined by the following equations, where $M_i, i = 1, \dots, 5$ is an encoding fully connected layer.

$$X'_{ij}{}^{(t+1)} = M_1(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (11)$$

$$a'_{ij}{}^{(t+1)} = M_2(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (12)$$

$$X_{ji}^{(t+1)} = M_3(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (13)$$

$$a_{ji}^{(t+1)} = M_4(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (14)$$

$$E_{i \rightarrow j}^{(t+1)} = M_5(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)}), \quad (15)$$

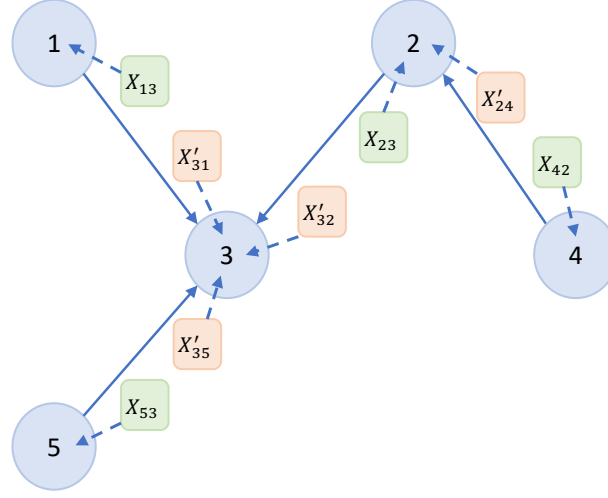


Figure 5. Graph attention. Circles denote nodes and solid arrows denote edges. Squares are the calculated features, and dash arrows represent feature aggregation. The orange/green squares denote the features to be aggregated in the target/source nodes of the edges. The edge embedding and the attention are not shown.

Notation	Meaning
$\mathcal{E}$	The set of edges in the graph
(i, j)	Edge from node i pointing to node j
$X_i^{(t)}$	Representation of node i at layer t
$E_{i \rightarrow j}^{(t)}$	Representation of the edge (i, j) at layer t
$X_{ij}^{(t)}$	Representation for node i associated with edge (i, j)
$X'_{ij}^{(t)}$	Representation for node i associated with edge (j, i)
$a_{ij}^{(t)}$	Raw attention associated with representation $X_{ij}^{(t)}$
$a'_{ij}^{(t)}$	Raw attention associated with representation $X'_{ij}^{(t)}$
$\alpha_{ij}^{(t)}$	Normalized attention associated with $a_{ij}^{(t)}$
$\alpha'_{ij}^{(t)}$	Normalized attention associated with $a'_{ij}^{(t)}$
$\mathcal{T}_i$	Target neighbors of node i : $\{j (i, j) \in \mathcal{E}\}$
$\mathcal{S}_i$	Source neighbors of node i : $\{j (j, i) \in \mathcal{E}\}$

Table 5. The notations in GEAN.

In words, the node-edge-node triplet, $(X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)})$, associated with edge (i, j) , is encoded by fully connected layers to output 5 features, including $X_{ij}^{(t+1)}$ and $a'_{ij}^{(t+1)}$ (a representation and attention to be aggregated in node i), and $X'_{ji}^{(t+1)}$ and $a_{ji}^{(t+1)}$ (a representation and attention to be aggregated in node j), and the updated edge representation $E_{i \rightarrow j}^{(t+1)}$. Note that the features to be aggregated to a target node are marked with the ', and those to a source node are without the ' (see Figure 5). After the feature encoding, we perform an attention-weighted neighborhood aggregation for each node, which can be mathematically described by the following equations.

$$\left[\alpha_{ij}^{(t+1)} \right]_{j \in \mathcal{T}_i} \parallel \left[\alpha'_{ij}^{(t+1)} \right]_{j \in \mathcal{S}_i} = \text{Softmax} \left[\left[a_{ij}^{(t+1)} \right]_{j \in \mathcal{T}_i} \parallel \left[a'_{ij}^{(t+1)} \right]_{j \in \mathcal{S}_i} \right] \quad (16)$$

$$X_i^{(t+1)} = \sum_{j \in \mathcal{T}_i} \alpha_{ij}^{(t+1)} X_{ij}^{(t+1)} + \sum_{j \in \mathcal{S}_i} \alpha'_{ij}^{(t+1)} X'_{ji}^{(t+1)} \quad (17)$$

where $\parallel$ denotes concatenation.

Learning Compiler Pass Orders using Coreset and Normalized Value Prediction

In comparison, GAT only outputs an attention score by encoding the node-edge-node triplet: $d_{ij}^{(t+1)} = (X_i^{(t)}, E_{i \rightarrow j}^{(t)}, X_j^{(t)})$, and the feature for neighborhood aggregation is only conditioned on the neighbors: $X_{ij}^{(t+1)} = \text{MLP}(X_j^{(t)})$. To summarize, our encoding approach can ensure that the GNN model is aware of the direction of the edge and that the edge embedding is updated in each layer, which helps improve the performance (as shown in Table 3).

C. Detailed Results

Dataset	Oracle	Top-45	Autophase-RL-PPO	Autophase-NVP	GEAN-RL-PPO	GEAN-NVP
anghabench-v1	0.7%/1.011	-1.0%/0.996	-15.9%/0.974	-0.2%/1.002	-0.8%/0.996	-0.0%/1.003
blas-v0	2.6%/1.028	-0.4%/0.997	-1.7%/0.984	2.1%/1.023	-1.0%/0.990	2.4%/1.026
cbench-v1	3.5%/1.041	-2.4%/0.984	-10.1%/0.925	-0.1%/1.008	-1.6%/0.998	2.2%/1.028
chstone-v0	9.3%/1.106	1.2%/1.016	1.3%/1.018	8.3%/1.095	5.4%/1.060	8.8%/1.101
clgen-v0	5.4%/1.060	3.1%/1.034	-0.5%/0.998	4.6%/1.051	0.3%/1.005	5.0%/1.056
csmith-v0	21.2%/1.320	-96.3%/0.851	-116.0%/0.954	21.1%/1.320	-124.6%/0.965	21.1%/1.320
github-v0	1.0%/1.011	0.2%/1.002	0.1%/1.001	0.9%/1.010	-0.2%/0.999	0.9%/1.010
linux-v0	0.6%/1.007	-0.4%/0.998	-0.5%/0.997	0.6%/1.006	-2.3%/0.989	0.6%/1.007
llvm-stress-v0	6.3%/1.087	-18.9%/0.885	-67.0%/0.731	0.7%/1.035	-17.5%/0.888	2.1%/1.045
mibench-v1	1.7%/1.020	0.0%/1.003	-2.8%/0.976	-5.8%/0.963	-0.3%/1.000	-0.1%/1.003
npb-v0	9.8%/1.159	5.7%/1.085	0.9%/1.035	5.1%/1.079	3.7%/1.068	5.5%/1.085
opencv-v0	5.2%/1.061	1.0%/1.013	0.5%/1.007	4.2%/1.051	0.3%/1.004	4.8%/1.057
poj104-v1	7.8%/1.105	3.9%/1.055	-17.5%/0.876	6.1%/1.080	-0.7%/1.008	6.3%/1.082
tensorflow-v0	6.1%/1.077	-0.2%/0.998	0.2%/1.004	5.9%/1.075	-0.2%/0.998	5.9%/1.075
Average	5.8%/1.075	-7.5%/0.992	-16.3%/0.960	3.8%/1.054	-10.0%/0.997	4.7%/1.062

Table 6. Detailed evaluation results on held-out test sets.

D. Trajectory comparison

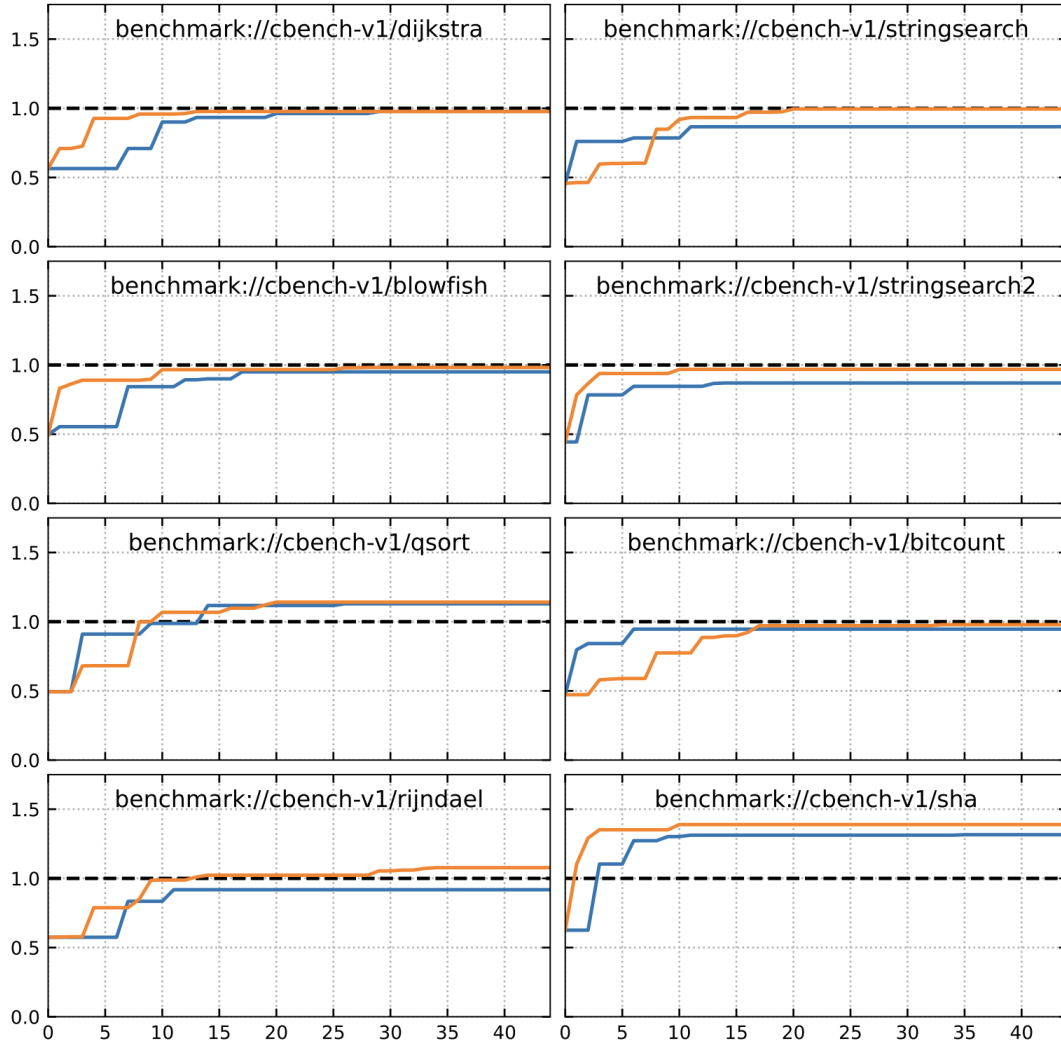


Figure 6. Program optimization example over many steps comparing the **Autophase-RL-PPO** (blue) approach with our **GEAN-NVP** (orange) approach. The dashed line represents the compiler default `-Oz` performance and higher is better.

E. Compiler passes

Index	Flag	Index	Flag	Index	Flag
0	-add-discriminators	42	-globalsplit	84	-lower-expect
1	-adce	43	-guard-widening	85	-lower-guard-intrinsic
2	-aggressive-instcombine	44	-hotcoldsplit	86	-lowerinvoke
3	-alignment-from-assumptions	45	-ipconstprop	87	-lower-matrix-intrinsics
4	-always-inline	46	-ipsccp	88	-lowerswitch
5	-argpromotion	47	-indvars	89	-lower-widenable-condition
6	-attributor	48	-irce	90	-memcpypopt
7	-barrier	49	-infer-address-spaces	91	-mergefunc
8	-bdce	50	-inferattrs	92	-mergeicmps
9	-break-crit-edges	51	-inject-tli-mappings	93	-mldst-motion
10	-simplifcfg	52	-instsimplify	94	-sancov
11	-callsite-splitting	53	-instcombine	95	-name-anon-globals
12	-called-value-propagation	54	-instnamer	96	-nary-reassociate
13	-canonicalize-aliases	55	-jump-threading	97	-newgvn
14	-consthoist	56	-lcssa	98	-pgo-memop-opt
15	-constmerge	57	-licm	99	-partial-inliner
16	-constprop	58	-libcalls-shrinkwrap	100	-partially-inline-libcalls
17	-coro-cleanup	59	-load-store-vectorizer	101	-post-inline-ee-instrument
18	-coro-early	60	-loop-data-prefetch	102	-functionattrs
19	-coro-elide	61	-loop-deletion	103	-mem2reg
20	-coro-split	62	-loop-distribute	104	-prune-eh
21	-correlated-propagation	63	-loop-fusion	105	-reassociate
22	-cross-dso-cfi	64	-loop-guard-widening	106	-redundant-dbg-inst-elim
23	-deadargelim	65	-loop-idiom	107	-rpo-functionattrs
24	-dce	66	-loop-instsimplify	108	-rewrite-statepoints-for-gc
25	-die	67	-loop-interchange	109	-sccp
26	-dse	68	-loop-load-elim	110	-slp-vectorizer
27	-reg2mem	69	-loop-predication	111	-sroa
28	-div-rem-pairs	70	-loop-rollback	112	-scalarizer
29	-early-cse-memssa	71	-loop-rotate	113	-separate-const-offset-from-gep
30	-early-cse	72	-loop-simplifcfg	114	-simple-loop-unswitch
31	-elim-avail-extern	73	-loop-simplify	115	-sink
32	-ee-instrument	74	-loop-sink	116	-speculative-execution
33	-flattencfg	75	-loop-reduce	117	-slsr
34	-float2int	76	-loop-unroll-and-jam	118	-strip-dead-prototypes
35	-forceattrs	77	-loop-unroll	119	-strip-debug-declare
36	-inline	78	-loop-unswitch	120	-strip-nondebug
37	-insert-gcov-profiling	79	-loop-vectorize	121	-strip
38	-gvn-hoist	80	-loop-versioning-licm	122	-tailcallelim
39	-gvn	81	-loop-versioning	123	-mergereturn
40	-globaldce	82	-loweratomic		
41	-globalopt	83	-lower-constant-intrinsics		

Table 7. A list of LLVM compiler pass indices and their corresponding command line flag.



Efficient compiler optimization by modeling passes dependence

Jianfeng Liu¹ · Jianbin Fang¹ · Ting Wang¹ · Jing Xie¹ · Chun Huang¹ · Zheng Wang²

Received: 7 July 2024 / Accepted: 30 September 2024 / Published online: 18 December 2024
© The Author(s) 2024

Abstract

Selecting the optimal combination of compiler passes is a significant challenge to enhance performance and reduce the code size of compiled binaries. While a well-selected sequence of compiler passes can yield considerable benefits, the large number of potential combinations and the scarcity of effective ones make this task prohibitively complex. To tackle this problem, we propose a novel approach to group compiler passes into a small set of sub-sequences. This approach translates the task of identifying the right compiler passes combination into determining the appropriate combination of these sub-sequences. We apply our approach to CBench and PolyBench, demonstrating remarkable performance improvements. Our approach enhances runtime performance by 22% compared to the default LLVM ‘O3’ option, and achieves a code size reduction of 24% compared to the ‘Oz’ option. Our approach also outperforms state-of-the-art across various optimization tasks and hardware platforms.

Keywords Compiler phase ordering · Passes clustering · Iterative compilation

1 Introduction

Modern compilers offer a wide range of optimization passes to enhance the performance of compiled code. To facilitate their usage, compilers provide tuning flags to customize individual optimization passes, such as turning on or off a pass, changing the threshold for vectorization and loop unrolling. While the compiler’s default options like ‘O3’ for performance optimization and ‘Oz’ for code size reduction provide a bundle of default compiler passes, prior studies have demonstrated that customizing compiler flag combinations can lead to significant performance improvements compared to the default settings (Ansel et al. 2014; Fursin et al. 2011).

However, the search space for potential compiler pass combinations is vast, making it challenging to identify the optimal pass sequence. Modern compilers such as LLVM offer over 100 general-purpose optimization passes,¹ which can be applied iteratively. This results in an exponentially growing search space with regard to the sequence length. The vast optimization space makes it a highly daunting task for discovering the right compiler pass sequence that enhances performance beyond the default settings. Despite the potential for substantial performance gains, the execution time required to explore this space based on searching algorithms can be prohibitively long, limiting their practical adoption. This problem is also known as *phase-ordering* (Vegdahl 1982).

The prior approaches to tackling the phase-ordering problem in compiler optimization include search-based algorithms and iterative compilation techniques. The search-based algorithms, such as genetic algorithms and simulated annealing, have been used to explore the optimization space (Fursin et al. 2011; Cooper et al. 1999). However, these methods can be computationally expensive and time-consuming due to the large number of possible pass combinations. Iterative compilation frameworks, like OpenTuner (Ansel et al. 2014), aim to reduce the search space by iteratively refining the optimization passes based on

✉ Jianfeng Liu
liujianfeng14@nudt.edu.cn

✉ Jianbin Fang
j.fang@nudt.edu.cn

✉ Ting Wang
tingwang@nudt.edu.cn

Jing Xie
xiejing@nudt.edu.cn

¹ National University of Defense Technology, Changsha, China

² Northwest University, Xi’an, China

¹ <https://llvm.org/docs/Passes.html>.

performance feedback. While these frameworks have shown promise, they still face challenges in efficiently navigating the vast search space to find optimal pass sequences.

This paper presents Dependence-based Compiler Optimization framework (DCO), a new approach for reducing the search space of phase ordering. Our key insight is that some compilation passes exhibit mutual benefits or dependencies, making it beneficial to consider and use them together, e.g., incorporating loop unrolling and instruction scheduling is likely to benefit a vectorization pass. By identifying and organizing these related passes into *sub-sequences* to leverage their interdependencies, we can translate the problem of finding a sequence of individual compiler passes into seeking a combination of subsequences. Since the number of sub-sequences is significantly smaller than the number of individual compiler passes, doing so can substantially reduce the overall search space and greatly diminish the search overhead.

The core of our work is modeling two fundamental types of relations to capture compiler pass dependence: *source code dependence* and *pairwise performance dependence*. These dependences are determined by analyzing the order in which compilation passes are invoked and by evaluating the performance of pairwise combinations of optimization passes. By effectively capturing these dependences, we construct an optimization dependence graph (ODG) to further generate sub-sequences. Our approach generates sub-sequences by clustering this optimization dependence graph, which are then specifically designed to be applied within two iterative compilation frameworks [i.e., OpenTuner (Ansel et al. 2014) and LaMCTS (Wang et al. 2020)]. Our approach facilitates more targeted and efficient reduction of the optimization space, leading to improved performance within the iterative compilation frameworks.

Our experimental results demonstrate the performance advantages of DCO over state-of-the-art approaches. In the two iterative compilation frameworks, DCO achieves an up to 1.22x speedup compared to the ‘O3’ optimization sequence, and a 1.24x improvement in code size reduction compared to the ‘Oz’ sequence. These results confirm the effectiveness of our approach in enhancing both running-time performance and code size optimization.

The key contributions of our paper are summarized as follows.

- We introduce a novel approach to model compiler pass dependence through source code dependence and pairwise performance dependence, and construct an optimization dependence graph (ODG) to serve as the basis for generating optimization sub-sequences (Sects. 3.1–3.4).
- We implement our approach within two iterative compilation frameworks, OpenTuner and LaMCTS,

demonstrating its applicability and effectiveness (Sect. 3.5).

- Our experimental results show substantial improvements in both runtime performance and code size optimization tasks, validating the superiority of our method over state-of-the-arts (Sect. 4).

2 Background and motivation

2.1 Problem definition

Our work targets the classical compiler optimization problem of *phase-ordering* (Ashouri et al. 2018; Wang and O’Boyle 2018; Leather et al. 2020). This problem considers the selection and ordering of compiler passes to be applied to the target problem, typically observed in LLVM optimizations. This differs from compiler pass selection, which focuses on selecting the optimal set of compiler passes without considering their order, as seen in GCC.

The designed space of compiler phase ordering can be vast. In this work, we let Ω denote the whole optimization space, N denote the number of available compiler passes, and M denote the maximum length of compiler optimizations. We use these notations to define the problem size for *pass selecting* in Eq. 1.

$$\Omega_{select} = 2^M \quad (1)$$

Note that some optimization passes in compilers exist in intermediate states beyond a simple on or off setting. For instance, configurable parameters like loop tiling factors or loop vectorization factors can take on multiple values. Thus, Eq. 1 only offers a lower-bound estimate of the optimization space, while the actual space can be significantly larger. GCC currently incorporates over 200 optimization passes (flags), and attempting to encompass the entire optimization space would result in an exceedingly vast search space.

Meanwhile, determining a suitable optimization sequence for LLVM will introduce an even larger search space. Note that merely selecting a set of optimization passes for LLVM is inadequate because how to arrange the order of these passes can also impact performance. The problem size for phase ordering can be quantified with Eq. 2.

$$S|\Omega_{Phase}| = \sum_{i=1}^M N^i \quad (2)$$

Optimizing codes in LLVM is particularly challenging, as it offers more than 100 compiler passes for code optimization. Considering the length of the predefined ‘O3’ sequence in LLVM, which is approximately 300 (version 12.0.1), the size of the optimization space becomes overwhelmingly large, exceeding 10^{600} according to the Eq. 2. This immense

search space makes it infeasible to exhaustively explore and find the optimal solution to the phase ordering problem.

Our work mainly focuses on the phase-ordering problem and tries to reduce the optimization space by clustering optimization passes into several sub-sequences.

2.2 Motivation

Previous studies have investigated using *sub-sequences* to address the extensive search space linked with compiler optimization passes. A sub-sequence, which comprises one or multiple optimization passes arranged in a specific order, acts as the basic unit. Categorizing compiler passes into k groups leads to k larger passes, effectively reducing the overall search space. Several compiler optimization works are based on sub-sequences.

MiCOMP (Ashouri et al. 2017) generates optimization sub-sequences from the ‘O3’ sequence. It models the ordering relationship within the sequence as pass dependence and clusters all passes occurring in ‘O3’ into several sub-sequences. By analyzing each pass’s dependence, MiCOMP effectively groups passes to create sub-sequences that maintain the optimization benefits of ‘O3’.

CORL (Mammadli et al. 2020) also derives optimization sub-sequences from the ‘O3’ sequence. It follows the observation that the optimization sequence consists of smaller logical sub-sequences ending with a ‘simplifycfg’ pass. This generation is empirical and requires human effort to identify and segment these logical sub-sequences.

POSET-RL (Jain et al. 2022) constructs optimization sub-sequences using IR2Vec embeddings and the ‘Oz’ optimization sequence. The framework identifies critical paths and nodes with high degrees in the Oz dependence graph and creates paths representing sub-sequences of optimization passes. These sub-sequences are designed to efficiently optimize both binary size and execution time.

Table 1 Comparison among sub-sequence based methods

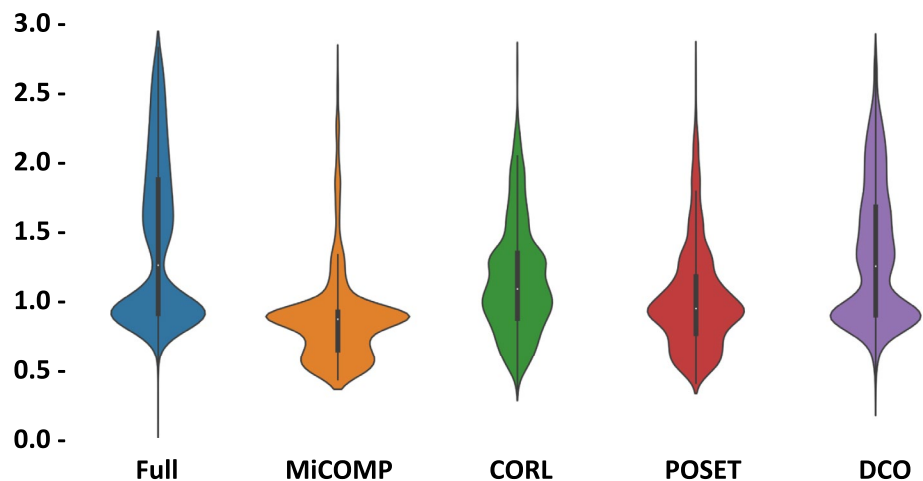
Method	Representative	Evenness	Automation
MiCOMP	★	★	★★
CORL	★★	★★★	★
POSET-RL	★★	★★★	★
DCO	★★★	★★★	★★★

However, the prior works based on sub-sequence have drawbacks in *representatives*, *evenness* and *automation*, which are summarized in Table 1.

The generated sub-sequences of the prior approaches are not sufficiently representative In this paper, we use the term *representatives* to describe the optimization potential of compiler sub-sequences. Figure 1 presents the performance results of pairwise combinations of units from different optimization spaces, where ‘Full’ represents the entire optimization space without applying any sub-sequence method. We evaluate the differences using two metrics: the mean value of the combination results and the threshold of cover range. The figure clearly shows that the ‘Full’ space outperforms the current sub-sequence methods on both metrics.

Not all combinations of sub-sequences produce favourable results. For instance, most combinations of sub-sequences in MiCOMP do not perform better than the non-optimized code. This implies that the sub-sequences in MiCOMP may lack the required diversity or effectiveness to achieve significant optimizations. Conversely, our approach, named DCO, achieves performance comparable to the ‘Full’ space by effectively combining different sub-sequences and covering a wider optimization space. This suggests that DCO can generate more representative sub-sequences within the optimization space, thus striking a better balance between optimization potential and computational efficiency.

Fig. 1 Pairwise combination of sub-sequences from different search spaces, where the ‘Full’ space means the entire space formed by all optimization passes



The generated sub-sequences show a notable issue of unevenness We found that the *unevenness* of sub-sequence construction not only affects its effectiveness but also influences the compilation time and tuning process of iterative compilation. We will discuss the details in Sect. 4. The reason behind this could be that when sub-sequences are unevenly constructed, there tends to be one longer sub-sequence compared to the others. A longer sub-sequence has a more significant chance of being chosen during iterative compilation, leading to increased compilation times.

The sequences of optimization passes in MiCOMP show significant unevenness in their distribution. Most passes are concentrated in one long sub-sequence, while other sub-sequences usually contain only one or two passes. This disparity primarily comes from the usage of the standard ‘O3’ optimization sequence, which typically includes most passes only once. Additionally, the lack of clear distinctions among compiler passes may introduce inconsistencies in the analysis of pass dependencies. For example, some passes with no direct relationship might be sequenced together in ‘O3’, which could result in erroneous information, misleading the analysis into suggesting that these passes are highly related. Moreover, the frequent co-occurrence of passes leads to extensive chains within the compiler, creating a clustering phenomenon where numerous passes accumulate in a single long sub-sequence. This disproportionate distribution inadvertently extends the overall length of the optimization sequence, potentially resulting in prolonged compilation times and sub-optimal performance.

The sub-sequences are handcrafted and have limited applicability Sub-sequences can be generated either manually or automatically. Manual construction of sub-sequences has been explored in CORL and POSET-RL. These predefined sub-sequences have shown promising performance on specific optimization tasks but require human effort for generation, limiting their applicability across different scenarios. Another limitation is the restricted set of optimization passes used in these frameworks. For instance, CORL and POSET-RL both rely on

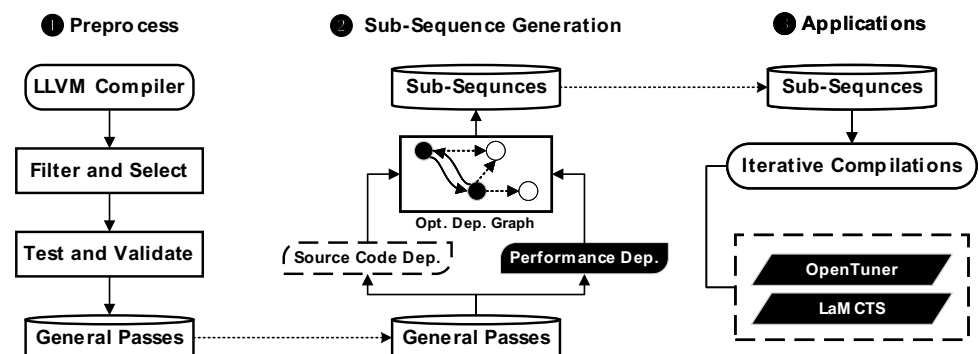
predefined sequences like ‘O3’ or ‘Oz’ in the compiler, which represent only a subset of the available optimization passes. This constraint might lead to a loss in optimization potential as some useful passes may not be included in the optimization space. On the other hand, MiCOMP demonstrates the automatic construction of optimization sub-sequences. However, these sub-sequences are typically designed for specific optimization tasks and may not be suitable for other optimization tasks, e.g., MiCOMP is only applicable to the running-time optimization task. Consequently, when encountering new optimization situations or even the same problem on different architectures, the effectiveness of handcrafted sub-sequences may diminish. Therefore, the generality of handcrafted sub-sequences is a significant constraint, necessitating more adaptive and versatile approaches.

Our analysis indicates that current approaches have limitations in these areas, often resulting in unreliable optimizations. These shortcomings are primarily due to the lack of well-defined boundaries between compiler passes and ambiguity in defining sub-sequences, hindering the effectiveness of optimization strategies. To address these limitations, our work introduces a new approach that uses in-depth dependence analysis and constrained clustering to automatically generate sub-sequences. By focusing on these key features, our method aims to overcome the drawbacks of existing methods and provide more robust and effective optimization strategies. The in-depth dependence analysis ensures that our sub-sequences are highly representative and evenly distributed, while the constrained clustering enhances adaptability to various optimization contexts and tasks.

3 Our DCO approach

The large number of available compiler optimization passes can result in an exceedingly large search space, making it difficult to find the optimal solution. To address this challenge, we propose a novel method that generates groups of

Fig. 2 The overall framework of our DCO approach



reliable sub-sequences through constraint clustering, shown in Fig. 2. Our method is based on a comprehensive dependence analysis that considers both source code dependence and performance dependence, allowing us to identify and cluster related passes into meaningful sub-sequences. DCO has three main steps: passes preprocessing (❶), sub-sequence generation (❷), and sub-sequence application (❸).

3.1 Preprocess compiler optimization passes

3.1.1 Compiler passes analysis

Compiler optimization passes are used to control the behavior of the compiler and influence the quality of generated codes. LLVM has a diverse set of optimization passes, encompassing various aspects of code optimization. These passes can be classified into three categories: *analysis passes*, *transform passes*, and *utility passes*.

Analysis passes They serve the purpose of gathering information that can be used for subsequent optimization tasks. These passes do not directly transform the code but provide insights into the program's structure, characteristics, and dependencies. They lay the foundation for informed decision-making during the optimization process.

Transform passes They are responsible for code transformation. These passes apply various code transformations, such as loop unrolling, function inlining, or vectorization, to change the code structure. Applying transform passes has a significant impact on the performance of the resulting executable, in terms of execution time, code size, power consumption, and many others.

Utility passes They are designed to provide auxiliary information or debugging messages to programmers. While these passes do not directly influence programs' performance, they serve as useful tools for developers in understanding the behavior of the compiler and aiding the debugging process.

Being unrelated to performance enhancements, the utility passes are excluded from our consideration. Our work focuses primarily on the *analysis passes* and *transform passes*, because they directly contribute to performance optimization.

3.1.2 Automated pass filtering

Figure 3 shows that we have developed an automatic tool to extract the relevant optimization passes from the LLVM command line utility 'opt'. Our tool uses a three-step filtering

process to identify the general-purpose passes that are applicable for compiler optimization.

❶ **Filtering unrelated passes** Initially, we obtain a comprehensive list of all available passes provided by LLVM by invoking the 'opt -help' command. To distinguish compiler optimization-related passes from the broader set, we employ pattern recognition techniques. Specifically, we target passes with descriptions that contain particular keywords or phrases indicative of their purpose. First, we filter out utility passes with keywords such as 'Print', 'Annotation', 'Configure', or 'Prepare', which are typically used for auxiliary functions like debugging, logging, and setting up the compilation environment. Next, we separate analysis passes from the remaining pool by identifying keywords like 'Analysis', 'Information', 'Profile', or 'Tracker' in their descriptions. These passes are primarily concerned with gathering and reporting data about the program's structure and behavior, which can be useful for further optimization but do not directly contribute to the transformation of the code itself. By applying these filters, we can focus on the actual optimization passes that directly alter the code to improve performance or reduce size, thereby refining our selection process and enhancing the efficiency of our optimization strategies.

❷ **Filtering architecture-specific passes** In addition to the initial filtering process, we undertake an essential step to pinpoint and exclude passes tailored to specific hardware architectures. Since LLVM supports a wide range of architecture-dependent optimizations, it is crucial to identify and exclude passes that are specific to architectures such as 'aarch64', 'arm', 'mips', 'x86', and others. The procedure for this architectural filtering step involves employing string pattern matching techniques to detect architecture-specific keywords in the pass descriptions. By carefully analyzing the descriptions and identifying these keywords, we can effectively filter out passes that are not universally applicable. This ensures that our optimization strategy remains architecture-agnostic and can be applied across a broad spectrum of target platforms. This step not only simplifies the optimization process but also enhances the portability and flexibility of the compiled code, making it suitable for diverse computing environments.

❸ **Validation and filtering of application-specific passes** We will rigorously test the remaining passes across all available benchmarks to determine their practical suitability. Our goal is to identify any passes that are ineffective, inapplicable, or cause compilation errors for any of the benchmark programs. If a pass shows no noticeable benefits or causes compilation errors across all benchmarks, it will be systematically removed. This validation process guarantees that the selected



Fig. 3 Procedure of the passes filtering tool

passes positively contribute to the optimization and maintain the stability of the compilation process.

The generated passes are shown in Appendix 1, which are regarded as general-purpose and conducive to compiler optimization. Once we have obtained all the valid general purpose optimization passes, we construct an optimization dependency graph based on two kinds of dependence: source code dependence (Sect. 3.2) and performance dependence (Sect. 3.3).

3.2 Identifying source code dependence

Many compiler optimization passes are designed to function in concert with one another, and their isolated application often fails to achieve the optimal results. This interdependence arises because certain transform passes rely on the output of other analysis or transform passes. For example, effectively utilizing the ‘-loop-unroll’ transform pass relies on the insights provided by the ‘-loops’ analysis pass, which provides crucial information about the structure of loops in the code. These relationships among compiler tuning passes, characterized by their dependence on each other’s output, are called *source code dependencies*. They represent the direct relations between various compiler passes and are pivotal in guiding the optimization process. By accounting for these source code dependencies, the compiler can make informed decisions regarding the order and application of different passes, ensuring that optimizations are executed in a manner that maximizes their performance.

In this work, we harness the LLVM APIs to systematically construct the graph of source code dependence relationships among compiler passes, shedding light on their intricate interactions and interdependencies within the optimization process. To achieve this, we employ the LLVM command line parameter ‘-debug-pass=Arguments’ while applying each pass individually. This will generate the printing of dependent passes in an ordered sequence. Typically, this sequence culminates with commands such as ‘-verify’ and ‘-write-bitcode’, which serve validation and data-writing purposes. Any pass that appears before the target optimization pass in this sequence is considered the source code-dependent pass for that particular target. These APIs help us understand how various passes within LLVM’s optimization pipeline depend on one another.

Figure 4 illustrates an example of the ‘-loop-unroll’ pass. By using the ‘-debug-pass=Arguments’ pass, LLVM generates a comprehensive report that discovers the dependence information associated with the ‘-loop-unroll’ pass. This report effectively shows the pass’s dependencies on its preceding and subsequent passes, offering insights into the intricate inter-dependencies that govern the optimization process. The resultant source code dependence graph is depicted in Fig. 6, where the rounded rectangles represent analysis passes, and the others denote transform passes. For simplicity, all dependent connections are assigned a weight of 1. This graphical representation overviews the intricate dependencies influencing the optimization process. We show the extracted source code dependence in Fig. 5.

3.3 Building performance dependence

Section 3.2 has successfully constructed the initial portion of the overall Optimization Dependence Graph (ODG) by establishing the source code dependence graph among compiler passes. But it is important to note that the source code dependence could solely capture the code-level dependencies among the optimization passes. Recognizing the substantial impact of the order in which transform passes are applied on the performance of the target code, we extend the ODG by incorporating the performance dependence relationship between transform passes. This will allow for a more accurate representation of the interdependence among the optimization passes.

In this work, the term ‘performance dependence’ refers to how the collective application of multiple compiler passes influences performance. We define performance improvement as the ratio of the observed performance difference when combining two compiler passes compared to applying them individually. To quantify this performance dependence, we introduce the metric $e(p, a)$ to represent the performance of the target program p when compiled with pass a . Similarly, we use the metric $e(p, a, b)$ to denote the performance of p when compiled with pass a followed by pass b . Note that in both performance optimization and code size compression tasks, a smaller e indicates better performance. The performance dependence can be formulated in Eq. 3. By comparing the metric values, we can assess the performance impact of different combinations of compiler

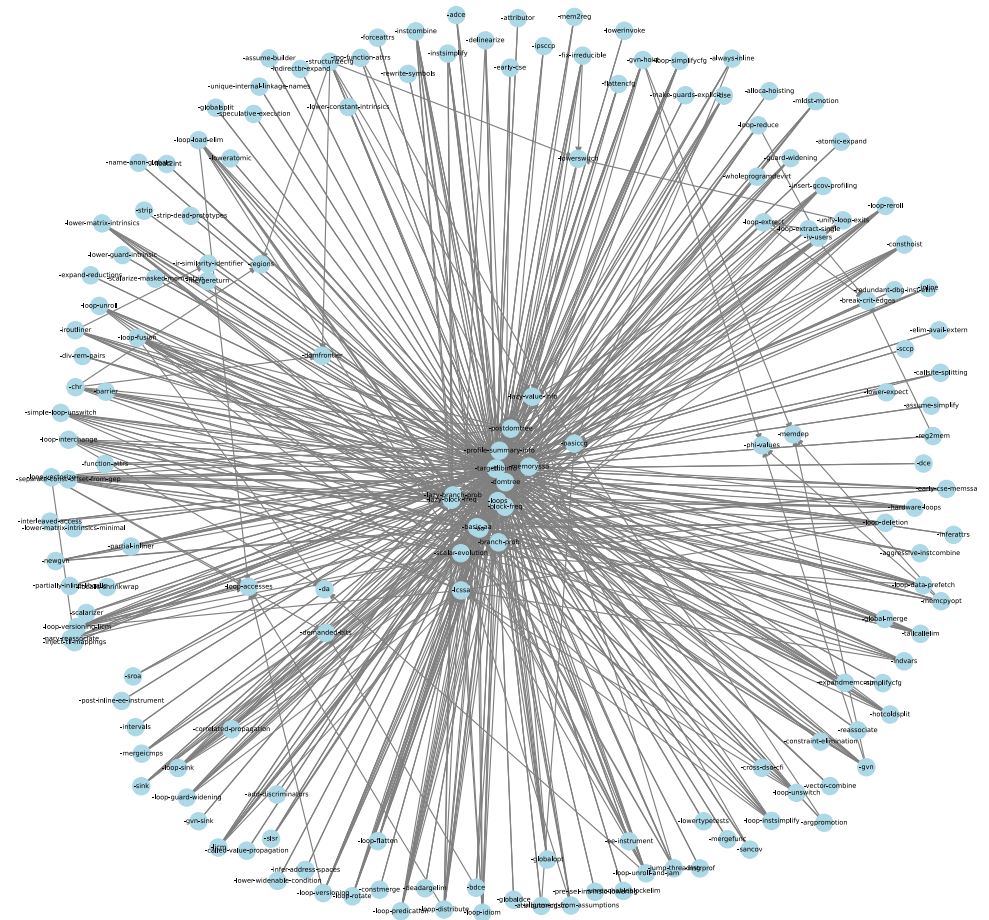
Fig. 4 Emitting the dependent passes

```

1  > opt -debug-pass=Arguments -loop-unroll test.ll -o
    ↪ test_opt.ll
    Pass Arguments: -targetlibinfo -tti -targetpassconfig
3  -assumption-cache-tracker -domtree -loops -loop-simplify
    ↪ evolution
5  -lcssa-verification -lcssa -basic-aa -aa -scalar-
    -loop-unroll -verify -write-bitcode

```

Fig. 5 Source code dependences extracted from LLVM compiler. Each directed edge means the back pass depends on the result of the front pass



passes. This analysis helps us understand how various passes work together to affect the overall performance of the compiled program.

$$dep_{per}(p, a, b) = \theta \cdot \left[\frac{e(p, a)}{e(p, a, b)} \right]^2 \quad (3)$$

where θ is used to control the effect of $\frac{e(p,a)}{e(p,a,b)}$ coming from the combination of passes a and b instead of an individual pass a or b . Supposing $e(p,a) < e(p,b)$, we consider θ under three cases shown as follows.

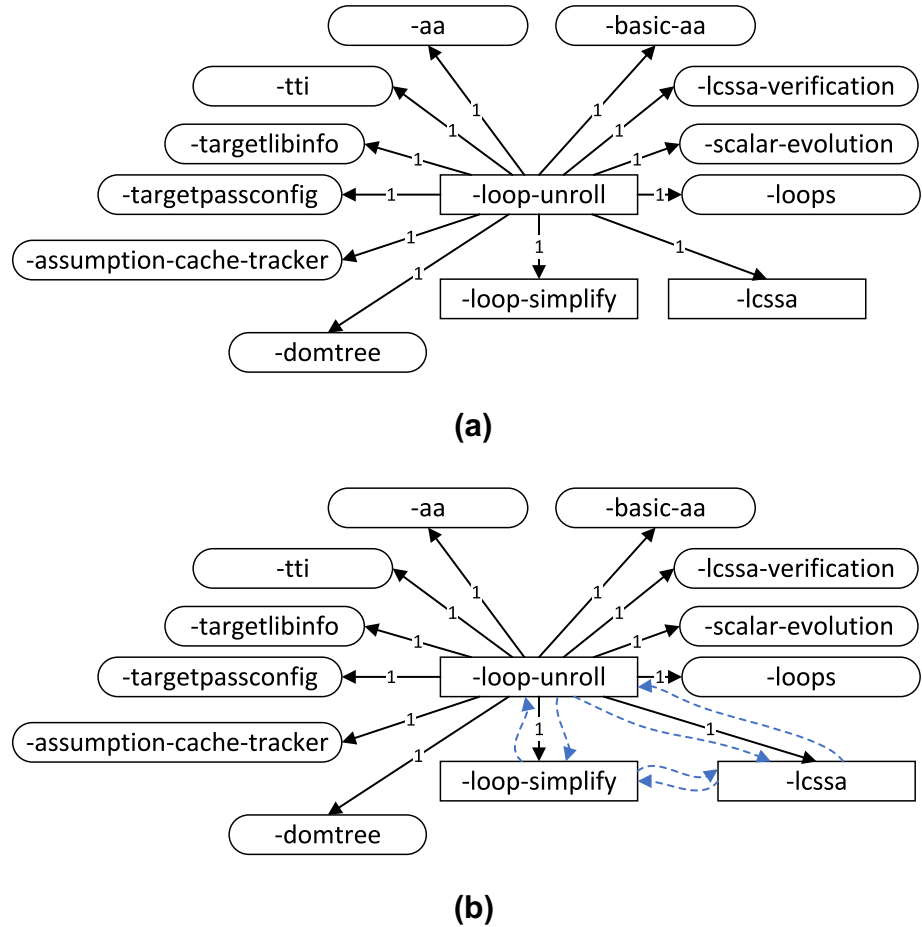
- When $e(p, a, b) < e(p, a) < e(p, b)$ indicating that the combined application of passes outperforms the individual application of each pass, we assign θ to be 1 to represent a positive performance impact.
- When $e(p, a) < e(p, b) < e(p, a, b)$, conversely, suggesting that using the combined passes results in a negative performance impact, we set θ to be -1.
- When $e(p, a) < e(p, a, b) < e(p, b)$, making it challenging to determine whether the combined pass yields a positive performance impact, we assign θ as 0 to denote the uncertainty.

This equation enables us to quantitatively measure the performance increase or decrease of applying the combination of passes (a, b) compared to applying pass a alone. By incorporating performance dependence into DCO, we obtain valuable insights into how compiler passes interact with each other and collectively impact performance. To illustrate this concept, we add dashed lines in Fig. 6 to indicate performance dependence among passes. These dashed lines represent the relationships between passes and depict how the combination of passes affects overall performance.

3.4 Clustering dependence graphs

Once the source code dependence and performance dependence are extracted from both Intel and ARM platforms, we will construct a dependence graph for all compiler optimization passes. This graph serves as a visual representation of the dependence relationships among the passes and aids developers in understanding the intricate interdependencies. We refer to this graph as the Optimization Dependence Graph (ODG).

Fig. 6 Construct optimization dependence graph for a sample optimization pass *-loop-unroll*, including constructing source code dependence graph (a) and adding performance dependence (b)



In the ODG, each node represents a compiler pass, while the edges denote the dependence relationships between pass pairs. Each edge of ODG carries information about the performance dependence or source code dependence between the related passes. The weight assigned to each edge quantifies the degree of dependence between two passes. By leveraging the ODG, developers can gain a deeper understanding of how different compiler passes interact and impact the overall optimization process.

Based on the constructed ODG, we employ a clustering algorithm to generate sub-sequences by grouping the graph nodes. We use the well-established K-means clustering algorithm. The standard K-means clustering algorithm has inherent limitations, particularly in its inability to ensure that clusters adhere to specific constraints, such as a maximum size or balanced distribution of elements. This shortcoming can lead to uneven sub-sequences where certain clusters become disproportionately large, while others remain underpopulated. Such imbalances may adversely affect the performance of subsequent processes that rely on these clusters. To address this issue, we introduce a constrained version of the K-means algorithm,

referred to as *Constraint K-means*. This modified algorithm (Algorithm 1) imposes a maximum threshold on the size of each cluster, ensuring that no single cluster exceeds this limit. The constraint is integrated into the clustering process by iteratively adjusting the assignment of points to clusters, particularly when a cluster reaches its maximum allowable size. By doing so, the algorithm not only maintains balanced clusters but also improves the overall effectiveness of the optimization process by avoiding the pitfalls of excessively large or small sub-sequences. The introduction of these constraints is crucial for applications that require a more uniform distribution of elements across clusters, as it enhances the stability and predictability of the resulting clustered sub-sequences. This constraint ensures that the resulting sub-sequences exhibit a balanced compiler pass distribution. We use the average link approach to measure affinity calculated in Eq. 4.

$$\phi_{x_i, \zeta_j} = \frac{1}{n_j} \sum_{x_k \in \zeta_j} e_{ik} \quad (4)$$

Algorithm 1 Constraint K-means Clustering

Require: A set of n sample points $X = \{x_1, x_2, \dots, x_n\}$, the target number of clusters K , and the maximum threshold of each cluster T .

- 1: **function** AddToNearestCluster(x_i):
- 2: Compute the affinity measure ϕ_{x_i, c_j} between x_i and each cluster c_j , forming a set $\Phi_{x_i} = \{\phi_{x_i, c_1}, \phi_{x_i, c_2}, \dots, \phi_{x_i, c_K}\}$ with descending order.
- 3: **for** $j = 1, 2, \dots, K$ **do**
- 4: **if** $\phi_{x_i, c_j} > T$ **then**
- 5: **if** $\phi_{x_i, c_j} > \max \phi_x$ **then**
- 6: Continue;
- 7: **else**
- 8: Delete the boundary point z from c_j ;
- 9: Add z in c_j ;
- 10: AddToNearestCluster(z);
- 11: **end if**
- 12: **else**
- 13: Add sample x_i to c_j ;
- 14: **end if**
- 15: **end for**
- 16: **end function**
- 17: Randomly form K initial cluster centers $C = \{c_1, c_2, \dots, c_K\}$.
- 18: **for** $i = 1, 2, \dots, n$ **do**
- 19: Select current sample point x_i ;
- 20: AddToNearestCluster(x_i);
- 21: **end for**

3.5 Sub-sequence application

This section applies the DCO sub-sequences within iterative compilation frameworks, highlighting the ability of these frameworks to explore and identify the extreme performance potentials of the generated sub-sequences. Iterative compilation frameworks, such as OpenTuner and LaMCTS, excel at systematically refining optimization sequences through repeated iterations, allowing them to uncover the highest-performing combinations of sub-sequences. By leveraging the iterative nature of these frameworks, DCO can achieve substantial performance improvements by dynamically adjusting and optimizing the sequence of compiler passes based on real-time feedback.

3.5.1 Enabling DCO in OpenTuner

OpenTuner is an advanced open-source tool renowned for its efficacy in general-purpose compiler auto-tuning [1]. It incorporates a wide range of search heuristics (e.g., Nelder-Mean, dynamic programming, evolution, and gradient ascent algorithms), making it a versatile and powerful solution. But it was originally designed specifically for GCC

compiler optimization, primarily focusing on the pass selection problem.

To apply DCO to OpenTuner with the LLVM compiler, several modifications are necessary. First, we define the search space as the collection of all possible optimization sequences, considering the unique characteristics and flexibility of the LLVM compiler. Unlike GCC, which adheres to a fixed optimization sequence, LLVM allows for a more adaptable and dynamic optimization sequence. To harness the capabilities of OpenTuner within the LLVM framework, we leverage the open APIs provided by OpenTuner. We use a sequence of *IntegerParameter* to identify a specific optimization sequence. By integrating OpenTuner into our workflow, we can systematically explore the vast search space of optimization sequences and identify the most optimal sequence based on the defined objectives.

3.5.2 Enabling DCO in LaMCTS

LaMCTS (Wang et al. 2020) operates in an iterative manner, managing a ‘good’ space and a ‘bad’ space based on observations made during the optimization process. The distinction between the ‘good’ and ‘bad’ spaces is determined by the performance outcomes achieved in different

optimization passes. This separation allows for targeted exploration in subsequent iterations to improve the optimization process. LaMCTS uses the Upper Confidence Bound for Trees (UCT) algorithm to intelligently select the next optimization sequence based on accumulated knowledge. The boundaries of the ‘good’ and ‘bad’ spaces are adaptively updated based on the outcomes of the current exploration. This dynamic adjustment process effectively refines the search regions, ultimately aiding in converging towards a stable state and identifying an optimal sequence of optimization passes.

By incorporating LaMCTS into the compiler optimization framework, we can effectively navigate the complex search space. Through a process of iterative exploration and refinement, LaMCTS efficiently identifies and takes advantage of promising areas within the search space, resulting in the discovery of the best sequence of optimization passes. This approach significantly enhances the overall performance and efficiency of the compiler optimization process, leading to improved outcomes for various optimization tasks. By utilizing its iterative and adaptive exploration capabilities, LaMCTS provides a promising solution for optimizing the selection of optimization passes, ultimately enhancing the performance of the compiler optimization process.

4 Performance evaluation

This section presents our experimental methodology and the evaluation setting used to gauge the performance of our DCO method. We outline the steps and considerations to ensure a rigorous and comprehensive evaluation. We first

outline the experimental settings, including the benchmark programs, target platforms, and compiler suite used in our study. Following that, we introduce the comparison baselines, which consist of current sub-sequence generation methods. These baselines are considered the most advanced sub-sequence-based methods in compiler optimization and will be used as a point of comparison to highlight the improvements achieved by our DCO method. Finally, we present the experimental details and analysis of the results. This involves a step-by-step explanation of our experimental procedure, from data collection to analysis. We then discuss the metrics used to evaluate performance, such as execution time, compilation time, and code size. The results will then be analyzed to identify trends, strengths, and weaknesses of our DCO method in comparison to the baselines.

4.1 Experimental setup

We use two benchmark suites, CBench and PolyBench, to evaluate the efficacy of DCO. These benchmarks are selected due to their widespread acceptance in compiler optimization. Table 2 details the programs used in our experiments.

- **CBench**, part of the Collective Knowledge framework, is a comprehensive collection of open-source programs for compiler research and auto-tuning. It has diverse real-world applications and computational kernels from different domains such as signal processing and numerical methods. This diversity in CBench ensures that our evaluation covers a wide spectrum of computational patterns, making it an excellent benchmark for assessing the adaptability and performance improvements offered by compiler optimizations across varied real-world scenarios.
- **PolyBench** is a benchmark suite that primarily focuses on evaluating the effectiveness of polyhedral model optimizations in compilers. It consists of programs characterized by intensive use of nested loops and numerical computations, which are typical in scientific and engineering workloads. PolyBench is mainly used for examining loop transformations and data locality optimizations in a controlled setting, providing insights into the optimization of computational kernels.

The experiments are run on two distinct platforms: (1) an Intel CoreX i9-10900X CPU operating at 3.70 GHz with 64 GB memory, and (2) a Phytium 2000+ ARM CPU running at 2.3 GHz with 64 GB memory. We use Ubuntu 20.04 and LLVM 12.0.0.

Table 2 Benchmark programs used in evaluation

CBench	CBench	PolyBench	PolyBench
bitcount	dijkstra	correlation	cholesky
qsort1	particia	covariance	durbin
susan_c	bzip2d	gemm	gramschmidt
susan_e	bzip2e	gemver	lu
susan_s	blowfish_d	gesummv	ludcmp
ghostscript	blowfish_e	symm	trisolv
rsyth	pgp_d	syr2k	deriche
stringsearch	pgp_e	syrk	floyd-warshall
jpeg_c	rijndael_d	trmm	nussinov
jpeg_d	rijndael_e	2 mm	adi
lame	sha	3 mm	fdtd-2d
tiff2bw	adpcm_c	atax	heat-3d
tiff2rgba	adpcm_d	bicg	jacobi-1d
tiffdither	CRC32	doitgen	jacobi-2d
tiffmedian	gsm	mvt	seidel-2d

4.2 State-of-the-art baselines

We compare DCO to several state-of-the-art methods based on sub-sequences.

- MiCOMP uses weak interactive relationships in generating sub-sequences. We evaluate its effectiveness in producing reliable subsequences for further optimization tasks (Ashouri et al. 2017).
- CORL uses a cooperative approach for the sub-sequence generation. CORL considers the inter-dependencies among tuning passes and uses reinforcement learning techniques (Mammadli et al. 2020).
- POSET-RL uses partially ordered sets to guide the sub-sequence generation process. It exploits the inherent order and dependencies among tuning passes to achieve improved optimization results (Jain et al. 2022).
- Full space. We compare the results obtained from these methods with the full optimization space, representing the complete set of tuning passes in LLVM.

4.3 Evaluation on representatives

Exploration and auto-tuning become easier and faster with sub-sequences, even though they cover only a limited part of the optimization space. However, simplifying the process could affect overall performance because some effective combinations of optimization passes may be left out for specific programs. Therefore, it is crucial to check the *representatives* of the generated sub-sequences.

4.3.1 Pairwise performance

To tackle this problem, we conducted a practical assessment to determine the effectiveness of combining different optimization units in sub-sequences, which we refer to as 'Pairwise Performance'. We treat each sub-sequence as a single comprehensive optimization pass and explore the connections between these sub-sequences. We think that analyzing the pairwise relationships between different sub-sequences can demonstrate the potential of these optimizations.

Table 3 Pairwise performance of different sub-sequences

Method	Performance coverage	Mean performance
Full	[0.11–2.95]	1.17
MiCOMP	[0.36–2.73]	0.94
CORL	[0.29–2.82]	1.08
POSET-RL	[0.33–2.83]	1.05
DCO	[0.16–2.91]	1.16

The bold values highlight wider coverage and higher mean performance. Among the sub-sequence methods, DCO demonstrates the best performance, achieving results close to those of the full optimization space

This evaluation aims to strike a balance between minimal exploration and optimal performance. In our tests, we closely examine the interconnection of various optimization units. We use two primary metrics to gauge performance: the range of covered performance and the average optimization performance. Through analyzing these metrics, we can assess the effectiveness of the sub-sequences compared to a more comprehensive exploration of all possible optimizations.

Figure 1 and Table 3 shows the performance results. We see that the different methods exhibit varying degrees of performance coverage and mean performance. The *Full* method entails exhaustive exploration of the optimization space and demonstrates the widest performance range, from 0.11 to 2.95, with a mean performance of 1.17. On the other hand, *MiCOMP* shows a narrower performance range, from 0.36 to 2.73, with a mean performance of 0.94. This suggests that while *MiCOMP* may simplify the search process, it might miss some high-performing combinations identified by the Full method. In comparison, the *CORL* and *POSET-RL* methods have similar performance coverages, ranging from 0.29 to 2.82 and 0.33 to 2.83, respectively, with mean performances of 1.08 and 1.05. These methods strike a balance between exploration efficiency and performance gain, making them effective in identifying good optimization sequences without exhaustive searching. Finally, our DCO demonstrates a performance coverage from 0.16 to 2.91 and a mean performance close to that of the Full method at 1.16. This indicates that DCO can achieve near-optimal performance while reducing the complexity of the exploration process.

4.3.2 Iterative performance

Another important aspect to showcase the potential of sub-sequences is the actual improvement they bring to applications. We compare results in two different scenarios:

- **Constrained Resource Scenario:** This scenario represents situations where only a limited number of iterative compilation steps are allowed, such as in embedded sys-

Table 4 Iterative performance on OpenTuner under different platforms (Intel/ARM)

Method	Execution Time		Code Size	
	iter=50	iter=1000	iter=50	iter=1000
MiCOMP	0.97 1.03	1.02 1.10	0.98 1.04	1.02 1.07
CORL	1.03 1.06	1.05 1.17	1.08 1.07	1.12 1.18
POSET-RL	1.05 1.06	1.07 1.18	1.07 1.06	1.13 1.21
DCO	1.07 1.08	1.11 1.19	1.08 1.09	1.14 1.23

We compare iterative performance across two frameworks to evaluate the optimization potential of different sub-sequence methods. The bold values represent the best results under the same conditions

Table 5 Iterative performance on LaMCTS under different platforms (IntellARM)

Method	Execution Time		Code Size	
	iter=50	iter=1000	iter=50	iter=1000
MiCOMP	0.84 1.01	0.99 1.08	0.97 1.03	1.02 1.11
CORL	0.97 1.05	1.03 1.18	1.01 1.06	1.09 1.15
POSET-RL	1.05 1.03	1.07 1.17	1.04 1.07	1.11 1.22
DCO	1.04 1.08	1.08 1.22	1.05 1.07	1.13 1.24

We compare iterative performance across two frameworks to evaluate the optimization potential of different sub-sequence methods. The bold values represent the best results under the same conditions

tems or mobile applications. These environments often have restricted computational resources and need to optimize performance within tight constraints.

- **Exhaustive Exploration Scenario:** This scenario pertains to programs that are compiled once and run continuously, such as high-performance computing applications or server-side software. These applications can allocate more time to extensive optimization in order to achieve the best performance possible. For example, optimizing the code for a scientific simulation running on a super-computer, where the initial compilation time is less critical compared to the runtime performance.

We examine the outcomes in two different scenarios. The first scenario involves allowing only minimal effort in the iterative compilation steps, to illustrate that these efforts do not have unlimited compiler optimization resources. The second scenario involves representing programs that compile once and run indefinitely. In these cases, the applications can withstand extensive exploration of the optimization space to achieve the best performance. For these two scenarios, we restrict the tuning steps of two iterative compilation frameworks, OpenTuner and LaMCTS, to 50 steps for the constrained resource scenario and 1000 steps for the exhaustive exploration scenario.

Tables 4 and 5 shows the performance results on both Intel and ARM platforms. In a scenario with limited resources and 50 iterations, we observe several trends. Among the methods, MiCOMP shows the least favorable performance, indicating that its sub-sequences are less effective in compiler optimization. POSET-RL shows strong performance, with execution times of 1.05 in OpenTuner and 1.05 in LaMCTS, along with code size values of 1.07 and 1.04 respectively. While not as high as DCO, POSET-RL is still effective in optimizing performance. CORL shows

improvements of 1.03 in execution time in OpenTuner and 0.97 in LaMCTS, with code size reduction improvements of 1.08 and 1.01 respectively on the two frameworks. While it may be less effective than POSET-RL and DCO, CORL still shows good potential in performance optimization. DCO consistently outperforms other sub-sequence methods in both execution time and code size optimization. With an improvement of 1.07 in OpenTuner and 1.04 in LaMCTS on execution time, and an improvement of 1.08 and 1.05 on code size respectively, DCO demonstrates better efficiency even with minimal tuning steps. This indicates that DCO is highly effective at quickly finding near-optimal solutions, making it an excellent choice when resources are limited. The ARM platform shows similar trends and our DCO outperforms other sub-sequence methods on the all scenarios.

In the exhaustive exploration scenario with 1000 iterations, DCO demonstrates even more significant performance improvement. Although MiCOMP shows improvement with increased iterations, its relative performance remains the lowest among the methods. It achieves execution times of 1.02 in OpenTuner and 0.99 in LaMCTS, with code size reduction of 1.02 and 1.02 respectively. POSET-RL maintains strong performance with execution time improvement of 1.07 in OpenTuner and 1.07 in LaMCTS, and code size reduction of 1.13 and 1.11 respectively. CORL also improves with more iterations, achieving an execution improvement of 1.05 in OpenTuner and 1.03 in LaMCTS, with code size reductions of 1.12 and 1.09 respectively. However, it still performs worse than DCO. DCO continues to excel with extensive tuning, achieving the largest execution time improvement of 1.11 in OpenTuner and 1.08 in LaMCTS. Our code size reductions are also the most significant, with an improvement of 1.14 and 1.13 respectively. This robust performance underlines DCO's capability to effectively explore the optimization space, making it the most powerful method for achieving optimal performance. Our DCO gets even better results on the ARM platform for achieving the optimal performance improvement on all scenarios.

4.4 Evaluation on Evenness

While the evenness of optimization is less related to optimization performance, we found that it significantly influences compilation time, which is crucial in the tuning process of iterative compilations.

Assuming that,

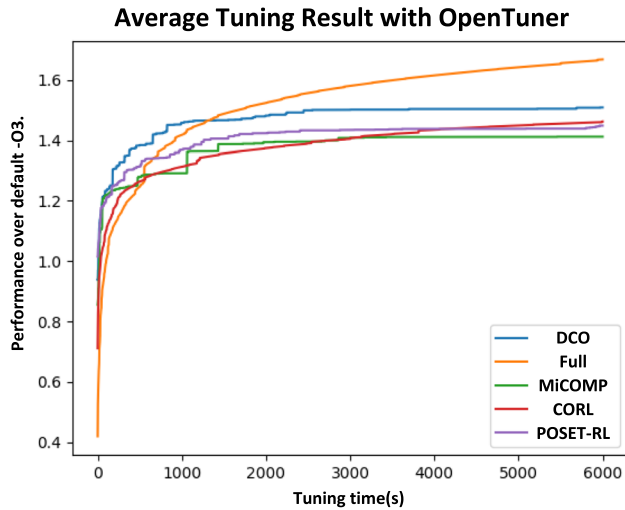


Fig. 7 Tuning performance of OpenTuner on different optimization space with same time cost

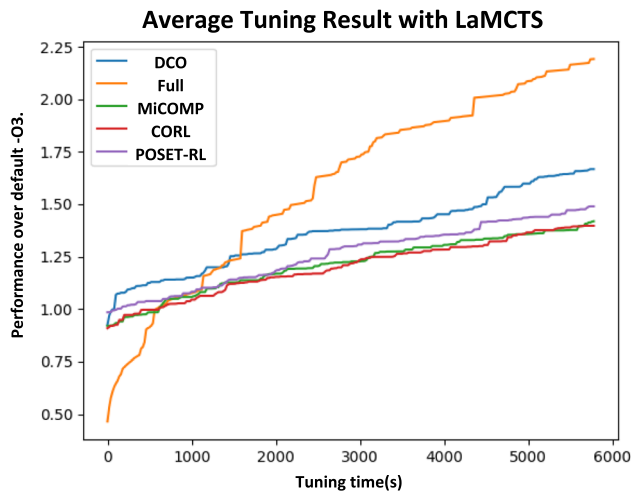


Fig. 8 Tuning performance of LaMCTS on different optimization space with same time cost

1. Each optimization pass O_i (where $i = 1, 2, \dots, n$) has the same compilation time t .
2. Each optimization pass O_i has an equal probability $p = \frac{1}{n}$ of being selected during auto-tuning process.

Now, we can calculate some interesting numbers that indicate the expected compilation time.

Let S_j be a sub-sequence consisting of k_j optimization passes.

Expected Compilation Time for a Single Optimization Pass:

$$E(T) = t \quad (5)$$

Expected Compilation Time for a Sub-sequence S_j :

$$T_{S_j} = k_j \cdot t \quad (6)$$

Probability of Selecting a Sub-sequence S_j :

$$P(S_j) = \frac{k_j}{\sum_{i=1}^m k_i} \quad (7)$$

where m is the total number of sub-sequences.

Expected Compilation Time for the Auto-tuning Process:

$$E(T_{auto}) = \sum_{j=1}^m P(S_j) \cdot T_{S_j} = t \cdot \sum_{j=1}^m \frac{k_j^2}{\sum_{i=1}^m k_i} \quad (8)$$

Hence, the anticipated compilation time is determined by the squared lengths of the sub-sequences. Greater lengths of sub-sequences lead to a larger contribution to the overall expected compilation time, showing that the uniformity of sub-sequence lengths directly affects the total compilation time. Although this is a simplified estimate (as actual compilation times and selection probabilities may vary), it demonstrates the influence of sub-sequence lengths on the observed overall compilation time in real applications.

During our testing, we assessed the performance of various sub-sequences using iterative compilation frameworks while maintaining a consistent time limit, rather than a fixed number of iteration steps. This method takes into consideration the compilation time, as shorter times facilitate a more thorough exploration of the optimization space. The results are shown in Figs. 7 and 8. Our method, referred to as DCO, exhibited the most significant performance enhancement among all sub-sequence-based techniques. Specifically, DCO demonstrated a notable performance improvement compared to other methods, including DCO, MiCOMP, CORL, and POSET-RL. This improvement was evident across both the OpenTuner and LaMCTS frameworks. For instance, in the OpenTuner framework (Fig. 7), DCO achieved superior performance early in the process and

Table 6 Compiler version iteration and changes to optimization passes and the default ‘O3’ and ‘Oz’

Compiler Version	O3 Length	O3 Number	Oz Length	Oz Number	# Passes
v10.0.0	276	88	261	83	147
v11.0.0	288	91	265	85	147
v12.0.0	294	93	271	87	147

maintained this advantage as tuning progressed. Likewise, in the LaMCTS framework (Fig. 8), DCO consistently outperformed the other methods.

We observe that exhaustive exploration of the ‘Full’ space surpasses our approach after approximately 1500 s of tuning. However, it is important to note that our method achieves competitive performance with significantly less tuning time, highlighting its efficiency and practicality for real-world applications.

4.5 Evaluation on automation

Our DCO can automatically extract available optimization passes from the compiler’s API, rather than relying on predefined optimization sequences such as ‘O3’ and ‘Oz’. One notable difference between these two approaches is the stability of the generated sub-sequences.

Since its initial release in 2003, the LLVM compiler has undergone several significant updates. For instance, version 10.0.0 was released in March 2020, followed by version 11.0.0 in October 2020, and version 12.0.0 in April 2021. Table 6 illustrates the default settings of optimization sequences for the LLVM compiler over time. The ‘O3’ and ‘Oz’ optimization sequences have shown rapid improvement across different versions. Surprisingly, the optimization passes have remained unchanged despite three major compiler version changes. This indicates that our method remains relatively stable over time, unlike approaches reliant on predefined optimization sequences such as ‘O3’ and ‘Oz’.

Frequent software releases can lead to inconsistencies in predefined optimization sequences. These sequences may vary in effectiveness with each new version of the compiler. In contrast, DCO extracts and uses the current set of optimization passes in the compiler’s API dynamically. This approach ensures that the generated optimization sub-sequences are always aligned with the latest capabilities of the compiler, resulting in a more stable and adaptable optimization process.

Furthermore, DCO’s method enables the discovery of new optimization paths that predefined sequences might overlook, potentially resulting in greater performance improvements. This flexibility is especially advantageous in quickly evolving software environments, where keeping up with the latest optimization techniques is essential for maintaining high performance.

5 Related work

This section introduces the work on iterative compilation and optimization space.

Iterative compilation Iterative compilation is one approach of releasing programmers and has shown inspiring

results in the past few decades. Programs are auto-tuned with a sequence of iterations to improve performance iteratively. As each iteration requires an online compilation and an execution, iterative compilation is very time-consuming. Thus there are many works proposed to improve the efficiency of iterative compilation by customizing heuristics.

Almagor et al. use a genetic algorithm to select efficient optimization sequences for an adaptive compiler (Almagor et al. 2004). They construct self-steering and self-tuning compilers, which use multiple compilations in a feedback loop to discover good configurations for each combination of input program and target machine. To lower the cost of iterative compilation, Leather et al. provide a method to filter out those ‘bad’ samples through statistic analysis (Leather et al. 2009). This approach can reduce the number of runs of generating statistically rigorous results on compiler performance optimizations. OpenTuner [1] is one of the state-of-the-art models for general-purposed compiler optimization. It is a framework for building domain-specific multi-objective program auto-tuners. OpenTuner achieves to obtain an optimal solution by allowing ensembles of various search techniques on programming auto-tuning. It integrates exhaustive searching, dynamic programming and several evolutionary algorithms. Recently, Chen et al. treat the problem of compiler auto-tuning as a black-box and apply Bayesian Optimization to solve it (Chen et al. 2021). Moreover, auto-tuning methods have been employed across various domains and backend architectures (Zheng et al. 2019; Zhang et al. 2018). Compared to traditional iterative compilation techniques applying black-box optimization techniques in compiler, auto-tuning has shown a significant improvement in terms of search efficiency and overall performance.

Optimization space One major solution of the size problem is to specify a subset of the optimization space. For example, NeuroVectorizer (Haj-Ali et al. 2020) focuses on a loop optimization pass ‘-loop-vectorize’ in LLVM compiler and the whole optimization space can be formed by the combination of two integers.

A promising approach is to use sub-sequences of compiler passes, which are combinations of compiler optimizations that are known to be effective in certain scenarios. Ashouri et al. (2017) propose a technique called MiCOMP that uses optimization sub-sequences and machine learning to mitigate the compiler phase-ordering problem. They show that by selecting the optimization sub-sequence based on the ‘O3’ sequence and pairwise occurrence relationship, they could improve the performance for executing a program and outperform the standard LLVM ‘O3’. Similarly, Mammadli et al. (2020) use a deep reinforcement learning-based approach for static neural compiler optimization, which uses a sub-sequence of optimization passes to optimize LLVM IR code. Jain et al. present POSET-RL (Jain et al. 2022), which optimizes the size

and execution time of programs by learning a policy for selecting a sub-sequence of optimization passes. Their approach involves selecting the right combination of compiler optimizations through LLVM ‘Oz’ sequence, and using reinforcement learning to learn the optimal order of these optimizations.

Compared to these approaches, DCO is a purely automatic method that effectively harnesses the in-depth dependence of compiler passes to deliver reliable sub-sequences.

6 Conclusion and future work

Our work addresses the limitations of existing methods by capturing in-depth pass dependence relationships and then using the constraint clustering. By harnessing the power of these techniques, we have successfully constructed reliable and efficient sub-sequences that significantly reduce the optimization space while maintaining performance gains.

We have showcased the efficacy of our DCO approach in the optimization tasks of running-time performance and code size reduction. The results have shown that DCO can outperform the standard optimization sequences and the state-of-the-art sub-sequence generation methods.

Our work contributes to the broader understanding of the intricacies in compiler optimization. By demonstrating the potential of sub-sequences, we provide valuable insights for improving iterative compilation processes and enhancing software performance. Our findings indicate that the intricate relationships between compiler passes is key to unlocking more optimization potentials.

For future work, we will investigate the adaptability of our method to diverse software contexts and hardware architectures. Ultimately, our work serves as a stepping stone towards more efficient and intelligent compiler

optimizations that can accommodate the ever-moving landscape of software and hardware technologies.

Appendix 1. LLVM optimization passes

Our experiment is carried on LLVM version 12.0.1, we collect all analysis passes and general purpose transform passes and show them in Table 7 and 8.

Table 7 Analysis passes

Analysis passes		
-aa	-aa-eval	-basic-aa
-block-freq	-branch-prob	-cfl-anders-aa
-cfl-steens-aa	-cg-profile	-cost-model
-da	-demanded-bits	-divergence
-domfrontier	-domtree	-external-aa
-globals-aa	-instcount	-lazy-block-freq
-lazy-branch-prob	-lazy-value-info	-loop-accesses
-loops	-machine-branch-prob	-memdep
-module-summary-analysis	-module-summary-info	-pa-eval
-phi-values	-polyhedral-info	-postdomtree
-profile-summary-info	-prune-eh	-regions
-scalar-evolution	-scev-aa	-scoped-noalias-aa
-stack-safety	-stack-safety-local	-targetlibinfo
-tbaa	-tti	

Table 8 Transform passes

Transform passes		
-adce	-add-discriminators	-aggressive-instcombine
-alignment-from-assumptions	-alloca-hoisting	-always-inline
-argpromotion	-assume-builder	-assume-simplify
-atomic-expand	-attributor	-attributor-cgsc
-barrier	-basiccg	-bdce
-break-crit-edges	-called-value-propagation	-callsite-splitting
-chr	-consthoist	-constmerge
-constraint-elimination	-correlated-propagation	-cross-dso-cfi
-dce	-deadargelim	-delinearize
-div-rem-pairs	-dse	-early-cse
-early-cse-memssa	-ee-instrument	-elim-avail-extern
-expand-reductions	-expandmemcmp	-fix-irreducible
-flattencfg	-float2int	-forceattrs
-function-attrs	-global-merge	-globaldce
-globalopt	-globalsplit	-guard-widening
-gvn	-gvn-hoist	-gvn-sink
-hardware-loops	-hotcoldsplit	-indirectbr-expand
-indvars	-infer-address-spaces	-inferattrs
-inject-tli-mappings	-inline	-insert-gcov-profiling
-instcombine	-instrprof	-instsimplify
-interleaved-access	-intervals	-ipscpp
-ir-similarity-identifier	-iroutliner	-iv-users
-jump-threading	-lcssa	-libcalls-shrinkwrap
-licm	-loop-data-prefetch	-loop-deletion
-loop-distribute	-loop-extract	-loop-extract-single
-loop-flatten	-loop-fusion	-loop-guard-widening
-loop-idiom	-loop-instsimplify	-loop-interchange
-loop-load-elim	-loop-predication	-loop-reduce
-loop-reroll	-loop-rotate	-loop-simplifycfg
-loop-sink	-loop-unroll	-loop-unroll-and-jam
-loop-unswitch	-loop-vectorize	-loop-versioning
-loop-versioning-licm	-lower-constant-intrinsics	-lower-expect
-lower-guard-intrinsic	-lower-widenable-condition	-lower-matrix-intrinsics
-lower-matrix-intrinsics-minimal	-loweratomic	-lowerinvoke
-lowerswitch	-lowertypetests	-make-guards-explicit
-mem2reg	-memcpyopt	-memoryssa
-mergefunc	-mergeicmps	-mergereturn
-mldst-motion	-name-anon-globals	-nary-reassociate
-newgvn	-partial-inliner	-partially-inline-libcalls
-post-inline-ee-instrument	-pre-isel-intrinsic-lowering	-reassociate
-redundant-dbg-inst-elim	-reg2mem	-rewrite-symbols
-rpo-function-attrs	-safe-stack	-sancov
-scalarize-masked-mem-intrin	-scalarizer	-sccp
-separate-const-offset-from-gep	-simple-loop-unswitch	-simplifycfg
-sink	-slsr	-speculative-execution
-sroa	-stack-protector	-strip
-strip-dead-prototypes	-structurizecfg	-tailcallem
-type-promotion	-unify-loop-exits	-unreachableblockelim
-unique-internal-linkage-names	-vector-combine	-wholeprogramdevirt

Appendix B. Sub-sequences for compiler optimization

Here we provide current sub-sequence-based methods designed for compiler optimization. CORL and POSET-RL's sub-sequences are extracted from their paper. As

MiCOMP uses an early version of LLVM compiler, we reconstruct the sub-sequences follow all steps in its paper based on LLVM version 12.0.1.

These sub-sequences are shown in Table 9, Table 10 and Table 11 (see Table 12).

Table 9 MiCOMP's sub-sequences

MiCOMP Sub-Sequences ¹	
1	-tti -tbaa -scoped-noalias-aa -assumption-cache-tracker -verify -ee-instrument -sroa -early-cse -lower-expect -profile-summary-info -annotation2metadata -forceattrs -inferattrs -callsite-splitting -ipsccp -called-value-propagation -globalopt -mem2reg -deadargelim -instcombine -basiccg -globals-aa -prune-eh -inline -openmpopt -function-attrs -argpromotion -memoryssa -early-cse-memssa -speculative-execution -lazy-value-info -jump-threading -correlated-propagation -aggressive-instcombine -libcalls-shrinkwrap -postdomtree -branch-prob -block-freq -pgo-memop-opt -tailcallelim -reassociate -lcssa -scalar-evolution -loop-rotate -licm -loop-unswitch -loop-idiom -indvars -loop-deletion -loop-unroll -mldst-motion -phi-values -memdep -gvn -memcpyopt -sccp -demanded-bits -bdce -adce -dse -barrier -elim-avail-extern -rpo-function-attrs -globaldce -float2int -lower-constant-intrinsics -loop-accesses -loop-distribute -inject-tli-mappings -loop-vectorize -loop-load-elim -slp-vectorizer -vector-combine -transform-warning -alignment-from-assumptions -strip-dead-prototypes -constmerge -cg-profile -loop-sink -instsimplify -div-rem-pairs -annotation-remarks
2	-targetlibinfo -simplifycfg
3	-domtree -aa
4	-basic-aa
5	-loops
6	-lazy-branch-prob
7	-lazy-block-freq
8	-opt-remark-emitter
9	-loop-simplify
10	-lcssa-verification

¹ This is our reconstruction of MiCOMP sub-sequence based on LLVM version 12.0.1

Table 10 CORL's sub-sequences

CORL Sub-Sequences	
1	-tti -verify -tbaa -scoped-noalias-aa -simplifycfg -sroa -early-cse -lower-expect
2	-targetlibinfo -tti -forceattrs -tbaa -scoped-noalias-aa -inferattrs -ipsccp -globalopt -mem2reg -deadargelim -instcombine -simplifycfg
3	-globals-aa -prune-eh -inline -function-attrs -argpromotion -sroa -early-cse -jump-threading -correlated-propagation -simplifycfg
4	-instcombine -tailcallelim -simplifycfg
5	-reassociate -loop-rotate -licm -loop-unswitch -simplifycfg
6	-instcombine -indvars -loop-idiom -loop-deletion -loop-unroll -mldst-motion -gvn -memcpyopt -sccp -bdce -instcombine -jump-threading -correlated-propagation -dse -licm -adce -simplifycfg
7	-instcombine -barrier -rpo-function-attrs -elim-avail-extern -globals-aa -float2int -loop-rotate -loop-vectorize -instcombine -slp-vectorizer -simplifycfg
8	-instcombine -loop-unroll -instcombine -licm -alignment-from-assumptions -strip-dead-prototypes -globaldce -constmerge

Table 11 POSET-RL's sub-sequences

POSET-RL's Sub-Sequences	
1	-ee-instrument -simplifycfg -sroa -early-cse -lower-expect -forceattrs -inferattrs -mem2reg
2	-ipsccp -called-value-propagation -attributor -globalopt
3	-deadargelim -instcombine -simplifycfg
4	-prune-eh -inline -function-attrs -barrier
5	-sroa -early-cse-memssa -speculative-execution -jump-threading -correlated-propagation
6	-simplifycfg -instcombine -tailcallelim -simplifycfg -reassociate
7	-loop-simplify -lcssa -loop-rotate -licm -loop-unswitch -simplifycfg -instcombine
8	-loop-simplify -lcssa -indvars -loop-idiom -loop-deletion -loop-unroll
9	-mldst-motion -gvn -memcpyopt -sccp -bdce -instcombine -jump-threading -correlated-propagation -dse
10	-loop-simplify -lcssa -licm -adce -simplifycfg -instcombine
11	-barrier -elim-avail-extern -rpo-function-attrs -globalopt -globaldce -float2int -lower-constant-intrinsics
12	-loop-simplify -lcssa -loop-rotate -loop-distribute -loop-vectorize
13	-loop-simplify -loop-load-elim -instcombine -simplifycfg -instcombine
14	-loop-simplify -lcssa -loop-unroll -instcombine -loop-simplify -lcssa -licm -alignment-from-assumptions
15	-strip-dead-prototypes -globaldce -constmerge -loop-simplify -lcssa -loop-sink -instsimplify -div-rem-pairs -simplifycfg

Table 12 DCO's sub-sequences

DCO's Sub-Sequences	
1	-instrorderfile -loop-extract -loop-extract-single -called-value-propagation -lower-guard-intrinsic -lower-matrix-intrinsics -lower-expect -lowerswitch -interleaved-load-combine -inject-tli-mappings
2	-lcssa -loop-flatten -loop-simplify -memoryssa -instcombine -newgvn -gvn -licm -sroa -mem2reg
3	-hotcoldsplit -extract-blocks -deadargelim -globalsplit -div-rem-pairs -chr -delinearize -inline -instrprof -expand-reductions -constraint-elimination -cross-dso-cfi -dce -mergeicmps -function-attrs -expandmemcmp -forceattrs -interleaved-access -make-guards-explicit -callsite-splitting -lower-constant-intrinsics -global-merge -guard-widening -barrier -ee-instrument
4	-loop-reroll -loop-sink -add-discriminators -argpromotion -loop-versioning -loop-simplifycfg -globalopt -loop-unswitch -assume-simplify -loop-load-elim -tailcallelim -loop-unroll-and-jam -aggressive-instcombine -loop-rotate -jump-threading
5	-atomic-expand -always-inline -sccp -loop-instsimplify -strip-dead-prototypes -ipsccp -speculative-execution -loop-unroll -rpo-function-attrs -loop-predication -scalarize-masked-mem-intrin -mergefunc -rewrite-symbols -attributor-cgsc -loop-guard-widening -mldst-motion -vector-combine -alignment-from-assumptions -loop-vectorize -partially-inline-libcalls -loop-reduce -alloca-hoisting -flattencfg -whole-programdevirt -assume-builder
6	-indirectbr-expand -loop-data-prefetch -reg2mem -constmerge -loop-distribute -float2int -instsimplify -correlated-propagation -consthoist -lower-widenable-condition -irce -libcalls-shrinkwrap -safe-stack -lower-matrix-intrinsics-minimal -globaldce -fix-irreducible -lower-atomic -loop-deletion -load-store-vectorizer -lint -attributor -loop-versioning-licm -lowerinvoke -indvars -bdce
7	-memcpyopt -scalarizer -sancov -instcount -reassociate -rewrite-statepoints-for-gc -stack-protector -partial-inliner -loop-interchange -loop-fusion -redundant-dbg-inst-elim -loop-idiom -slsr -type-promotion -mergereturn -strip -adce -nary-reassociate -post-inline-ee-instrument -lowertypetests -unique-internal-linkage-names -strip-gc-relocates -name-anon-globals -elim-avail-extern -tti
8	-dse -break-crit-edges -aa -assumption-cache-tracker -basic-aa -domtree -lazy-block-freq -lazy-branch-prob -loops -scalar-evolution
9	-aa-eval -cg-profile -cost-model -divergence -external-aa -globals-aa -intervals -liveintervals -liveregmatrix -memprof -memprof-module -module-summary-analysis -pa-eval -prune-eh -reaching-deps-analysis -regions -sample-profile -scev-aa -scoped-noalias-aa -stack-safety -stack-safety-local -tbaa
10	-basiccg -block-freq -branch-prob -da -demanded-bits -domfrontier -loop-accesses -memdep -phi-values -postdomtree

Acknowledgements We thank anonymous reviewers for their constructive comments. This work was partially supported by the National Key Research and Development Program of China under Grant No.2023YFB3001503, and the National Natural Science Foundation of China under Grant No. 61972408.

Data Availability The datasets used and analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no Conflict of interest.

SPRINGER BRIEFS IN APPLIED SCIENCES AND
TECHNOLOGY • POLIMI SPRINGER BRIEFS

Amir H. Ashouri

Gianluca Palermo

John Cavazos

Cristina Silvano

Automatic Tuning of Compilers Using Machine Learning



POLITECNICO
MILANO 1863



Springer

Contents

1	Background	1
1.1	Introduction	1
1.2	Compiler Optimizations	4
1.2.1	A Note on Terminology and Metrics	4
1.2.2	Compiler Optimization Benefits and Challenges	5
1.2.3	Compiler Optimization Problems	5
1.3	Machine Learning Models	8
1.3.1	Supervised Learning	8
1.3.2	Unsupervised Learning	13
1.3.3	Other Machine Learning Methods	14
1.4	Conclusions	16
	References	16
2	Design Space Exploration of Compiler Passes: A Co-Exploration	
	Approach for the Embedded Domain	23
2.1	VLIW	23
2.2	Background	25
2.3	Methodology for Compiler Analysis of Customized VLIW Architectures	26
2.3.1	Custom VLIW Architecture Selection	28
2.3.2	Compiler Transformation Statistical Effect Analysis	30
2.4	Conclusions and Future Work	38
	References	38
3	Selecting the Best Compiler Optimizations: A Bayesian	
	Network Approach	41
3.1	Introduction	41
3.2	Previous Work	43
3.3	Proposed Methodology	44

3.3.1	Applying Program Characterization	46
3.3.2	Dimension-Reduction Techniques	47
3.3.3	Bayesian Networks	49
3.4	Experimental Evaluation	52
3.4.1	Benchmark Suites	52
3.4.2	Compiler Transformations	54
3.4.3	Bayesian Network Results	55
3.4.4	Comparison Results	58
3.4.5	A Practical Usage Assessment	63
3.4.6	Comparison with State-of-the-Art Techniques	65
3.5	Conclusions	68
	References	68
4	The Phase-Ordering Problem: An Intermediate Speedup Prediction Approach	71
4.1	Introduction	71
4.2	Related Work	73
4.3	The Proposed Methodology	74
4.3.1	Compiler Phase-Ordering Problem	76
4.3.2	Application Characterization	76
4.3.3	Intermediate Speedup Prediction	77
4.4	Experimental Evaluation	79
4.5	Conclusions	82
	References	82
5	The Phase-Ordering Problem: A Complete Sequence Prediction Approach	85
5.1	Intermediate Versus Full-Sequence Speedup Prediction	85
5.2	Related Work	87
5.3	The Proposed Methodology	88
5.3.1	Application Characterization	90
5.3.2	Constructing Compiler Sub-sequences	90
5.3.3	The Proposed Mapper	93
5.3.4	Predictive Modeling	94
5.3.5	Recommender Systems Heuristic	97
5.4	Experimental Results	98
5.4.1	Analysis of Longer Sequence Length	101
5.4.2	MiCOMP Prediction Accuracy	102
5.4.3	MiCOMP Versus the Ranking Approach	104
5.5	Comparative Results	105
5.5.1	Comparison with Standard Optimization Levels	105
5.5.2	Comparisons with State-of-the-Art Models	107
5.5.3	Comparison with Random Iterative Compilation	109

5.6 Conclusion	111
References	112
6 Concluding Remarks	115
6.1 Main Contributions	115
6.2 Open Issues and Future Directions	116
Index	117

Chapter 1

Background

Abstract Since the mid-1990s, researchers have been trying to use machine-learning based approaches to solve a number of different compiler optimization problems. The techniques primarily enhance the quality of the obtained results and, more importantly, make it feasible to tackle two main compiler optimization problems: optimization selection (choosing which optimizations to use) and phase-ordering (choosing the order of applying optimizations). The compiler optimization space continues to grow due to the advancement of applications, increasing compiler optimizations, and new target architectures. Generic optimization passes in compilers cannot fully leverage newly introduced optimizations and, therefore, cannot keep up with the pace of increasing options. This chapter summarizes and classifies the recent advances in using machine learning for the compiler optimization field, particularly on the two major problems of (i) selecting the best optimizations and (ii) the phase-ordering of optimizations. The chapter highlights the approaches taken, the obtained results, the holistic comparisons among different approaches and finally, the visionary path towards the near future.

1.1 Introduction

Recent developments in silicon production and fabrication led to the creation of much faster computational units such as CPUs, GPUs, FPGAs, and similar devices with different instruction set architectures (ISAs). Software (SW) programming paradigms including OpenMP, MPI, OpenCL, and OpenACC allow software developers to exploit Hardware (HW) parallelism to port legacy serial codes on these emerging platforms to attain application speedups. Compilers struggle to keep up with the increasing development pace of ever-expanding hardware and software programming paradigms. Additionally, the growing complexity of modern compilers and the concern over security are among the most important problems that compilers should answer. Moore's law [1] states that transistor density should double every two years; however, the rate of compilers, which are faced with many open-research problems, have not been able to improve more than a few percentage points each year [2].

Usually, software applications are developed in a high-level programming language (e.g. C, C++) and then passed through the compiler to emit an executable binary. Compilers have been used for the past 50 years [2, 3] for generating machine-dependent executable binary from high-level programming languages. Compiler developers typically design optimization passes in order to transform each code segment of a program to produce an optimized version of an application. The optimizations can be applied at different stages of the compilation process since compilers have three main layers: (i) *front-end* (ii) *intermediate-representation* (IR) and (iii) *backend*. At the same time, optimizing source code by hand is a tedious task. Compiler optimizations provide an automatic method to transform code. To this end, optimizing the intermediate phase plays an important role on the performance metrics. Enabling compiler optimization parameters (e.g. loop unrolling, register allocation, etc.) might substantially benefit several performance metrics. Depending on the objectives, these metrics could be execution time, code size, or power consumption. A holistic exploration approach to trade-off these metrics represents a challenging problem [4].

Autotuning addresses automatic code-generation and optimization by using different scenarios and architectures. It constructs techniques for automatic optimization of different parameters to maximize or minimize the satisfaction of an objective function. Historically, optimizations were mostly in the backend where *scheduling*, *resource-allocation*, and *code-generation* are done [5, 6]. The constraints form a linear problem (ILP), thus one should solve an ILP to find an optimized design. Recently, researchers have shown increased effort in introducing frontend and IR optimizations. The reason is twofold: (i) the complexity of a backend compiler requires strict and exclusive knowledge of compiler designers, and (ii) frontend optimizations have lower overheads compared with the optimizations performed in the back-end. Nonetheless, each has its benefits and drawbacks and is subject to further analyses.

The major challenge in choosing the right set of compiler optimizations is the fact that the optimizations are programming language, application, and architecture dependent. Additionally, the word optimization is a misnomer; there is no guarantee that the transformed code will perform better than the original version. Aggressive optimizations can degrade the performance of the code to which they are applied [7]. Understanding the behavior of the optimizations, the perceived effects on the source-code, and the interaction of the optimizations with each other are complex modeling problems. This understanding is particularly difficult because compiler developers must consider hundreds of optimizations that can be applied during the different compilation phases. This optimization ordering dilemma creates the phase-ordering problem.

There were a number of long-standing compiler optimization problems have not yet been adequately addressed. These problems comprise of knowing *what* optimizations (by which configuration (i.e., the tile size in loop tiling)), and in *which order* the optimizations should be used to obtain the best improvement. The former yields the so-called problem of selecting the best compiler optimizations and the latter is the phase-ordering problem of compiler optimizations. The *phase-ordering* problem

has been an open problem for many decades [8–11]. The inability of researchers to solve the phase-ordering problem has led to advances in the less complex problem of selecting the right set of optimizations, but even this has yet to be fully resolved [12–14].

In order to understand the difficulty of the problems, one should consider that the process of selecting the right optimizations for each code segment is typically done manually and the sequence of optimizations is constructed with little insight into the interaction between the preceding compiler optimizations in the sequence. The task of constructing heuristics to select the right sequence of compiler optimizations is infeasible given the ever-growing number of compiler optimizations being integrated into compiler frameworks. As an example, GCC has more than 200 compiler passes, referred to as compiler *options*,¹ and LLVM-clang and LLVM-opt both have more than 100 transformation passes² each. Additionally, these optimizations are applied at very different phases of the compilation, including analysis passes and loop-nest passes. Most optimization flags are turned off by default and compiler developers rely on software developers to know, which optimizations will be beneficial for their code. Compiler developers provide standard optimization levels, i.e., -O1, -O2, -Os, etc. to introduce a fixed-sequence of compiler optimizations that, on average, bring good performance on a set of benchmarks compiler developers tested the optimization levels with. However, using predefined optimizations usually is not good enough to bring the best achievable application-specific performance. One of the key approaches used recently in literature to find the best optimizations to apply to an application is inducing prediction models using different classes of machine learning [15]. Approaches which leverage machine learning to find the best optimizations to apply will be the center focus of this book.

Contribution. In this chapter, we provide an overview of techniques and approaches appeared so far to tackle the aforementioned problems. We elaborate many recent papers proposed for compiler autotuning when Machine Learning (ML) was involved. To the best of our knowledge, the first application of machine learning for the compiler autotuning problem was done by [16–18]. However, there were other original works which tackled the problem of compiler autotuning without the use of machine learning technique [10, 19–22], and we believe them to be the driving force of using machine learning to the existing problems. Thus we decided to consider the past 25 years as it covers the whole time span of the literature on the very field. Additionally, this chapter can be a connecting point for the already available surveys [23, 24] on the compiler optimization field.

We first discuss the motivation and challenges involved in the compiler optimization research in Sect. 1.2, followed by an analysis of the optimization space for the two major optimization problems. Then, we discuss the machine learning models used in Sect. 1.3 and provide a full classification of different techniques used in the recent research. Finally, we conclude the chapter with discussion and future trends.

¹<https://gcc.gnu.org/onlinedocs/gcc/Option-Summary.html>.

²<http://llvm.org/docs/Passes.html>.

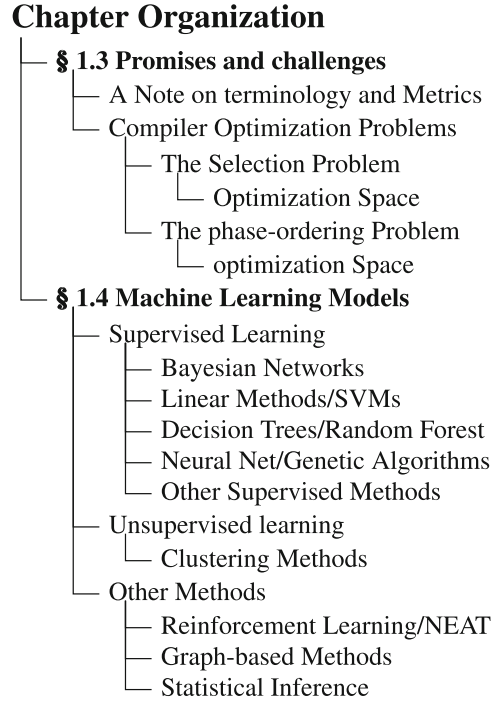


Fig. 1.1 Organization of the chapter in different sections

We hope that this chapter will be useful for a wide range of readers, including computer architects, compiler developers, researchers and technical marketing professionals.

Organization of the chapter. In these sections, we organize the state-of-the-art works in different categories to underscore their similarities and differences. Note that the works presented in these sections are deeply intertwined. While we study a certain work under a single group, several of these works belong to multiple groups. We organize the chapter in a way that all research papers corresponding to a specific type of classification are cited. However, we selectively focused on the most notable works under each section and we provide more elaboration on their contribution. Fig. 1.1 represents our organization of the chapter.

1.2 Compiler Optimizations

1.2.1 A Note on Terminology and Metrics

Since publications mentioned in this chapter originated from varying authors, terminologies were locally defined and might not be strictly defined. We clarify terms used in this chapter here and relate them to the publications discussed. Compiler optimization field has been referred to as compiler autotuning, compilation, code optimization, collective optimization, and program optimization. To maintain

clarity, we do not use all these terms but rather use optimizing compilers, or compiler autotuning. Moreover, under each classified subsection, we will point out the other nomenclatures that have been used widely and our reference subsequently.

1.2.2 Compiler Optimization Benefits and Challenges

The majority of potential speedup no longer arrives at the increase of processor core clock frequencies. Automatic methods of exploiting parallelism and reducing dependencies are needed. Compiler optimizations [20] allow a user to affect the generated code without changing the original high-level source code. When these optimizations are applied may result in a program running better on a target architecture. Since a user is not able to manual tune a large code, automatic methods need to be introduced. Additionally, manual tuning is not portable – transformations applied to code running on one architecture is not guaranteed to yield the same performance increase on another architecture. We discuss the challenges and benefits at two levels:

Research Level

One clear benefit of optimization passes is their portability – if necessary, they can be easily adapted to newer architectures. However, there are some optimizations that have been researched more than others. Specifically, we still have not reached the once “holy grail” of auto-parallelizing compilers, but we have made significant progress. Polyhedral loop analysis and transformations paved the way for safe transformations leading to auto-parallelizable code segments. The polyhedral model also aided with the generation of architecture-dependent, cache-friendly access patterns.

User’s Level

Compiler writers expose general-purpose transformations to end users. Ultimately, a subset of these transformations leads to better architecture fitting given source code. Over time we have seen an overall improvement of compilers and related tools. New high-level languages and languages extensions make additional optimizations possible. Directive-based programming languages, such as OpenMP [25] and OpenACC [26], automatically transform users’ code to exploit parallelism. One of the most advantageous contributions has been the introduction of easily expandable compiler infrastructures such as LLVM. An advanced compiler infrastructure makes it possible to introduce new optimizations with minimal effort.

1.2.3 Compiler Optimization Problems

The problem of interdependency among phases of compiler optimizations is not unique to compiler optimization field. Phase inter-dependencies have been noted in

Table 1.1 A classification based on the type of the problem

Classification	References
The selection problem	[16, 29–80]
The phase-ordering problem	[7–11, 22, 33, 81–88]

traditional optimizing compilers between constant folding and flow analysis, and between register allocation and code generation [10, 27].

In optimization theory [28], a feasible set, search space, or solution space is the set of all possible points (sets of values of the choice variables) of an optimization problem that satisfy the problem’s constraints, potentially including inequalities, equalities, and integer constraints. A feasible set is the initial set of candidate solutions to the problem before the set of candidates has been narrowed down. Compiler optimization problem polarizes over two major sub-problems based on (i) whether we take into account the enabling/disabling the optimizations only (optimization selection problem) or (ii) changing the ordering of those optimizations (phase-ordering problem). Here we briefly discuss the different optimization space of the two.

Table 1.1 classifies the existing literature based on the type of the problem. As we mentioned earlier, the inability of researchers to completely solve the phase-ordering problem has led to some advances in the problem of selecting the right set of compiler optimizations. Thus, we see more research focusing on the selection problem recently.

1.2.3.1 The Problem of Selecting the Best Compiler Optimizations

Several compiler optimizations form an optimization sequence. When we disregard the ordering of these optimizations and focus on whether or not to apply the optimization, we define the scope of selecting the best compiler optimizations. Previous researchers have shown that the interdependencies and interaction between enabling/disabling optimizations in a sequence can dramatically alter the performance of a running code even by ignoring the order of phases [12, 29].

Optimizations Space

Let us define a Boolean vector o whose elements o_i are the different compiler optimizations. Each optimization o_i can be either enabled $o_i = 1$ or disabled $o_i = 0$. A compiler optimization sequence to be *selected* is represented by the vector o belongs to the n dimensional Boolean space of:

$$|\Omega_{Selection}| = \{0, 1\}^n \quad (1.1)$$

For the application a_i being optimized and n represents the number of compiler optimizations under study. Therefore, the mentioned research problem consists of an

exponential space as its upper-bound. Having $n = 10$, drive us to a total space (2^n) up to $|\mathcal{O}_{selection}| = 1024$ options to select among per interested target application a_i to be optimized and this number itself would be multiplied by different applications $A = a_0 \dots a_N$ under study.

Extended version of the current definition in Eq. 1.1 is the case where we have more than a binary choice (enabling/disabling). Certain compiler optimizations offer multiple levels of optimization to choose among, i.e. `-loop-unrolling`, `loop-tiling`, etc. in many compiler frameworks with different values such as 4, 8, 16, m etc. Consequently, we turn the previous equation as:

$$|\Omega_{Selection_Extended}| = \{0, 1, \dots, m\}^n \quad (1.2)$$

where m defines the number of different optimization levels a compiler optimization has.

1.2.3.2 The Phase-Ordering Problem

Compiler designers must consider the order in which optimization phases are performed; a pair of phases may be interdependent in the sense that each phase could benefit from information produced by the other. When both the selection and the ordering are of importance, the phase-ordering problem is formed. It is one of the longstanding problems of compilation field and has its peer problems in numerous other sub-fields of compiler design such as register allocation, code-generating and compaction [10, 27].

Optimizations Space

A phase-ordering optimization sequence represented by the vector $\mathbf{o}$ belongs to the n dimensional factorial space of:

$$|\Omega_{Phases}| = n! \quad (1.3)$$

where n represents the number of compiler optimizations under study. However, the mentioned bound is for a simplified phase-ordering problem having a fixed length optimization sequence length and no repetitive application of optimizations. Enabling optimizations to be repeatedly applied and a variable length sequence of optimizations will expand the problem space to:

$$|\Omega_{Phases_Repetition_variableLength}| = \sum_{i=0}^m n^i \quad (1.4)$$

where n is the number of optimizations under study and m is the maximum desired length for the optimization sequence. Even for reasonable values of n and m , the

entire search space is enormous. For example, assuming n and m are both equal to 10, this leads to an optimization search space of more than 11 billion different optimization sequences to select from for each piece of code being optimized [82].³

1.3 Machine Learning Models

Machine learning explores the study and construction of algorithms that can learn from and make predictions on data [89]. Many types and sub-fields of machine learning exist and here we classify them based on the broad categories: (i) Supervised learning (ii) Unsupervised learning, and (iii) Other methods (including reinforcement learning, graph-based technique, and statical methods). The classification of all machine learning models used is depicted in Table 1.2. In each subsection we provide an overview of the method and we mention the major tools and works involved. The general formulation of the optimization problem is to construct a function that takes as input the features of the unoptimized program being compiled. In other words, this model takes as an input a tuple (F, T) where F is the feature vector of the collected instrumentation of the program being optimized; and T is one of the several possible compiler sequences predicted to perform well on this program. Its output is a prediction of the speedup T should achieve when applied to the original code.

1.3.1 Supervised Learning

Supervised learning is the machine learning task of inferring a function from labeled training data [89, 112]. The learner receives a set of labeled examples as training data and makes predictions for all unseen points. This is the most common scenario associated with classification, regression and ranking problems.

1.3.1.1 Bayesian Networks

Bayesian Networks (BN) [113, 114] are powerful classifiers to represent the probability distribution of different variables that characterize a certain phenomenon such as the optimality of compiler optimization sequences. A Bayesian Network is a direct acyclic graph whose nodes represent variables and whose edges represent the dependencies between these variables. The probability distributions of the two optimizations depend on the program features represented by α . Additionally, the

³The phase-ordering problem does not have a deterministic upper-bound when we have an unbounded optimization length.

Table 1.2 A classification based on machine learning models

Classification		References
Supervised learning	Bayesian networks	[12, 90]
	Linear models/SVMs	[12, 68, 72, 82, 86, 91]
	Decision trees/Random forests	[45, 46, 57, 63, 77, 92–94, 94–97]
	Neural networks/Genetic algorithm	[8, 9, 16, 29, 30, 40, 52, 54, 55, 62, 66, 71, 73, 78, 83–85, 87, 95, 96, 98–107]
	Others	[12, 35, 48, 53, 59, 71, 75–78, 86, 91, 104, 106–109]
Unsupervised	Clustering methods	[75, 84, 87, 110, 111]
Others methods	Reinforcement learning/NEAT	[9, 95, 107]
	Graph-based methods	[62, 84, 111]
	Statistical methods	[9, 12, 29, 31, 32, 34, 35, 40–48, 50, 51, 56, 59, 61, 64, 68, 70, 71, 74, 77, 78, 90, 93, 95, 96, 98, 101, 107, 108, 110]

probability distribution of o_2 depends on whether the optimization o_1 is applied. Nodes representing observed variables whose value can be input as evidence to the network.

Ashouri et al. [12, 90] proposed a Bayesian network approach to address the problem of selecting the best compiler optimizations suitable for an embedded processor to gain speedup versus the fixed standard optimizations available at different levels of GCC compiler. They used static, dynamic and hybrid features to construct an application feature vector and evaluated their approach with Cbench [115] and Polybench [116, 117] using BN to focus on iterative compilation and showed using the inferred compiler passes by the BN they could outperform GCC's -O2 and -O3 by around 50%.

1.3.1.2 Linear Models and SVMs

Linear models are one of the most popular supervised learning methods to be widely used by researchers in tackling many machine learning applications. Linear regression, nearest neighbor, and linear threshold algorithms are very stable [112]. Algorithms whose output classifier does not undergoes major changes in response to small changes in the training data. Moreover, SVMs are a supervised machine learning technique, used for both classification and regression, and it can apply linear techniques to non-linear problems. First, SVM transforms data into a linear space by using kernel functions and uses a linear classifier to separate data with a hyperplane. SVM not only finds a hyperplane to separate data, but also finds the best hyperplane, namely the maximum margin hyperplane, and shows the largest separation from the set of hyperplanes [91].

1.3.1.3 Decision Trees and Random Forests

A binary decision tree is a tree representation of a partition of the feature space. Decision trees can be defined using more complex node questions resulting in partitions based on more complex decision surfaces [89]. Random forests or random decision forests are an ensemble learning methods for classification, regression and other tasks, that operate by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees. Random decision forests correct decision trees' habit of overfitting to their training set [112].

Fraser et al. [93] proposed to use machine learning to perform code compression. It uses IR structure of the codes to automatically infer a decision tree that separate IR code into a stream that compress better. They evaluated their approach with GCC and used Opcodes which can also help predict elements of the operand stream.

Monsifrot [97] addressed the automatic generation of optimization heuristics for a target processor by machine learning. They evaluated the potential of this method on an always legal and simple transformation: loop unrolling. They used decision trees to learn the behavior of the loop unrolling optimization on the code being studied and drive to decide whether to unroll on UltraSPARC and IA-64 machines.

Fursin and Cohen [94] built an iterative and adaptive compiler framework on the SPEC applications using a modified GCC. Building a transparent framework which reuses all the compiler program analysis routines (from a program transformation database) to avoid duplications in external optimization tools.

Lokuciejewski et al. [96, 118] proposed an adaptive worst-case execution time (WCET)-aware compiler framework using Random forests for an automatic search of compiler optimization sequences that yield highly optimized code. Besides the objective functions ACET and code size, they consider the WCET which is a crucial parameter for real-time systems. To find suitable trade-offs between these objectives, stochastic evolutionary multi-objective algorithms identifying Pareto optimal solutions for the objectives (WCET, ACET) and (WCET, code size) are exploited.

Luo et al. [57] proposed a technique to select a minimal set of representative optimization variants (function versions) for such frameworks while avoiding performance loss across available datasets and code-size explosion. They developed a novel mapping mechanism using a decision tree namely, rule induction based machine learning techniques to rapidly select best code versions at run-time based on dataset features and minimize selection overhead.

1.3.1.4 Neural Networks and Genetic Algorithms

Neural networks (NN) are frequently employed to classify patterns based on learning from examples. Different neural network paradigms employ different learning rules, but almost all in some way determine pattern statistics from a set of training samples and then classify new patterns on the basis of these statistics [119]. A NN is a network inspired by biological neural networks which are used to estimate or

approximate functions that can depend on a large number of inputs that are generally unknown. Artificial neural networks are typically specified using three components: (i) architecture, (ii) activity rule, and (iii) learning rule. Genetic Algorithm (GA) is a meta-heuristic inspired by the process of natural selection and can be paired with any other machine learning technique or work independently. A notable GA heuristic is NSGA which is a popular method for many optimization problems [120].

Cooper et al. [16, 101] in one of the early related work of literature, addressed the code size of the generated binaries by using genetic algorithm to find optimization sequences that generate small object codes. The solutions generated by this algorithm are compared to solutions found using a fixed optimization sequence and solutions found by testing random optimization sequences. Based on the results found by the genetic algorithm, a new fixed sequence is developed to reduce code size.

Knijnenburg et al. [54] proposed an iterative compilation approach to tackle the selection size of the tiling and the unrolling factors in an architecture independent manner. They evaluated their approach using several iterative strategies based on genetic algorithms, random sampling, and simulated annealing and compared the results with static-techniques. The targeted compiler was native Fortran77 or g77 compiler with full optimization on. The benchmarks used were Matrix-Matrix Multiplication ($M \times M$), Matrix-Vector Multiplication ($M \times V$) and Forward Discrete Cosine Transform.

Stephenson et al. [71] introduced Meta Optimization, a methodology for automatically fine-tuning compiler heuristics. Meta Optimization uses machine-learning techniques and specifically genetic programming to automatically search the space of compiler heuristics. The authors targeted IMPACT compiler with Spec and Mediabench [121] applications to evaluate their approach.

Cavazos and O’Boyle [99] developed a genetic algorithm based approach to automatically tune a dynamic compiler’s internal inlining heuristic. The approach used a program’s performance to guide the search. Genetic algorithms have been used as candidates and the geometric mean of the performance of the SPECjvm98 benchmarks was used as their fitness function.

Agakov et al. [29] introduced machine-learning models to focus on the exploration of the compiler optimizations for the most promising region. Their methodology exploits a Markov chain oracle and an independent identically distributed (IID) probability distribution oracle. These two models learned offline bias using only a certain optimizations rather than using the uniform probability distribution they applied earlier for the RIC reference methodology. The authors reported significant speedup by coupling these machine-learning models with a nearest-neighbor-classifier. When predicting the probability distribution of the best compiler optimizations for a new application, the classifier first selects the training application having the smallest Euclidean distance in the feature vector space (derived by PCA). Then it learns the probability distribution of the best compiler optimizations for this neighboring application either by means of the Markov chain model or by using an IID model. The probability distribution learned is then used as the predicted optimal distribution for the new application. It has been reported that the Markov chain oracle outper-

forms the IID oracle, followed by the RIC methodology using a uniform probability distribution.

Kulkarni et al. [8] used a depth-first search algorithm to produce the next sequence to evaluate in an exhaustive exploration of the phase ordering problem. To evaluate their method, the authors used hill-climbing, simulated annealing, genetic-algorithm and a random search on an embedded architecture.

Leather et al. [104] used grammatical evolution based on the genetic algorithm to describe the feature space and used predictive modeling on GCC 4.3.1 to evaluate the approach on Pentium 6 with mediabench to determine the loop-unrolling factor.

Purini et al. [87] have defined a machine learning based approach to downsample the compiler optimization sequences in LLVM's `-O2` and then applied machine learning to learn a model. The authors introduced a clustering algorithm to clustering sequences based on Sequence Similarity matrix by calculating the Euclidean distance between the two sequence vectors. In the experimental evaluation, they have mentioned the most frequent optimization passes with their fitness function (execution speedup) as well.

1.3.1.5 Other Supervised Methods

For conciseness proposes, we decided to classify other supervised learning methods in this subsection. These include Lazy learning, ANOVA, K-nearest neighbor, Gaussian process learning [89], and others.

Moss et al. [18] showed how to cast the instruction scheduling problem as a learning task, obtaining the heuristic scheduling algorithm automatically. They focused on the narrower problem of scheduling straight-line code (also called basic blocks of instructions). They used static and IR features of the basic block with the SPEC benchmark to experimentally evaluate their approach by using Geometric mean as fitness function and fold-cross-validation.

Cavazos and Moss [35] used JIT Java compiler and SPECjvm98 benchmark and rule set induction learning model to decide whether to schedule. The induced function chooses for each block between list scheduling or not scheduling the block. Using the induced function the authors obtained over 90% of the improvement of scheduling every block but with less than 25% of the scheduling effort.

Tournavitis et al. [76] proposed a technique using profile-driven parallelism detection in which they could overcome the limitations of static analysis, and allowing the identification of more application parallelism. This approach only relied on the user for final approval. It integrated profile-driven parallelism detection and machine-learning based mapping in a single framework. Moreover, the authors replaced the traditional target-specific and inflexible mapping heuristics with a machine-learning based prediction mechanism, resulting in better mapping decisions while providing more scope for adaptation to different target architectures. Finally, they have experimentally evaluated their approach against NAS and SPEC-OMP.

1.3.2 *Unsupervised Learning*

Unsupervised learning is the machine learning task of inferring a function to describe hidden structure from unlabeled data. Since the examples given to the learner are unlabeled, there is no error or reward signal to evaluate a potential solution [89, 122]. Unsupervised learning is closely tightened with the problem of density estimation in statistics [123]. However, unsupervised learning also encompasses many other techniques that seek to summarize and explain key features of the data.

1.3.2.1 Clustering Methods

One of supervised learning's key subclasses is clustering. Clustering helps to down-sample the chunk of unrelated compiler optimization passes into meaningful clusters that correspond to each other, i.e., targets loop-nests or scalar values, or they should follow each other in the same sequence. Another benefit of clustering/downsampling is to reduce the compiler optimization space which as mentioned in Sect. 1.2.3 are in tens of thousands orders of magnitudes.

Thomson et al. [75] present a new approach to reduce the training time of a machine learning based compiler. They focused on the programs which best characterized the optimization space and proposed to use a clustering technique, namely GustafsonKessel algorithm, after applying the dimension reduction process. They evaluated the clustering approach on the EEMBCv2 benchmark suite and show that they can reduce the number of training runs by more than a factor of seven.

Ashouri et al. [32, 110] developed a hardware/software co-design toolchain to explore compiler design space jointly with microarchitectural properties of a VLIW processor. The authors have used clustering to derive to four (4) good hardware architectures followed by mitigating the selection of promising compiler optimization with statistical techniques such as kruskal-wallis test and pareto-optimal filtering. (This method involved with statistical methods as well. Refer to Sect. 1.3.3.3.)

Martins et al. [84, 111] tackled the problem of phase-ordering by a DSE approach that uses a clustering-based selection method for grouping functions with similarities and exploration of a reduced search space resulting from the combination of optimizations previously suggested for the functions in each group. Authors used DNA encoding where program elements (e.g., operators and loops in function granularity) are encoded in a sequence of symbols, and followed by calculating the distance matrix and a tree construction of the optimization set. Consequently, they applied the compiler optimization passes already included in the DSE to measure the reduction in the total exploration time of the search space such as Genetic algorithm.

1.3.3 *Other Machine Learning Methods*

In this section, we present the recent literature involved with using machine learning methods that could not be classified by supervised or unsupervised learning methods. Examples of these methods are reinforcement learning, graph-based methods and the statistical methods [89].

1.3.3.1 **Reinforcement Learning and NEAT**

Reinforcement learning (RL) is an area of machine learning inspired by behaviorist psychology, concerned with how software agents ought to take actions in an environment to maximize some notion of cumulative reward. The problem, due to its generality, is studied in many other disciplines, such as game theory, control theory, operations research, information theory, simulation-based optimization, multi-agent systems, swarm intelligence, statistics, and genetic algorithms [89]. The interesting difference in RL is that the training and testing phases are intermixed. To collect information the learner should actively interact with the environment.

We decided to group RL with a neuroevolution approach called NEAT as suggested by [124] since NEAT can be a strong method on the pole-balancing benchmark reinforcement learning tasks. NEAT alters both the weighting parameters and structures of networks, attempting to find a balance between the fitness of evolved solutions and their diversity. It is based on applying three key techniques: (i) tracking genes with historical markers to allow crossover among topologies, (ii) applying speciation (the evolution of species) to preserve innovations, and (iii) developing topologies incrementally from simple initial structures (complexifying).

Kulkarni et al. developed two approaches to tackle both problems of selection [95] and the phase-ordering [9] of compiler optimizations. The approach for selecting the good compiler passes is done using NEAT and static features to tune Java hotspot server compiler with SPEC Java benchmarks (using two benchmarks for training and two for testing). The authors used NEAT to train decision trees for the caller and the callee whether to inline. The approach for the phase-ordering problem, formulates it as a Markov process and uses a characterization of the current state of the code being optimized to creating a better solution to the phase ordering problem. Here, they uses NEAT to construct an artificial neural network that is capable of predicting beneficial optimization ordering for a piece of code that is being optimized. By using a stop-condition, the NEAT knows when to stop expanding the sequence and converging to the final sequence.

1.3.3.2 **Graph-Based Methods**

Recently, graph-based methods have emerged as a powerful means of exploiting many different machine learning applications on a wide range of applications from

semi-supervised learning [125] to clustering and classification [126]. We decided to place them in the section related to the other machine learning methods to be more precise on our classification.

Park et al. [62] introduced a novel way of both characterizing programs using a graph-based characterization, which uses the program's intermediate representation and an adapted learning algorithm to predict good optimization sequences. In order to construct the feature vectors, they used graph-kernels. The authors evaluated different characterization techniques, focusing on loop-intensive programs. They constructed prediction models that drive polyhedral optimizations, such as auto-parallelism and various loop transformations.

Nobre et al. [105] proposed an iterative compilation approach using graph-based features of the optimized code to mitigate the selecting and the ordering of the compiler optimization passes in LLVM. After iterative evaluations, the authors could find solutions with a speedup factor over the baseline and they provided a clustering method to organize their finding.

1.3.3.3 Statistical Methods

Terminology across fields is quite varied for statistical methods [127]. In statistics, where classification is often done with logistic regression or a similar procedure, the properties of observations are termed explanatory variables (or independent variables, regressors, etc.), and the categories to be predicted are known as outcomes, which are considered to be possible values of the dependent variable. In machine learning [15, 89], the observations are often known as instances, the explanatory variables are termed features (grouped into a feature vector), and the possible categories to be predicted are classes. Here we refer to the group of work in the literature which involved with the most frequent procedures or multivariate distribution. Some references considered Bayesian networks, decision trees or random forests as a form of statistical methods, but we decided to have an individual section for each to classify in a more fine-grained manner.

Pinker et al. [64] proposed an automatic iterative procedure to turn on or to turn off compiler options. This procedure is based on orthogonal arrays that are used for a statistical analysis of profile information to calculate the main effect of the options. This approach can be used on top of any compiler that allows a collection of options to be set by the user. They showed that the proposed approach outperforms -O3 of GCC on some SPEC benchmarks.

Haneda et al. [51] introduced a statistical technique to derive a methodology which trims down the search space considerably, thereby allowing a feasible and flexible solution for defining high performance optimization strategies. They show that the technique finds a single compiler setting for a collection of programs SPECint95 that performs better than the standard settings of GCC.

Namolaru et al. [59] proposed a general method for systematically generating numerical features from a program. The authors implemented the approach in a production compiler. This method does not place any restriction on how to logically

and algebraically aggregate semantical properties into numerical features, offering a virtually exhaustive coverage of statistically relevant information that can be derived from a program. They have used static features of MilePost GCC and MiBench to evaluate their approach.

Ashouri et al. [32, 110] introduced a statistical technique to cluster and choose the best compiler optimizations in a software/hardware co-design manner. The authors used multi-objective optimization and Pareto-filtering to derive their micro-architectural parameters followed by the ANOVA and Kruskal-wallis tests. Using a performance distribution graphs as their fitness function, they selected a number of compiler parameters to be used for a VLIW [128, 129] architecture.

1.4 Conclusions

Using compiler optimizations to exploit large-scale parallelism available on architectures and power-aware hardware is an essential task. In the coming decades, research on compilation techniques and code optimization will play a key role in alleviating various challenges within computer science and the high performance computing field. This includes auto-parallelization, security, exploiting multi/many-core processors, reliability, reproducibility, and energy efficiency.

By the advancement we have seen in GPUs, deep learning has emerged as a viable mean of addressing many classification problems. These complex learners allow automated systems to efficiently perform tasks with minimal programmer effort.

In this chapter, we have synthesized the research work on compiler autotuning using machine learning and we showed the broad spectrum of the use of machine learning techniques and their key research ideas and applications. We surveyed research works at different levels of abstraction when machine learning models were used. We discussed both major problems of compiler autotuning, namely the selection and the phase-ordering problem along with the benchmark suits proposed to evaluate them. In the next chapters, we tackle the aforementioned problems by means of a design space explorations and machine learning approaches.

References

1. Schaller RR (1997) Moore’s law: past, present and future. *IEEE Spectr* 34(6):52–59
2. Hall M, Padua D, Pingali K (2009) Compiler research: the next 50 years. *Commun ACM* 52:60–67
3. Aho AV, Sethi R, Ullman JD (1986) *Compilers, principles, techniques*. Addison-Wesley, Boston
4. Palermo G, Silvano C, Zaccaria V (2005) Multi-objective design space exploration of embedded systems. *J Embedded Comput* 1(3):305–316
5. Chaitin GJ, Auslander MA, Chandra AK, Cocke J, Hopkins ME, Markstein PW (1981) Register allocation via coloring. *Comput Lang* 6(1):47–57

Chapter 2

Design Space Exploration of Compiler Passes: A Co-Exploration Approach for the Embedded Domain

Abstract Very Long Instruction Word (VLIW) processors represent an attractive solution for embedded computing, offering significant computational power with reduced hardware complexity. However, they impose higher compiler complexity since the instructions are executed in parallel based on the static compiler schedule. Therefore, finding a promising set of compiler transformations and defining their effects have a significant impact on the overall system performance. In this chapter, we provide a methodology with an integrated framework to automatically (i) generate optimized application-specific VLIW architectural configurations and (ii) analyze compiler level transformations, enabling application-specific compiler tuning over customized VLIW system architectures. We based the analysis on a Design of Experiments (DoEs) procedure that statistically captures the higher order effects among different sets of activated compiler transformations. Applying the proposed methodology onto real-case embedded application scenarios, we show that (i) only a limited set of compiler transformations exposes high confidence level (over 95%) in affecting the performance and (ii) using them we could be able to achieve gains between 16–23% in comparison to the default optimization levels. In the next chapters, we go deeper in building machine learning models to tackle the problem.

2.1 VLIW

Embedded system design traditionally exploits the knowledge of the target domain, e.g., telecommunication, multimedia, home automation etc., to customize the HW/SW coefficients found onto the deployed computing devices. Although the functionalities of these devices differ, the computational structure and design are tightly connected with the platform they rely on. Platform-based designs have been proposed as a promising alternative for designing complex systems by redefining the problem of tuning specific design parameters of the platform template.

The scientific and commercial urge to use VLIW technology seems to be raised again after three decades of existence [1]; VLIW processor templates are being used particularly in embedded processors, designed to perform special-purpose functions, usually for real-time or hardware acceleration. Being able to use VLIW power-saving

cores in CPUs seems to be using day by day. However, the trade-offs between right parallel execution and the speedup managed by compiler instead of hardware are becoming a very complex task. VLIW can potentially achieve greater performance, offering high degree of Instruction Level Parallelism (ILP) with low silicon and power costs. On the one hand, architecture configurability of VLIW platforms offers significant advantages regarding portability, sizing and parameter tuning provided to the designer [1, 2]. On the other hand, it introduces a lot of complexity during optimization due to multi-objective nature of the solution space and the multi-parametric structure of the design space.

Although a significant amount of research has been conducted on exploring and optimizing VLIW architectural parameters [3] and introducing specific compiler optimization for VLIW processors [4, 5], there are limited references regarding the analysis of the impacts of conventional compiler transformations onto VLIW architectures and moreover how these transformations are correlating with the underlying architectural configuration. Nowadays, the existence of modular and reusable compiler tool-chains LLVM and ROSE [6] raises the opportunity for system designers to exploit sophisticated compiler passes and customize their compiler infrastructure accordingly. Given the large optimization space provided by the modern compiler infrastructures, the designer has to traverse to find the best trade-off points, thus a fine-grained and automatic characterization of the effects that each compiler transformation has onto the application's behavior, is considered of great importance. Empirical evaluation of the effects, by simply activating and deactivating compiler passes cannot be considered adequate, since a lot of inter-transformation interactions and second order effects are neglected. Due to the complexity of characterizing the solution space, there is a necessity to extend conventional exploration approaches by applying sophisticated analysis and data-mining for extracting knowledge from statistical results [7]. The problem becomes more demanding in the embedded computing domain, which requires different optimizations related to each platform configuration customized for a specific application domain. The main contribution of this chapter consists of proposing a compiler/architecture methodology that provides to the designer an integrated environment to automatically (i) generate optimized application specific architectural configurations of VLIW-based platforms, and (ii) a statistical analysis of the effects of compiler level transformations.

We target the design problem of compiler/architecture co-exploration in embedded computing. Thus, we focus on enabling application-specific compiler tuning over customized VLIW system architectures. First, a multi-objective exploration loop targeting application-specific micro-architectural customization is applied for extracting the best VLIW architecture candidates. We utilize the newly introduced *Roofline* processor architecture model [8] for characterizing the differing architectural solutions onto various resource constraints. The optimized VLIW architectural configurations are then propagated to the compiler analysis phase in which the statistical effects of the applied compiler transformations are characterized in a fine-grained manner. The developed exploration framework integrates the LLVM compiler infrastructure [9] as a source to source code transformation tool together with the VEX compiler-simulator for mapping the transformed code onto custom VLIW

architecture instances. We evaluated the overall methodology (customized architecture selection and statistical compiler level analysis) using a GSM codec application as the driving use case. We show that only a limited set of compiler transformations has a significant effect on optimizing performance across a set of GSM specific VLIW processors. In addition to the application specific scenario, we present results regarding the multiple embedded applications onto a single VLIW instance, showing that the proposed analysis can be used to extract promising compiler transformations in cross-application manner.

The rest of the chapter is organized as follows. Section 2.2 provides a brief discussion on related work and the current state of the art in the field. In Sect. 2.3, we introduce the basic methodology for architecture customization and statistical compiler level analysis. Section 2.3.2.1 presents the experimental evaluation of the proposed methodology on differing customized VLIW architectures and benchmark applications. Section 2.4 summarizes the work and concludes this chapter.

2.2 Background

Although we have entered the era of multi-core systems, the high degree of instruction parallelism offered by VLIW architectures seems to make them an interesting alternative for a large set of commercial embedded systems [1, 10, 11]. VLIW architectures are also emerging in the modern many-core embedded accelerator devices, i.e., KALRAY MPPA256 for image and signal processing applications.

Several research works have targetted the generation of Pareto optimal VLIW architectural configurations [3, 11] by exploring the space using pre-allocated compiler sequences over differing architecture instances. Towards the same direction of VLIW architectural configuration, Wong et al. [12] introduced r-VEX, a reconfigurable and extensible VLIW processor. The source code is mapped using the VEX (VLIW Example) environment [13], which forms a compilation-simulation system that targets a broad class of VLIW processor architectures, and enables compiling, simulating, analyzing and evaluating C programs [2].

In current literature, there is a lot of attention on iterative compilation and predictive compiler modeling to predict the potential speedup of compiler transformed programs utilizing code features provided by static program analysis as mentioned in the Chap. 1. However, there is a lack of comprehensive analysis regarding the impact of applying differing conventional compiler transformations on customized VLIW architectures. Although, in VLIW compilation infrastructures [13] there are available batch compiler optimization modes, fine-grained analysis of compiler effects for VLIW architectures and its relation with architecture customization is not adequately targeted.

2.3 Methodology for Compiler Analysis of Customized VLIW Architectures

In this section, we describe the proposed methodology for compiler analysis of customized VLIW architectures. The proposed methodology comprises of two phases: (i) Customized VLIW architecture selection and (ii) Statistical analysis of compiler transformations. From a high-level point of view, we first generate a set of promising VLIW architectural candidates that tailor to the characteristics of the target application, optimizing on the performance-intensity trade-off curve with respect to the overall hardware allocated resource. Then, statistical analysis of distributions generated over the compiler transformation space is performed on the set of these selected customized VLIW solutions. This enables the designer to characterize the effects of each compiler transformation in both an architecture specific manner and a cross-architecture manner.

We used the Roofline performance model [8] as the basis for both generating the custom architecture configurations and characterizing the effect of the compiler passes. Roofline relates processor performance to off-chip memory traffic. It characterizes processor architectures in a two-dimensional space, i.e., performance (Mops/sec) versus operational intensity (ops/Byte). *Operational intensity* is defined as operations per byte of DRAM traffic, defining total byte accessed as those bytes that go to the main memory after been filtered by the cache hierarchy. The advantage of using Roofline model is twofold: (i) it provides the designer with an intuitive insight visual metric for fast evaluation of the architectural optimality of the configuration and (ii) it is useful to characterize the impact of applied compiler transformations onto a specific architecture. For example, Fig. 2.1 depicts the Roofline model of a specific VLIW configuration and the superposition of application configurations

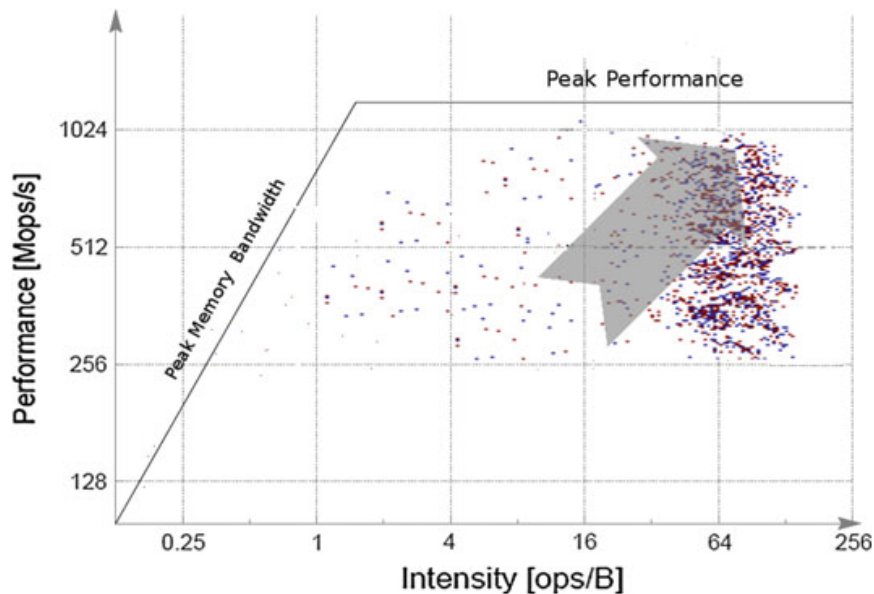


Fig. 2.1 Roofline example

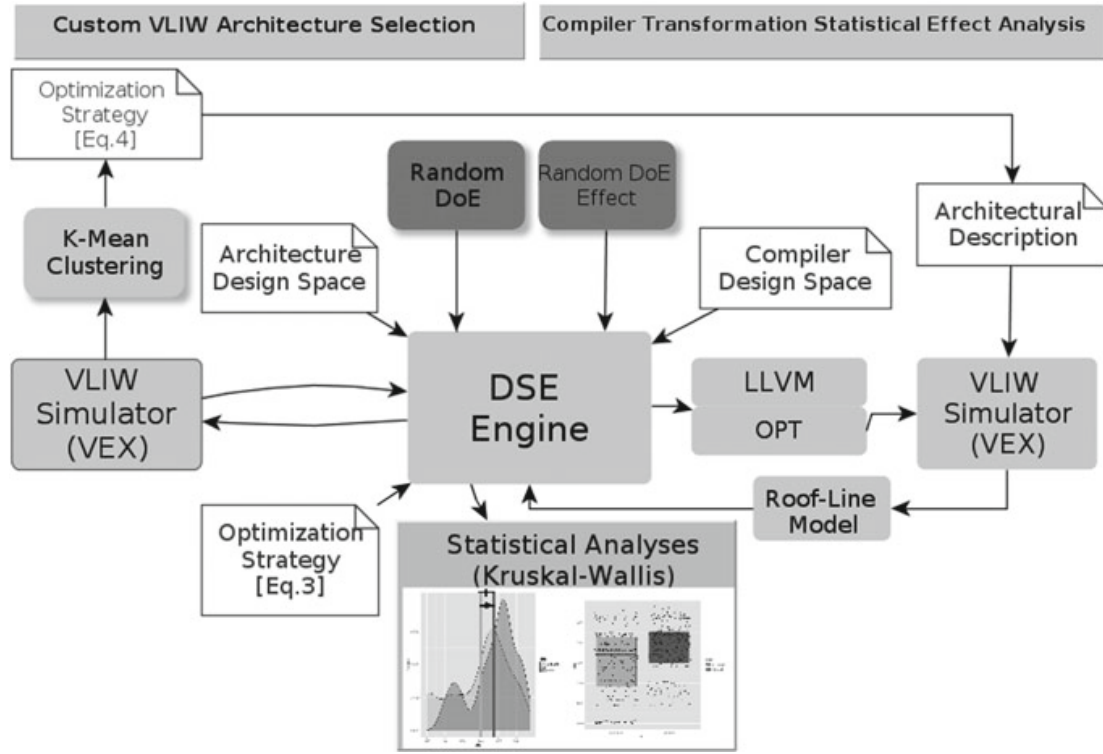


Fig. 2.2 Tool-chain implementing the proposed methodology

derived from an experimental campaign of 4 K different compiler parameter combinations. A general trend (highlighted by the arrow in Fig. 2.1) can be easily detected towards higher performance and operational intensity points. Given this visual representation, a designer can identify promising compiler passes to be applied. A custom exploration and analysis framework (Fig. 2.2) has been developed based on the integration of open source tools to implement the proposed methodology. Specifically, we used Multicube Explorer [14, 15] as the central DSE engine. Given the architectural and compiler design space descriptions, it manages to automatically generate configuration vectors according to the specified DoE – random DoE during the phase of custom VLIW architecture selection and random effect DoE during the compiler transformation analysis phase. The LLVM compiler infrastructure¹ is integrated within the framework – specifically the LLVM C front-end and the opt tool – as a source to source transformation tool of the original application code after applying the compiler transformations instructed by the DSE engine. The transformed code of the application is mapped onto the VLIW processor using the VEX [13] VLIW compiler-simulator tool, which is used for both generating different VLIW architectural configurations and mapping code onto these custom VLIW processors. Custom scripts have been developed to evaluate each examined configuration according to the Roofline model. Statistical analysis and visualization of results are performed using the statistical language R [16].

¹LLVM projected supported its C source-to-source compiler frontend till v2.8.

2.3.1 Custom VLIW Architecture Selection

Application-specific customization of architecture's parameters is one of the early system design optimization phases for defining platform configurations that meet the desired performance specifications. Given the large number of parameters that usually defines a processor architecture and the delay required for simulating each possible configuration, the task of optimal micro-architectural parameter selection forms an extremely challenging exploration problem that for reasonably representative design space definitions becomes intractable, regarding the time required for exhaustive evaluation. Several research works utilizing well-known meta-heuristics [3, 17] have been already proposed for generating the Pareto optimal sets of the aforementioned optimization problem.

In this chapter, however, we slightly shift the focus of exploration from delivering the optimal set of architectural configurations to discover custom architecture configurations that do not correspond to the boundaries of Pareto regions, i.e., very low-cost architectures with very poor performance or very expensive architectures that deliver very high gains regarding performance. Thus, here we invoke a relaxed optimization search strategy that is based on a random sampling of the targeted design space rather than on an optimization oriented strategy, e.g. simulated annealing or NSGA-II genetic optimization [17] etc.

Table 2.1 shows the micro-architectural design space, Ω , considered for the custom VLIW architecture selection phase. In the first step, we randomly sample the Ω design space. Each explored solution is stored in the database of explored solutions, X after being characterized according to the performance and operational intensity metrics defined within the Roofline model, where:

$$Performance(\mathbf{x}) = \frac{\#Operations(\mathbf{x})}{\#NumCycles(\mathbf{x}) \times ClkFreq(\mathbf{x})} \quad (2.1)$$

$$Intensity(\mathbf{x}) = \frac{\#Operations(\mathbf{x})}{\#CacheMisses(\mathbf{x}) \times CacheLineSize(\mathbf{x})} \quad (2.2)$$

After the formation of the X , we are interested in finding those explored architectures that maximize the performance and operational intensity of the application while using minimum computational and memory resources. In order to extract the desired architectural configurations, we perform Pareto filtering on the solution space defined with the X , by considering the following multi-objective optimization problem:

$$\min_{\mathbf{x} \in \Omega} \left[\begin{array}{c} \frac{1}{Performance(\mathbf{x})} \\ \frac{1}{Intensity(\mathbf{x})} \\ \#CompResources(\mathbf{x}) \\ \#MemResources(\mathbf{x}) \end{array} \right] \quad (2.3)$$

Table 2.1 VLIW microarchitectural design space

Parameters	Values (integer range)
lg2CacheSize	[11–30]
lg2Sets	[0,3]
lg2LineSize	[5,9]
lg2ICacheSize	[11,30]
lg2ICacheSets	[0,3]
lg2ICacheLines	[5,9]
ClkFreq	[300,500]
NumCaches	[1,2]
IssueWidth	[1,16]
NumAlus	[1,16]
NumMuls	[1,4]
RegisterFile	[32,128]
BranchRegister	[32,128]

where computational resources are (i) number of ALUs and (ii) number of multipliers, while memory resources are (i) data cache size, (ii) instruction cache size and (iii) register file size. Although, in Eq. 2.3 we present the unconstrained version of the target optimization problem, we note that our exploration infrastructure permits also the inclusion of arbitrary constraints either on the objectives itself or on specific parameter combinations that the designer has a priori evaluated as not interesting.

The outcome of the optimization procedure defined in Eq. 2.3 is a Pareto surface, X_p , of the explored X , thus exhibiting a large number of VLIW architectural configurations. In order to restrict the number of VLIW configuration that will be characterized as the representative customized VLIW solutions that will be propagated to the statistical compiler analysis phase, we perform a clustering on the performance - intensity solution space. We used k-means [18] clustering for the aforementioned procedure, with a configurable number of clusters, k , decided by the designer. The clustering procedure partitions the X_p solution space into k regions of interest, X_p^{ci} , e.g. region of high intensity and high performance, or region of low intensity and high performance etc. Eventually, each cluster should deliver one representative VLIW architecture, that forms the optimal solution within the cluster. We define this optimal solution per cluster as the architectural configuration that minimizes area cost of the processor while maximizing both the metrics of performance and operational intensity. In order to extract this optimal configuration from each cluster, we iteratively apply the following single-objective minimization problem in every X_p^{ci} produced by the k-means clustering:

$$\min_{\mathbf{x} \in X_p^{ci}} \frac{Area(\mathbf{x})}{Performance(\mathbf{x}) \times Intensity(\mathbf{x})} \quad (2.4)$$

For the calculation of the area cost in Eq. 2.4, the area model provided by the McPAT [19] micro-architecture framework has been used, assuming a processor technology of 90 nm.

Finding an architectures which is optimized by using the right set of compiler optimizations is an essential task to mitigate. However, reaching this goal has its own tolerance and trade-off. Occasionally it happens to sacrifice the code size for better performance or portability versus code size. Consequently, there should be a precaution when using these options otherwise it ends up heavier and less-usable. Without any optimization option, the compiler's goal is to reduce the cost of compilation and to make debugging produce the expected results. Turning on optimization flags makes the compiler attempt to improve the performance and/or code size at the expense of compilation time and possibly the ability to debug the program. Compilers perform optimization based on the program knowledge. Not all optimizations are controlled directly by an optimization pass. In this work, we select 15 compiler passes supported by LLVM compiler are as described in the Table 2.2.

2.3.1.1 DoE

Given a huge multi-objective optimization problem, it is necessary to use the design of experiment (DoE) methods, i.e., Taguchi Design of experiment [20]. DoEs are the basic components for building the exploration strategies. The DoE used in this work was based on random factors which generated a set of random designed points. In addition, the optimization algorithm used here was parallel DoE (PDoE) which was based on the possibility of performing concurrent evaluation of the different design points, i.e, in the experimental analyses, for each compiler transformations per benchmark, the number of exploration was 500, therefore, it would have given enough points for the system to use for DoE and Optimizer to generates the effects and metrics besides the Pareto points (if exists).

2.3.2 *Compiler Transformation Statistical Effect Analysis*

The second phase of the proposed methodology receives as input the generated custom VLIW architectures as described in the previous section, and for each of the set of micro-architectural points, it evaluates the statistical effects of the compiler transformations in a fine-grained manner. In this research work we focus on 15 of the compiler passes supported by LLVM (see Table 2.2).

As a first step in our analysis, we have to determine a reasonable number of samples to produce a robust analysis of the main effects associated with the 15 compiler parameters. In the following, each configuration of these compiler parameters, or set of compiler options, will be defined as a vector of 15 values, where each value represents a compiler pass option.

Table 2.2 Selected compiler transformations from LLVM framework

Compiler transformation	Abbreviation	Short description
Constant propagation	Constprop	Constant operands instructions are replaced with a constant value and propagated
Dead code elimination	Dce	Checks instructions that were used by removed instructions to see if they are newly dead
Function integration/Inlining	Inline	Bottom-up inlining of functions into callees
Combine redundant instruction	Instcombine	Combine instructions to form fewer, simple instructions. This pass does not modify the CFG and applies algebraic simplification
Loop invariant code motion	Licm	Removes as much code from the body of a loop as possible. It does this by either hoisting code into the pre-header block, or by sinking code to the exit blocks if it is safe
Loop strength reduction	Loop-reduce	Strength reduction on array references inside loops that have as one or more of their components the loop induction variable
Rotates loops	Loop-rotate	A simple loop rotation transformation
Unroll loops	Loop-unroll	A simple loop unrolling
Unswitch loops	Loop-unswitch	Transforms loops that contain branches on loop-invariant conditions to have multiple loops
Promote memory to register	Mem2reg	It promotes memory references to be register references
Memory copy optimizations	Memcpyopt	Performs various transformations related to eliminating memcpy calls, or transforming sets of stores into memset's
Reassociate expressions	Reassociate	It reassociates commutative expressions in an order that is designed to promote better constant propagation
Scalar replacement of aggregates	Scalarrepl	It breaks up alloca instructions of aggregate type (structure or array) into individual alloca instructions for each member if possible
Sparse cond const propagation	Sccp	It assumes values are constant and Basic Blocks are dead unless proven otherwise. It proves values to be constant, and replaces them with constants and Proves conditional branches to be unconditional
Simplify the control flow graph	Simplycfg	Performs dead code elimination and basic block merging

To accommodate our goal, we defined a randomized design of experiments $D_N(p)$ for each compiler parameter p . $D_N(p)$ is a list of *options sets*:

$$D(p) = [o_{1+}, o_{1-}, o_{2+}, o_{2-}, \dots, o_{N+}, o_{N-}] \quad (2.5)$$

where o_{n+} corresponds to the n -th random option set in which compiler pass $p \in \{OFF, ON\}$ is set to its maximum value (*ON*) while all the others compiler passes are randomly chosen. In a dual way, o_{n-} is equal to o_{n+} except that p assumes its minimum value (*OFF*).

By applying this DoE, we can easily measure how much the impact of the transition ($- \rightarrow +$) for parameter p impacts (in average over all the considered options sets) on the performance without requiring a full-factorial design. As an example, Fig. 2.3 depicts the generated performance distributions by activating and deactivating the ‘licm’ and ‘reassociate’ compiler transformations for a GSM codec application. It can be observed that while the activation of ‘licm’ has a clear positive effect on performance—the median is shifted towards higher performance values, this is not the case for the ‘reassociate’ transformation for which the activation and deactivation distributions have almost the same shape and density, thus not permitting the designer to recognize a clear trend.

As the second step, for each optimization set in $D(p)$ we evaluate the vector of performance responses with the actual architecture synthesis after the compilation and simulation of the target application. We consider the hypothesis whether the mean of the performance given by the options sets where p was minimum (or *off*) is *different* from the mean where p was maximum (or *on*). In practice, this is framed as a *null-hypothesis statistical test*, which, given the *non-parametric* (or non-gaussian)

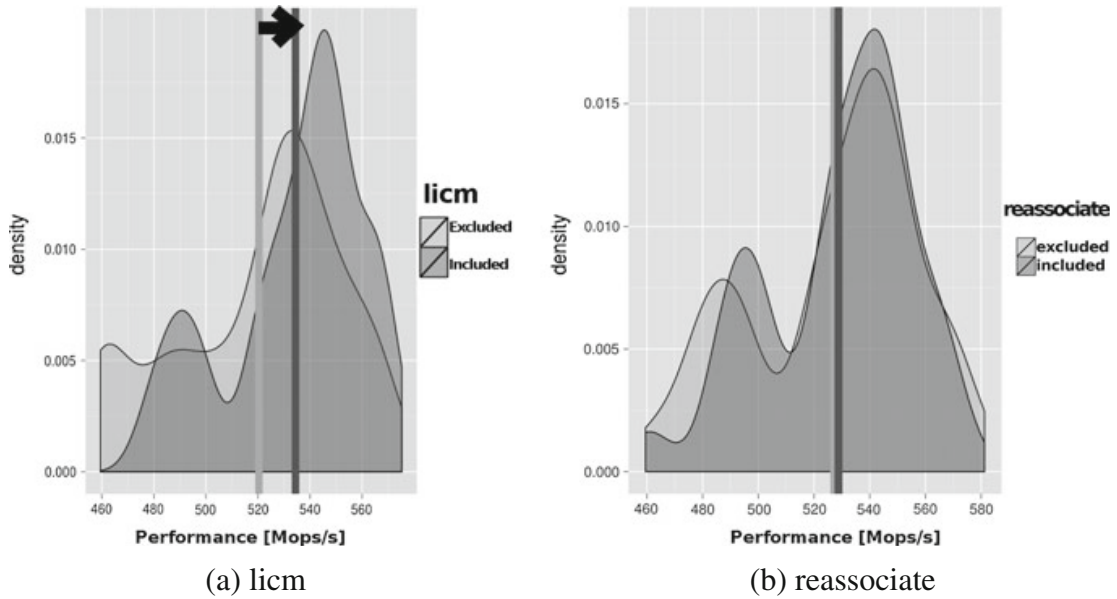


Fig. 2.3 **a** licm’s having significant positive effect, **b** reassociate’s causing no significant effect

nature of the underlying distributions,² cannot be assessed with as a simple ANOVA but, instead, with a Kruskal-Wallis test [21]. To complete the hypothesis test, the designer sets an acceptance ratio of $p - value\%$ meaning that the probability of ‘measuring’ different means when the underlying distributions are equal (or the chance of a false positive) is less than 5%.

2.3.2.1 Statistical Analysis

As mentioned in Sect. 2.2, there have been several works involving the machine learning techniques and predictions [22–24]. In this research work we have focused on analyzing the effects of applying the specific compiler transformations on the design space. The probability of certainty about the effects of a specific compiler transformation on performance metric could be done using some statistical tests; ANOVA, Kruskal-Wallis. ANOVA [25] test has been widely used as a reliable tester for normal distributions. In addition, using Kruskal-Wallis [21], is a good test tool as it assumes the distribution to be non-parametric. This method is used for testing whether samples originated from the same distribution or not. In this work, since dealing with empirical data on experimental results, we assumed the models as non-parametric, therefore, Kruskal-wallis was employed. The algorithm goes as:

- 1- Rank all the groups from 1 to N together
- 2- Statistical test is elaborated among the group to calculate the value K which contains the square of the average ranks
- 3- Finally the $p-value$ is approximated as $Pr(\chi^2_{g-1} \geq K)$
- 4- If the statistic is not significant, then there is no real evidence of difference between samples and could be deduced the samples comply with the model.

In this chapter, the global threshold was set as high as 5% in order to increase the robustness of the results. Therefore, a test is deduced as passed regarding Kruskal-wallis test in which it has the p-value smaller than 0.05. In this case, a model is passed if and only if it had confidence threshold over 95%; experimental analyses represented in Fig. 2.5 will be focused later in this chapter.

In this section, we experimentally evaluate the proposed methodology. We consider the GSM codec embedded application as the driving use case, automatically generating four representative application specific architectures after applying the custom VLIW architecture selection. We use these VLIW architectures for statistically analyzing the effects of compiler transformations across differing VLIW configurations. Furthermore, we analyze the compiler transformation effects in a cross-application manner, by considering a larger set of embedded applications mapped onto a default (application independent) VLIW processor configuration.

The first subsection, introduce the experimental setup and the framework. The second subsection will contain the architectural selection based on the method described

²Since the distributions are built based on empirical/experimental data, the distribution is considered in general non-parametric.

in Sect. 2.3.1 and exploration on standard benchmark regarding the derived configurations will be presented. Eventually, there will be a comparison of the default architecture among 5 other benchmarks will be discussed and depicted with the statistical consolidations.

We apply the overall proposed methodology considering the GSM codec as the driving application. We apply the custom VLIW architecture selection phase to generate optimized representative VLIW architectures in an application specific manner. The considered architectural design space is depicted in Table 2.1. We configure the search procedure to randomly generate and evaluate 30 K configurations, using a uniform sampling over the targeted configuration space (Table 2.1). Applying the multi-objective optimization problem defined in Eq. 2.3 over the 30 K solutions, the Pareto surface of the configurations that maximize performance and operational intensity while minimizing resources is generated. Without loss of generality, we consider the generation of $k = 4$ clusters over the generated Pareto surface, aiming at generation of four GSM-specific VLIW architectures. Figure 2.4 shows results of clustering of the extracted Pareto surface and its mapping onto the two-dimensional performance versus intensity space. Each cluster has been characterized according to its position on the performance versus intensity space as: (i) HH for the cluster placed to the high intensity and high performance region, (ii) LH for the low intensity and high performance region, (iii) LL for the low intensity and low performance region, and (iv) HL for the high intensity and low performance region, respectively.

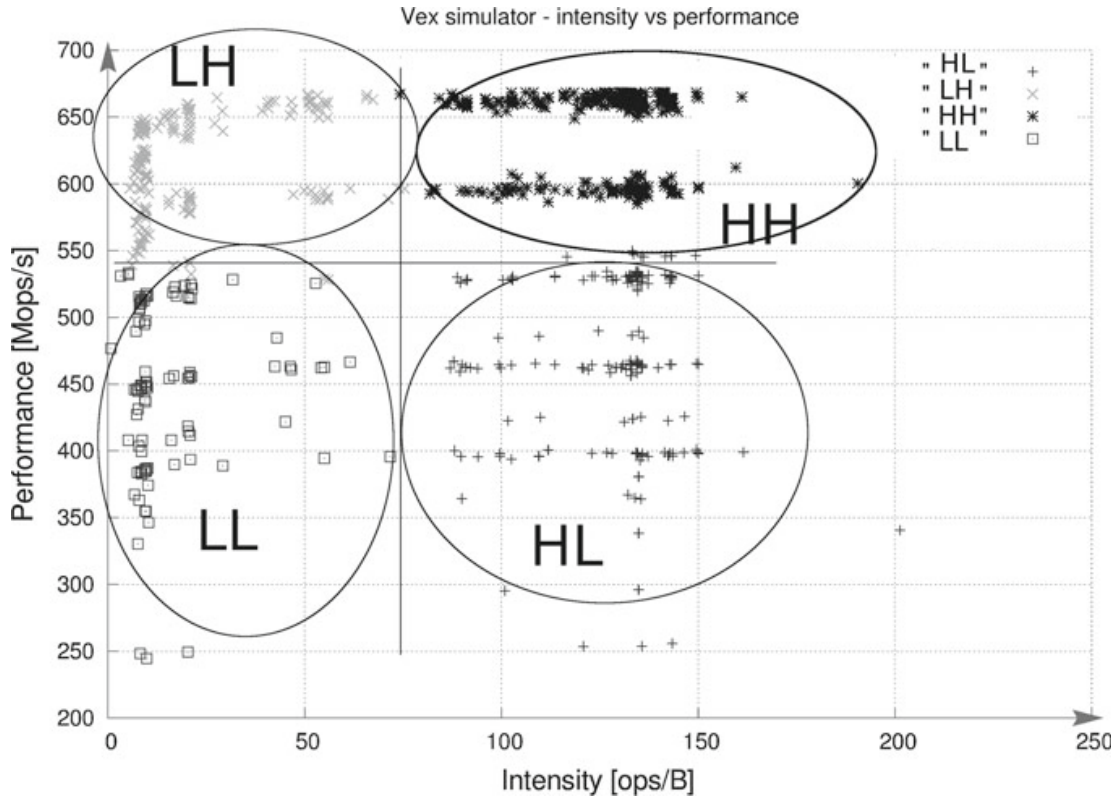


Fig. 2.4 Four clustered Pareto-sets

Table 2.3 VLIW architecture configurations

Parameters	Arch-HL	Arch-LH	Arch-HH	Arch-LL	Arch-User
lg2CacheSize	15	12	13	12	16
lg2Sets	1	3	0	1	2
lg2LineSize	7	5	5	5	5
lg2ICacheSize	16	14	16	14	16
lg2ICacheSets	1	3	3	2	2
lg2ICacheLines	6	8	7	5	6
ClkFreq	400	450	450	300	500
NumCaches	2	1	1	1	1
IssueWidth	6	6	14	9	8
NumAlus	4	6	7	3	8
NumMuls	1	4	4	14	2
MemLoad	4	3	6	5	4
MemStore	2	8	4	6	4
RegisterFile	104	100	32	76	64
BranchRegister	76	84	88	48	64

The final $k = 4$ representative VLIW architectures are derived after applying within each cluster the optimization operator of Eq. 2.4. Table 2.3 reports the architectural configuration for each of the $k = 4$ application specific VLIW architectures.

For each of the $k = 4$ application specific VLIW architectures, we explore the compiler level design space, defined in Table 2.2. We generate the non-parametric distribution of the performance and intensity for each compiler transformation considering 500 samples per transformation. As described in Sect. 2.3.2, the non-parametric distributions are analyzed based on Kruskal-Wallis test to specify the statistical effect, i.e. if the inclusion or exclusion of a specific transformation impacts in a specific and robust manner the two considered metrics. Table 2.4 summarizes the results of Kruskal-Wallis statistical tests for each compiler transformation over the four examined architecture configurations. As shown, four compiler passes (*inline*, *licm*, *loop-reduce* and *loop-rotate*), over the fifteen initially considered, had a significant impact on performance when activated. In addition, Fig. 2.5, shows the confidence level for each of the considered compiler transformations. It is shown that the four mentioned compiler transformations exhibit a high confidence level $>99\%$. Therefore, it could be implied that activating these specific transformations, the designer can be around 99% confident that the effect on performance will be the same as the one determined by the exploration.

In the second set of experiments, we perform statistical analysis in a cross-application manner. For this experimental campaign, we assume a larger set of applications (namely GSM, AES encryption engine, ADPCM codec, JPEG decoder and

Table 2.4 Summary of Kruskal-Wallis analysis on performance for GSM-specific VLIW architectures

Compiler transformation	Arch-HL	Arch-LH	Arch-HH	Arch-LL
Constprop	–	–	–	–
Dce	–	–	–	–
Inline	✓	✓	✓	✓
Instcombine	–	–	–	–
Licm	✓	✓	✓	✓
Loop reduce	✓	✓	✓	✓
Loop rotate	✓	✓	✓	✓
Loop unroll	–	–	–	–
Loop unswitch	–	–	–	–
Mem2reg	–	–	–	–
Memcpyopt	–	–	–	–
Reassociate	–	✓	✓	✓
Scalarrepl	–	–	–	–
Sccp	–	–	–	–
Simplifycfg	–	–	–	–

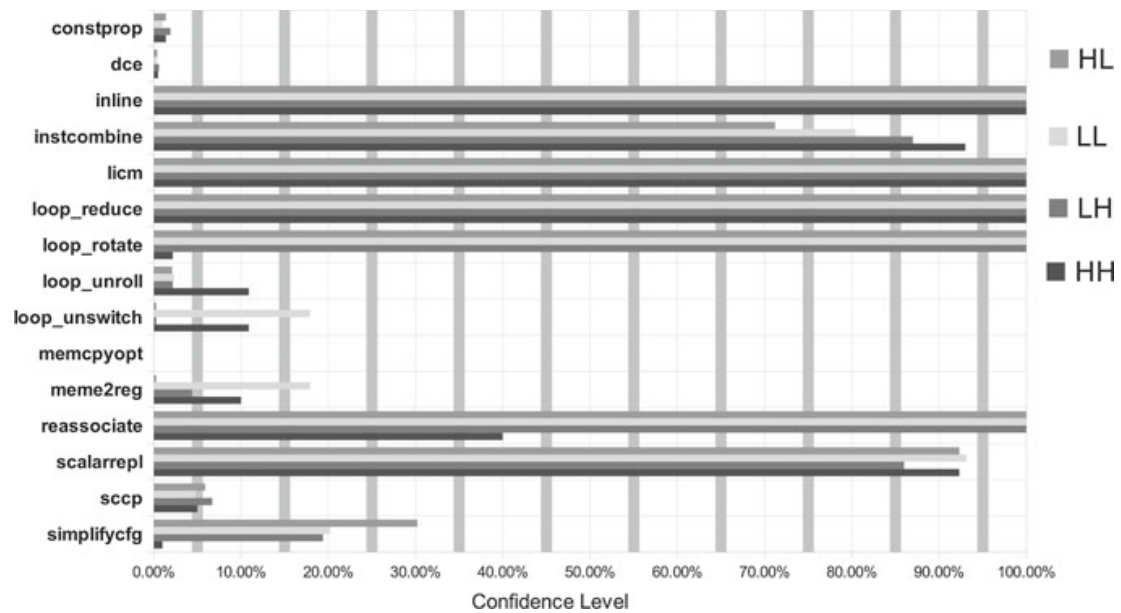
**Fig. 2.5** Confidence level characterization of compiler transformations regarding the effect on performance for each one of the GSM specific VLIW architectures, resulted after Kruskal-Wallis statistical test

Table 2.5 Kruskal-Wallis analysis on performance for multiple applications

Compiler transformation	GSM	AES	ADPCM	JPEG	Blowfish
Constprop	–	–	–	–	–
Dce	–	–	–	–	–
Inline	✓	–	✓	✓	–
Instcombine	✓	–	✓	✓	✓
Licm	✓	✓	✓	✓	✓
Loop reduce	✓	✓	✓	✓	✓
Loop rotate	✓	✓	✓	✓	–
Loop unroll	–	–	–	–	–
Loop unswitch	–	–	–	–	–
Mem2reg	✓	✓	✓	✓	✓
Memcpyopt	–	–	–	–	–
Reassociate	✓	–	–	–	–
Scalarrepl	✓	–	–	–	✓
Sccp	–	–	–	–	–
Simplyfycfg	–	–	–	–	–

Blowfish block cipher). The performance of each applications has been evaluated considering a user specified VLIW architecture, Arch-User, defined in the last column of Table 2.3. For each benchmark the compiler transformation statistical effect analysis (Sect. 2.3.2) is applied, considering distributions of 500 samples per compiler transformation. Table 2.5 summarizes in an aggregated manner the results of the Kruskal-Wallis analysis considering in each case a confidence level $\geq 5\%$. For the specific setup, we observe that there is a set of four compiler parameters (*licm*, *loop reduce*, *loop rotate* and *mem2reg*) with significant effect on the performance and with a high confidence level over all the examined application use cases. Furthermore, examining each application in isolation, the designer can derive which are the compiler parameters that need to be pre-allocated, thus reducing significantly the design-time required to optimize the performance of the targeted application during iterative compilation exploration. As an example, we depict in the Fig. 2.6, the normalized speedup gains achieved by activating the compiler transformations proposed by our methodology in comparison with several well-known compilation strategies. It is shown that the proposed methodology defined speedup gains in all the examined cases between 16 and 23%.

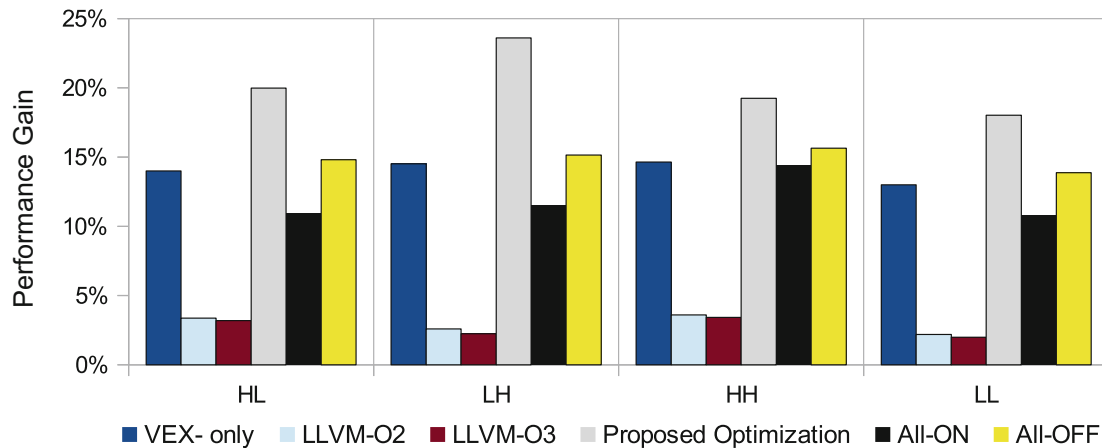


Fig. 2.6 The gained speed-up we gained comparing to the default LLVM-O1 optimization level in GSM benchmark

2.4 Conclusions and Future Work

This chapter presented a methodology for a compiler/architecture co-exploration of VLIW platform design. It provides the designer with an integrated environment to automatically (i) generate optimized application specific VLIW architectural configurations and (ii) analyze in a fine-grained manner the effects of compiler level transformations regarding the performance and operational intensity trade-offs. Being focused more on the analysis part, we showed that the adoption of the specific methodology either in a cross-architecture and/or cross-application manner, can deliver significant application specific insights thus enabling the designer to guide through decisions regarding the architecture and the compilation optimization strategy. Future work is aligned with our strong belief that the proposed methodology can be exploited in a straightforward manner within automated design frameworks focusing on performance optimization through iterative compilation and architecture specialization.

References

1. Fisher JA, Faraboschi P, Young C (2009) VLIW processors: once blue sky, now commonplace. *IEEE Solid-State Circuits Mag* 1(2):10–17
2. Fisher JA, Faraboschi P, Young C (2004) *Embedded computing: a VLIW approach to architecture, compilers and tools*. Morgan Kaufmann, Burlington, MA
3. Ascia G, Catania V, Palesi M, Patti D (2005) A system-level framework for evaluating area/performance/power trade-offs of vliw-based embedded systems. *Design automation conference*. In: *Proceedings of the ASP-DAC 2005. Asia and South Pacific*, vol 2., pp 940–943
4. Fisher JA (1981) Trace scheduling: a technique for global microcode compaction. *IEEE Trans Comput* 30(7):478–490
5. Hwu WMW, Mahlke SA, Chen WY, Chang PP, Warter NJ, Bringmann RA, Ouellette RG, Hank RE, Kiyohara T, Haab GE et al (1993) The superblock: an effective technique for VLIW and superscalar compilation. *J Supercomput* 7(1–2):229–248

Chapter 3

Selecting the Best Compiler Optimizations: A Bayesian Network Approach

Abstract After presenting our DSE approach for finding good compiler optimizations, we present our autotuning framework to tackle the problem of selecting the best compiler passes. It leverages machine learning and an application characterization to find the most promising optimization passes given an application. This chapter proposes *COBAYN*: Compiler autotuning framework using Bayesian Networks. An autotuning methodology based on machine learning to speed up application performance and to reduce the cost of the compiler optimization phases. The proposed framework is based on the application characterization done dynamically by using independent micro-architecture features and Bayesian networks. The chapter also presents an evaluation based on static analysis and hybrid feature collection approaches. Besides, we compare our approach against several state-of-the-art machine-learning models. Experiments are carried out on an ARM embedded platform and GCC compiler by considering two benchmark suites with 39 applications. The set of compiler configurations selected by the model (less than 7% of the search space), demonstrated an application performance speedup of up to $4.6\times$ on Polybench ($1.85\times$ on average) and $3.1\times$ on Cbench ($1.54\times$ on average) with respect to standard optimization levels. Moreover, the comparison of the proposed technique with (i) random iterative compilation, (ii) machine learning-based iterative compilation and (iii) non-iterative predictive modeling techniques, shows on average, $1.2\times$, $1.37\times$ and $1.48\times$ speedup, respectively. Finally, the proposed method demonstrates $4\times$ and $3\times$ speedup, respectively on cBench and Polybench, in terms of exploration efficiency given the same quality of the solutions generated by the random iterative compilation model.

3.1 Introduction

Usually, software applications are developed in a high-level programming language (e.g. C, C++) and then passed through the compilation phase to get the executable binary. Optimizing the second phase (*compiler optimization*) plays an important role for the performance metrics. In other words, enabling compiler optimization parameters (e.g., loop unrolling, register allocation, etc.) might lead to substantial benefits in performance metrics. Depending on the strategy, these performance metrics could

be *execution time*, *code size* or *power consumption*. A holistic exploration approach to trade-off these metrics also represents a challenging problem [1].

Application developers usually rely on compiler intelligence for software optimization, but they are unaware of *how* the compiler itself does the job. Compiler interface usually has some standard optimization levels which enable the user to automatically include a set of predefined optimization sequences for the compilation process [2]. These standard optimizations (e.g. -O1, -O2, -O3 or -Os) are known to be beneficial for performance (or code size) in most cases. In addition to the above-mentioned standard optimizations, there are other compiler optimizations which are not included in the predefined optimization levels. Their effects on the software are quite complex and mostly depend on the features of the target application. Therefore, it is rather hard to decide whether to enable specific compiler optimizations on the target code. Considering an application-specific embedded system domain; constructing an optimized compiler optimization heuristic becomes even more crucial because the application is compiled once and then deployed on millions of devices on the market.

So far, researchers proposed two main approaches for tackling the problem of identifying the best compiler optimizations: (i) *iterative compilation* [3] and (ii) *machine-learning predictive modeling* [4–6]. The former approach relies on several recompilation phases and then selecting the best set of optimizations. Despite being effective, this approach has high overhead as it needs to be evaluated iteratively over all the enumerations of an optimization space. The latter approach focuses on building machine-learning predictive models to predict the best set of compiler optimizations. It relies on software features that are collected either *offline* or *online*. Once the model has been trained, given a target application, it can predict a sequence of compiler optimization options to maximize performance. Machine learning approaches normally require fewer compilation try-outs, however, their downside is the predicted optimizations lead to an execution binary that performs worse than the one found with iterative compilation.

In this chapter, we propose an approach to tackle the problem of identifying the compiler optimizations that maximize the performance of a target application. In contrast, our proposed methodology starts by applying a statistical methodology to infer the probability distribution of the compiler optimizations to be enabled. Then, we perform an iterative compilation process by sampling from this probability distribution. We use two major sets of training application suites to learn the statistical relations between application features and compiler optimizations. To the best of our knowledge, in this work, *Bayesian Networks* (BN) are used for the first time in this field to build the statistical model. Given a new application, its features are fed into the machine-learning algorithm as *evidence* on the distribution. This evidence imposes a bias on the distribution, and because compiler optimizations are correlated with the software features, we can iteratively sample the distribution obtaining the most promising compiler optimizations, by then exploiting an *iterative compilation* process.

The experiments carried out on an embedded ARM-based platform outperformed both standard optimization levels and the state-of-the-art iterative and not iterative

(based on prediction models) compilation techniques while using the same number of evaluations. Moreover, the proposed techniques demonstrated significant exploration efficiency improvement of up to $4\times$ speedup compared with random iterative compilation when targeting the same performance. To summarize, here are our contributions:

- The introduction of a BN capable of capturing the correlation between the application features and the compiler optimizations. BN allows to represent the relation by an acyclic graph, which can be easily analyzed graphically.
- The integration of the BN model in a compiler optimization framework. Given a new program, the probability distribution of the best compiler optimizations can be inferred by means of BN to focus on the optimization itself.
- The integration of both dynamic and static analysis feature collections in the framework as hybrid features.

Furthermore, the experimental evaluation section reports the assessment of the proposed methodology on an embedded ARM-based platform and the comparison of the proposed methodology against several state-of-the-art machine learning algorithms on 39 different benchmark applications.

The remainder of the chapter is organized as follows. Section 3.2 presents a quick review of the recent related literature. Readers are referred to Chap. 1 for a more comprehensive review on the state-of-the-art. Section 3.3 presents how the BN model can infer the probability of the distribution. Section 3.3.1 presents different techniques for collecting program features. Section 3.4 elaborates on the proposed framework. Sections 3.4.4 and 3.4.5 will introduce the results obtained on the application suites selected. Finally, Sect. 3.4.6 presents the comparison of the proposed methodology with state-of-the-art models.

3.2 Previous Work

Optimizations carried out at compilation have been broadly used, mainly in embedded computing applications. We refer to the holistic review on the survey in the Chap. 1. However, for the chapter completeness, we would like to remind a few related works. There are two major classes of optimization in the field of compilation research: (i) The problem of *selecting the best compiler optimizations* and (ii) The *phase-ordering* problem of compiler optimizations. As the target of this work is in the scope of *selection*, here we mostly refer to these areas. However, there are notable works to be mentioned that support the seminal concepts of the current work.

The related work in this field can be categorized into two sub-classes: (a) *iterative compilation* [7] and (b) *machine-learning* based approaches [8, 9]. Nonetheless, these two approaches have also been combined in many ways [4] that they cannot be distinguished easily.

Iterative compilation techniques were introduced as a mean to outperforming static handcrafted optimization sequences, those usually exposed by compiler interfaces as *optimization levels*. Since its introduction [7, 10], the goal of iterative compilation has been to identify the most appropriate compiler passes for a target application.

Other authors exploit iterative compilation jointly with architectural design space exploration for VLIW architectures [11]. The intuition was that the performance of a computer architecture depends on the executable binary which in turn, depends on the optimizations applied at compilation time. Thus, by studying the two problems jointly, the final architecture is optimal and the effects of different compiler optimizations are identified at the early design stages.

Given that compilation is a time-consuming task, several groups proposed techniques to predict the best compiler optimization sequences rather than applying a trial-and-error process, such as in iterative compilation. These prediction methodologies are generally based on *machine-learning* techniques [4, 6, 8, 12, 13].

More recent literature on using different program features with machine learning have been proposed by [14] where the authors collected *IR (intermediate representation)* of the kernels and utilized *graph-kernels* to derived the similarities between those fetched *IRs*. Our approach is significantly different from the previous ones given that it applies a statistical methodology to learn the relationships between application features and compiler optimizations as well as between different compiler optimizations where *machine-learning* techniques are used to capture the probability distribution of different compiler transformations. In this work, we propose the use of *BN* as a framework enabling statistical inference on the probability distribution given the evidence of application features. Given a target application, its features are fed to *Bayesian Networks* to induce an application-specific bias on the probability distribution of compiler optimizations.

Most recent machine-learning works aim at the generation of prediction models that, given a target application, predict the performance of the application for any set of compiler transformations applied to it. In contrast, in our work the machine-learning methodology aims directly at predicting the best compiler optimizations to be applied for a target application without going through the predictions of the resulting application performance.

Additionally, in our approach, program features are dynamic and obtained through micro-architecture-independent characterization [15] and compared with the results using the static profiling [16]. The adoption of dynamic profiling provides insight into the actual program execution with the purpose of giving more weight to the code segments executed more often (i.e. code segments whose optimization would lead to higher benefits according to Amdahl’s law).

3.3 Proposed Methodology

The main goal of the proposed approach is to identify the best compiler optimizations to be applied to a target application. Each application is passed through a characterization phase that generates a parametric representation of the application

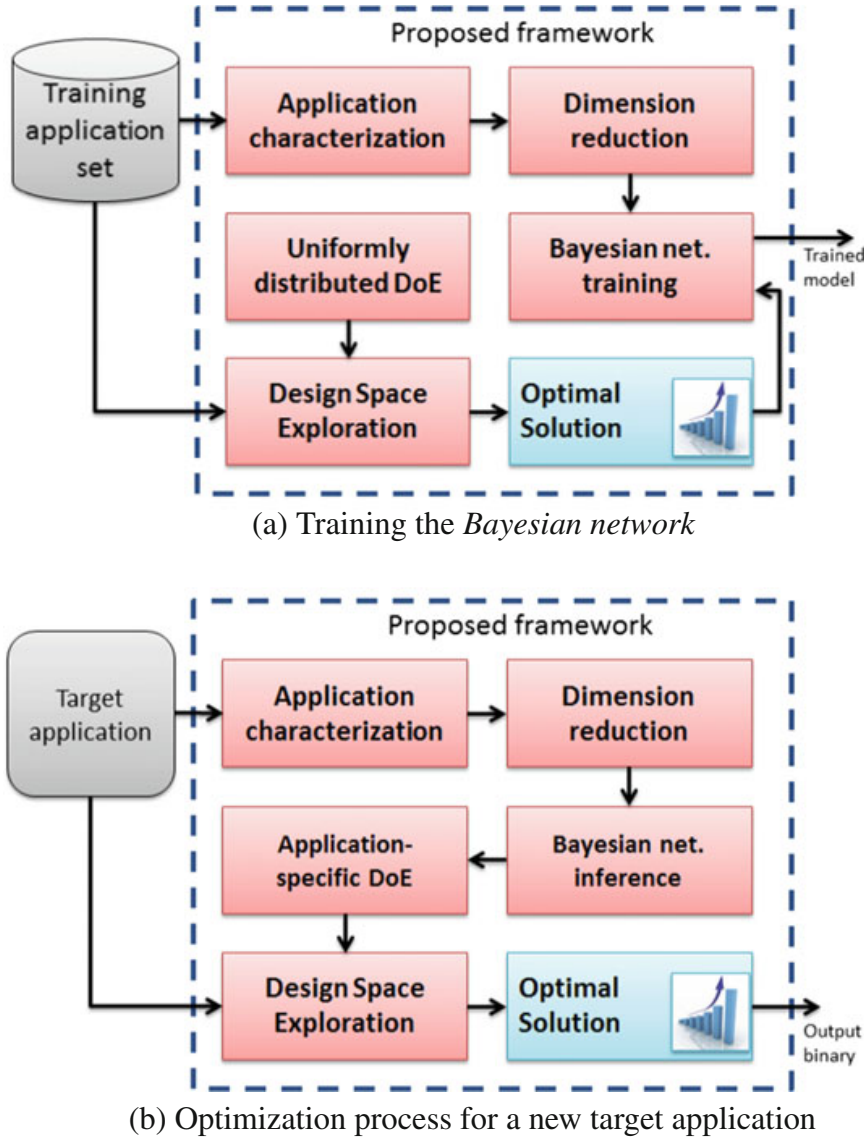


Fig. 3.1 Overview of the proposed methodology

under analysis in terms of its main features. With little information loss, these features are pre-processed by means of statistical *dimension reduction* techniques to identify a more compact representation. A statistical model based on *BN* correlates these reduced representations to the compiler optimizations to maximize application performance.

The optimization flow is shown in Fig. 3.1 and consists of two main phases. During the initial *training phase*, the *Bayesian network* is learned on the base of a set of training applications (see Fig. 5.1a). During the *exploitation phase*, new applications are optimized by exploiting the knowledge stored in the *Bayesian Network* (see an example of a BN topology in Fig. 3.3).

During both phases, an optimization process is necessary to identify the best compiler optimizations to achieve the best performance. This is done for learning purposes during the *training phase* and for optimization purposes during the *exploitation*

phase. To implement the optimization process, a Design Space Exploration (DSE) engine has been used. The DSE engine automatically compiles, executes and measures application performance by enabling/disabling different compiler optimizations. Which compiler optimizations will be enabled is decided in the Design of Experiments (DoE) phase. In our approach, the DoE is obtained by sampling from a given probability distribution that is either a *uniform distribution* (during the training phase as in Fig. 5.1a) or an *application-specific distribution* inferred through the BN (during the *exploitation phase* as in Fig. 5.1b).

The *uniform distribution* adopted during the *training phase* allows us to uniformly explore the compiler optimization space $\mathcal{O}$ to learn about the most promising regions of the space. The *application-specific distribution* during the *exploitation phase* allows us to speed up the optimization by focusing on the most promising region of the compiler optimization space $\mathcal{O}$.

3.3.1 Applying Program Characterization

The classic *supervised* Machine Learning (ML) approach deals with fitting a model exploiting a function f of program characterization. Function f might use a variety of comparison/similarity functions, such as *nearest-neighbor* and *graph-kernels*. To obtain a more accurate fitting, compiler researchers have been trying to understand the behavior of programs/kernels better and derive a *feature vector* that represents pair functionality efficiently. As a rule of thumb, the derived feature vector must be (i) representative enough of its program/kernel, and (ii) different programs/kernels must not have the same feature vectors as this will confuse the subsequent machine-learning process. Thus, building a huge non-efficient feature vector slows down the ML process and obtain less-precision.

Another goal of this work is to exploit the efficient use of different program characterization techniques and demonstrate their performance and effectiveness. Three characterization techniques have been selected among state-of-the-art works, namely, (i) *dynamic feature selection* using MICA [15], (ii) *static analysis* using MilePost [16] framework, and (iii) our handcrafted combination of those two as hybrid analysis.

MICA. *Microarchitecture-independent workload characterization* represents a recent work on dynamic workload characterization [15]. It is a plugin for the Linux-PIN tool [17] and is capable of characterizing the fed kernels *independently* from its running architecture as it monitors the *non-hardware* features of the kernels. This feature is of interest for targeting embedded domain as one might not be able to exploit PIN tools on the board. The main categories of MICA include *Instruction-Level-Parallelism (ILP)*, *Instruction Mix (ITypes)*, *Branch Predictability (PPM)*, *Register Traffic (REG)*, *Data Stream Stride (Stride)*, *Instruction and Data Memory Footprint (MEMFootprint)* and *Memory Reuse Distances (MEMReusedist)*.

MilePost. This recent tool [16, 18] was built as a plugin on top of GCC to capture static features of the programs. One advantage of *static analysis* is that the

compiler researchers do not have to run the actual binary just like what they have to do by a dynamic feature technique. On the other hand, *static analysis* fails to capture any correlations between the source code and memory hierarchy and different data streams fed as input dataset.

Hybrid. The third characterization technique consists of the combination of the two previous ones. We believe that, in some cases, hybrid feature selection can capture the kernel behaviors better as it takes into account both feature-selection methods.

3.3.2 Dimension-Reduction Techniques

In the proposed approach, the *dimension-reduction* process is important for two main reasons: (a) it eliminates the noise that might perturb further analyses, and (b) it significantly reduces the training time of the BN. The techniques used are *Principal Component Analysis* (PCA) and *Exploratory Factor Analysis* (EFA). The experimental results show that the selection of a good dimension-reduction technique has a significant impact on the final model quality. In the original work proposed in [5], PCA was used. In this chapter, we changed the model by exploiting *Exploratory Factor Analysis* (EFA) as explained in the following paragraphs. Experimental results will show the benefits of using EFA with respect to PCA for the specific problem addressed. For a quantitative comparison the readers can refer to Sect. 3.4.4, Table 3.5.

Let γ be a characterization vector storing all data of an application run. This vector stores l variables to account for either the static, dynamic or both analyses. Let us consider a set of known application profiles A consisting of m vectors γ . The application profiles can be organized in a matrix P with m rows and l columns. Each vector γ (i.e. a row in P) includes a large set of characteristics, such as the instruction count per instruction type (for both static and dynamic analysis), information on the memory access pattern and information characterizing the control flow (e.g. the number and length of the basic blocks, average and maximum loop nesting, etc.). Many of these application characteristics (columns of matrix P) are correlated to each other in a complex way. A simple example of this correlation is the instruction mix information collected during the static analysis and the instruction mix information collected during the dynamic profiling (even though these are not completely the same). A less intuitive example is between the distribution of basic block lengths and data related to the instruction memory reuse distance. The presence of many correlated columns in P implies that the information stored in a vector γ can be well represented with a vector α of smaller size.

Both PCA and EFA are statistical techniques aimed at identifying a way to represent γ with a shorter vector α while minimizing the information loss. Nevertheless, they rely on different concepts for organizing this reduction [19, 20]. In both cases, output values are derived by applying the dimension reduction and are no longer directly representing a certain feature. While in PCA the components are given by a combination of the observed features, in EFA the factors are represent-

ing the hidden process behind the feature generation. In both techniques, the output columns cannot be called by their header name and are not directly observable.

In PCA, the goal is to identify a summary of γ . To this end, a second vector ρ of the same length of γ (i.e. l) is organized by a variable change. Specifically, the elements of ρ are obtained through a linear combination of the elements in γ . The way to combine the elements of γ for obtaining ρ is decided upon the analysis of the matrix P , and is such that all elements in ρ are orthogonal (i.e. uncorrelated) and are sorted by their variance. Thus the first elements of ρ (also named principal components) carry most of the information of γ . The reduction can be obtained by generating a vector α to keep only the first most significant principal components in ρ , because the least significant ones carry little information content. Note that principal components in ρ (thus in α) are not meant to have a meaning; they are only used to summarize the vector γ as a signature.

In EFA, the elements in the vector of reduced size α are meant to explain the structure underlying the variables γ , while α , represents a vector of *latent variables* that cannot be directly observed. The variables γ are expected to be a linear combination of the variables in α . In EFA, this relationship explains the correlation between the different variables in γ ; that is, correlated variables in γ are likely to depend on the same hidden variable in α . The relationship between the latent α and the observed variables is regressed by exploiting the maximum likely method based on the data in matrix P .

When adopting PCA, each variable in α tends to be a mixture of all variables in γ . Therefore, it is hard to tell what a component represents. When adopting EFA instead, the components α tend to depend on a smaller set of elements in γ that are correlated with each other. That is, when applying EFA, α is a compressed representation of γ , where elements in γ that are correlated (i.e. that carry the same information) are compressed into a reduced number of elements in α . Note that reducing the profile size by means of EFA results in a α that better describes the type of application under analysis in reference to PCA [21].

Consequently, having obtained γ through any of the characterization techniques, a pre-processing filtering should be applied to ensure that the least noise has come through and the final P is eligible to be summarized by EFA. That implies manually (i) removing the zero columns in l and (ii) removing the redundant columns of l given that no column l is a linear combination of another l . In contrast, the algorithmic approach is that P needs to be transformed, as to obtain the final γ in *positive-definite covariance* form [22]. Different techniques have been described in the literature on how to transform a *non-positive-definite* matrix to a *positive-definite* one which exceeds the scope of this chapter, but interested readers can refer to [23, 24] or use packages in *R* statistical tool [25] i.e., *nearPD* to compute nearest positive definite matrix.

3.3.3 Bayesian Networks

Bayesian Networks are powerful to represent the probability distribution of different variables that characterize a certain phenomenon. The phenomenon to be investigated in this work is the optimality of compiler optimization sequences.

Let us define a Boolean vector o , whose elements o_i are the different compiler optimizations. Each optimization o_i can be either enabled, $o_i = 1$, or disabled, $o_i = 0$. In this chapter, the *phase ordering* problem [26] is not taken into account. We rather consider how different optimizations o_i are organized in a predefined order embedded in the compiler. A compiler optimization sequence represented by the vector o belongs to the n size Boolean space $\mathcal{O} = \{0, 1\}^n$, where n represents the number of compiler optimizations under study.

An application is parametrically represented by the vector α of the k reduced components computed either via PCA or EFA from its software features. Elements α_i in vector α generally belong to the continuous domain.

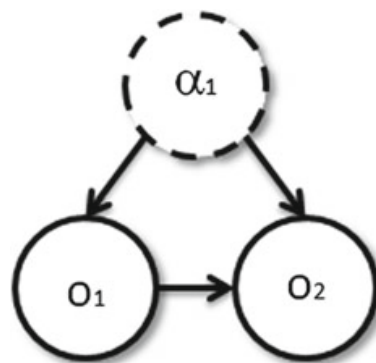
The optimal compiler optimization sequence $\bar{o} \in \mathcal{O}$ that maximizes the performance of an application is unknown. However, it is known that the effects of a compiler optimization o_i might depend on whether another optimization o_j has been applied in the optimization sequence. Additionally, it is known that the compiler optimization sequence that maximizes the performance of a given application depends on the application itself.

The reason why the optimal compiler optimization sequence $\bar{o}$ is unknown a priori is because it is not possible to capture, in a deterministic way, the dependencies among the variables in the vectors $\bar{o}$ and α . There is no way to identify an analytic model to exactly fit the vector function $\bar{o}(\alpha)$. As a matter of fact, the best optimization sequence $\bar{o}$ depends also on other factors that are somewhat outside our comprehension, *the unknown*. It is exactly to deal with *the unknown* that we propose not to predict the best optimization sequence $\bar{o}$ but rather to infer its probability distribution. The uncertainty stored in the probability distribution models the effects of *the unknown*.

As underlying probabilistic model, we selected *BN* because of the following features of interest for the target problem:

- Their expressiveness allows one to include heterogeneous variables in the same framework such as Boolean variables (in the optimization vector o) and continuous variables (in the application characterization α).
- Their capabilities to model *cause-effect* dependencies. Representing these dependencies is suitable for the target problem, as we expect that the benefits of some compiler optimizations (effects) are due to the presence of some application features (causes).
- It is possible to graphically investigate the model to visualize the dependencies among different compiler optimizations. If needed, it is even possible to manually edit the graph for including some a priori knowledge.
- It is possible to bias the probability distribution of some variables (the optimization vector o) given the *evidence* on other variables (the application characterization α).

Fig. 3.2 A Bayesian Network example



This enables us to infer an *application-specific distribution* for the vector o from the vector α observed by analyzing the target application.

A *Bayesian Network* is a direct acyclic graph whose nodes represent variables and whose edges represent the dependencies between these variables. Figure 3.2 reports a simple example with one variable α_1 representing the application features and two variables o_1, o_2 representing different compiler optimizations. In this example, the probability distributions of the two optimizations depend on the program features represented by α . Additionally, the probability distribution of o_2 depends on whether the optimization o_1 is applied. Dashed lines are used for nodes representing observed variables whose value can be input as evidence to the network. In this example, the variable α_1 can be observed and, by introducing its evidence, it is possible to bias the probability distributions of other variables.

Training the Bayesian model. Tools exist to construct *BN* automatically by fitting the distribution of some training data [27]. To do so, first the graph topology is identified and then the probability distribution of the variables including their dependencies is estimated.

The identification of the graph topology is particularly complex and time consuming. The *dimension reduction* technique applied on the SW features plays a key role in obtaining reasonable training times by limiting to k elements in the vector α , thus reducing the number of nodes in the graph.

For efficiency reasons, the algorithm used for selecting the graph topology is a heuristic algorithm, named *K2*, initialized with the *Maximum Weight Spanning Tree* (MWST) ordering method as suggested in the Matlab toolbox in use [27]. The initial ordering of the nodes for the MWST algorithm is given to let the elements α to appear first and then the elements of o . Even if the final topological sorting of the nodes changes according to the algorithm described in [28], by using this initialization criterion, it always happens that the dependencies are directed from elements of α to elements of o and not vice versa. When using the *K2* algorithm, the network topology is selected as follows. The graph is initialized with no edges to represent the fact that each variable is independent. Then, for each variable i , following their initial ordering, each possible edge from j to i (where $j < i$) is considered as a candidate to be added to the network. A candidate edge is added to the topology if it increases the probability that the training data were generated from the probability distribution

the new topology describes. This method has a polynomial complexity with respect to the number of variables involved and the number of lines in the training data set.

During the model training, we consider the *softmax* function for modeling the cumulative probability distribution of the Boolean elements in vector o [27]. This is a mathematical necessity to map in the *Bayesian framework* the dependencies of Boolean variables in o with respect to continuous variables in α . In particular, thanks to the use of *softmax* variables, we can express the conditional probability $P(o_i = b \mid \alpha_j = x)$, where o_i is a Boolean variable and α_j is a continuous variable.

The coefficients of the functions describing the probability distribution of each variable as well as their dependencies are tuned automatically to fit the distribution in the training data [27]. Training data are gathered by analyzing a set A of training applications (Fig. 5.1a). First, application features are computed for each application $a \in A$ to enable the principal component analysis. Thus, each application is characterized by its own principal component vector α . Then, an experimental compilation campaign is carried out for each application by sampling several compiler optimization sequences from the compiler optimization space $\mathcal{O}$ with a uniform distribution. For each application, we select the 15% best-performing compilation sequences among the sampled ones. The distribution of these sequences is learned by the *Bayesian Network* framework in relation to vector α characterizing the application.

Inferring an application-specific distribution. Once the *Bayesian Network* has been trained, the principal component vector α obtained for a new application can be fed as evidence to the framework to bias the distribution of the compiler optimization vector o . To sample a compiler optimization sequence from this biased distribution, we proceed as follows. The nodes in the direct acyclic graph describing the *Bayesian Network* are sorted in topological order, i.e. if a node at position i has some predecessors, those appear at positions j , $j < i$. At this point, all nodes representing the variables α appear at the first positions.¹ The value of each compiler optimization o_i is sampled in sequence by following the topological order such that all its parent nodes have been decided. Thus, the marginal probability $P(o_i = 0 \mid \mathcal{P})$ and $P(o_i = 1 \mid \mathcal{P})$ can be computed on the basis of the parent node vector value $\mathcal{P}$ (each parent being either an evidence α_j or a previously sampled compiler optimization o_j). Similarly, by using the maximum likelihood method, it is possible to compute the most probable vector from this biased probability distribution. When sampling from the application-specific probability distribution inferred through the *Bayesian Network*, we always consider finding the most probable optimization sequence as the first sample.

¹This is by construction due to the initialization of the MWST and the K2 algorithms used to discover the network topology.

3.4 Experimental Evaluation

The goal of this section is to assess the benefits of the proposed methodology. In this work, we run the experimental campaign on an ARMv7 Cortex-A9 architecture as part of a TI-OMAP 4430 processor [29] with *ArchLinux* and *GCC-ARM 4.6.3*.

3.4.1 Benchmark Suites

To assess the proposed methodology, we have used two major benchmark suites: (i) *cBench* [30], and (ii) *PolyBench* [31, 32]. Each consists of different classes of applications and kernels ranging from security and cryptography algorithms to office and image-processing applications. Readers can refer to Table 3.1 for the list of applications selected in the two benchmark suites.

3.4.1.1 cBench

The *cBench* suite [30] is a collection of open-source programs with multiple data sets assembled by the community to enable realistic workload execution and targeted by many different compilers such as *GCC*, *LLVM*, etc. The source code of individual programs is simplified to facilitate portability; therefore, it has been targeted in *autotuning* and *iterative compilation* research work. Of the available data sets for every individual kernel, we have selected five and sorted them in a way that dataset1 is always the smallest and dataset5 the largest. This ensures that for every kernel we have exposed enough of the input load to be able to measure fair runtime executions.

3.4.1.2 PolyBench

The *PolyBench* benchmark suite [31, 32] consists of benchmarks with static control parts. The purpose is to make the execution and monitoring of applications uniform. One of the main features of the *PolyBench* suite is that there is a single file per application, tunable at compile-time and used for kernel instrumentation. It performs extra operations such as cache flushing before the execution and can set real-time scheduling to prevent OS interference. We have defined two different data sets for each individual application to expose the main function with different input loads. PolyBench has a variety of benchmarks, i.e. 2D and 3D matrix multiplication, vector decomposition, etc. This suite is also suitable for parallel programming, which is beyond the focus of this work.

Table 3.1 Benchmark suites used in this work

cBench list	Description
(a) <i>cBench</i> applications selected for this work	
automotive_bitcount	Bit counter
automotive_qsort1	Quick sort
automotive_susan_c	Smallest univalue segment assimilating nucleus corner
automotive_susan_e	Smallest univalue segment assimilating nucleus edge
automotive_susan_s	Smallest univalue segment assimilating nucleus S
security_blowfish_d	Symmetric-key block cipher decoder
security_blowfish_e	Symmetric-key block cipher encoder
security_rijndael_d	AES algorithm Rijndael decoder
security_rijndael_e	AES algorithm Rijndael encoder
security_sha	NIST secure hash algorithm
telecom_adpcm_c	Intel/dvi adpcm coder/decoder coder
telecom_adpcm_d	Intel/dvi adpcm coder/decoder decoder
telecom_CRC32	32 BIT ANSI X3.66 crc checksum files
consumer_jpeg_c	JPEG kernel
consumer_jpeg_d	JPEG kernel
consumer_tiff2bw	convert a color TIFF image to grey scale
consumer_tiff2rgba	convert a TIFF image to RGBA color space
consumer_tiffdither	convert a TIFF image to dither noisespace
consumer_tiffmedian	convert a color TIFF image to create a TIFF palette file
network_dijkstra	Dijkstra's algorithm
network_patricia	Patricia Trie data structure
office_stringsearch1	Boyer-Moore-Horspool pattern match
bzip2d	Burrows—Wheeler compression algorithm
bzip2e	Burrows—Wheeler compression algorithm

(continued)

Table 3.1 (continued)

PolyBench list	Description
(b) Linear-algebra/applications of the <i>PolyBench</i> suite selected for this work	
2 mm	2 Matrix multiplications ($D = A \times B$; $E = C \times D$)
3 mm	3 Matrix multiplications ($E = A \times B$; $F = C \times D$; $G = E \times F$)
atax	Matrix transpose and vector multiplication
bicg	BiCG Sub Kernel of BiCGStab linear solver
cholesky	Cholesky decomposition
doitgen	Correlation computation
gemm	Matrix-multiply $C = \alpha A \times B + \beta C$
gemver	Vector multiplication and matrix addition
gesummv	Scalar, vector and matrix multiplication
mvt	Matrix vector product and transpose
symm	Symmetric matrix-multiply
syr2k	Symmetric rank—2k operations
syrk	Symmetric rank—k operations
trisolv	Triangular solver
trmm	Triangular matrix-multiply

3.4.2 Compiler Transformations

The compiler transformations analyzed have been reported in Table 3.2. We based our design space on the work of [3]. The authors implemented sensitivity analysis over a vast majority of the compiler optimizations and defined with a list of promising passes. Building upon their work, we selected the compiler optimizations with a speedup factor greater than 1.10. They are applied to improve application performance beyond the standard optimization level -O3 and have not yet been included in any prior optimization level. The optimizations can be enabled or disabled using their respective compiler optimization flags. The standard optimization level -O3 has been also used to collect the dynamic software-features for each application on both *training* and *inference* phases.

The application execution time has been estimated by using the Linux-perf tool. The execution time is done by averaging five loop-wraps of the specific compiled binary with one second of sleep in between five different executions of those loop-wraps. Therefore, in total, each transformed binary has been executed 25 times as five packages of five loop-wraps to ensure better accuracy of estimations and fairness among the generation of executions. This technique is used both in the *training* and the *inference* phases.

Table 3.2 Compiler optimizations under analysis (beyond -O3)

Compiler transformation	Abbreviation	Short description
-funsafe-math-optimizations	<i>math-opt</i>	Allow optimizations for floating-point arithmetic that (a) assume valid arguments and results and (b) may violate IEEE or ANSI standards
-fno-guess-branch-probability	<i>fn-gss-br</i>	Do not guess branch probabilities using heuristics
-fno-ivopts	<i>fn-ivopt</i>	Disable induction variable optimizations on trees
-fno-tree-loop-optimize	<i>fn-tree-br</i>	Disable loop optimizations on trees
-fno-inline-functions	<i>fn-inline</i>	Disable optimization that inline all simple functions
-funroll-all-loops	<i>funroll-lo</i>	Unroll all loops, even if their number of iterations is uncertain
-O2	<i>O2</i>	Overwrite the -O3 optimization level by disabling some optimizations involving a space-speed trade-off

3.4.3 Bayesian Network Results

In this work, Matlab environment [27, 33] has been used to train the *Bayesian Network*. We have used *Exploratory Factor Analysis* (EFA) of application features for the seven compiler optimization flags listed in Table 5.2. As stated in Sect. 3.3.2, one of the features of using EFA is that the factors are linear combinations that maximize the shared portion of the variance. Therefore, as a prerequisite, the covariance matrix should be positive definite. The pre-processing helps purify the highly correlated application characterization columns that are linearly correlated. In theory, *PCA* accepts any matrix ignoring the aforementioned condition and that is why we think applying factor analysis as our dimension reduction technique tends to obtain the most important factors and correlate them with the compiler optimizations. The decision on the numbers of factors has been derived from the *Kaiser* test [34]. The test implies taking only the factors having greater than 1 in the covariance matrix. In other words, the Kaiser rule is to drop all components with eigenvalues under 1, this being the eigenvalue equal to the information accounted for by an average single item. Table 3.3 reports the factors derived for each benchmark and characterization method.

Table 3.3 represents the number of features that have been produced both originally by the different feature selection techniques and by the Kaiser test. The third column is the *original number of features* and the last one refers to *range of selected factors* in each specific benchmark suite/feature selection method. Note that the last column reports the range of selected factors rather than a number as we have used cross-validation approach in the experimental campaign, thus different applications/datasets/feature selection techniques can result in a different number of factors to be used in COBAYN's framework.

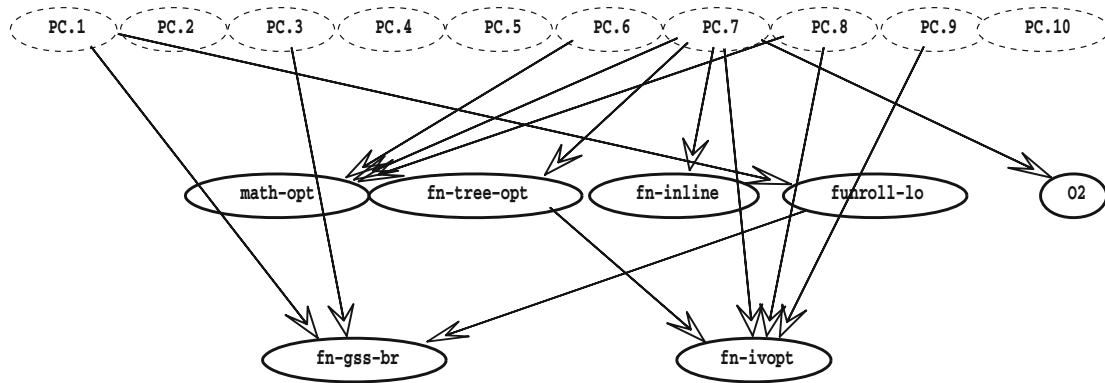
Table 3.3 Kaiser test results

Application	Characterization method	Original no. of factors	Range of selected factors by Kaiser test
cBench	MICA (Dynamic)	99	[7–11]
cBench	MILEPOST (Static)	53	[4–6]
cBench	Hybrid	143	[8–10]
polyBench	MICA (Dynamic)	99	[5–7]
polyBench	MILEPOST (Static)	53	[4–6]
polyBench	Hybrid	143	[4–5]

In this work, while training has been carried out using each application/dataset pair separately, the validation has been done through an application-level cross-validation (*Leave-One-Out* cross-validation, LOO). We train different BNs, each by excluding an applications (together with all its input dataset) from the training set.

Using *BN* enables us to investigate graphically the dependencies between the variables involved in the compiler optimization problem and to correlate them with the selected factors of the program characterization. We train a final *Bayesian Network* including all applications in the training set. The resulting network topology is a directed acyclic graph *DAG*, as shown in Fig. 3.3. By removing *security_rijndael_e* application from the training set, the graph topology slightly changes, mainly in terms of the different edges connecting the *Principal Components (PC)/program factors (FA)* nodes to the compiler optimization nodes. This is due to the change in the program features and its factors, which are computed in a different way. For the sake of conciseness, we do not report all graph topologies derived by the LOO technique for each individual trained *Bayesian Network*.

The nodes of the topology graph reported in Fig. 3.3 are organized in layers. The first layer reports the FAs that are the observable variables (reported as dashed lines). The second layer contains the compiler optimizations whose parents are the PC nodes (or FA nodes depending whether PCA or EFA is used). Therefore the effects

**Fig. 3.3** Topology of the *Bayesian Network* if *security_rijndael_e* is left out of the training set

of these compiler optimizations depend only on the application characterization in terms of its features. In the third layer, the compiler optimization nodes whose parents include optimization nodes from the second layer are listed. Once a new application is characterized for a target application data set, the evidence related to the PCs (or FAs) of its features is fed to the network in the first layer. Then, the probability distributions of other nodes can be inferred in turn on the second and third layers. There are two nodes in the third layer of Fig. 3.3. The first one is the *fn-gss-br* node that depends on *funroll-lo* because unrolling loops impacts the predictability of the branches implementing these loops. Moreover, *funroll-lo* impacts the effectiveness of the heuristic branch probability estimation, thus *fn-gss-br*. The second node in the third layer is the *fn-ivopt* node, which depends on *fn-tree-opt* as parent node in the second layer. Both these optimizations work on trees and therefore their effects are interdependent. While sampling compiler optimizations from the *Bayesian Network*, the decisions of whether to apply *fn-gss-br* and *fn-ivopt* are taken after deciding whether to apply *funroll-lo* and *fn-tree-opt*.

Table 3.4 shows the fine-grain breakdown of the timing when we use COBAYN framework. We have reported the time spent for each phase of the proposed technique, both on the training phase (done offline) and on the inference phase (done online). Constructing COBAYN's network is a one-time process and depends on the number of applications in the training set. The time needed to collect the training data is on the other side, depends not only on the number but also on the applications and data-set used for the training. The same applies to the time needed for compiling and executing the target application during the online compiler autotuning phase. To this end, Table 3.4 reports the numbers for each specific phase considering the Cbench as training set and Susan as the target application. During the offline training-phase, the time needed for data collection on the case of Cbench, is around 2 days. It includes the time needed for each benchmark to compile and execute, considering all set of configurations and the feature collection phase. The time needed to post-process the data and to generate the Bayesian Network model is around 70 s.

During the online phase (inference phase), the time needed for extracting the software features from the target application is 14.4 s while querying BN is less than 1 s. The compilation and execution-time on the target platform for Susan are 4.5

Table 3.4 COBAYN timing breakdown for offline training and online inference for *Susan* application

Phase	Tag and category	Time (s)
Offline training	(A) Offline data-collection	2 days
	(B) Construct BN	70 s
Online inference	(C) SW Feature collection	14.4 s
	(D) BN Inference	0.85 s
	(E) Susan compilation	4.5 s
	(F) Susan execution	8.9 s

and 8.9 s, respectively. Those numbers show that the initial overhead in adopting the proposed methodology on the user-side (composed of the software feature extraction and BN inference) is less than 2 compilation/executions pairs, for this specific example.

3.4.4 Comparison Results

It is well known that *Random Iterative Compilation* (RIC) can improve application performance compared with static handcrafted compiler optimization sequences [4]. Additionally, given the complexity of the *iterative compilation* problem, it has been proved that drawing compiler optimization sequences at random is as good as applying other optimization algorithms such as genetic algorithms or simulated annealing [3–5, 12]. Accordingly, to evaluate the proposed approach, we compared our results with (1) standard optimization levels -O2 and -O3 (2) the Random Iterative Compilation (RIC) methodology that samples compiler optimization sequences from the uniform distribution, and (3) two state-of-the-art methodologies coupling machine learning with an iterative methodology, and a optimization speedup predictor methodology, respectively.

The proposed methodology samples different compiler optimization sequences from the BN. The performance achieved by the best application binary depends on the number of sequences sampled from the model. In this section, the results of applying the proposed methodology using two benchmark suites with respect to standard optimization levels and the *random iterative compilation* have been reported. The performance speedup on the first comparison section is measured in reference to -O2 and -O3, which are the optimization levels available for GCC. In addition, we show the speedup of the proposed methodology with respect to our previous work [5].

3.4.4.1 Bayesian Networks Performance Evaluation

Table 3.5 reports COBAYN’s speedup achieved over the standard optimization levels of -O2 and -O3 and *Random Iterative Compilation* (RIC). The last column represents the average speedup achieved by revising our *Bayesian Network* engine and using the *Explanatory Factor Analysis* (EFA) described in the Sect. 3.3.2 with respect to PCA in [5]. Note that all speedup values have been averaged using harmonic mean. It is observed that in all categories, COBAYN outperforms standard optimization levels and the previous approach. The comparison with respect to the RIC has been reported by the harmonic average over the speedup data derived by dividing the COBAYN’s performance data by the RIC data in full space. It can be seen that dynamic feature selection brings best results followed by the hybrid and static method. However, in certain cases (cBench using hybrid SW features), it narrowly reaches the performance of dynamic feature selection.

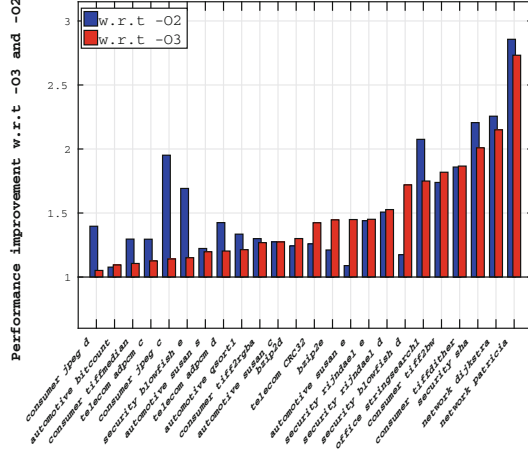
Table 3.5 COBAYN (BN using EFA) speedup w.r.t standard optimization levels (-O2 and -O3) and *Random Iterative Compilation* (RIC) and our previous approach of BN in [5] using PCA

Benchmarks	Features	COBAYN Speedup w.r.t			
		-O2	-O3	RIC	BN w/ PCA
Cbench	Dynamic	1.6093	1.528	1.2029	1.0744
Cbench	Static	1.5447	1.478	1.1143	1.0543
Cbench	Hybrid	1.5858	1.5066	1.2086	1.0654
Cbench average		1.5795	1.5035	1.1743	1.0617
PolyBench	Dynamic	1.9845	1.8387	1.3230	1.0921
PolyBench	Static	1.9353	1.8215	1.1518	1.0724
PolyBench	Hybrid	1.9441	1.7726	1.2333	1.1078
PolyBench average		1.9541	1.8101	1.2350	1.0901
Overall (Harmonic mean)		1.7669	1.6571	1.2052	1.0771

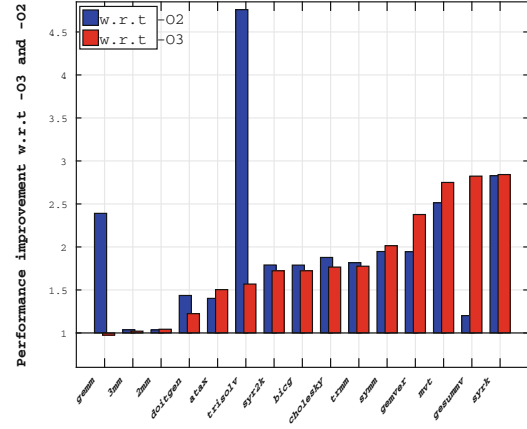
Using two major benchmarks and three different application characterization techniques, we report six different plots showcasing the benefits of the proposed methodology with respect to the GCC standard optimization levels. Figure 3.4, reports the speedups by considering a sample of eight different compiler optimization sequences. For each benchmark, the results have been averaged on the different data sets. All results have been sorted by the speedup values of -O3 and have been matched with their corresponding -O2 value. The bar plot is colored in blue and red, respectively, for the speedup achieved with respect to -O2 and -O3. All applications have achieved a speedup in reference to the performance of -O2 and -O3. This happens with the exception of *gemm* in reference to -O3 for static and hybrid feature-selection techniques and *consumer-jpeg-d* in reference to -O3 when using the dynamic method for feature selection. These applications reach their best performance using -O3 for two data sets out of five, and it was not possible to surpass this maximum by relying on the compiler transformations under consideration. On average for *Cbench*, the speedups are of **1.57** and **1.5** in reference to -O2 and -O3, respectively, and **1.95** and **1.81** for *PolyBench*. The maximum speedup observed is **3.1 $\times$** and **4.7 $\times$** . Table 3.5 reports the speedup gained using COBAYN compared with the standard optimization levels, *Random Iterative Compilation* (RIC) and our previous approach exploiting PCA as dimension-reduction method.

3.4.4.2 COBAYN's Portability Analysis

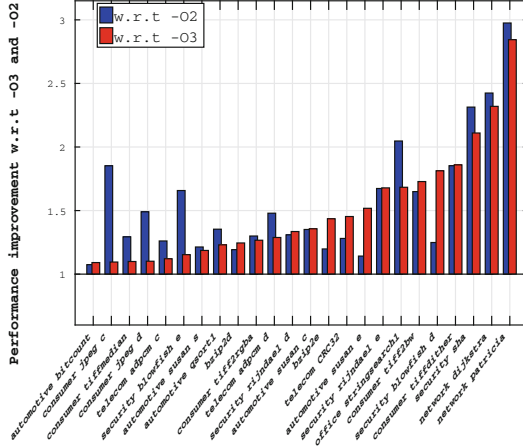
The results reported in this section are computed by means of LOO cross-validation on the two individual benchmark suites separately, one with 24 and the other with 15 applications. As the nature of these two benchmark suites is totally different, we believed it would be unfair to train on one and test on the other, so we analyzed the feasibility of mixing these applications in a fair heterogeneous set of *BigSet* so



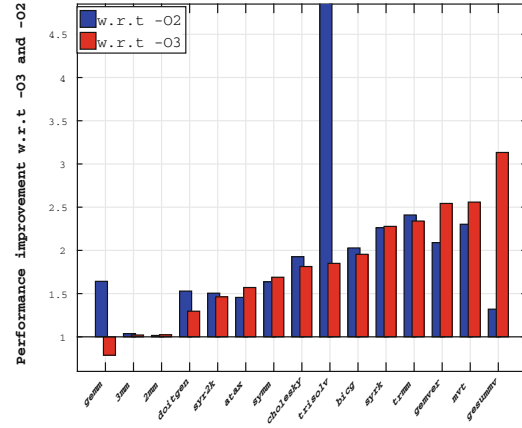
(a) BN with dynamic features on cBench



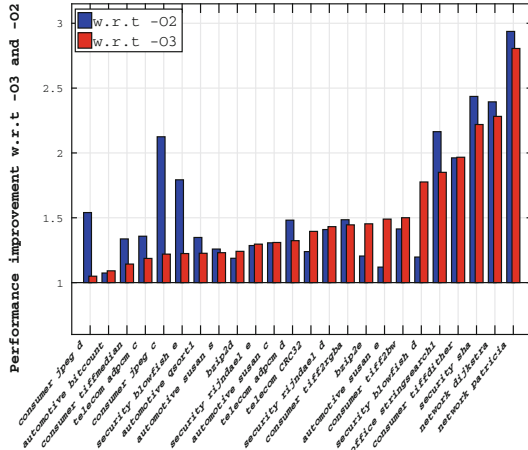
(b) BN with dynamic features on PolyBench



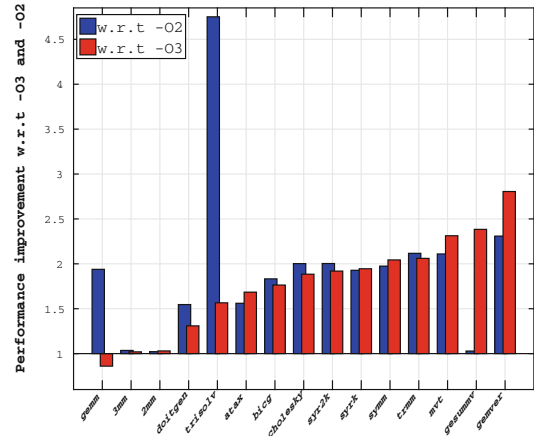
(c) BN with static features on cBench



(d) BN with static features on PolyBench



(e) BN with dynamic+static features on cBench



(f) BN with dynamic+static features on PolyBench

Fig. 3.4 Performance speedup w.r.t -O2 and -O3

Table 3.6 Evaluation of different BigSet formation in COBAYN Model construction. Note that COBAYN’s default refers to the version of COBAYN trained on a single benchmark set

BigSet combination		Speedup w.r.t. COBAYN’s default
Cbench	PolyBench	
24 (All)	15 (All)	1.1143
15	15	1.0743
10	10	1.0432
5	5	0.9896

that COBAYN’s engine gets evaluated. To this end, we tried 4 different scenarios, where the BigSet is obtained by: (i) including all 39 available applications, (ii) 15 applications of Cbench and 15 applications of PolyBench, (iii) selecting 10 Cbench and 10 PolyBench and finally (iv) 5 applications from each of those. Therefore, the BigSet was initialized with 39, 30, 20 and 10 different applications, and LOO cross-validation was carried-out. Table 3.6 reports the speedup gained in these scenarios. It is observed that COBAYN framework benefits from having (a) more applications, and (b) heterogeneous applications in the training set. The speedup listed in Table 3.6 is higher when BigSet accounts for more applications and, even just 10 applications per benchmark suite, it is higher than one (the default setting for the experimental results in this work refers to the COBAYN trained only on one of the two benchmark suites).

3.4.4.3 Performance Improvement

Let us define the *Normalized Performance Improvement* (NPI) as the ratio of the performance improvement achieved over the potential performance improvement:

$$NPI = \frac{E_{ref} - E}{E_{ref} - E_{best}} \quad (3.1)$$

where E is the execution time achieved by the methodology under consideration, E_{ref} is the execution time achieved with a reference compilation methodology and E_{best} is the best execution time computed through an exhaustive exploration of all possible compiler optimization sequences (in our case 128 different sequences). As the execution time E of the iterative compilation methodology under analysis gets closer to the reference execution time E_{ref} , the NPI gets closer to 0, reporting that no improvement is returned. In the same way, as E gets closer to the best execution time E_{best} , while NPI gets close to 1, reporting that the entire potential performance improvement has been achieved.

Figure 3.5 reports for six different benchmark/feature selection methods. The NPI achieved by the *proposed optimization* technique and by the RIC technique in refer-

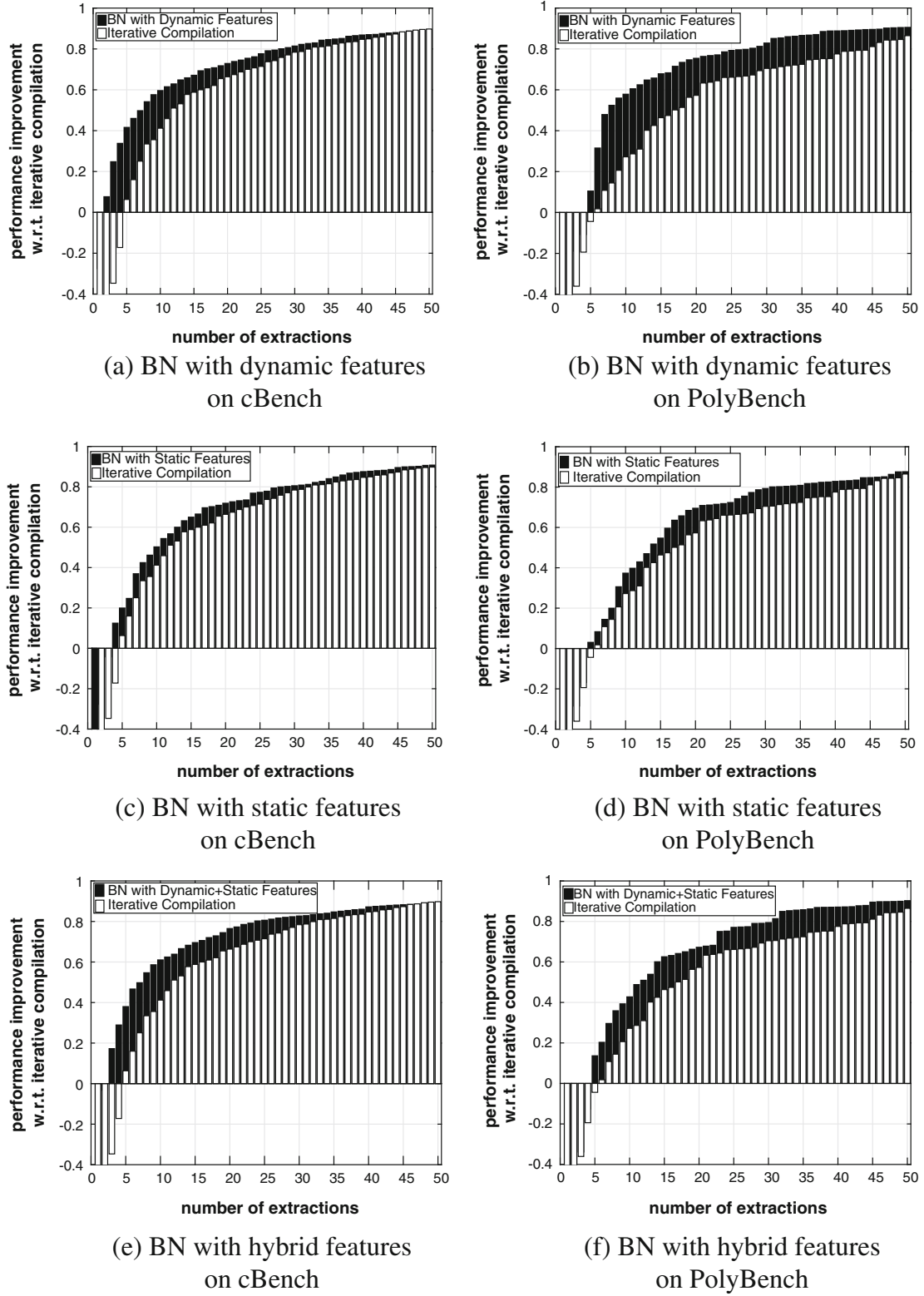


Fig. 3.5 Normalized performance improvement (NPI) w.r.t. RIC model

ence to the execution time obtained by -O3 (E_{ref}). It is noticeable that NPI has the upper-hand on performance on every number of extractions with respect to RIC. For readability purposes, we have only reported the first 50 extraction of the design space. The trend is continuously applied to the rest of the extractions until both get the maximum performance value of 1 at extraction no. 128, which accounts for the optimal compiler sequence given the specific application (also it is the optimal performance using exhaustive search). The comparisons reported in Fig. 3.5 were carried out by considering the same number of compiler optimization sequences sampled for both the RIC and the *proposed* approach. We acknowledge the fact that there is still room for improvement in future work. However, NPI figures show that in all cases, the proposed method was superior in terms of performance and that 30 extractions, on the current scale, reach 80% of the optimality.

3.4.5 A Practical Usage Assessment

When using *iterative compilation* in realistic cases, we need to decide how much effort should be spent on the optimization itself. This effort can be measured in terms of optimization time, which is directly proportional to the number of compilations to be executed. Thus, in this section, we evaluate the proposed optimization approach in terms of the application performance reached after a fixed number of compilations. In particular, we fix this number to eight which represents 6.25% of the overall optimization space. Our model has been compared with RIC and in Fig. 3.6, we report the *violin* plot for application speedup, while keeping the compilation effort of the proposed methodology to eight compilations (or extractions) and varying the compilation efforts of the RIC to explore more compiler space in the long run. Each individual distribution in Fig. 3.6 represents the performance of the proposed work with respect to *RIC* across different extractions. The red cross marks the *mean* and the green square marks the *median* of each violin distribution. It can be seen that the proposed methodology with BN inference achieved an at least $3\times$ reduction in exploration process effort compared with the same extraction of RIC. Here we define *exploration speedup* as the factor measuring the aforementioned metrics, enabling the researchers to traverse the compiler design space more efficiently.

Accordingly, by increasing the compilation efforts on *RIC*, while keeping the exploration efforts of the proposed approach constant, the application speedup of COBAYN decreases. On average, RIC needs 24-32 extractions to achieve the application performance obtained with eight extractions by COBAYN. This means that COBAYN provides a speedup of $3-4\times$ in terms of optimization efforts, that is only slightly impacted by the initial overhead (less than 2 evaluations) reported in Sect. 3.4.3. Furthermore, at the most extreme case, when RIC exhaustively enumerates and explores the full-space, 8 extractions of COBAYN, on average, still could gain up to 91% of the optimal solution. This is shown on the final distribution of each *violin* plot separated by a vertical dashed-line.

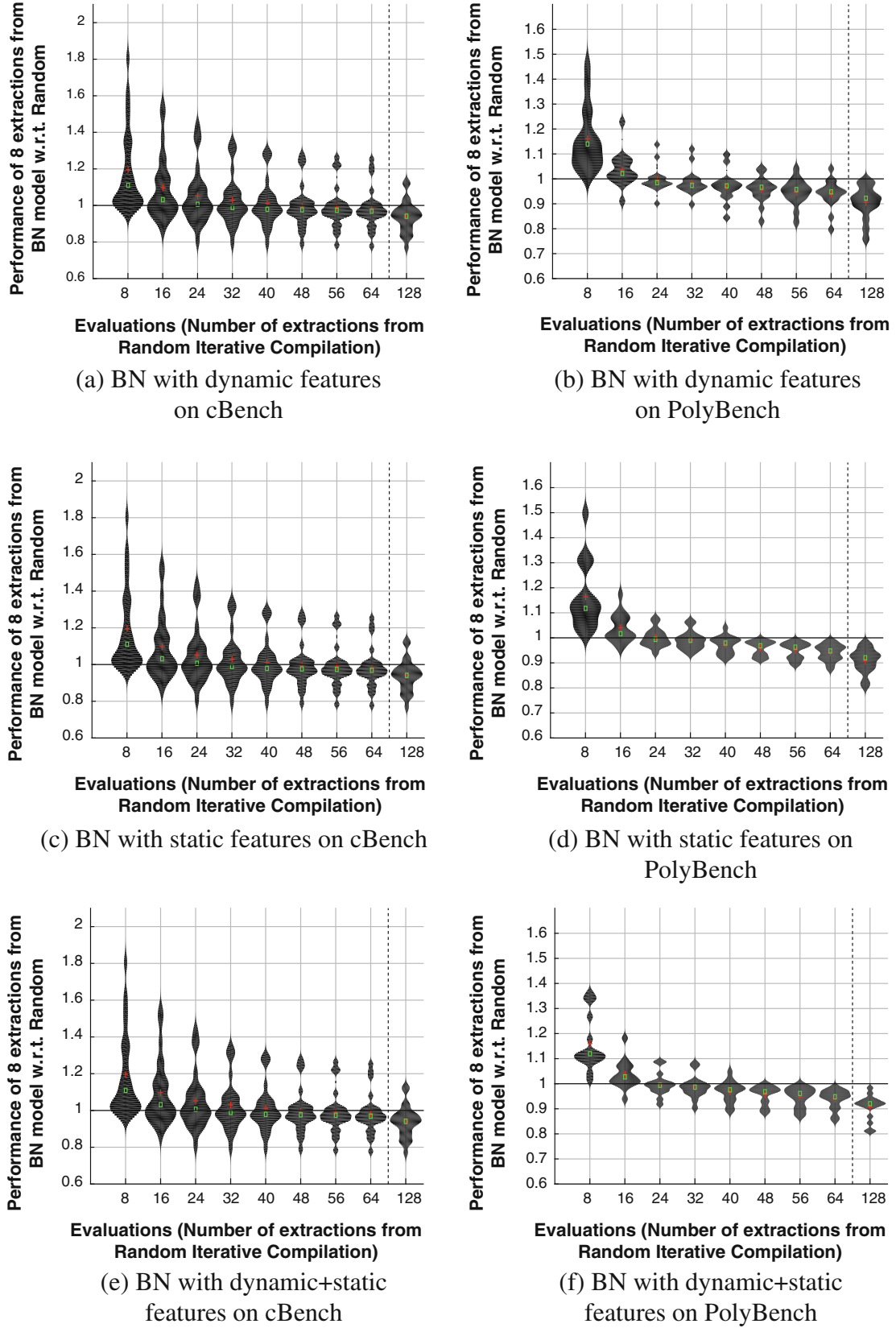


Fig. 3.6 Exploration speedup of 8 extractions w.r.t different evaluations of RIC

3.4.6 Comparison with State-of-the-Art Techniques

In this section, we compare the quality of the COBAYN results with respect to approaches that derived from (A) an *iterative compilation* and (B) a *non-iterative compilation* methodology.

3.4.6.1 Comparison to a Iterative Compilation Methodology

Agakov et al. [4] leveraged machine-learning models to focus on the exploration of the compiler optimization. Their methodology exploits a Markov chain oracle and an independent identically distributed (IID) probability distribution oracle. These two models learned offline bias certain optimizations over others and replace the uniform probability distribution we applied earlier for the RIC reference methodology. Their work reports significant speedup by coupling these machine-learning models with a nearest-neighbor-classifier. When predicting the probability distribution of the best compiler optimizations for a new application, the classifier first selects the training application having the smallest Euclidean distance in the feature vector space (derived by PCA). Then it learns the probability distribution of the best compiler optimizations for this neighboring application either by means of the Markov chain model or by using an IID model. These probability distribution learnt is then used as the predicted optimal distribution for the new application. It has been reported that the Markov chain oracle outperforms the IID oracle, followed by the RIC methodology using a uniform probability distribution.

We constructs the $P(S)$ probability matrix reported in Sects. 4.2 and 4.3 of [4] as:

$$P(S_{IID}) = s_1, s_1, \dots, s_L = \prod_{i=1}^L P(s_i) \quad (3.2)$$

$$P(S_{Markov}) = P(s_1) \prod_{i=2}^L P(s_i | s_{i-1}) \quad (3.3)$$

where $P(S_{IID})$ and $P(S_{Markov})$ define the probability of the specific sequence with IID and Markovian property for the optimization $t_1, t_1, \dots, t_L$. Using LOO cross-validation, we find the closest neighbor for each cBench application trained by the two oracles, and we sample from their probability distributions. To comply with the original work in [4], we consider only the five most relevant principal components PCs and account only for the *static program features* (also when applying COBAYN). The results are depicted in Fig. 3.7. It shows that COBAYN is faster in reaching higher speedup values. The results are scaled and normalized with respect to -O3 by using the NPI value (Eq. 5.5). COBAYN is able to capture a more realistic probability matrix of the compiler optimization problem and achieves with faster convergence

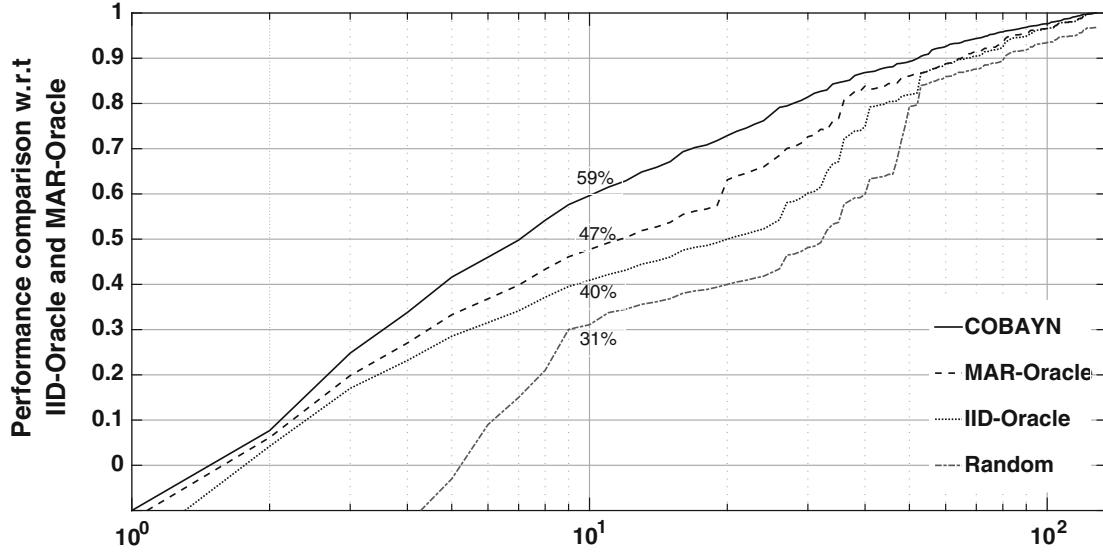


Fig. 3.7 NRI-scales speedup comparison of COBAYN with IID-oracle and MAR-oracle reported in [4]

towards the optimal result. It brings $1.25\times$ and $1.47\times$ speedup with respect to IID and the Markov oracle, respectively.

3.4.6.2 Comparison to a Non-iterative Compilation Methodology

Park et al. [35] used a polyhedral compiler framework capable of predicting the speedup for an unseen application. They used certain loop-optimizations in their *design-space* and surfed the *full-search* of the space. They reported the average speedup gained with respect to standard optimization O3 by using different machine-learning models on WEKA [36] machine-learning environment.

(i) Their predictive models are based on performance counters that are collected from the underlying architecture while running the applications. Therefore, the program features to be exposed to the model are architecture dependent, and the model loses its portability when it is used for a different architecture.

(ii) We also explore a different compiler optimization space. They have explored polyhedral optimization space including loop transformations, whereas we focus on GCC optimization space including loop transformations and other optimizations such as *inlining*, *math optimizations*, etc.

(iii) Furthermore, our model is based on a statistical analysis and BN, whereas they use a different set of machine-learning techniques, namely, predictive models. [35].

Nonetheless, we compare their methodology by applying it to the problem at hand. Table 3.7 reports the results obtained by using the machine-learning models in [35] on our compiler optimization space. We reproduced the data on both 1-shot and 8-shot scenarios to conform with the number of inference (predictions) COBAYN has in the current work. We use the Harmonic mean to average the speedup here.

Table 3.7 COBAYN speedup w.r.t. the average speedup gained with predictive modeling in both 1-shot and 8-shot scenarios reported in [35]

Algorithm and parameter configuration	COBAYN Speedup	
	w.r.t 1-shot	w.r.t 8-shot
LR -S 0	1.7551	1.6673
LR -S 1	1.7590	1.6710
LR -S 2	1.7191	1.6331
SVM NormalizedPolykernel -C 1.0 -E 8.0	1.5437	1.4665
SVM RBFKernel -C 2.0 -G 0.0	1.5206	1.4445
SVM RBFKernel -C 2.0 -G 25.0	1.5082	1.4327
SVM RBFKernel -C 2.0 -G 50.0	1.5045	1.4292
SVM RBFKernel -C 2.0 -G 75.0	1.5029	1.4277
SVM RBFKernel -C 2.0 -G 30.0	1.4927	1.4180
SVM RBFKernel -C 4.0 -G 30.0	1.5073	1.4319
SVM RBFKernel -C 0.01 -G 30.0	1.5073	1.4374
SVM RBFKernel -C 4.0 -G 50.0	1.5045	1.4292
IBk -K 1	1.4447	1.3724
IBk -K 2	1.4667	1.3933
IBk -K 5	1.4887	1.4142
M5P -M 1.0	1.4281	1.3566
M5P -M 2.0	1.4282	1.3567
M5P -M 4.0	1.4282	1.3568
M5P -M 10.0	1.4575	1.3846
M5P -M 50.0	1.4913	1.4167
K* -B 0 -M a	1.5192	1.4432
K* -B 20 -M a	1.5216	1.4455
K* -B 25 -M a	1.5258	1.4495
K* -B 50 -M a	1.4740	1.4003
K* -B 75 -M a	1.4737	1.4001
K* -B 100 -M a	1.5172	1.4413
K* -B 0 -M n	1.5208	1.4447
K* -B 20 -M n	1.5216	1.4455
K* -B 25 -M n	1.5258	1.4495
K* -B 50 -M n	1.4740	1.4003
MLP -L 0.3 -N 500 -H a	2.0435	1.9413
MLP -L 0.05 -N 500 -H a	1.6738	1.5901
MLP -L 0.1 -N 500 -H a	1.7246	1.6383
MLP -L 0.5 -N 500 -H a	1.8138	1.7231
MLP -L 0.9 -N 500 -H a	1.5250	1.4487

(continued)

Table 3.7 (continued)

Algorithm and parameter configuration	COBAYN Speedup	
	w.r.t 1-shot	w.r.t 8-shot
MLP -L 0.4 -N 500 -H a	1.9426	1.8454
MLP -L 0.5 -N 1000 -H a	1.8535	1.7608
MLP -L 0.5 -N 1500 -H a	1.7388	1.6518
MLP -L 0.5 -N 500 -H t	1.5579	1.4801
AVERAGE (Harmonic mean)	1.5622	1.4841

Note that Harmonic mean is always less than or equal to the arithmetic mean [37]. In all cases, COBAYN outperforms the reference methodology; specifically we have at least $1.3\times$ and up to $2.04\times$ speedup compared with the best achieved results reported in [35].

3.5 Conclusions

This chapter presented COBAYN, a methodology to infer by means of a *Bayesian framework* the best compiler optimizations to be applied for optimizing the performance of a target application. The methodology uses target independent software features to sample a statistical model built using Bayesian Networks to extract a set of suitable compiler configurations. Feature reduction techniques have been adopted to reduce the complexity and training time of the Bayesian model while also eliminating possible noise in the data and improving the quality of the results. The proposed approach has been evaluated on an ARM-based platform, using GCC compiler. The experimental results demonstrated that the proposed technique outperforms both standard optimization levels and state-of-the-art iterative and not iterative compilation techniques while using the same number of evaluations.

In the following chapter, we move towards tackling the phase-ordering problem of compiler optimizations using machine learning.

References

1. Palermo G, Silvano C, Zaccaria V (2005) Multi-objective design space exploration of embedded systems. *J Embed Comput* 1(3):305–316
2. Hoste K, Eeckhout L (2008) Cole: compiler optimization level exploration. In: *Proceedings of the 6th annual IEEE/ACM international symposium on code generation and optimization*, pp 165–174
3. Chen Y, Fang S, Huang Y, Eeckhout L, Fursin G, Temam O, Chengyong W (2012) Deconstructing iterative optimization. *ACM Trans Archit Code Optim (TACO)* 9(3):21

Chapter 4

The Phase-Ordering Problem: An Intermediate Speedup Prediction Approach

Abstract This chapter presents the first of two methods to tackle the phase-ordering problem of compiler optimizations. Here, we present an intermediate speedup prediction approach followed by a full-sequence prediction approach in the next chapter and we show pros and cons of each approach in detail. Today's compilers offer a vast number of transformation options to choose among, and this choice can significantly impact on the performance of the code being optimized. Not only the selection of compiler options represents a hard problem to be solved, but also the ordering of the phases is adding further complexity, making it a long-standing problem in compilation research. This chapter presents an innovative approach to tackling the compiler phase-ordering problem by using predictive modeling. The proposed methodology enables (i) to efficiently explore compiler exploration space including optimization permutations and repetitions and (ii) to extract the application dynamic features to predict the next-best optimization to be applied to maximize the performance given the current status. Experimental results are done by assessing the proposed methodology with utilizing two different search heuristics on the compiler optimization space and it demonstrates the effectiveness of the methodology on the selected set of applications. Using the proposed methodology on average we observed up to 4% execution speedup with respect to LLVM standard baseline.

4.1 Introduction

Selecting the best ordering of compiler optimizations for an application has been an open problem in the field for many decades and the problem is known to be NP-hard. The unrealistic exhaustive search is the only solution that seems appealing to achieve the optimal solution. Compiler researchers rely on their insights on the compiler *backend* to come up with some predefined sequences and ordering. This process is usually done tentatively and the selected pass is constructed with little insight on the interaction between the selected compiler options. However, to come up with an optimal solution, researchers might have to spend several years to run different code variants and this is simply unfeasible, given the growing design space composed of different architectures and software models that rely on modern com-

pilers frameworks. As an example, GCC compiler has more than 200 compiler passes and LLVM-OPT has more than 100, and these optimizations are working on different layers of application e.g. *analysis passes*, *loop-nest passes*, etc. Most of the passes are usually turned off by default and compiler developers rely on software developers to know which optimization can be beneficial for their code. The so-called *average case* has been defined as to get certain *standard optimization levels*, e.g. O1, O2, Os, etc. to introduce a fix sequence of compiler options, that on average can bring good results for most applications. Given the peculiarity of the problem, this certainly is not enough.

Exploiting compiler optimizations in application-specific *embedded domains*, where applications are compiled once and then deployed on the market on millions of devices is troublesome. The reason why is because embedded systems are usually designed with tight extra-functional properties constraints. Second, the large variety of embedded platforms cannot be faced with the average case provided by standard optimization levels, thus custom compiler optimization sequences might lead to substantial benefits in reference to several performance metrics (e.g. execution time, power consumption, memory footprint).

In the *High Performance Computing* (HPC) domain, parallel computer systems are increasingly more complex. Currently, HPC systems offering a peak performance of several Petaflops have hundreds of thousands of cores to be managed efficiently. Those machines have deep software stack, which has to be exploited by the programmer to tune the program. Moreover, to reduce the power consumption of those systems, advanced hardware and software techniques are applied, such as the usage of GPUs that are highly specialized for regular data parallel computations via simple processing cores and high bandwidth to the graphics memory. Numerous scientific and engineering compute-intensive applications spend most of their execution time in loop nests that are suitable for high-level optimizations. Typical examples include dense linear algebra codes and stencil-based iterative methods [25]. *Polyhedral compilation* is a recent attempt to bring mathematical representation focusing on the loop-nest of the *polyhedral model* including many different tools [4, 5, 16, 20].

In this chapter, we tackle the phase-ordering problem by using predictive modeling. Our predictive model can predict the next-best compiler optimization to apply given the current status of the application under analysis. The status of the application is defined by a *vector of representative features* that has been collected dynamically and it is independent from the architecture the code is running on. We call this approach an intermediate speedup prediction as it finds the best available optimization to apply at each intermediate state of an application. The proposed predictive model has been trained off-line with different permutations of the compiler flags (allowing repetitions and dynamic sequence length). Thus, the proposed method receives as input the *program features* and it generates the next-best compiler option to maximize the performance of the application. We selected a set of benchmark applications to assess the benefits of the proposed approach and to prove its feasibility.

In this chapter, we propose a predictive modeling methodology to mitigate the phase-ordering problem, In particular, the main contributions are:

- Predictive modeling methodology capable of capturing the correlation between the program features and the compiler optimization at each state.
- The integration of the predictive modeling within a compiler framework. The generated model is trained by means of Machine Learning to focus on the next-immediate best compiler optimization to be applied given the current status of the application for any new previously unobserved program.
- Tackling the phase-ordering problem on utilizing different relative positioning of the sequences of compiler options previously acquired as good sequences from LLVM standard optimization level and explore the design space by using a larger set of compiler flags rather than the individual options.
- Intermediate speedup predictive modeling, capable of iteratively predicting the next-best compiler option using two search heuristics to be applied given the current status of the application under optimization.

We apply prediction modeling techniques originally proposed in [9] for selecting the best compiler optimizations. However, the original work was mostly performing predictions on fixed optimization vectors length, while our proposed model is able to iteratively call the function and generate the next-best optimization to be applied, given the current status of the application. This feature is certainly vital for the phase-ordering problem because of: (i) it opens up to complete the new states towards exploring more regions of interest in the design space and (ii) it enables us to apply *repetitions* on the application being optimized. Moreover, the original work was tackling the problem of *selection of best compiler optimization*, while the current work is targeting the substantially harder problem of *phase-ordering*.

The rest of the chapter has been organized as follows. Section 4.2 provides a brief discussion on the related work. In Sect. 4.3, we introduce the predictive modeling approach to tackle *phase-ordering*. Section 4.4 presents experimental evaluation of the proposed methodology on an Unix-based Intel platform. Finally, Sect. 4.5 summarizes the outcome of the work and some future paths.

4.2 Related Work

We have extensively presented the literature in the Chap. 1. However, for the sake of completeness, we reiterate on a few important related work here. *Phase-ordering* problem is closely tightened with the *selection of the best compiler options* problem. Therefore, study on the literature can be classified in two main classes: (i) *autotuning and iterative compilation approaches* [1, 3, 7, 8] and (ii) *applying machine learning to compilation* [2, 24]. Nevertheless, these two approaches have been amalgamated in many ways by exploiting different techniques and methodologies.

There are quite a few studies that have tackled the *phase-ordering* problem. Authors in [19] have applied *Neuro-Evolution for Augmenting Topologies* (NEAT) on Jikes dynamic compiler and come up with sets of good ordering of phases. Other works have approached the problem by exploiting compiler backend optimizations

and using statistical tests to reduce code-size [12]. Authors in [18] exhaustively exploring the ordering space at functions' granularity level and evaluate their methodology with search tree algorithms and in [22] the authors have exploited iterative compilation with the information relying on relative passes in previously generated compiler options in the sequence in function level.

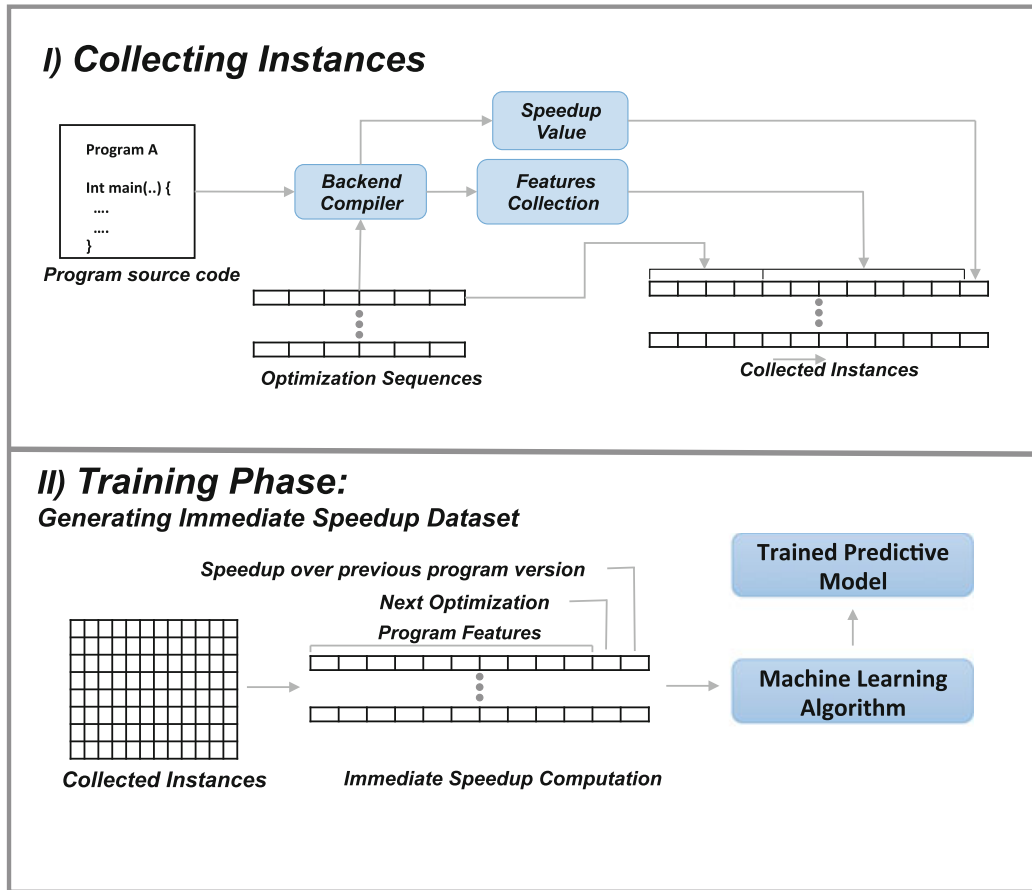
Our approach is rather different with respect to the literature, given that we propose a predictive modeling methodology utilizing an independent micro-architecture characterization features for all different *permutations with repetitions* of the compiler options and come up with the prediction of the *next-best compiler option to be applied* on the application given the current status. This is called an intermediate speedup predictor and is able to predict good set of compiler options even with *dynamic lengths* and it is not just limited to fixed vector length. We use previously acquired relative positioning of the promising sequences utilized on LLVM standard optimization levels and treat each of those acquired sequences as one whole. In this case, we could apply phase ordering feasibility on a larger set of compiler options, while generating less design space in the problem.

4.3 The Proposed Methodology

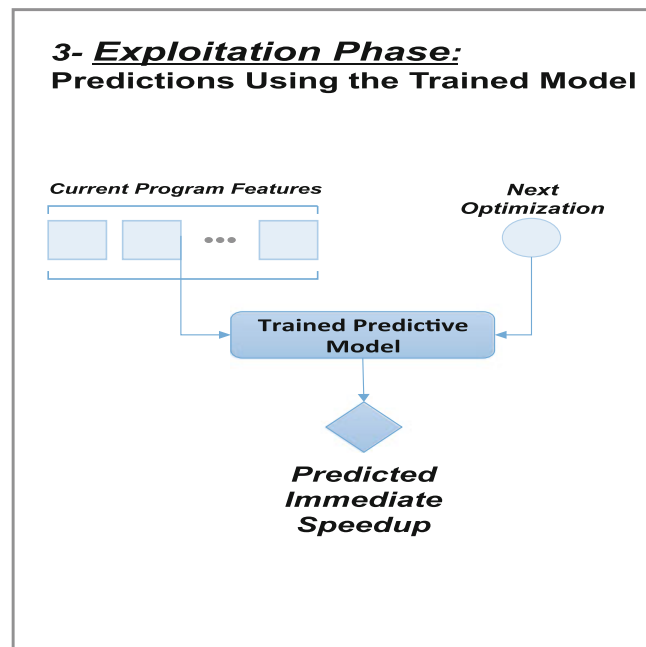
Main goal of the proposed methodology is to identify the feasibility of tackling the phase ordering problem using a predictive modeling methodology. Each application optimized with a unique compiler options sequence is passed through a characterization phase, that generates a parametric representation of its dynamic features. A model based on predictive modeling correlates these features to the compiler optimizations applied such as to predict the application speedup by using the next-best compiler optimization at each level.

The optimization flow is represented in Fig. 4.1. It consists of three main phases: (1) *Data collection* where different instances of the application are executed and an application characterizations is collected with the speedup achieved by utilizing the specific compiler pass, (2) *Training phase* where a predictive modeling is learned on the base of a set of training applications, and (3) *Exploitation phase*, where new applications are optimized by exploiting the knowledge stored in the trained predictive model. The model is able to predict, given the current program characterization, the intermediate speedups associated with each of the compiler optimization under analysis.

During the second and the third phases, an optimization process is necessary to identify the best compiler optimizations to be enabled to get the best performance. This is done for learning purposes during the *training phase* and for optimization purposes during the *exploitation phase*. To implement the optimization process, a Design Space Exploration (DSE) engine has been used. This DSE engine compiles, executes and measures application performance by enabling and disabling different permutations with repetitions of compiler optimizations.



(a) Data Collection and Training



(b) Testing Prediction Model

Fig. 4.1 Proposed framework: offline-training phase which is done once, and online-prediction phase for optimizing new unseen applications

In our approach the DoE is obtained by *exhaustive exploration* including all permutations with repetitions of compiler configurations (during the training phase as shown in Fig. 4.1) either by means of the *whole sequence* at once or *the current compiler optimization* that has been applied to the previous state. On the other side, *exploitation phase*, they are generated by means of predicting the *whole sequence* at once or as *the next-best configuration* to be applied given the current status. These two different techniques will be elaborated more in Sects. 4.3.3.1 and 4.3.3.2 respectively.

4.3.1 Compiler Phase-Ordering Problem

We have formulated the optimization space of the phase-ordering problem in Chap. 1 in details. Here is a quick recap of what we discussed earlier in this book.

The *phase-ordering* problem, let us define a Boolean vector o whose elements o_i are the different compiler optimizations. A *Phase-ordering* compiler optimization sequence represented by the vector o belongs to the n dimensional factorial space $|\mathcal{O}phases| = n!$, where n represents the number of compiler optimizations under study. However, the mentioned bound is for a simplified *phase-ordering* problem given fixed vector length without repetitions. Enabling repetitions and dynamic length will expand the design space size to:

$$|\mathcal{O}phases_repetition| = \sum_{i=0}^m n^i \quad (4.1)$$

where n is the number of interesting optimizations under study and m is the maximum desired length for the optimization sequence length. In this case, assuming the same n and m equal to 10, $|\mathcal{O}phases_repetition|$ will drive up to more than 11 *Billion* different configurations to select per each application.

The o_i in this work consists more than one single compiler optimizations. These set of optimizations are derived from the LLVM standard optimization level. Reader can refer to the specific o_i in Sect. 4.4; Table 4.2. We treat each of the whole sequence (which being referred to as *genes*) as a discrete variable, so that each optimization o_i can be either enabled $o_i = 1$ or disabled $o_i = 0$ and enabling the o_i will enable all its contained sub-optimizations respectively.

4.3.2 Application Characterization

For our predictive modeling presented in this chapter, again we exploit PIN [21] based *dynamic profiling* framework to analyze the behavior of the different applications at execution time. Refer to the previous chapters for more information on how MICA can characterize an application.

Upon collecting the features from an application, we use PCA. To reiterate, PCA is a technique to transform a set of correlated parameters (application features) into a set of orthogonal (i.e. uncorrelated) principal components. The PCA transformation aims at sorting the principal components by descending order based on their variance [15]. For instance, the first components include the most of the input data variability, i.e. they represent the most of the information contained in the input data. To reduce the number of input features, while keeping most of the information contained in the input data, it is simply needed to use the first k principal components as suggested in [14]. In particular, we set $k = 10$ to trade off the information stored in the application characterization and the time required to train the *predictive modeling*.

4.3.3 Intermediate Speedup Prediction

We present our prediction approach in this section. We call this approach *intermediate* speedup prediction. The general formulation of the optimization problem is to construct a function that takes as input the features of the *current status* of a program being optimized to generate as output the *the next-best optimization* to be applied that maximize the *immediate predicted speedup*. We used the prediction model originally proposed in [9]. However, the original work was mostly performing predictions on fixed optimization vectors length, while our proposed model is able to iteratively call the function and generate the next-best optimization to be applied, given the current status of the application. This feature is certainly vital for the phase-ordering problem because of: (i) it opens up to complete the new states towards exploring more regions of interest in the design space and (ii) it enables us to apply *repetitions* on the application being optimized.

An application is parametrically represented by the vector ρ , whose elements α_i are the first k principal components of its dynamic profiling features. Elements α_i in the vector ρ generally belong to the continuous domain. The optimal compiler optimization sequence $\bar{o} \in \mathcal{O}$ that maximizes the performance of an application is generally unknown. However, it is known that the effects of a compiler optimization o_i might depend on whether or not another optimization o_j is applied.

Our models predict optimizations to apply to unseen programs that were not used in training the model. To this purpose, we need to feed as input a characterization of the unseen program. The model is able to predict the speedup of each possible optimization set $\mathcal{O}$ in our predictive optimization space, given the characteristics of the unseen program. We order the predicted speedups to determine which optimization set is predicted best, and we apply the predicted best optimization set(s) to the unseen program. In the experimental Sect. 4.4, we use a leave-one-benchmark-out cross-validation procedure for evaluating the models. The proposed predictive modeling is going to introduce two different heuristics on predictive modeling.

4.3.3.1 DFS Search Heuristic

Depth-First Search (DFS) and its optimized version Depth-First Iterative Deepening [17] are well-known tree traversing algorithms. DFS starts at the root and explores as far as possible along each branch before backtracking. Adapting the heuristic on the current problem, we propose to start from an empty optimization sequence o_o . Considering sequence o_i , for each of the possible optimizations we predict the immediate speedup δ_i derived from applying o_i after o_i in the compilation process. The *Intermediate speedup* is computed as:

$$e_i = \text{Exec_time}(o_i) / \text{Exec_time}(o_j) \quad (4.2)$$

where e_i is the ratio between the execution time of the program compiled using o_i and the execution time of the version of the program generated using o_j . We define o_j as o_i followed by o_i .

Then, we order the possible optimizations by the value of the predicted immediate speedup δ . If none of the optimizations o_i to be explored has an associated immediate speedup δ_i greater than 1, we choose o_i as the next sequence to test, and we use it to compile the program and measure corresponding execution time. Otherwise, we repeat the same exploration process starting from sequence o_j , that is o_i followed by the still unexplored o_i maximizing predicted immediate speedup δ_i .

Once all the possible optimizations o have been explored, and the original sequence o_i tested, the algorithm backtracks to the previously considered sequence o_k , that is, o_i without its last optimization. If a sequence o_i has reached the maximum optimization sequence length N to be considered, we stop applying further optimizations after it. We then evaluate o_i it and backtrack to the previous node. The algorithm explores the complete optimization space using this policy and terminates when reaching the backtracking point for the initial empty sequence o_o .

4.3.3.2 Exhaustive Search Heuristic

The second iterative approach is tackling the exploration with exhaustive search. A model trained using machine learning techniques produces speedup predictions for all the configurations in the complete considered sequence space. Ordered by decreasing predicted speedup values, the sequences are then applied to the program, and their actual speedup is measured. This approach has been successfully used in selecting of the best compiler sequences problem, but lacks of applicability in the phase-ordering problem, given the complexity increase of the configuration space.

In our specific case, we modify this methodology to adapt it to our application scenario. In particular, as described previously at Sect. 4.3.3.1; Eq. 4.3, the model we trained is able to predict only intermediate speedups δ (i.e. the speedup of o_j over o_i , where o_j is the optimization sequence obtained by applying o_i after o_i). We are able to predict the actual speedup of optimization sequence o by *Multiplying* the immediate speedups δ_i predicted at each optimization $o_i \in o$:

$$o_o : \prod_{O_i \in o} \delta(o_i) \quad (4.3)$$

where o_i is each individual optimization options to be explored. The proposed exhaustive exploration computes the immediate speedups starting from the initial empty sequence o_o to the complete optimization space. In this way, we are able to predict the speedup of every optimization sequence $o \in \mathcal{O}$ and map our system to the classic exhaustive predictions methodology we described.

4.4 Experimental Evaluation

We assess the proposed methodology of the *intermediate speedup* on quad-core Intel-Xeon E1607 running at 3.00 GHz. We have used LLVM compilation tool v3.8 within our framework. A subset of Cbench benchmark suite [10] consisting of six different applications has been integrated within the framework. The list of selected applications has been reported in Table 4.1. Table 4.2 presents the utilized sets of LLVM optimizations categories in 4 different genes. The utilized passes are part of the LLVM standard optimization levels and we exploited the phase-ordering scenario having them relatively fixed internally while altering the whole sequence externally at each phase. In this mode, we speculated that we could explore more interesting regions of the design space thus being able to reach higher potential speedups. The utilized 4 different genes, consists of 30 compiler optimizations in total and 13 unique optimizations. One of the features of the proposed methodology is that it supports repetitions in the compiler design space. Table 4.3 represents the maximum achievable speedup enabling repetition feature on every individual application. We observe that excluding two applications that coincidentally had their gene having the best achievable speedup without repetitions (thus enabling repetition was converging to the very same result), the other four are gaining benefits from enabling this features. We used harmonic mean to better report the average of the speedups rather than arithmetic mean here [13].

Table 4.1 Applications used in this work

Applications	Description
automotive_bitcount	Bit counter
automotive_qsort1	Quick sort
automotive_susan_c	Smallest univalue segment assimilating nucleus corner
automotive_susan_e	Smallest univalue segment assimilating nucleus corner
network_dijkstra	Dijkstra's algorithm
network_patricia	Patricia trie data structure

Table 4.2 Compiler optimizations under analysis: derived from LLVM-opt

Gene	Abbreviation	Relative positioning of the optimizations
A	<i>domtreeRULE</i>	-domtree -memdep -dse -adce -instcombine -simplifycfg -domtree -loops -loop-simplify -lcssa -branch-prob
B	<i>simplifycfgRULE</i>	-simplifycfg -reassociate -domtree -loops -loop-simplify
C	<i>memdepRULE</i>	-memdep -domtree -memdep -gvn -memdep -memcpypopt -sccp
D	<i>loopsRule</i>	-loops -loop-simplify -lcssa -branch-prob -block-freq -scalar-evolution -loop-vectorize
Total number of optimizations		30
Unique number of optimizations		13

Table 4.3 Best found optimization passes by enabling repetition

Application	With repetition	Without repetition	Speedup (%)
automotive-susan-c	CDD	CD	9.34
network-patricia	CDCD	AD	60.37
automotive-qsort1	CBA	CBA	–
automotive-bitcount	CCBB	CDB	2.41
network-dijkstra	ACAB	AD	36.59
automotive-susan-e	CD	CD	–
Harmonic mean			7.68

The application execution time is estimated by using the *Linux-Perf* tool. Execution time required to process a given data set by a compiled application binary is estimated by averaging four different executions. In order to implement dimension reduction technique, we used PCA having set PCs to 10 in this work. The PCA components have been computed by using the MICA features collected from each application run, normalized by standard deviation across all data sets. Our exploration experiments have been generated by using the data previously collected offline.

WEKA Machine Learning tool [11] has been integrated into our framework to exploit predictive modeling algorithms. More in detail, in this work, we assessed the proposed methodology with *Linear Regression* (LR) Machine Learning algorithm activating the *M5* attribute selection method with default ridge parameter. Experimental results has been carried out by means of *leave-one-out cross-validation*. Given a new unseen application in the training set, the current program feature already obtained offline will impose a bias on the trained model and the predictive modeling will be able to predict the *next-best optimization* to be applied to maximize the immediate speedup in a greedy manner. As mentioned in Eq. 4.1, given the complexity of the problem, we assessed the feasibility of our proposed approach with four different

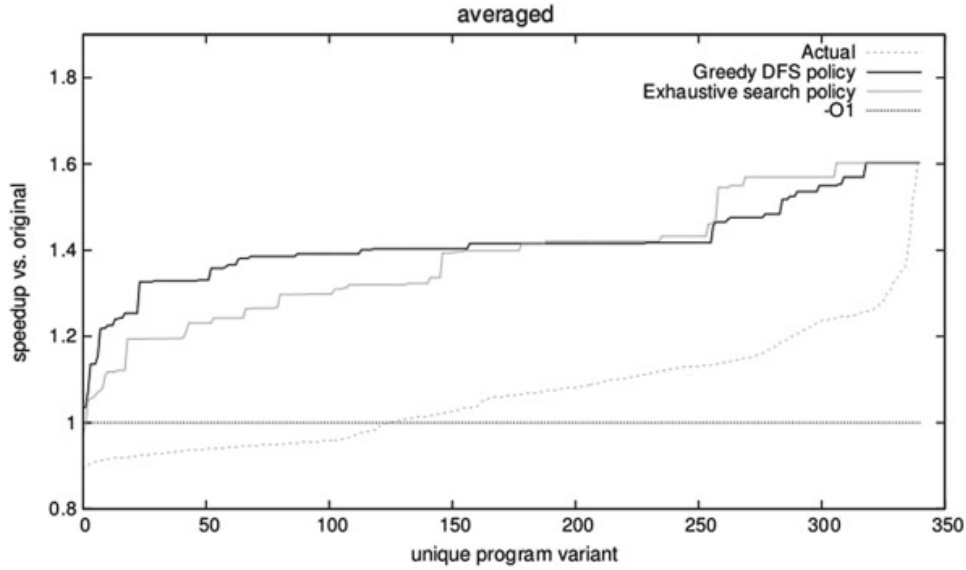


Fig. 4.2 Average Speedup of the proposed methodology across all applications

sequences of compiler options with a total of 30 compiler optimizations (13 unique optimizations) to be used in the design space. Generating different permutations with repetitions and enabling dynamic sequence length led to 341 different variations and 2046 for all considered applications.

The results obtained by exploring the phase-ordering space with the two proposed heuristics is reported in Fig. 4.2. It shows that the revised DFS search approach, namely an *Iterative Deepening First Search* policy (based on a greedy Depth First Search (DFS) heuristic) is doing slightly better with respect to the *exhaustive search heuristic* presented in Sect. 4.3.3. We define the *actual speedup* line as the speedup observed from running the application using the compiler optimization prediction of the machine learning model. Table 4.4 is presenting the quantitative values of utilizing the two predictive modeling algorithms within our framework. We evaluated our *iterative greedy approaches* with the classic 1-shot predictive speedup approach mentioned in [6, 9, 23] and the results reported in Table 4.4 are the average output of the one-shot approach per application. One-shot approach is extracting the prediction by means of one-extraction only and observe its speedup gain. In average, these two search algorithms demonstrated respectively 4 and 2% performance speedup over LLVM default performance.

The graph in Fig. 4.2 demonstrates that the performance of the greedy exploration policy in the early generation of the prediction is better than the exhaustive search methodology for the selected benchmarks. The horizontal axis represents different variations of the applications. This is rather interesting because in the phase-ordering problem the cardinality of the optimization sequence space $\mathcal{O}$ is too huge for an exhaustive search policy to be applied. On average, by traversing 15% of the compiler design space, we can reach up to 80% of the best found options in the design space.

Table 4.4 Average speedup of the one-shot prediction for both approaches w.r.t LLVM baseline

Application	Greedy DFS	Exhaustive search
automotive-susan-c	0.9808	0.9658
network-patricia	1.0069	1.002
automotive-qsort1	1.1255	0.9670
automotive-bitcount	1.0848	1.1506
network-dijkstra	0.9988	0.9988
automotive-susan-e	1.0617	1.1015
Average	1.0431	1.0242

4.5 Conclusions

This chapter presented a method based on predictive modeling to select the next-best compiler option (intermediate speedup prediction) to be applied to maximize the application performance. Experimental results exploiting two different search heuristics on the selected set of benchmarks demonstrated respectively 4 and 2% performance speedup with respect to the default LLVM compiler framework.

In the next chapter, we provide another prediction approach namely, full-sequence prediction to tackle the phase-ordering problem of optimizations.

References

1. Agakov F, Bonilla E, Cavazos J, Franke B, Fursin G, O'Boyle MFP, Thomson J, Toussaint M, Williams CKI (2006) Using machine learning to focus iterative optimization. In: Proceedings of the international symposium on code generation and optimization, IEEE Computer Society, pp 295–305
2. Ashouri AH, Mariani G, Palermo G, Park E, Cavazos J, Silvano C (2016) COBAYN: compiler autotuning framework using bayesian networks. *ACM Trans Archit Code Optim (TACO)* 13(2):21:1–21:25. <http://doi.acm.org/10.1145/2928270>
3. Ashouri AH, Zaccaria V, Xydis S, Palermo G, Silvano C (2013) A framework for compiler level statistical analysis over customized VLIW architecture. In: *VLSI-SoC*, pp 124–129. <http://dx.doi.org/10.1109/VLSI-SoC.2013.6673262>
4. Benabderrahmane M-W, Pouchet L-N, Cohen A, Bastoul C (2010) The polyhedral model is more widely applicable than you think. *Compiler constr*, 6011, pp 283–303
5. Bondhugula U, Hartono A, Ramanujam J, Sadayappan P (2008) PLuTo: A practical and fully automatic polyhedral program optimization system. In: Proceedings of the ACM SIGPLAN 2008 conference on programming language design and implementation (PLDI 08), Citeseer, Tucson, AZ, June 2008
6. Cavazos J, Dubach C, Agakov F (2006) Automatic performance model construction for the fast software exploration of new hardware designs. In: Proceedings of the 2006 international conference on compilers, architecture and synthesis for embedded systems, pp 24–34
7. Cavazos J, Fursin G, Agakov F (2007) Rapidly selecting good compiler optimizations using performance counters. In: International symposium on code generation and optimization (CGO'07)

Chapter 5

The Phase-Ordering Problem: A Complete Sequence Prediction Approach

Abstract This chapter proposes our second approach to tackle the phase-ordering problem. We already showed our intermediate speedup prediction method in Chap. 4. Here, we present our full-sequence speedup prediction method called MiCOMP. MiCOMP: Mitigating the Compiler Phase-ordering problem using optimization sub-sequences and machine learning, is an autotuning framework to mitigate the compiler phase-ordering problem based on machine-learning techniques effectively. The idea is to cluster the optimization passes of the LLVM O3 setting into different clusters to predict the speedup of the complete-sequence of all the optimization clusters. The predictive model uses (i) dynamic features, (ii) an encoded version of the compiler sequence and (iii) an exploration heuristic to tackle the problem. Experimental results using the LLVM compiler framework and the Cbench suite show the effectiveness of the encoding technique to application-based reordering of passes while using a number of predictive models. We performed statistical analysis on the prediction space and compared against (i) standard optimization levels O2 and O3, (ii) random iterative compilation, and (iii) two recent non-iterative approaches. We demonstrate that our proposed methodology outperforms the performance of -O1, -O2, and -O3 optimization levels in just a few iterations, reaching an average performance speedup of 1.26 (up to 1.51) on the Cbench benchmark suite.

5.1 Intermediate Versus Full-Sequence Speedup Prediction

Recent compilers offer a vast number of multilayered optimizations, capable of targeting different code segments of an application. Choosing among these optimizations can significantly impact the performance of the code being optimized. The selection of the right set of compiler optimizations for a particular code segment is a hard problem, but finding the best ordering of these optimizations adds further complexity. In fact, finding the best ordering is a long standing problem in compilation research called the phase-ordering problem. The traditional approach of constructing compiler heuristics to solve this problem simply can not cope with the enormous complexity of choosing the right ordering of optimizations for every code segment in an application.

Finding the best ordering of compiler optimizations can have substantial benefits for performance metrics such as execution time, power consumption and code-size. To this end, using predefined optimizations usually is not good enough to bring the best achievable application-specific performance. In this chapter, we propose a framework to mitigate the complexity of the phase-ordering problem. So far, there are two potential techniques we could use to predict good optimization orders for code being optimized:

- (i) *Intermediate Sequence Prediction*: This technique uses a model to predict the current best optimization (from a given set of optimizations) that should be applied based on the characteristics of code in its present state [1, 2].
- (ii) *Complete Sequence Prediction*: This technique uses a model to predict the complete sequence of optimizations that needs to be applied to the code just by looking at characteristics of the code. Although this technique has been extensively used in the selection problem of the compiler optimizations [3–6], there has been no work tackling the phase-ordering with the aforementioned method.

The framework proposed in this chapter, MiCOMP, falls under the second category. It uses predictive models on complete optimization sequences, rather than individual optimizations. We characterize applications as a vector of dynamic features that are independent of the target architecture. Predicting the complete optimization sequence to apply to a piece of code, i.e., complete sequence prediction, has the benefit of only requiring a single-round of feature collection of the code before any optimizations are applied to it. In order to use classic machine learning algorithms with the phase-ordering problem, we adapt an encoding scheme to transform variable-length vectors of optimizations into fixed-length vectors. Our prediction models are trained offline and program features and different compiler configurations are fed as inputs. As outputs, a prediction model predicts the speedup without the need to actually run the code on the target architecture. The dynamic characterization is independent from the architecture the code is running, thus it brings portability among different architectures. Additionally, we define exploration heuristics to find the best models in the shortest time. We refer to time as the minimum number of predictions from the model to obtain the best version of the code being optimized. The heuristic is based on Adjusted Cosine Similarity [7] to correlate different configurations of optimizations with their corresponding predicted speedups across all the training data. A recommendation algorithm enables us to explore only a fraction of the configuration space to reach the best speedups rather than a simple sorting/ranking [3–6]. In our experimental results, we show that our technique can outperform LLVM’s highest optimization level of `-O3` by just a few predictions. We also show competitive and quantitative comparisons with respect to state-of-the-art iterative and non-iterative models. We selected a variety of applications from the Ctuning Cbench benchmark [8] to assess and evaluate the benefits of the proposed approach and to prove its feasibility. The main contributions of the proposed approach are as follows:

- An independent predictive-modeling framework, capable of capturing the correlation between different compiler optimizations and their predicted speedup with-

out having to run optimized code variants on the target platform. Our autotuning framework can be paired with any desired predictive models.

- Dynamically reordering the optimizations within the LLVM optimization level `-O3`. We have clustered different compiler optimizations, all taken from LLVM's `O3` into 5 different groups. The order of optimizations within a group is internally fixed but the ordering of the groups can be altered. In this work, these groups are called sub-sequences and we exploit the phase-ordering by using these sub-sequences rather than the individual optimizations. By starting from no optimizations (as the baseline) and exploring different orderings of the sub-sequences using the same optimizations available to `-O3`, we outperformed `-O3`.
- Adapting a simple mapping technique to encode an optimization sequence into a bit string. The proposed technique allows us to apply traditional machine learning algorithms as they are mostly designed to cope with both fixed-length feature vectors.
- Adapting a Recommender System (RS) approach on the prediction space to use dynamic information. We show this can boost the exploration and help to obtain better speedups.

The rest of the chapter organized as follows: Sect. 5.2 presents related work. Section 5.3 introduces our proposed methodology including all its components. In Sect. 5.4, we present our experimental results and evaluate the results by means of several comparisons in the Sect. 5.5. We conclude this chapter with future work and the conclusion.

5.2 Related Work

As we discussed in the previous chapter, literature [9] on the phase-ordering problem is closely related to the problem of selecting the best set of compiler optimizations in a fixed ordering. Recent literature can be classified into two main classes: (i) autotuning and iterative compilation approaches and (ii) applying machine learning to the problem of optimization selection. (Readers can refer to Chap. 1 for the complete survey on the literature.)

Our approach, MiCOMP, is significantly different compared with those mentioned in the literature. Our work mostly resembles the approach of Park et al. [5, 6]. However, our techniques tackle the significantly harder problem of the phase-ordering. We introduce a mapping function that encodes an optimization sequence into a bit string. It preserves the ordering and the repetition of the optimizations. At the same time, the proposed work is able to predict the complete optimization sequence to apply to the unoptimized code, rather than predicting the best optimization to apply to the current state of the optimized code [1, 2]. An intermediate sequence approach needs multiple profiling of the application being optimized (based on the characteristics of code in its present state) while we need to profile just once, both in the offline-training and the online-prediction. We use dynamic architecture independent features to feed

into our model. Moreover, we used clustering over all passes in LLVM’s `-O3`, that tended to perform well, to significantly outperform the single optimization sequence performed by `-O3` itself. We do that by re-ordering these sub-sequences automatically based on the type of the application under optimization. To summarize, the proposed work is the first approach that uses machine-learning based techniques on the phase-ordering problem to predict the complete sequence of optimizations. In Sect. 5.4, we improve the machine-learning model through Recommender Systems technique and assess the experimental results we obtain against the state-of-the-art phase-ordering approaches.

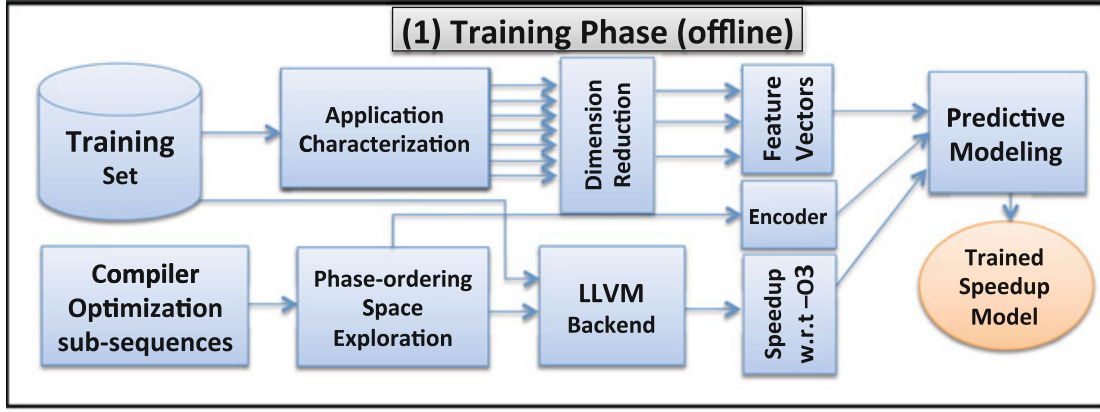
5.3 The Proposed Methodology

Compilers typically ship with standard optimization levels (e.g. `-O2`, `-O3` and `-Ofast`) each tuned at the compiler factory to obtain a certain level of performance on a standard set of benchmarks. These optimization levels do not always translate to good performance on other applications. The main objective of the proposed methodology is to introduce a compiler autotuning framework, which is able to dynamically reorder the compiler passes within LLVM’s optimization level `-O3`, to achieve the maximum speedup for the applications being optimized. We found that if we could reorder sub-sequences of optimizations that tended to perform well, we could significantly outperform the single optimization sequence performed by `-O3`. This process should be customized based on the features of the application under analysis. To mitigate the phase-ordering problem, a model has to be constructed in such a way that it can correlate the effect of using different compiler sequences and the corresponding achievable speedup. MiCOMP uses such a model, and it can (i) recommend good sequences of optimizations that maximize an application’s performance and (ii) these good-sequences are recommended with very few predictions.

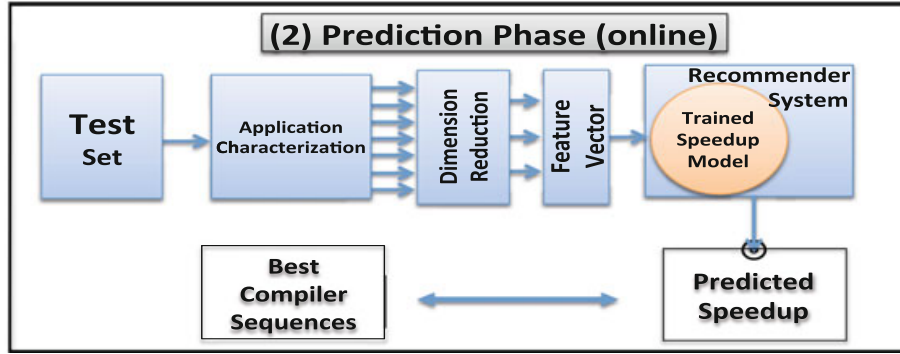
There are certain limitations facing the ever increasing complexity of the problem. The phase-ordering problem is also complicated by enabling the possibility of variable-length compiler sequences. State-of-the-art approaches for selecting the right set of optimizations used fixed length feature-vectors [3, 5, 6]. Therefore, in order to tackle the phase-ordering problem, we propose to encode the phase-ordering space into a conventional fixed-length feature vector space to apply traditional machine learning algorithms. We provide a simple encoding-scheme with mathematical deduction in Sect. 5.3.3. During the prediction phase, MiCOMP proposes an iterative process in which different solutions are explored by evaluating different optimization sequences with the potential of leading to higher speedups. We predict optimization sequences that will perform well against using state-of-the-art *ranking* [5, 6] techniques.

Figure 5.1 illustrates the two main phases of MiCOMP: (i) *offline training* and (ii) *online prediction*.

The *offline training phase* is used to learn about the effects of compiler optimizations when compiling an application. In particular, this phase is used to induce a



(a) Training MiCOMP



(b) Testing MiCOMP

Fig. 5.1 Proposed framework: offline-training phase which is done once, and online-prediction phase for optimizing new unseen applications

prediction model considering application features and applied optimizations (including order and repetitions). This phase is performed once for each compiler and the model is built on a set of representative applications. In this phase, each application is passed through a single round of feature collection to extract an application's characteristics. A dynamic profiler is used to generate a representation of the program in terms of its features. Since a very large set of features is extracted for each application, we apply a dimension-reduction technique to reduce the number of features that is fed as input to the prediction model (e.g. PCA (Principal Component Analysis) [10]). This speeds up the learning during the model construction process. Application-profiling and dimension-reduction techniques are extensively described in Sect. 5.3.1. Next, an application is compiled with different configurations of compiler optimizations, executed and profiled in terms of speedup with respect to LLVM's -O3. The speedup values together with the reduced program features and an encoded version of the used compiler optimizations (characterized by a fixed-length binary output, see Sect. 5.3.3) are fed to a machine learning algorithm to induce the speedup predictor (see Sect. 5.3.4). This model can then be used during the online phase.

The *online prediction phase*, is used every time a new application is optimized. To this end, we use the same feature extraction and dimension reduction techniques

described in the offline training phase. The collected features are used to query the speedup prediction model to predict the best set of compiler sequences to apply to an application. The goal of our method is to discover the fewest number of predictions that will be needed to obtain the optimization sequence that gives the best speedup possible. Thus, MiCOMP has been coupled with a heuristic derived from the field of Recommender Systems (see Sect. 5.3.5). This technique is used to obtain a predicted set of optimization sequences where each sequence is as diverse as possible to the other sequences in the set, thus guaranteeing coverage of a large part of the optimization configuration space, consequently obtaining a set of optimization sequences that are robust to model inaccuracies.

5.3.1 Application Characterization

In this work, we used a PIN-based [11] dynamic instrumentation framework to analyze and characterize the behavior of applications at execution-time. In particular, our framework provides a high level Micro-architectural Independent Characterization of Applications (MICA) [12] suitable for characterizing applications in a target architecture agnostic manner. There is no static syntactic analysis, but the framework is solely based on dynamic MICA profiling. Readers can refer to Chap. 3 for more details on the MICA framework.

In our experimental setup, an application is compiled and profiled on an Intel XEON machine, while the target architecture where the application will eventually execute (i.e. the architecture for which the application is being optimized) will be a different platform. That is, the machine learning model will be fed a high level abstraction of the application characterization carried out with MICA. This allows us to easily change the target architecture without the need of replicating the profiling infrastructure.

To reduce the number of input features, while keeping most of the information contained in the input data, one simply needs to use the first k principal components as suggested in previous work [12]. In particular, we set $k = 5$, which captures more than 98% of the overall variance across all training sets.

5.3.2 Constructing Compiler Sub-sequences

In this section, we briefly explain our novel idea behind clustering certain compiler optimizations as *sub-sequences*. A phase-ordering optimization sequence represented by the vector o belongs to the n dimensional factorial space $|\Omega_{phases}| = n!$, where n represents the number of compiler optimizations under study. However, the mentioned bound is for a simplified phase-ordering problem having a fixed length optimization sequence length and no repetitive application of optimizations. Enabling optimizations to be repeatedly applied and a variable length sequence of optimizations will expand the problem space to:

$$|\Omega_{phases_repetition}| = \sum_{i=0}^m n^i \quad (5.1)$$

where n is the number of optimizations under study and m is the maximum desired length for the optimization sequence. Even for reasonable values for n and m , the entire search space is enormous. For example, assuming n and m are both equal to 10, this leads to an optimization search space of more than 11 billion different optimization sequences to select from for each piece of code being optimized [1].

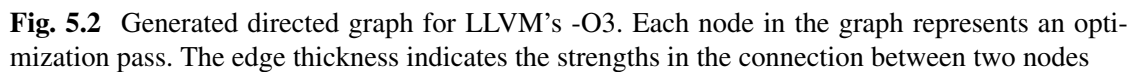
5.3.2.1 The Optimization Dependence Graph

Mitigating the phase-ordering problem with previous approaches is not practical due to a large number of different possible optimization sequences to select from each piece of code being optimized. MiCOMP, proposes to group optimizations into clusters of sub-sequences that are known to perform well, which reduces the size of the search space to explore and thus introduces scalability. There are 157 compiler passes in LLVM optimization level -O3 (more than 60 unique compiler passes) and selecting the most promising sub-sequences from these optimizations can positively affect the autotuning process. Among all these 157 compiler passes, some are analysis passes (i.e. basicaa, memdep, etc.) which do not transform the code directly, but instead provide analysis information to other compiler passes that follow them. The rest are transformation passes, i.e., Aggressive Dead Code Elimination (adce), Loop Invariant Code Motion (licm), loop-rotate, etc., which perform optimizations on the code.¹

In this chapter, we introduce the idea of clustering sub-sequences of all the passes available to the optimization level -O3 and adapt prediction models to order these sub-sequences in ways that improve the performance of a particular application. We show that this technique can improve the performance of an application over using -O3 by evaluating a few predicted orderings of the sub-sequences of optimizations.

Let $o = \{o_1, \dots, o_N\}$ be the set of all transformation passes from LLVM optimization level -O3. We can represent the optimizations in optimization level -O3, with a directed graph of $G = (V, E)$ where V is the set of nodes representing the optimizations and E is the set of edges, where we add an edge between two optimization nodes if that pair of optimizations appears in the optimization sequence in -O3. Further, we can annotate an edge with the number of times that each pair of optimizations consecutively appears in the sequence. The outcome of this is a graph. We term the optimization dependence graph. This graph can be represented by a *Weighted Adjacency Matrix* that has the size of $N \times N$. This matrix can be used for clustering optimizations into different sub-sequences. Figure 5.2 shows the constructed graph on -O3.

¹<http://llvm.org/docs/Passes.html>.



For our clustering of optimizations into sub-sequences, we could have used any number of the numerous clustering methods proposed in the literature related to Pattern Recognition (e.g. iterative, hierarchical, divisive, etc.,) [13]. We selected *agglomerative clustering* [14] which is an iterative clustering technique that merges smaller clusters and improves the complexity of k-mean clustering on graphs [13]. A key insight of this method is that it treats clusters as a dynamical system and its samples as states. The algorithm works as follows: Agglomerative clustering receives as input the matrix of the graph G and the number of desired clusters (n_T) and builds (i) the graph G with k-nearest-neighbors upon computing its Weighted Adjacency Matrix (W). (ii) The algorithm then calculates the transition probabilities and (iii) forms sample clusters $C = \{c_1, \dots, c_{nc}\}$. (iv) It enters a loop to iteratively try to add more sub-clusters to the already available clusters in C as long as the conditional sum of the all-path integrals within the new sub-clusters maximizes some objective function (*argmax*) [14]. A path integral is a metric to measure the stability of a dynamical system and is computed by summing the paths within the cluster on the directed graph weighted by transition probabilities. We used the algorithm and tentatively increased the number of max desired clusters until no clusters could be added. The final five clusters, namely, the best optimization sub-sequences the algorithm could find are reported in Sect. 5.4; Table 5.2.

5.3.2.3 Benefits of Sub-sequences

Clustering optimizations into sub-sequences makes sense. Certain analysis algorithms typically should be done before an optimization in order for the optimization to have any significant impact. For example, we may want to run analysis that performs basic block counts and predicts branch instruction outcomes before applying an optimization that reorders the code blocks in an application. Additionally, it is likely that -O3 will contain optimizations that should follow other optimizations in order to obtain the best performance. Thus, forming a cluster of optimizations that should be applied together makes a lot of sense.

5.3.3 The Proposed Mapper

Constructing prediction models for the problem of selecting the right compiler optimizations with fixed-length feature vectors has been extensively studied [3–6]. However, prediction models fall short when correlating program characterizations with the right compiler optimizations to apply when it comes to a variable optimization sequence length [15]. Therefore, we adapt a simple encoding technique which allows us to map the representation of the compiler phase-ordering sub-sequences to a fixed-length vector of optimizations and at the same time preserves the order of optimizations in the sequence. Our proposed mapping function encodes an optimization sequence into a bit string.

Let $\mathcal{A} = \{\alpha_1, \dots, \alpha_N\}$ be the set of all variables, which can be thought of as an *alphabet*. Every α_i is a *letter*. A finite string of not necessarily distinct letters is called a *word*. Thus, each word is a concatenation of the form $\alpha_{i_1}, \alpha_{i_2} \dots \alpha_{i_k}$, where $i_1, i_2, \dots, i_k \in \{1, \dots, M\}$. The integer k is the *length* of the word. We will also allow the empty word which by definition has length zero.

There is a simple way of encoding the space $\mathcal{W}$ of all words of length at most M using the space described by $\{0, 1\}^{N \times M}$ consisting of all binary strings of the fixed length $N \times M$. To see this, consider the mapping function $f : \mathcal{A} \rightarrow \{0, 1\}^N$ which maps each letter α_i to the binary string $f(\alpha_i) = b_1 \dots b_M$, where

$$b_j = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{if } j \neq i. \end{cases}$$

Now we define the mapping function $F : \mathcal{W} \rightarrow \{0, 1\}^{N \times M}$ by mapping each word $\alpha_{i_1}, \alpha_{i_2} \dots \alpha_{i_k}$ to the binary string

$$F(\alpha_{i_1}, \alpha_{i_2} \dots \alpha_{i_k}) = f(\alpha_{i_1})f(\alpha_{i_2}) \dots f(\alpha_{i_k}) \underbrace{\mathbf{0} \dots \mathbf{0}}_{N-k \text{ times}},$$

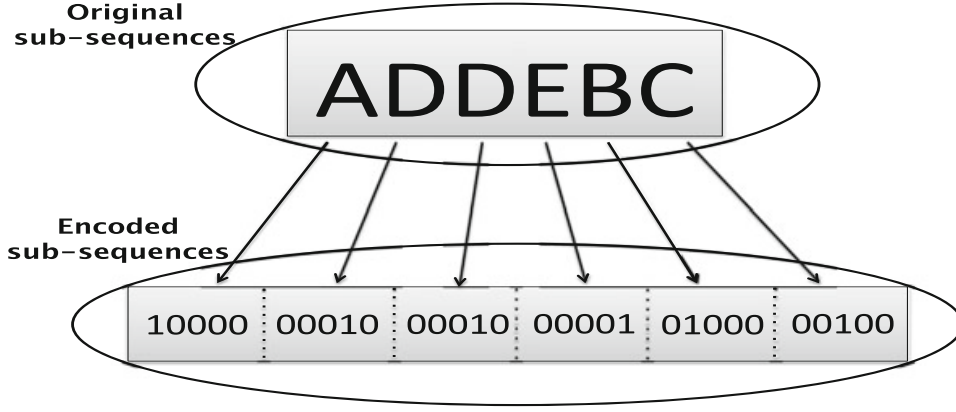


Fig. 5.3 An example of the proposed encoder where $\{N = 5, M = 6\}$: Each letter represents a compiler sub-sequences containing different compiler optimizations

where $\mathbf{0} = 0 \dots 0$ is the zero string of length N . Evidently the map F is one-to-one. The image $F(\mathcal{W})$ is much smaller than the target space $\{0, 1\}^{N \times M}$, as these sets have $\sum_{k=0}^M N^k$ and $2^{N \times M}$ elements, respectively.

If we identify each element of $\{0, 1\}^{N \times M}$ with a concatenation $s_1 \dots s_N$ of N elements of $\{0, 1\}^N$, the image $F(\mathcal{W})$ can be simply characterized by the following two requirements:

1. Each s_i has at most one non-zero binary digit.
2. If $s_i = \mathbf{0}$ and $s_j \neq \mathbf{0}$, then $i > j$.

Given the proposed mapping, there exists a one-to-one (1:1) mapping F for every instance of $\mathcal{A} = \{\alpha_1, \dots, \alpha_N\}$ with the binary size of $N \times M$ that has the same characteristics of the original presentation with the benefit of having a fixed $N \times M$ length. An example of the proposed mapping function is shown on the Fig. 5.3. Our adapted mapping function uses a *one-hot encoding* approach [16] for $N=5$ and $M=6$ to assign a single high (1) at each segment of the transformed binary while other bits are turned off (0). This technique can inexpensively preserve the order and the repetitions of optimizations in a sequence, at the same time it assures the transformed feature vector has fixed-length size.

5.3.4 Predictive Modeling

The methodology in Fig. 5.1 illustrates the use of predictive modeling in both the offline (training) and online (testing) phases of the of the process. We used the predictive modeling in the offline training phase to (i) construct the model and in (ii) the online prediction phase we exploit the constructed model on the target application to predict the speedup of a complete optimization sequence without the need to actually apply the sequence of optimizations to the code.

5.3.4.1 Constructing the Prediction Model

Predictive modeling is the process of constructing, testing and validating a model to predict an unobserved outcome based on a characterization of a state from which to predict the outcome. In this chapter, the state being characterized is the code being optimized and the predicted outcome corresponds to the speedup metric calculated by normalizing the execution time of the current optimization sequence by the execution time of the baseline optimization sequence. The general formulation of the optimization problem is to construct a function that takes as input the features of the unoptimized program being compiled. In other words, this model takes as an input a tuple (F, T) where F is the feature vector of the collected instrumentation of the program being optimized; and T is one of the several possible compiler sequences predicted to perform well on this program. Its output is a prediction of the speedup T should achieve when applied to the original code.

5.3.4.2 Analysis of Selecting the Compilation Baseline

As explained in Sect. 5.3.2, we do not use any of the default compilation optimization levels as a baseline to start from since we used all compiler optimizations passes that are used in `-O3` for our clustering purposes (see Sect. 5.3.2.2). Additionally, we found that using a baseline compiler optimization level to start from ultimately reduces the speedup achievable from the sequence we construct with predictive modeling. We empirically justify this argument by running a set of experiments, one without using a compiler optimization level as a baseline and a set of experiments where we use a sequence of optimizations to apply on top of different `-OX` baselines. Figure 5.4 illustrates the mean of MiCOMP's proposed optimization sequences using different compilation baselines. All four speedup lines have an upper-bound of sequences of length five. Results suggest that using MiCOMP optimization sequence without an optimization level as a baseline can lead to substantial benefits compared with using any of `-OX` optimization levels as a baseline. MiCOMP targets the region of interest where the highest achievable speedup values are located and it drives the prediction model to reach that part of the optimization sequence search space least number of predictions. Note that using a baseline of `-O1`, `-O2`, or `-O3` all converge to a sub-optimal speedup. Thus, applying certain sequences causes a degradation in performance as can be seen by using these standard optimization levels as a baseline. The better option is to not use a baseline sequence at all and to allow MiCOMP to predict the best sequence to apply on its own.

The insights of this experiment are threefold: (i) The clustering technique is beneficial; first, to gain better speedup values and second, to reduce the number of compiler optimization sequences needed to achieve the best results from around 50 to 5 so that our iterative compilation method is both scalable and practical. (ii) The sub-sequences can be coupled with machine-learning techniques so they can be reordered based on the applications being optimized while outperforming the highest

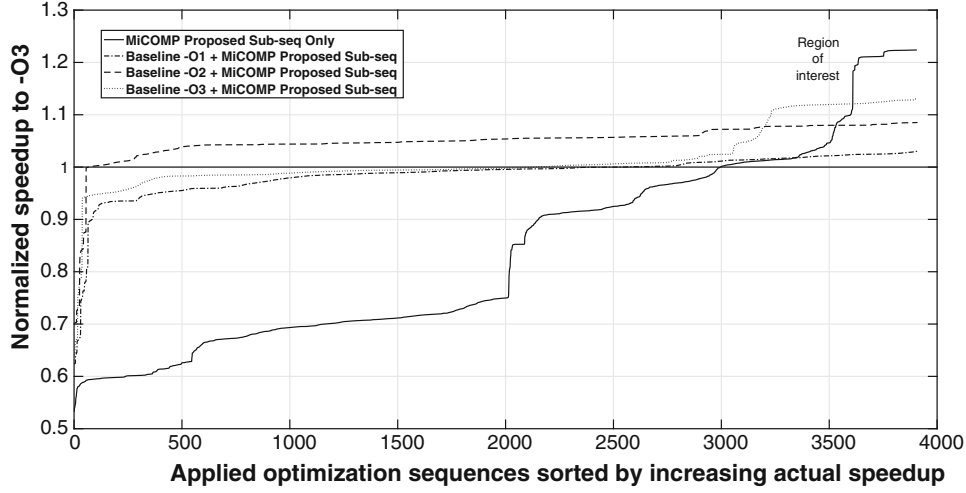


Fig. 5.4 Empirical analysis of having different compilation baseline across all CBench applications (Harmonic mean). Region of interest is depicted where MiCOMP sub-sequences outperformed other compiler sequences having a fixed standard compilation baseline

standard optimizations levels. (iii) Phase-ordering indeed does matter in the field of compilers; i.e., using the same set of optimization flags available to -O3, MiCOMP can significantly outperform -O3 itself.

5.3.4.3 Application-Specific Prediction

Our machine-learning constructed models can be used for unseen target applications to predict the speedup when applying compiler sequences to them. The predicted speedup values correspond to the optimization sequence applied to the program. For a given input program, first a feature vector containing dynamic instrumentation is collected. Then, our prediction model is fed the features of the program being compiled to predict the expected speedup if an optimization sequence T was applied to it. By predicting the performance of each possible optimization sequence that can be applied, it is possible to rank the optimization sequences according to their expected speedup and only select the sequences to actually apply that are predicted to give the highest speedups.

A state-of-the-art ranking approach [4, 6] was used to rank optimization sequences in descending order, and we only select the top N optimization sequences to evaluate their actual optimization quality. In this work, we propose an iterative process in which different solutions are explored to find those leading to higher speedups. In other words, our proposed exploration technique uses the output of our prediction model to generate an initial exploration strategy, and the exploration strategy dynamically updates itself in order to reach the highest speedup values in the least number of predictions.

5.3.5 Recommender Systems Heuristic

Mitigating the phase-ordering problem imposes a proper exploration strategy. In the initial steps taken by [1, 2], the authors defined iterative exploration heuristics, based on the current optimized state of the target application being compiled, to select the next best optimization to apply, which will bring the eventual best speedup. As the current state of the optimized application depends on the optimizations that were already applied, this previous approach required several rounds of feature collection. In this chapter, we propose a predictive approach that generates the complete optimization sequence for a program that has not been optimized, thus it needs to collect features only once before any optimizations are applied.

5.3.5.1 Adjusted Cosine Similarity

Many of the aforementioned state-of-the-art approaches, tackling both the selection and the phase-ordering problem, define exploration strategies on the optimizations design space. Yet, to the best of the authors' knowledge, none of them make use of information in order to dynamically improve the strategy itself. Dynamic information, in our particular case, is the predicted speedup on the sequences already explored and evaluated. The knowledge can be effectively used to improve the initial exploration. The technique we propose leverages the similarity between the unexplored and the explored optimization sequences. In particular, our proposed technique prioritizes the evaluation of solutions less similar to the ones already explored. is especially important for the phase-ordering problem where there are a plethora of optimization sequences that need to be explored. The similarity measure is based on how close the achieved speedup is for predicted solutions across all the training sets. As an example, let $S_{p,i}$ and $S_{p,j}$ be the predicted speedups of the sequence i and j when applied to program p in the set of programs P . We define an iterative process to look for predicted similarities in i and j .

In recommender system (RS), an algorithm called Basic Cosine Similarity [7] is used to correlate users and items. However, computing the similarity using this algorithm has one important drawback; the difference in rating scale are not taken into account. The Adjusted Cosine Similarity offsets this drawback by subtracting the corresponding user-average from each co-rated pair. Adapting this technique, we can compute the Adjusted Cosine Similarity between optimization sequence i and j as:

$$sim(i, j) = \frac{\sum_{p \in P} (S_{p,i} - \bar{S}_p)(S_{p,j} - \bar{S}_p)}{\sqrt{\sum_{p \in P} (S_{p,i} - \bar{S}_p)^2} \sqrt{\sum_{p \in P} (S_{p,j} - \bar{S}_p)^2}} \quad (5.2)$$

where $S_{p,i}$ is the speedup achieved by sequence i when applied to program p of all set of programs P , and $\bar{S}_p$ is the average speedup on program p . We use the computed measure to evaluate the correlation between a pair of optimization sequences to boost our exploration strategy.

Algorithm 1 Proposed heuristic using Adjusted Cosine Similarity

```

tmpTestedSet = EmptySet
while phases Not Tested Yet do
  tmpTestedSet = EmptySet
  for phases in prediction space do
    if new phases exist then
      if similar solution exists then
        Skip Phases
      else {Test phases:  $\text{sim}(i, j)$ }
        Add phases
      end if
    end if
  end for
end while

```

The pseudocode of the algorithm is shown in Algorithm 1. The exploration strategy is defined as follows:

1. Sort predicted speedup solutions in decreasing order in a list.
2. Test solutions in order. If the solution to test is too similar to one already tested in the current list iteration, skip it.
3. If the end of the list has been reached and there are still optimization sequences to test, go to 2., starting from the head of the list and excluding already tested solutions.

High values of Adjusted Cosine Similarity (ACS) for a pair of optimization sequences are the consequence of achieving pairwise similar speedups across all training data. We exploit this measure to give exploration priority to the solutions that are less similar to the ones already tested. This allows our ACS algorithm to boost exploration to cover different areas of the optimization sequence space quicker than it would have achieved by predictive modeling alone, thus achieving better speedups with fewer exploration steps.

5.4 Experimental Results

In this section, we evaluate our proposed methodology on an Intel Xeon architecture. We adapted our instrumentation and architecture-independent tool (Sect. 5.3.1) to extract characteristics from a large set of benchmarks from the Ctuning CBench suite [8]. We have used LLVM compilation framework v3.8 (Clang for the frontend/backend and Opt for the optimization passes). The training set consists of different applications ranging from automotive, security, office, and telecom. The list of applications we evaluated is reported in Table 5.1. Table 5.2 illustrates the list of different compiler optimizations that are clustered into 5 different *sub-sequences* (refer to Sect. 5.3.2.2) that are derived from LLVM’s -O3. We used the sub-sequences with

Table 5.1 Applications under analysis (CTuning CBench suite [8])

cBench list	Description
automotive_bitcount	Bit counter
automotive_qsort1	Quick sort
automotive_susan_c	Smallest Univalued Segment Assimilating Nucleus Corners
automotive_susan_e	Smallest Univalued Segment Assimilating Nucleus Edges
automotive_susan_s	Smallest Univalued Segment Assimilating Nucleus Smoothing
security_blowfish_d	Symmetric-key block cipher Decoder
security_blowfish_e	Symmetric-key block cipher Encoder
security_rijndael_d	AES algorithm Rijndael Decoder
security_rijndael_e	AES algorithm Rijndael Encoder
security_sha	NIST Secure Hash Algorithm
telecom_adpcm_c	Intel/dvi adpcm coder/decoder Coder
telecom_adpcm_d	Intel/dvi adpcm coder/decoder Decoder
telecom_gsm	gsm encoder/decoder
consumer_jpeg_c	JPEG kernel
consumer_jpeg_d	JPEG kernel
consumer_tiff2bw	convert a color TIFF image to grey scale
consumer_tiff2rgba	convert a TIFF image to RGBA color space
consumer_tiffdither	convert a TIFF image to dither noisepace
consumer_tiffmedian	convert a color TIFF image to create a TIFF palette file
network_dijkstra	Dijkstra's algorithm
network_patricia	Patricia Trie data structure
office_stringsearch1	Boyer-Moore-Horspool pattern match
bzip2d	Burrows-Wheeler compression algorithm
bzip2e	Burrows-Wheeler compression algorithm

no baseline in MiCOMP and generate the design space enabling orderings and repetitions of these sub-sequences. The optimizations are fixed within a sub-sequence, but sub-sequences are allowed to appear in any order in the full optimization sequence.

An application's execution time is measured by using the Linux *Perf* tool. The execution time of a compiled application binary is measured by averaging the execution times of three different executions. In order to implement a dimension reduction technique we applied PCA. This analysis reveals that by using a 5-D vector of features we can capture 98% of the variance available in the training set. The Principal Components (PCs) have been computed using the MICA features collected from application executions (it is required only once), normalized by standard deviation across all data sets. An application characterization phase takes between 15 to 50 seconds depending on the type of the application. We noticed a small factor of slowdown when we perform the feature collection phase versus measuring the pure application's execution time. The overhead is negligible first, as it is required once and second,

Table 5.2 Candidate clusters of compiler optimizations into sub-sequences (all derived from -LLVM -O3)

sub-seq	Compiler passes
A	-alignment-from-assumptions -argpromotion -barrier -bdce -block-freq -branch-prob -constmerge -deadargelim -demanded-bits -dse float2int -forceattrs -functionattrs -globaldce -globalopt -globals-aa -gvn -indvars -inferattrs -inline -ipsccp -jump-threading -lcssa -loop-accesses -loop-deletion -loop-idiom -loop-unroll -loop-unswitch -loop-vectorize -mldst-motion -prune-eh -reassociate -rpo-functionattrs -sccp -simplifyfycfg -sroa -strip-dead-prototypes
B	-licm -mem2reg
C	-instcombine -loop-rotate -loop-simplifyfy
D	-memcpyopt
E	-adce -loop-unswitch -slp-vectorize -tailcallelim

Table 5.3 List of the predictive models used in our experiments

Predictive model	Description
MultilayerPerceptron (MLP)	A feedforward artificial neural network model that maps sets of input data onto a set of appropriate outputs. A MLP consists of multiple layers of nodes in a directed graph, with each layer fully connected to the next one
Linear regression (LR)	An approach for modeling the relationship between a scalar dependent variable y and one or more explanatory variables (or independent variables) denoted X . In linear regression, the relationships are modeled using linear predictor functions whose unknown model parameters are estimated from the data
KStar	It is an instance-based classifier, that is the class of a test instance is based upon the class of those training instances similar to it, as determined by some similarity function. It differs from other instance-based learners in that it uses an entropy-based distance function

Note that the proposed methodology is independent from any specific machine-learning algorithm (classifier) and it can be paired with any algorithm desired

the speedup gained by using MiCOMP is far higher. The proposed methodology is prediction model independent, and we report the results using three different models described in Table 5.3. Machine learning algorithms we used include (i) a Linear Regression (LR) classifier using the M5 attribute selection method with default ridge parameter, (ii) a *Multilayer Perception* that back-propagates to classify instances and using the default configuration, and the (iii) K^* algorithm using default settings.

In this work, the WEKA machine learning tool [17] has been integrated into our framework. We trained different speedup predictors, each one by excluding from the training application set, one of the applications. This technique is called Leave-One-Out-Cross Validation (LOOCV) and ensures a fair evaluation of our trained models. Validation data is used on the application excluded from the training set for

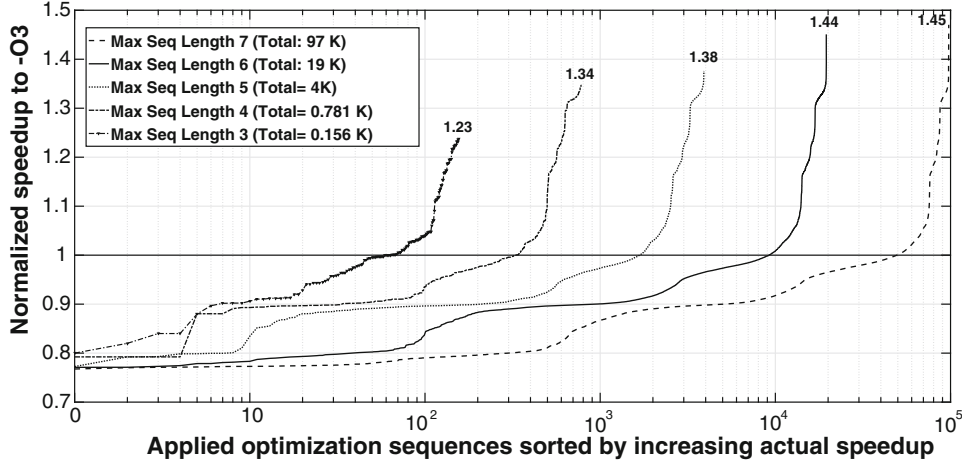


Fig. 5.5 Empirical analysis of having different compiler sequence lengths on 5 candidate applications: *telecom_adpcm_d*, *jpeg_d*, *bzipd*, *network_dijkstra*, *automotive_bitcount*. Note that X axis is in logarithmic scale

prediction purpose. In MiCOMP, cross validation is done in a few minutes for each application under analysis.²

5.4.1 Analysis of Longer Sequence Length

As described in Sect. 5.3, MiCOMP requires an upper bound on the sequence length for using the encoding scheme. To this end, we evaluate MiCOMP by different max values for the sequence length. A speedup prediction model requires a one time expensive training be done in order to construct an accurate model. We believe that the longer the sequence length, the better the chance of finding higher speedup values. To that end, we have tested our proposed sub-sequences with different maximum sequence lengths to empirically find the most effective length across all the training applications. This is done also with the goal of scalability and speeding up the training phase.

Figure 5.5 gives the Harmonic mean (as suggested by [18].³) values of the actual speedups using five selected applications each having different upper bound sequence lengths. We randomly selected an application from each of CBench categories (automotive, compression, telecom, consumer and network) since it was impractical to do this analysis with all applications. Having the upper bounds set to 3, 4, 5, 6 and 7 respectively, gives search spaces of 156, 781, 3909, 19 and 97k distinct permutations of sub-sequences with repetitions enabled (refer to Eq. 5.1 for the optimization

²Model construction is heavily correlated with the type of machine learning algorithm we use. We observed LR to be the fastest and MLP to be slowest for our data.

³We provide harmonic mean rather than arithmetic mean as we are dealing with averaging speedups. Note that harmonic-mean is always less than or equal to arithmetic-mean.

space). The five speedup lines show the trend of reaching a higher speedup value by iteratively exploring more fraction of optimization space. The maximum speedup found against $-O3$ using sequence lengths of 3, 4, 5, 6, and 7, respectively, are 1.23, 1.34, 1.38, 1.44 and 1.45. These results suggest to set the maximum length to 6 as this ensures achieving good speedups while avoiding a potential exploration of 100 K sequences per each application in the training set. For our optimization sub-sequences, this empirically found upper bound value is the right trade-off between choosing a good optimization sequence space and the possibility to explore efficiently.

5.4.2 *MiCOMP Prediction Accuracy*

Unlike sequence prediction models [6, 19, 20] in speedup prediction approaches, prediction quality is measured by means of prediction error. This metric demonstrates how close the prediction values were to the actual speedups given the same sequence. We use the following different error measurement techniques.

5.4.2.1 Mean Absolute Error

In statistics, the Mean Absolute Error (MAE) [21] is a quantity used to measure how close predictions are to the eventual outcomes. The mean absolute error is given by:

$$MAE = \frac{1}{n} \sum_{i=1}^n |f_i - y_i| \quad (5.3)$$

where we define e_i as $|f_i - y_i|$ given f_i as the prediction values and y_i the actual values. Consequently, the value e_i is inverse proportional to the accuracy of the prediction.

5.4.2.2 Approximation Error

Complementary to MAE, Approximation Error (AE) [22] is a common error measurement whereas in some data there is some discrepancy between an exact value and the approximation. An approximation error can occur because (i) certain measurements of the data are not precise (which we consider it can be the case for any computer scientific measurement) and (ii) approximated values are used instead of the real values (the iterative prediction way keeps using the predicted values). It is calculated as:

$$\delta = \frac{|\varepsilon|}{|v|} = \frac{|v - v_{approx}|}{|v|} \quad (5.4)$$

Table 5.4 Average error rate for the proposed mapping function versus an arbitrary mapping

M.L	MiCOMP mapping function		No mapping		Improvement factor	
	MAE	AE	MAE	AE	MAE	AE
MLP	0.06798	0.05479	0.10826	0.11838	1.59×	2.16×
LR	0.07525	0.07795	0.12879	0.13974	1.71×	1.79×
KStar	0.05179	0.05078	0.09188	0.10866	1.77×	2.13×

where the absolute error is the magnitude of the difference between the exact value and the approximation. These definitions can be extended to the case when v and $v_{approximate}$ are n -dimensional vectors, then by replacing the absolute error with an n -norm error.

5.4.2.3 Prediction Accuracy

We provide the prediction’s error rate in Table 5.4. We observe that the arbitrary mapping leads to higher error rates in the prediction values. Exploiting the adapted encoding scheme reduces the noise on the prediction and stabilizes the output trend with lower error-rate. Table 5.4 shows that the KStar model does slightly better in terms of accuracy compared with other models, it achieves around 5% error rate on average. In general, having a smaller error rate does not always guarantee higher performance gain but rather showcases the accuracy of the prediction model to capture the correlation between different compiler sub-sequences and the speedup values.⁴

5.4.2.4 Iterative Compilation Max Speedups

Iterative compilation is known to be able to achieve good performance results when compiling applications [23]. However, the approach is expensive and should be combined with more intelligent search algorithms [19, 20]. Table 5.5 reports the maximum speedups found by an iterative compilation approach using our proposed clustering while exploring the full optimization space. This experiment empirically confirms that the proposed clustering is useful on the phase-ordering space since we show that we can achieve on average a 26% speedup versus -O3. Figure 5.4 illustrates the trend when using MiCOMP sub-sequences with no baseline compared with having a baseline (e.g.: -O1, -O2 or -O3). The best optimization sequence for each of applications under-analysis and its speedup value are reported in the second

⁴We are aware of the many other encoding possibilities that are more efficient (currently having $N \times M$ length). However, we believe that extending the current encoding scheme to a more sophisticated version is out of scope of the work. Moreover, the proposed clustering technique can effectively reduce the number of N , thus the encoding scheme is scalable for higher orders.

Table 5.5 Best compiler optimization sub-sequences found using an iterative compilation and their related speedups

Application	Best sub-sequence	Speedup w.r.t. -O3
telecom_adpcm_c	ECDDCC	1.35
security_sha	ACCACE	1.06
security_blowfish_e	BCCEEA	1.03
automotive_susan_e	AABACA	1.15
consumer_tiffdither	DCEDCD	1.20
security_rijndael_e	CAEEC	1.10
consumer_tiff2bw	CCDCD	1.30
bzip2e	CBADCA	1.30
automotive_susan_s	ECCCDE	1.22
office_stringsearch1	ABCBAC	1.07
telecom_adpcm_d	DCAACA	1.13
consumer_jpeg_c	DDC	1.14
network_patricia	CECBAA	1.08
automotive_susan_c	BDBCCB	1.23
consumer_tiff2rgba	DEDDC	1.32
automotive_qsort1	CBAAAC	1.04
security_blowfish_d	DACECA	1.05
network_dijkstra	EECBBE	1.51
security_rijndael_d	ECEACD	1.05
bzip2d	CBDACCA	1.29
automotive_bitcount	BEACCA	1.07
consumer_jpeg_d	CCED	1.18
consumer_tiffmedian	BCBACB	1.15
telecom_gsm	BACBAC	1.07
Harmonic mean		1.26

and the third columns of Table 5.5. Readers can refer to Table 5.2 to find the exact set of compiler optimizations clustered in each sub-sequence.

5.4.3 MiCOMP Versus the Ranking Approach

Our proposed approach can improve the exploration to find the best optimization sequences in an optimization search space and to find the best speedups using a fewer number of predictions. Table 5.6 reports the comparison between the best speedup found by our approach and a state-of-the-art N-shot approach [4, 6]. The results, averaged using a Harmonic mean across all applications, show that using the same

Table 5.6 Prediction improvement of MiCOMP based on Adjusted Cosine Similarities against the Ranking (N-shot approach)

Exploration Techniques	Top-1	Top-5	Top-10	Top-15	Top-20
MiCOMP	1.01	1.06	1.09	1.10	1.12
Ranking	0.93	1.02	1.06	1.07	1.08

number of predictions from both models, our exploration technique can outperform the ranking approach on every number of predicted optimization sequences used (1, 5, 10, 15 and 20). This shows that our proposed methodology can effectively predicts the best compiler sequences to use and converges faster to better solutions in the space.

5.5 Comparative Results

In this section we evaluate the results of our model against three different techniques: **(i)** Standard optimization levels, **(ii)** Random Iterative Compilation (RIC) and, **(iii)** Non-iterative Models. We use our MiCOMP exploration policy and compare the performance of predictions to a previously published ranking approach. For each application under analysis, we tested the speedup gained using 1, 5 and 10 predictions and provide the Harmonic mean values. Table 5.7, reports the results. For each application and number of predictions, we provide two: i) achieved speedup compared with -O3 and, ii) achieved speedup compared with the optimal solution for the specific application. For example, one can see that for the `network_dijkstra` application we can gain a higher speedup values using MiCOMP and, on average even better than -O3 from just the first prediction. Moreover, we can achieve a 4% performance improvement over -O3 when we use 5 predicted optimization sequences from our model. Over all our benchmarks, using our model we can achieve 1%, 4%, and 9% speedups over -O3 using 1, 5, and 10 predicted optimization sequences, respectively. Consequently, our technique allows MiCOMP to outperform -O3 by high margins. Thus combining prediction models with iterative compilation achieves much better performance than using pure iterative compilation alone. For example, note in Fig. 5.5 that it takes a pure iterative compilation technique 100–5000 iterations to surpass the performance of -O3.

5.5.1 Comparison with Standard Optimization Levels

Standard optimization levels have been introduced to achieve good performance on average. However, they are coming short of the customized auto-tuning framework

Table 5.7 MLP’s speedup table against LLVM’s -O3. Reported numbers are A (B%): (A) speedup and (B) achieved speedup w.r.t. the optimal speedup value

Application	1 prediction	5 predictions	10 predictions
automotive_bitcount	1.04 (95.38%)	1.07 (98.12%)	1.08 (98.92%)
automotive_qsort1	1.01 (95.32%)	1.03 (96.93%)	1.03 (97.55%)
automotive_susan_c	1.04 (96.61%)	1.06 (98.53%)	1.06 (99.07%)
automotive_susan_e	1.04 (96.47%)	1.03 (98.41%)	1.04 (99.00%)
automotive_susan_s	0.99 (96.26%)	1.01 (98.42%)	1.02 (98.98%)
bzip2d	0.93 (92.77%)	0.96 (94.02%)	1.00 (94.37%)
bzip2e	1.09 (83.77%)	1.10 (86.02%)	1.12 (90.37%)
consumer_jpeg_c	1.01 (85.18%)	1.07 (90.35%)	1.10 (94.51%)
consumer_jpeg_d	1.09 (84.70%)	1.14 (88.97%)	1.17 (97.85%)
consumer_tiff2bw	0.96 (75.54%)	0.99 (80.59%)	1.02 (82.46%)
consumer_tiff2rgba	0.91 (80.61%)	0.95 (86.19%)	1.07 (88.08%)
consumer_tiffdither	1.02 (80.14%)	1.09 (85.86%)	1.11 (87.68%)
consumer_tiffmedian	0.94 (79.21%)	1.02 (85.72%)	1.06 (89.31%)
network_dijkstra	1.13 (60.00%)	1.29 (68.46%)	1.38 (73.00%)
network_patricia	0.91 (64.99%)	0.93 (70.79%)	0.97 (73.91%)
security_sha	0.91 (64.99%)	1.01 (70.79%)	1.03 (73.91%)
security_blowfish_e	0.92 (64.99%)	1.03 (70.79%)	1.03 (73.91%)
security_blowfish_d	0.91 (64.99%)	0.99 (70.79%)	1.02 (73.91%)
security_rijndael_e	0.92 (64.99%)	1.02 (70.79%)	1.01 (73.91%)
security_rijndael_d	0.89 (64.99%)	1.01 (70.79%)	1.04 (73.91%)
telecom_adpcm_c	0.91 (64.99%)	1.01 (70.79%)	1.02 (73.91%)
telecom_adpcm_d	0.92 (64.99%)	1.02 (70.79%)	1.01 (73.91%)
telecom_gsm_d	0.89 (64.99%)	1.03 (70.79%)	1.04 (73.91%)
Harmonic mean	1.01 (82.74%)	1.04 (87.51%)	1.09 (91.52%)

per architecture/application/dataset. As we showed in Table 5.7, MiCOMP can surpass the performance of -O3 with a few predictions on application bases. Here we provide Table 5.8 which reports more fine-grained speedup over all standard optimization levels. This demonstrates how fast (first number in the tuple) and in what percentage of the explored space (the second number), the framework is reaching a sequence which can outperform the specific standard optimization level. Each column is reporting two values: (i) in how many predictions and (ii) in what percentage of the whole configuration space the propped methodology can outperform OX levels.

Table 5.8 Average Speedup w.r.t LLVM -O3. Numbers are A (B%): **(A)** How fast (in terms of number of predictions) in average the proposed methodology outperforms LLVM standard Optimizations. **(B)** The percentage of the optimization space explored to satisfy the goal

Predictive modeling	-O1	-O2	-O3
MultilayerPerceptron	1 (0.01%)	1 (0.01%)	3 (0.02%)
LinearRegression	1 (0.01%)	1 (0.01%)	3 (0.02%)
KStar	1 (0.01%)	1 (0.01%)	2 (0.016%)

5.5.2 Comparisons with State-of-the-Art Models

In this section, we compare MiCOMP with two state-of-the-art intermediate-sequence prediction approaches proposed in [1, 2].

5.5.2.1 Intermediate Speedup Comparison Case. (A)

Kulkarni et al. [2] used *Neuro-Evolution for Augmenting Topologies* (NEAT) to predict the best compiler optimization to apply given the state of source-code being optimized by the dynamic JIT Jikes RVM compiler. They used static source-code features to characterize each state of application under optimization, as opposed to the technique we propose in this chapter where we obtain features of the code only once before it is optimized. Kulkarni et al., used NEAT, a machine-learning framework based on genetic evolution, to generate many neural-networks where each network was evaluated on the task of using static source code features to predict the next compiler optimization to apply. NEAT can make optimization predictions to any given maximum-length to predict the most beneficial sequence of optimizations for the target application being compiled. In NEAT training time was reported around 10 days while the current approach requires a few hours to construct the model. Another advantage of the current work is the fact that is supporting multiple predictions from the prediction-space while the NEAT approach can produce one-shot result based on the stop condition for each application and neural network configuration. We reproduced the work by Kulkarni et al. [2] by using 100 chromosomes and 500 generations on 12 cores Xeon(R) CPU E5-1650 v2 @ 3.50GHz with 12GB running on Ubuntu and we report the result in Table 5.9. We ran NEAT in parallel with average running time of 1.75 hours per model (the longest took 4 hours). For this comparison, we used the same training data of up to the length of 4 for both MiCOMP and Kulkarni et al. to be uniform on both comparisons as NEAT needs feature collection on each iteration and collecting 19 k feature vectors for the number of applications in our training set was impractical. The training/prediction is done with leave-one-out cross-validation to be uniform to the reported experimental results in this chapter. We used Harmonic-mean to average the speedup gains on both models and observed $1.0295 \times$ speedup (5% performance improvement) against the mentioned work.

Table 5.9 Performance comparison of the single prediction by MiCOMP against the intermediate speedup approach reported in in previous work [2]

Application	NEAT		MiCOMP
	Best NN Size	1 prediction	1 prediction
automotive_qsort1	1105	1.0336	1.0385
automotive_bitcount	1536	1.0923	1.0498
automotive_susan_c	607	1	1.0491
automotive_susan_e	613	1	1.0481
automotive_susan_s	1295	1.0135	1.0195
bzip2d	1159	1	0.9698
consumer_jpeg_c	1327	1.0205	1.1882
consumer_jpeg_e	596	1	1.0981
consumer_tiff2bw	1038	0.9522	0.9491
consumer_tiff2rgba	1147	0.9905	0.9295
consumer_tiffdither	612	1	1.0288
consumer_tiffmedian	1356	0.9097	0.9497
network_dijkstra	1343	1.0353	1.1382
network_patricia	622	0.7971	0.8585
Harmonic Mean		0.9742	1.0275

All values are normalized by $-O3$

5.5.2.2 Intermediate Speedup Comparison Case. (B)

Ashouri et al. [1] demonstrated a predictive methodology in order to predict the intermediate speedup obtained by an optimization from the configuration space, given the current state of the application. The fitness function for the intermediate speedup is the ratio between the execution times of the program before and after the optimization process. They exploited predictive models to correlate the current state of the dynamic features of the application under study with the current state of the compiler optimization to come up with a speedup value and utilize heuristics to search that space. Ashouri et al. used dynamic feature selections, however as mentioned in Sect. 5.5.2.1, a major downside in this work is that application feature should be collected on every state by means dynamic feature extraction and this makes the system impractical on large-scale data. There is an extension to the aforementioned work. Their comparison baseline was LLVM's default optimization, while here we provide a comparison against LLVM's $-O3$ (we show MiCOMP can outperform an aggressive optimization setting in LLVM, that is, $-O3$, in only a few predictions.). Figure 5.6 demonstrates the comparison. For this comparison, we used the same training data of up to the length of 4 for both MiCOMP and Ashouri et al. to be uniform on both comparisons. We observe that except the first two predictions, the proposed approach outperforms the intermediate speedup methodology reported in this work and on average MiCOMP brings 11% speedup gain.

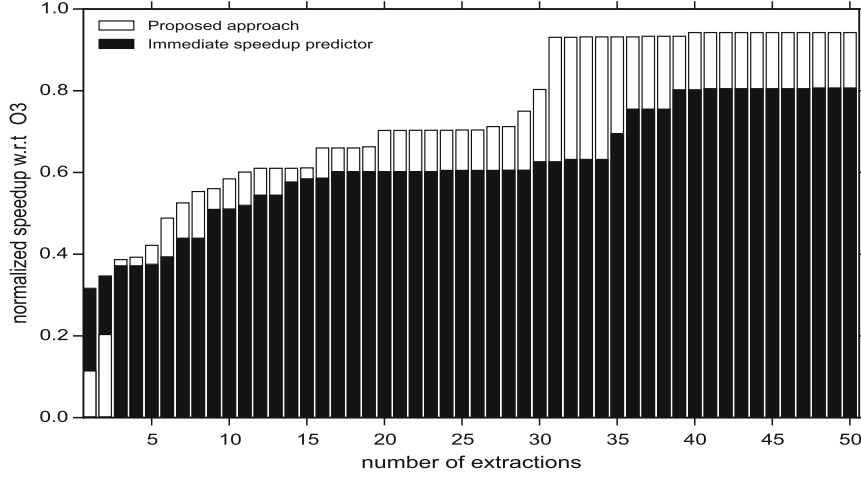


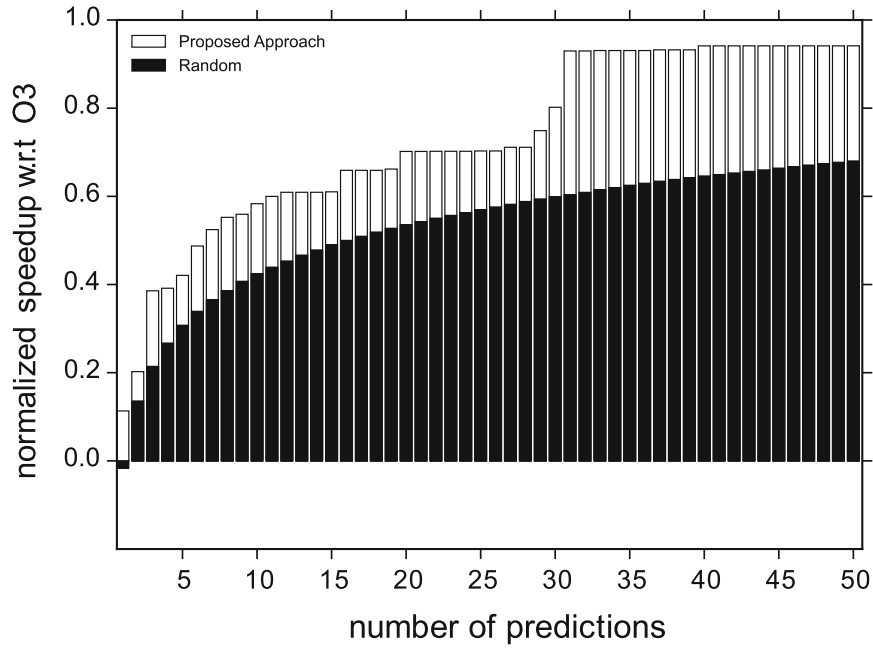
Fig. 5.6 Performance of MiCOMP w.r.t the performance of intermediate speedup predictor approach [1]

5.5.3 Comparison with Random Iterative Compilation

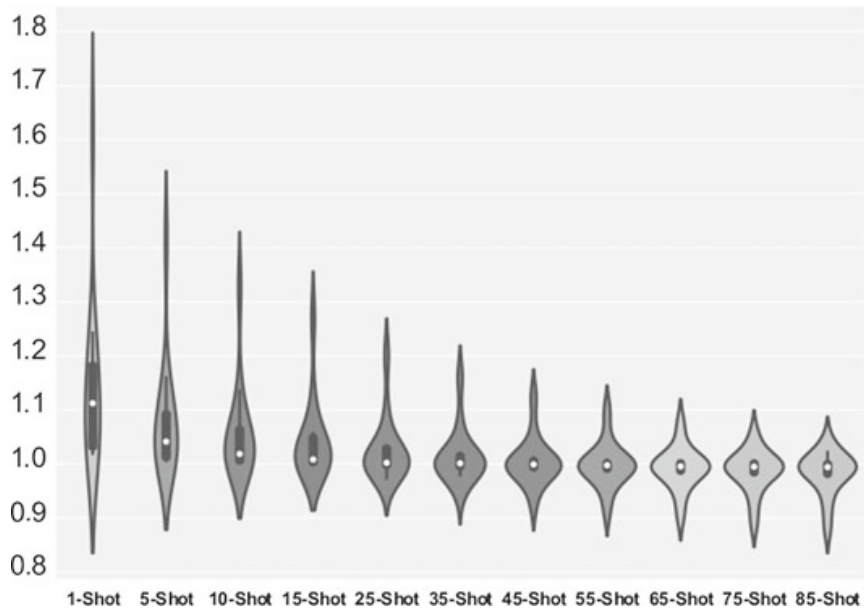
As we illustrated in Sect. 5.4.2.4, iterative compilation can improve application performance over standard compiler optimization sequences [19, 23]. Additionally, several published works have shown that drawing compiler optimization sequences at random can often be as good as using other more complicated search algorithms, such as genetic algorithms or simulated annealing [3, 19, 24]. In this section, we compare the effectiveness of MiCOMP to a Random Iterative Compilation (RIC) method that samples sequences from a uniform distribution. We randomized the distribution of predictions 10000 times to make sure the obtained model is totally uniform. Our results are presented in Fig. 5.7. To present our results, we define *Normalized Performance Improvement* (NPI) as the ratio of the performance improvement achieved over the potential performance improvement:

$$NPI = \frac{E_{ref} - E}{E_{ref} - E_{best}} \quad (5.5)$$

where E is the execution time achieved by the methodology under consideration, E_{ref} is the execution time achieved with a reference compilation methodology and E_{best} is the best execution time that can be obtained through an exhaustive exploration of all possible compiler optimization sequences in the optimization space we are exploring. As the execution time E of the iterative compilation methodology under analysis gets closer to the reference execution time E_{ref} , the value of NPI gets closer to 0, where 0 indicates no improvement was obtained. As E approaches the best execution time, E_{best} , the value of NPI approaches 1. An NPI value of 1 indicates that the optimal performance available was achieved. The goal of the evaluation in this section is to show how effective MiCOMP is at exploring the optimization sequence space compared to RIC. Figure 5.7a shows results for both MiCOMP and RIC with the same. The X axis pertains to the number of predicted optimization sequences



(a) Both MiCOMP and Random exploring the opt. space



(b) Only Random iterative compilation exploring the opt. space (MiCOMP is fixed at 5 predictions)

Fig. 5.7 MiCOMP performance comparison versus Random Iteration Compilation

used and the Y axis shows their corresponding speedup values. We used NPI (scaled within $[-\infty, 1]$) and the speedups are all normalized by $-O3$ performance. Thus, $Y = 0$ is the speedup line corresponding to $-O3$. We observe that the performance of MiCOMP outperforms Random Iterative Compilation for each number of predicted optimization sequences used. Note the larger the number of predicted sequences used, the more significant the performance difference between MiCOMP and RIC. Table 5.5 gives the the absolute speedup values.

Figure 5.7b displays another result where we compare a fixed number of predicted optimization sequences for MiCOMP, that is 5 predicted sequences, versus different number of predicted sequences from RIC. That is we observe the prediction quality of MiCOMP compared to different numbers of predicted optimization sequences drawn from a random distribution. Figure 5.7b depicts this scenario using a violin plot where the Y axis pertains to the speedup with respect to the RIC and the X axis corresponds to the different predicted optimization sequences obtained from RIC. Statistically, we observe that the quality of the 5-prediction of MiCOMP is as good as using 25 prediction optimization sequences from RIC. In our experiments, we observed that the predicted optimization sequences derived by MiCOMP can give up to $5\times$ exploration speedup versus the RIC method.

5.6 Conclusion

This chapter presented MiCOMP, a framework to exploit predictive modeling to solve the compiler phase-ordering problem. We proposed a clustering technique for all the compiler optimizations in LLVM's $-O3$ and clustered them in five different optimization sub-sequences to speedup the training and exploration phase. This method outperforms LLVM's $-O3$ optimization sequence. Moreover, MiCOMP has a simple mapping function that encodes an optimization sequence into a bit string, allowing us to apply standard machine learning techniques that require fixed length feature vectors. We incorporated analogies between the analyzed problem and the context of Recommender Systems, and integrate similarity measures to boost exploration efficiency. We show that MiCOMP outperforms LLVM's standard optimization levels with just a few predicted of optimizations sequences and achieves top 80% of the available speedup by traversing less than 5% of the optimization sequence space. This is rather crucial when the optimization space can consist of a variety of different optimization sequences.

Chapter 6

Concluding Remarks

In this book, we have tackled the major problems of compiler autotuning. We have used machine learning, DSE, and meta-heuristic techniques to construct efficient and accurate models to induce prediction models.

This chapter unfolds in two main sections. Section 6.1 summarizes the main contribution and Sect. 6.2 provides a list of open issues and our future direction towards tackling those challenges.

6.1 Main Contributions

This book presented the following contributions.

1. We extensively surveyed the literature of the past 25 years and classified the papers by the type of contribution, the prediction models, and many other sub-features. We hope the survey is useful for a wide range of researchers and industry professionals.
2. We provided a co-exploration framework to statistically analyzing the compiler level parameters over a customized VLIW architecture. Moreover, we applied several statistical tests, e.g., ANOVA, Kruskal-Wallis, and clustering techniques and we showed that not all the available compiler parameters are beneficial to use within the embedded domain.
3. Chapter 4 presented COBAYN framework for autotuning compiler passes using Bayesian networks. It uses application's characteristics to predict the best set of optimization passes on given application. We experimentally showed that this framework outperforms GCC's standard optimization levels, i.e., `-O2` and `-O3` and betters the state-of-the-art approaches.
4. An intermediate speedup predictor was presented towards tackling the phase-ordering problem in Chap. 4. Choosing the best local alternative as the next-best

optimization to use might lead to a local minimum, whereas other paths with less steep initials, might end up in a better global point. The main goal of this chapter was to familiarize with the difficult problem of the phase ordering of the compiler optimizations and an intuitive approach to tackling such.

5. Last but not least, MiCOMP was presented as a framework for mitigating the phase-ordering problem. It uses predictive modeling to form the best ordering of phases for an application under analysis. A major advantage of MiCOMP is the fact that it predicts the full optimization sequence at once. We experimentally showed that the framework outperforms state-of-the-art techniques, also within a few predictions it can outperform standard optimization levels.

6.2 Open Issues and Future Directions

We identify the challenges and open issues involved in optimizing compilers.

1. Leveraging COBAYN and MiCOMP address a few research questions. However, these approaches can be further improved and be more accurate. Inducing prediction models always deals with approximations; therefore there wouldn't be a definite optimal result for a given problem. By the advancement in new machine learning techniques, i.e., deep learning, etc. it is expected that we adapt more accurate models to achieve results, yet this remains an open question to be answered.
2. Parallel and heterogeneous computing brought both new applications and challenges for autotuning. Now, it is up to the compiler researchers to adapt and introduce novel tuning techniques for CPU, GPU, and other accelerators. This direction is indeed both challenging and interesting for future research.
3. Multi-objective optimization strategies need to be exploited for power-aware and energy efficient systems. Finding the right Pareto-curve representing the right set of optimization is still an open challenge.

We hope that the book paves the way to fine-grain knowledge and understanding of the discussed problems and light up more ideas for young researchers to carry on the path towards tackling the existing open issues.